

The background of the slide features the Kubernetes logo, which is a large, light blue hexagon containing a ship's steering wheel. The title text is centered over this logo.

Docker und Kubernetes

Alexander Erben
Gartenstr. 48
53721 Siegburg
info@it-erben.com
<https://linkedin.com/in/aerben>

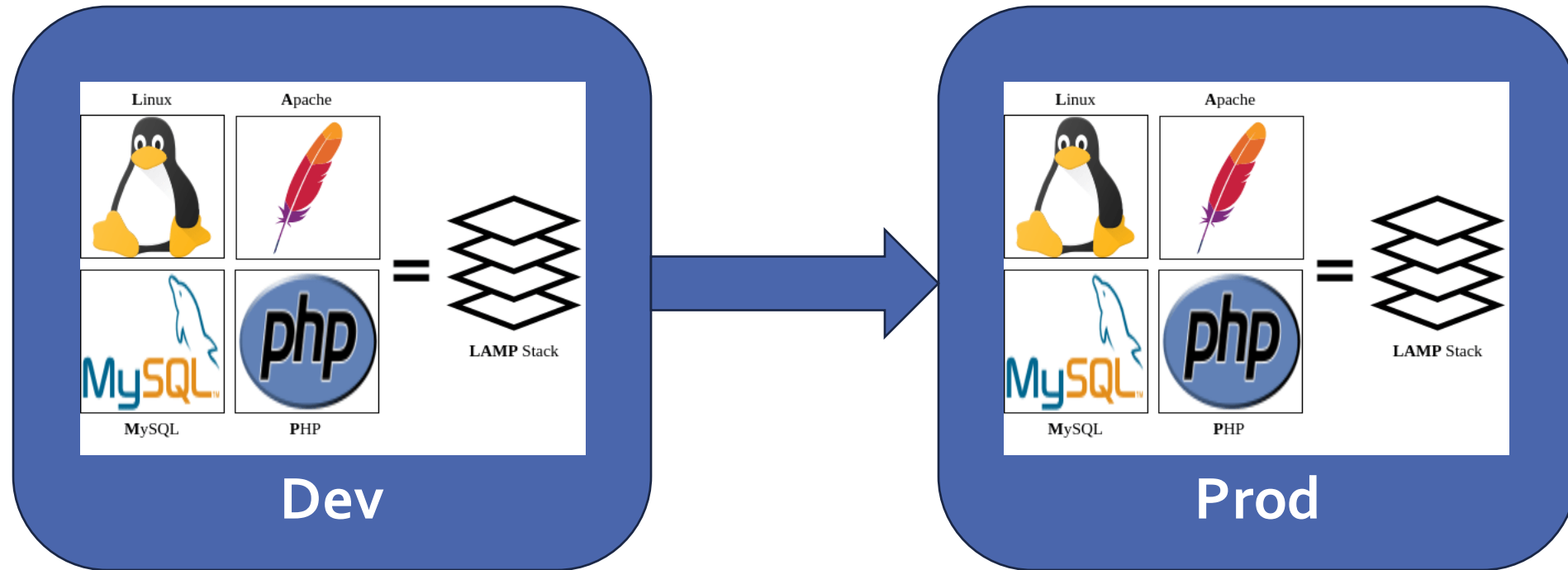
Container – Einführung

Motivation

- Anwendung mit einer **spezifischen Version** (z.B. von PHP) ausliefern.
- Tools mit **komplexer Einrichtung** (z.B. ImageMagick).
- Applikationen gleichförmig und **reproduzierbar** ausspielen

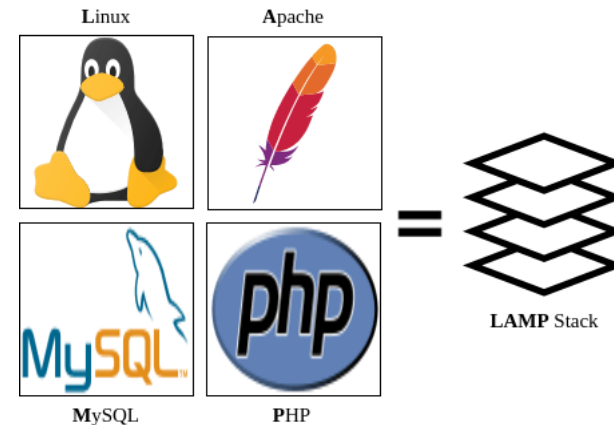


Herausforderung: Deployment



Herausforderungen beim Deployment

- Richtige **Versionen** der Komponenten
- Korrekte **Konfiguration** der Applikationen
- Richtige **Mounts**
- Richtige **Netzwerkregeln**

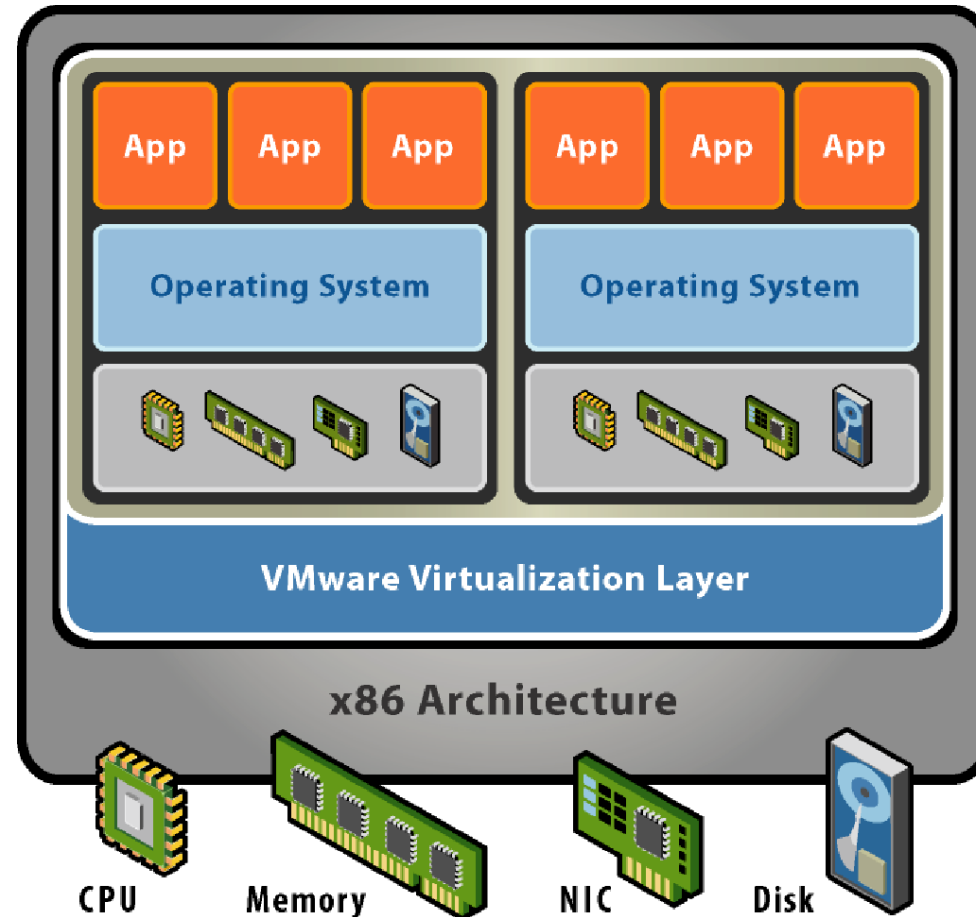


Gibt es dafür nicht VMs?

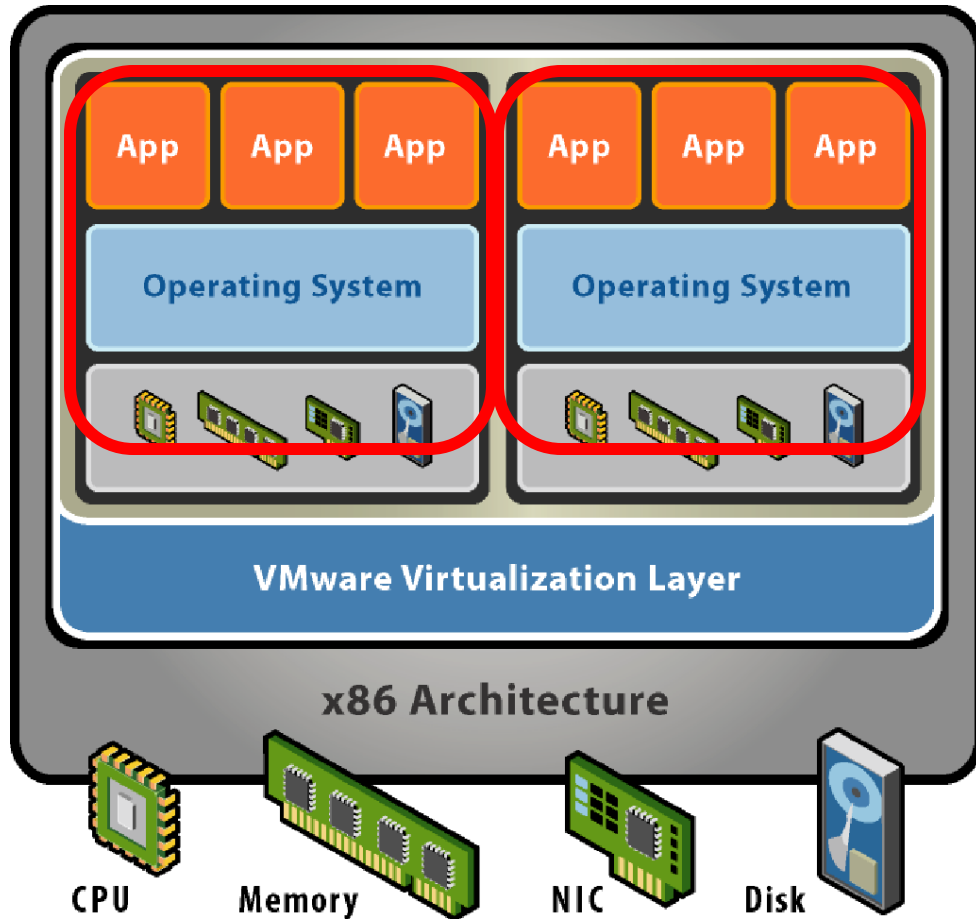
- Virtuelle Maschinen sind alles andere als neu
- Schon seit mehr als 70 Jahren!
- Sie machen die Systemumgebung vom Host unabhängig



Aufbau virtueller Maschinen (Beispiel VMware)



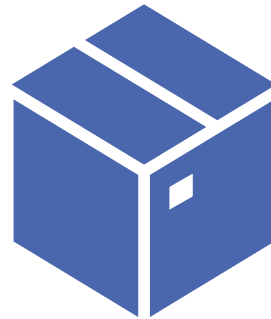
Nachteile von VMs



- Alles „doppelt gemoppelt“
- Hoher Ressourcenverbrauch
- Schlechte Entwicklungsergonomie
- „Alles ist langsam“ für Entwickler

Auftritt: Container

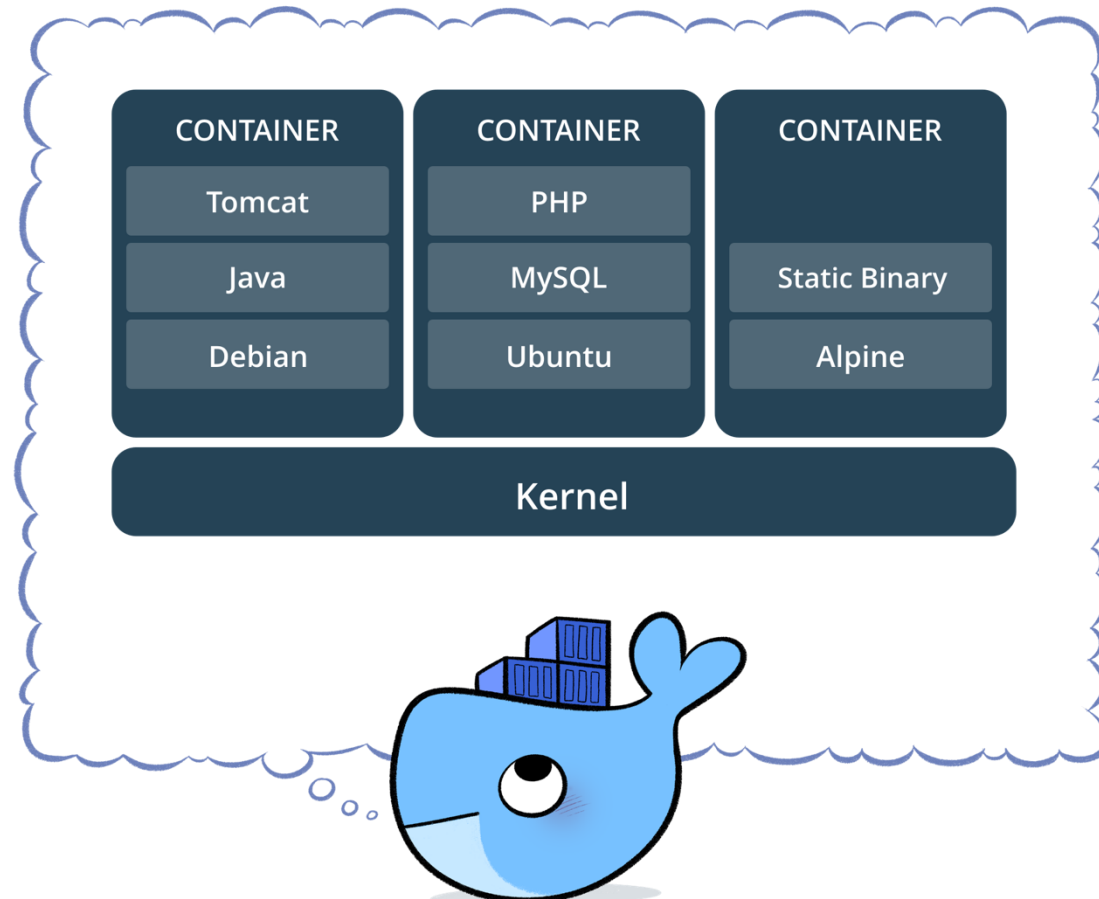
Container sind:
leichtgewichtige,
portable und
isolierte Einheiten,
die Anwendungen und
deren Abhängigkeiten verpacken.



Vorteile von Containern

- **Reproduzierbarkeit:** Container werden aus Images erstellt, die den Anwendungscode und dessen Abhängigkeiten enthalten.
- **Unabhängigkeit:** Der Container ist weitestgehend unabhängig von der Einrichtung des Hostsystems.
- **Entwicklerproduktivität:** Einfache Entwicklung und Testen von Funktionen in einer sauberen Umgebung.
- **Ressourceneffizienz:** Container teilen sich den Kernel

Ressourceneffizienz: Geteilter Kernel



Container Stack

Orchestrator

- Management
- Skalierung
- Erreichbarkeit

High Level Runtime

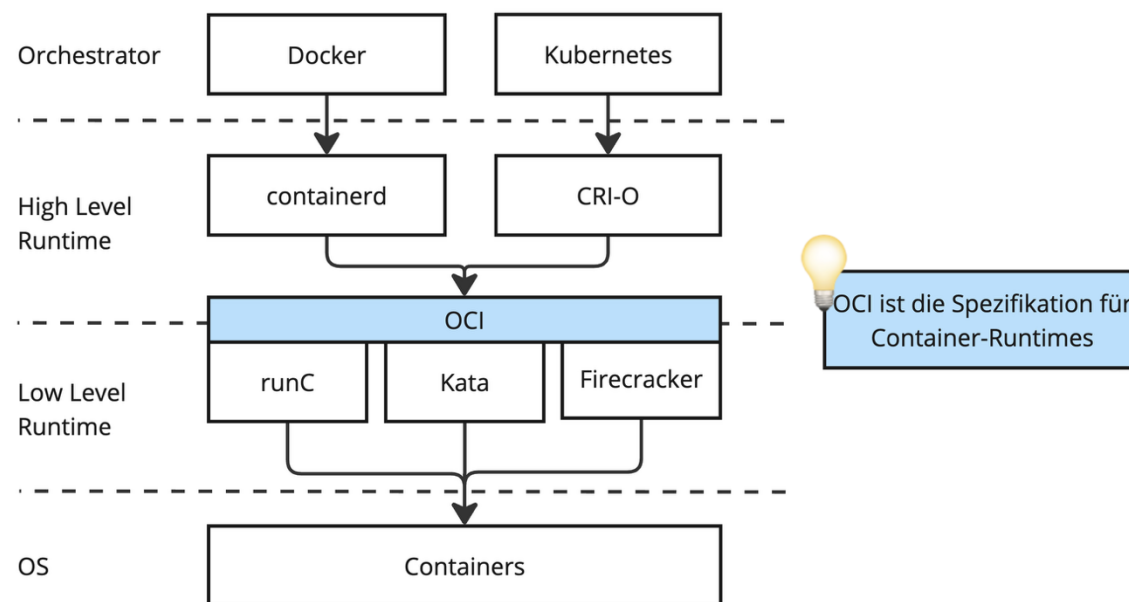
- Standard-API
- Image Handling
- Logging, Monitoring

Low Level Runtime

- Kernel-Interaktion
- Container-Prozesse
- Namespaces (PID, Netzwerk, User)

Container Stack

- Container werden von einem **Orchestrator** verwaltet. Er bestimmt, welche Container wo wann laufen.
- Die **High Level Runtime** bieten Schnittstellen zur Arbeit mit Containern unabhängig von der Implementierung
- **Low Level Runtimes** setzen die eigentliche Arbeit mit Images und Containern um
- **OCI** ist der Standard für solche Runtimes. Er wurde 2015 von Docker Inc. erarbeitet



Container Runtimes

- High Level (Docker, CRI)
 - **containerd** ist die Standard-Runtime für den Docker-Stack. Mittlerweile seltener im Einsatz, denn...
 - **cri-o** ist die neuere Standard-Runtime für den Einsatz in Kubernetes. Sie implementiert die Kubernetes-APIs für die Arbeit mit Containern
- Low Level (OCI)
 - **runC** wird sowohl von containerd als auch cri-o verwendet, um Container auf dem Linux-Kernel laufen zu lassen
 - **Kata** ist eine Runtime für das Betreiben von Containern in Lightweight VMs
 - **Firecracker** ist die Runtime für Serverless Computing in AWS

Container und der Linux-Kernel

Runtimes wie runC interagieren mit dem Linux-Kernel hauptsächlich durch Kernel-Features wie **cgroups, namespaces, capabilities und secComp-Profilen**

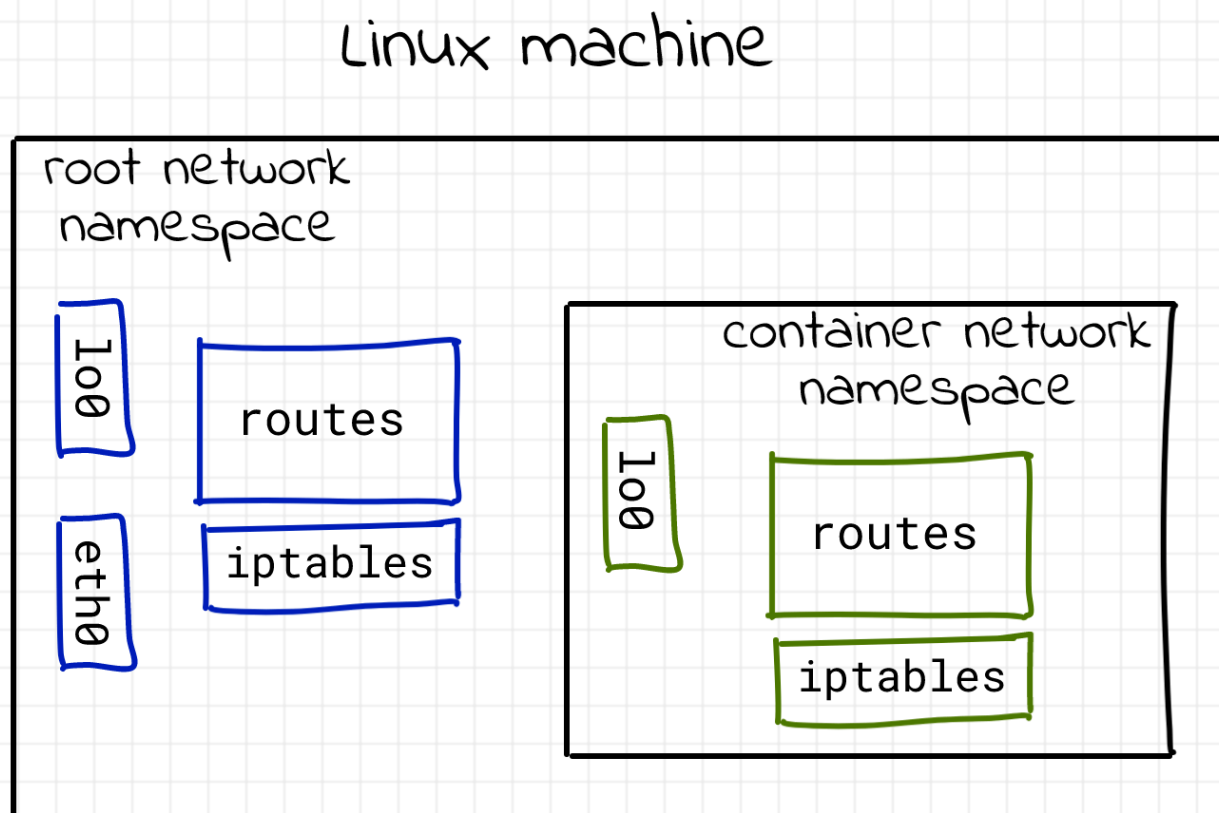
Es gibt auch eine Container-Laufzeit für mich. Dazu später mehr.



Linux Namespaces

- Ein **Linux Namespace** ist ein Kernel-Feature, das Isolation und Abtrennung von Systemressourcen für verschiedene Prozesse ermöglicht.
- Namespaces stellen sicher, dass Prozesse keine anderen Prozesse anderer Namespaces beeinflussen können
- Es gibt **verschiedene Arten** von Namespaces in Linux, die sich auf unterschiedliche Ressourcenaspekte beziehen (Prozesse, Netzwerk, Benutzer...)

Network Namespaces

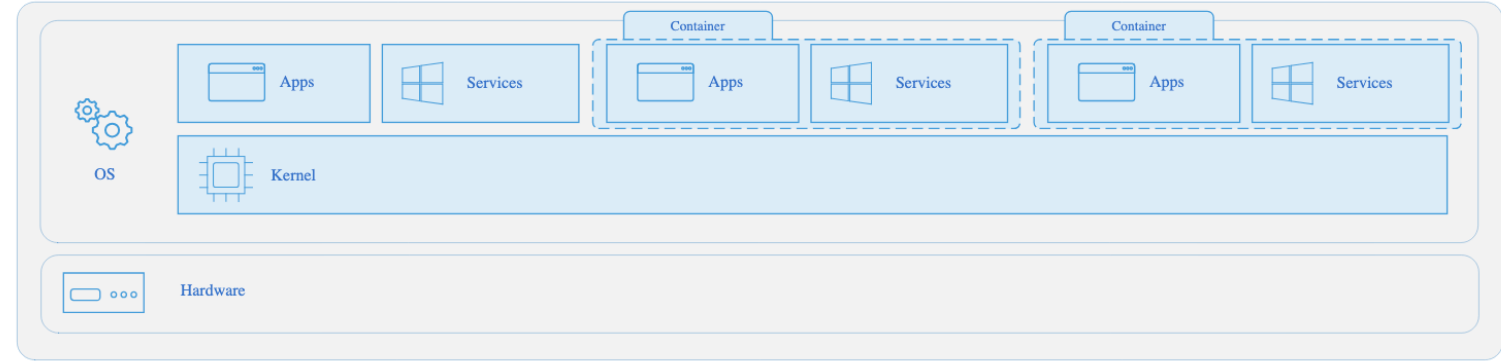


[Digging into Linux namespaces - part 1 \(quarkslab.com\)](https://quarkslab.com/blog/digging-into-linux-namespaces-part-1/)

Container auf Windows

Gegenstück zu runC ist auf Windows runhcs.

Windows hat damit mittlerweile eine native OCI-kompatible Runtime.



<https://learn.microsoft.com/de-de/virtualization/windowscontainers/about/containers-vs-vm>

Docker – Einstieg

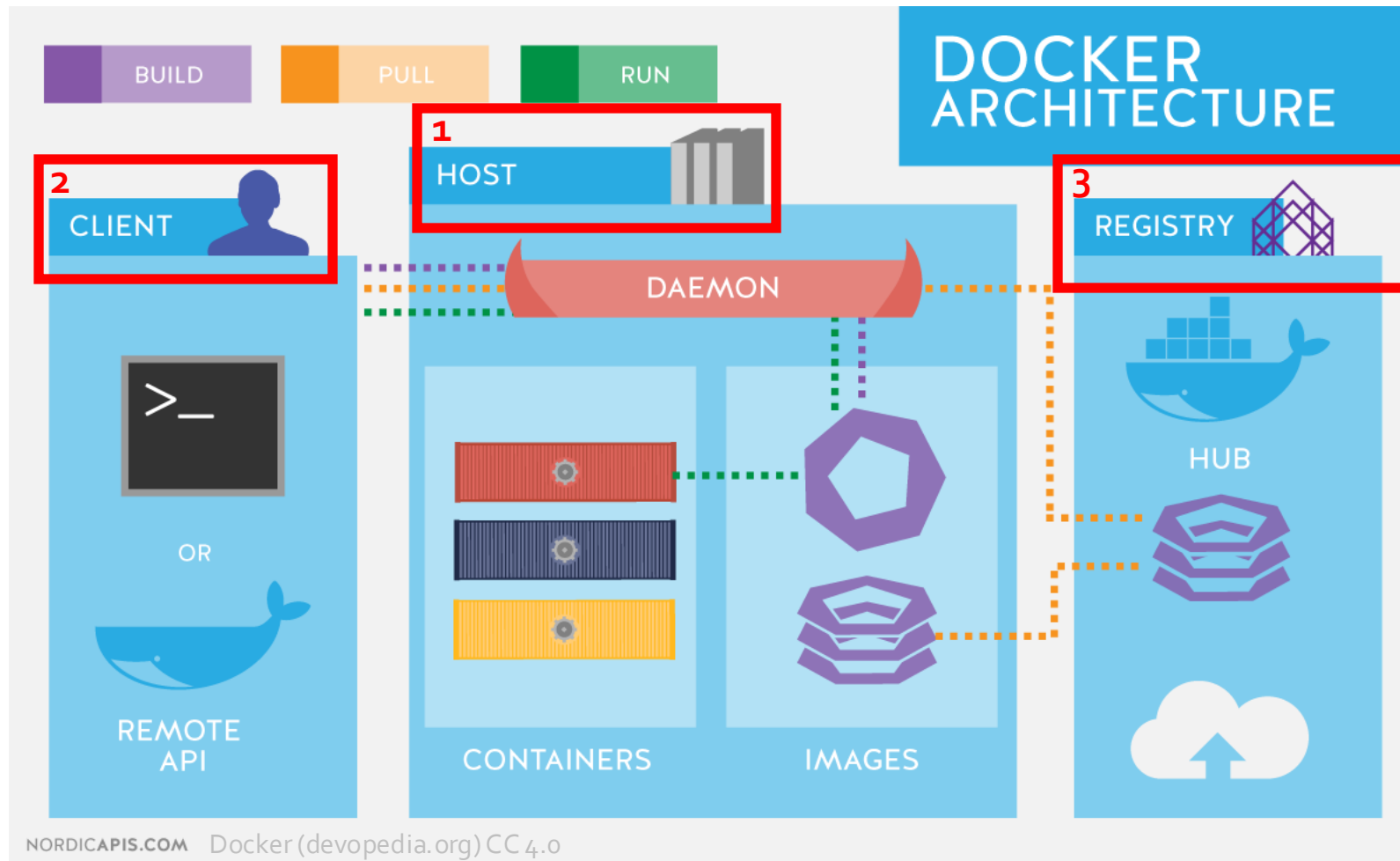
Docker

- Docker ist *nicht* einfach nur eine Laufzeit, um Container auszuführen
- Genau genommen wird oft gar nicht mehr Docker als Laufzeit benutzt (mehr dazu beim Thema Kubernetes)
- Docker ist eine **Plattform**, um **Container** und Container-Images zu **entwickeln**, zu **bauen**, zu **verwalten**, zu testen und zu verteilen
- Es enthält dazu **Client-** und **Server-Komponenten**

Warum Docker?

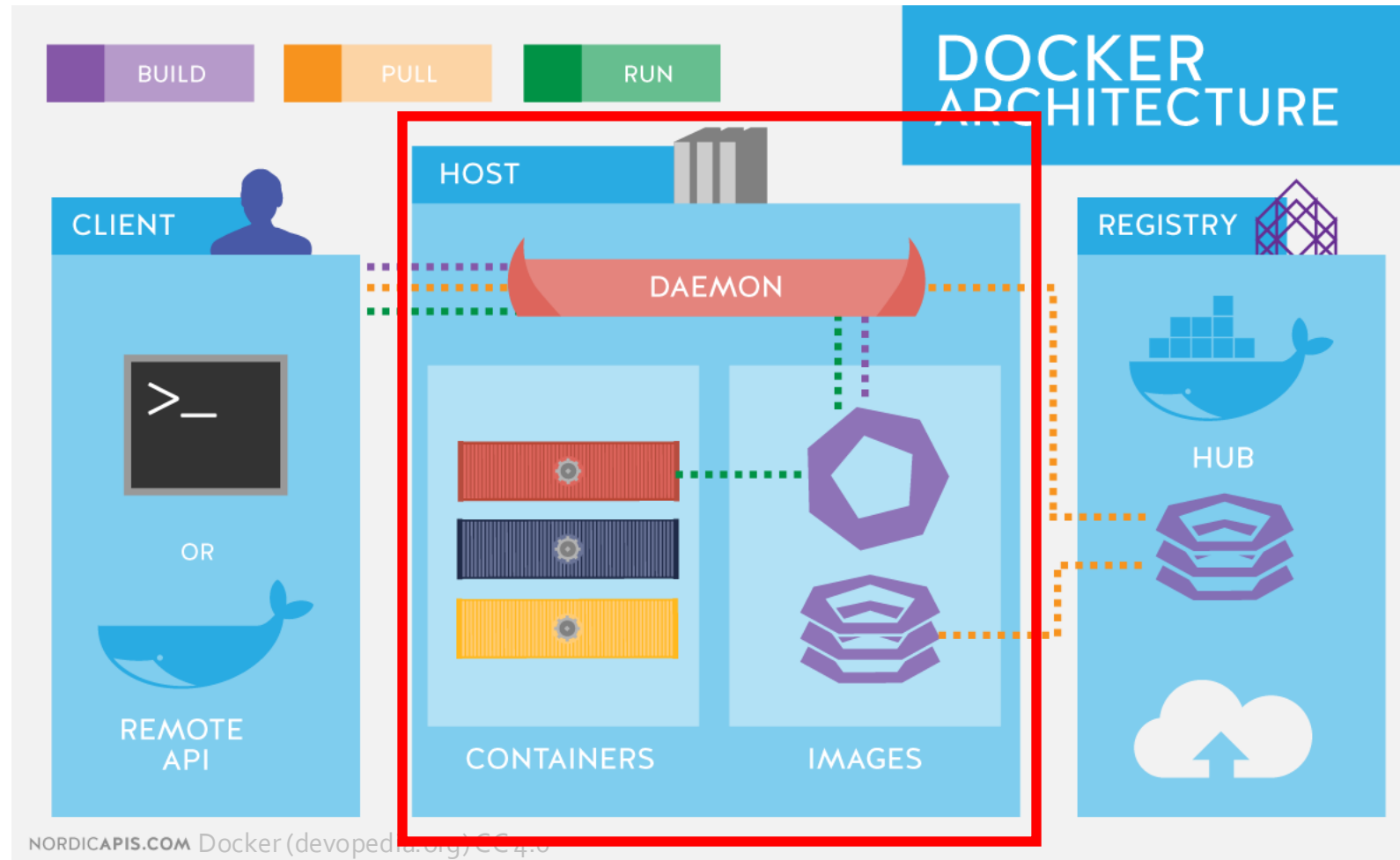
- Der Docker-Stack hat in Produktion an Bedeutung verloren
- **Kubernetes unterstützt Docker nicht mehr direkt**, stattdessen ist oft cri-o im Einsatz
- Für Entwickler ist Docker aber eine **sehr gute Entwicklungsumgebung** für Images und Container, besonders auf Windows
- **Docker Desktop** bietet viele praktische **Tools** und liefert direkt eine Runtime mit (containerd)
- Docker-Images und Container sind **OCI-kompatibel** und damit auch in Kubernetes über cri-o lauffähig

Docker Architektur



Docker Architektur

1



Docker Host

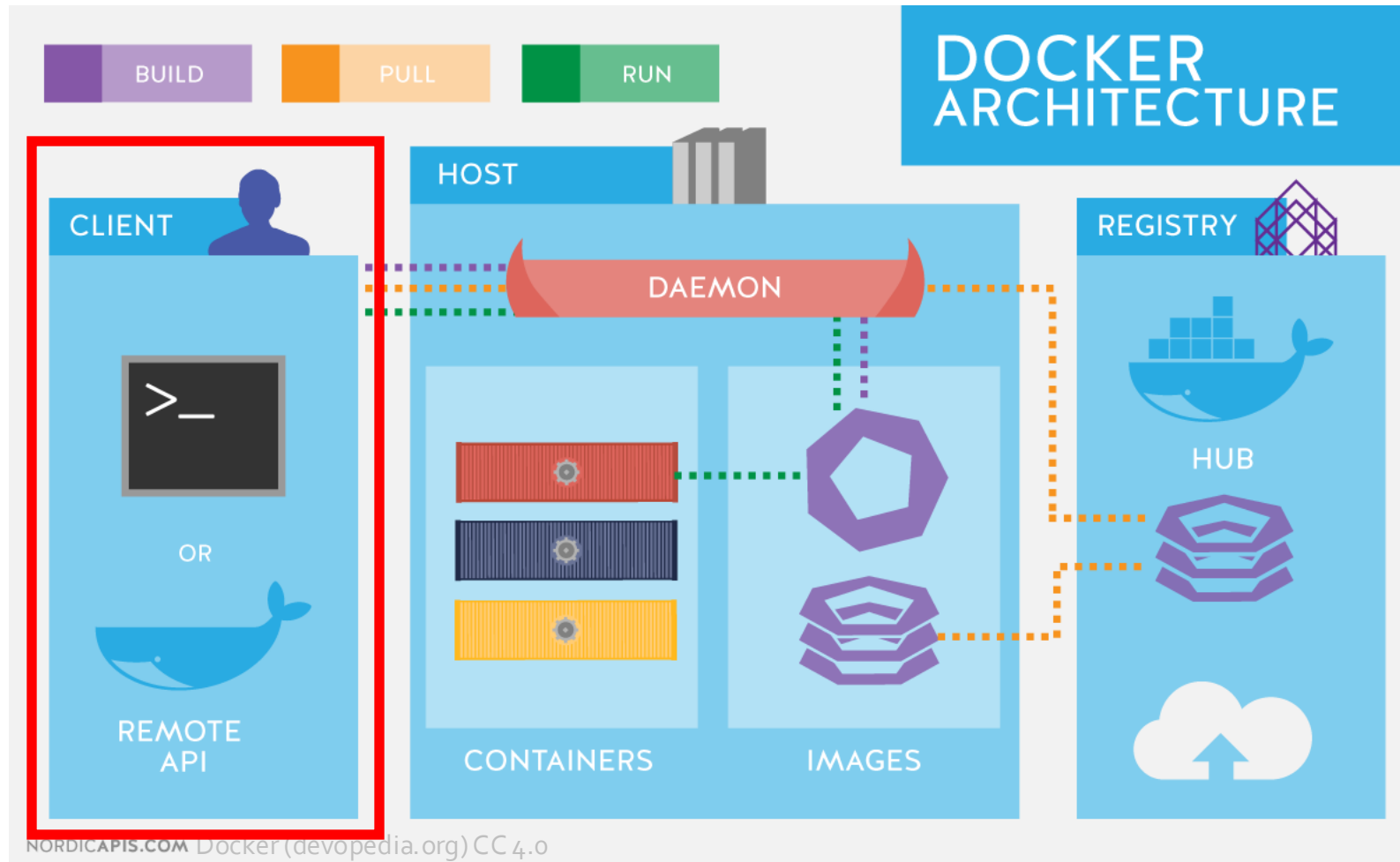
1

- Führt den **Docker Daemon** aus
- Ist für das **Erstellen, Starten** und **Verwalten** von **Containern** zuständig, indem es u.A. Images lädt
- Das Kommando `docker info` zeigt einige Informationen zum Daemon
- Docker Desktop liefert den Docker Daemon bereits mit, um lokal Container zu starten

Daemon: ein Prozess, der im Hintergrund arbeitet und Dienste bereitstellt

Docker Architektur

2



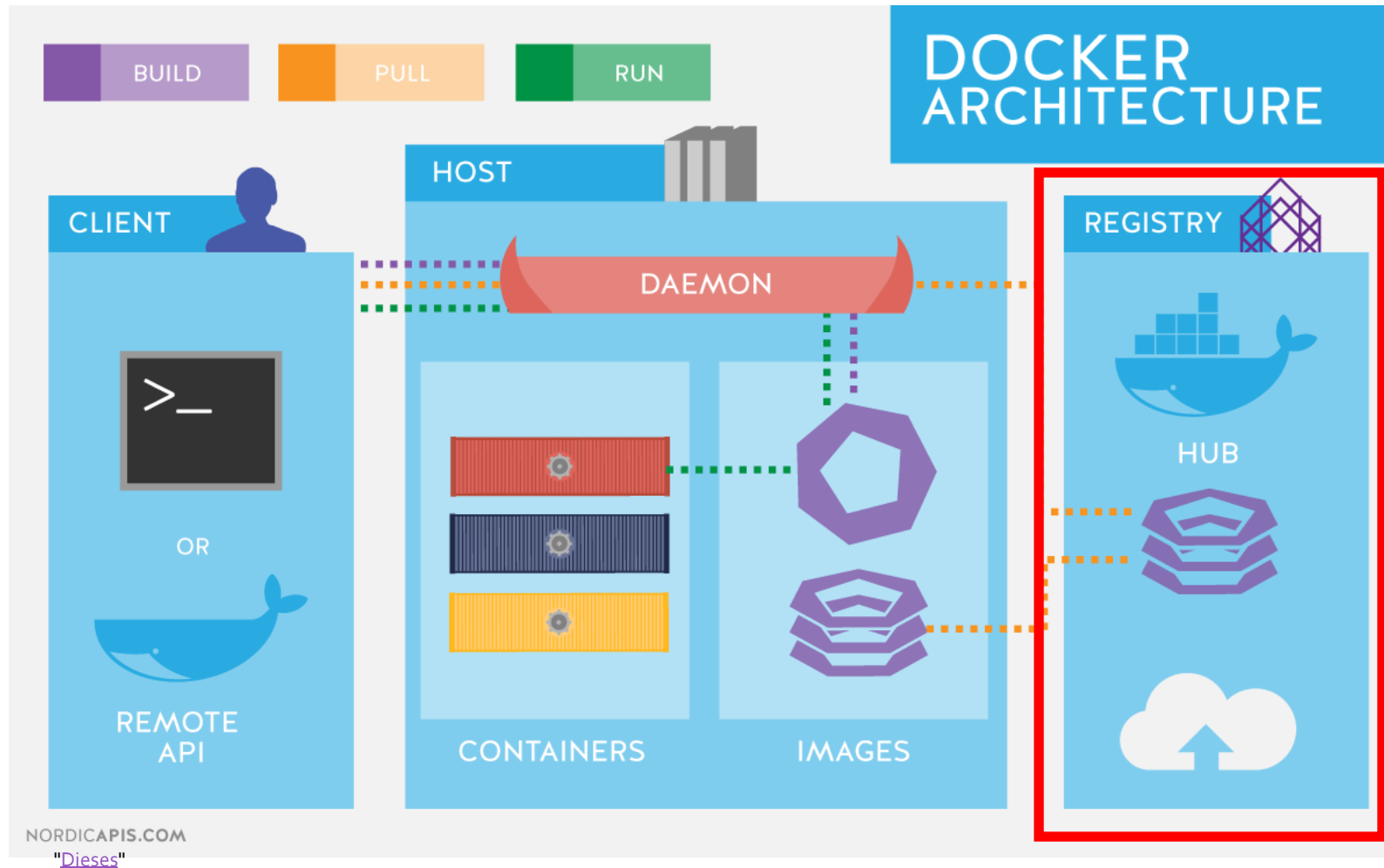
Docker Client

2

- Kommuniziert mit dem Docker Daemon über die **Docker API**
- Wird über ein CLI gesteuert
- Ebenfalls in Docker Desktop enthalten
- Bei Docker Host und Docker Client können auf dem **gleichen Host** laufen, oder auf **verschiedenen**

Container Registries

3



Container Registries

3

- Container Registries sind **Server**, die **Images bereitstellen**
- Beispiel Docker Hub: Plattform mit einer Vielzahl öffentlicher Images
<https://hub.docker.com/>
- Jede/r kann aber **eigene Registries** bereitstellen, auch private.

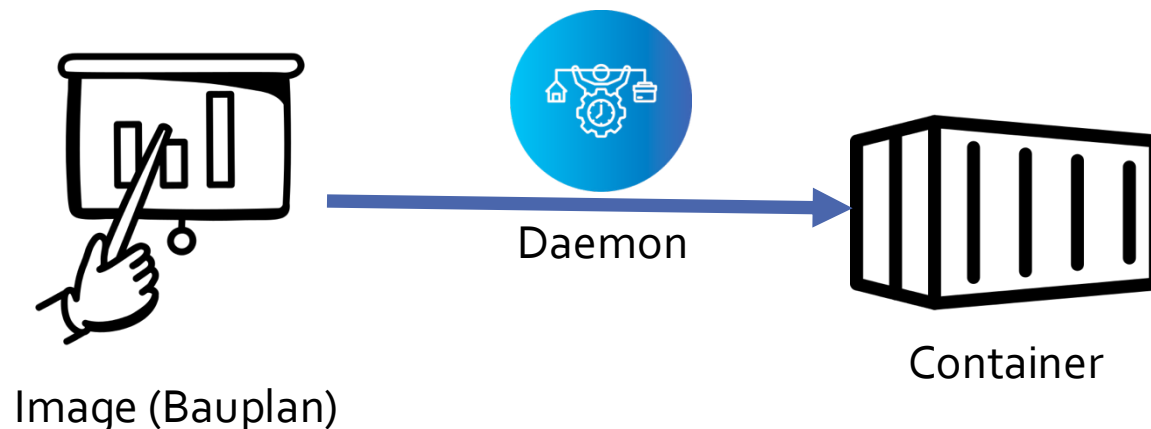
Praxisaufgabe

Docker Client, Host und Registries

Container Images

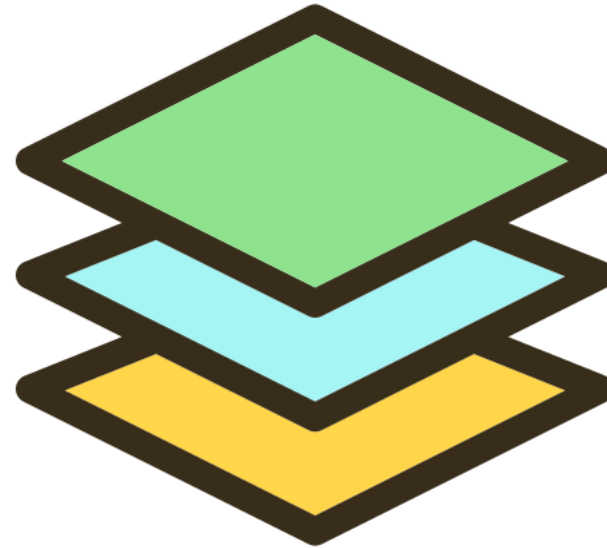
Container und Images

- **Container** werden auf der Basis von **Images** erstellt
- Ein Image ist ein **Bauplan** für Container
- Images enthalten alle erforderlichen Daten (Dependencies, Konfigurationen), um einen Container zu starten
- Images sind nach ihrem Erstellen **unveränderlich** (read-only-Dateisysteme)



Aufbau von Images

- Jedes Image besteht aus Schichten (**Layers**)
- Jeder Layer stellt theoretisch ein **eigenständiges Image** dar
- Layer bauen aufeinander auf. Sie speichern nur die **Differenz** zum letzten Layer
- Layer werden von Docker **gecached**, sodass sie nicht bei jedem Bauen neu geladen werden müssen



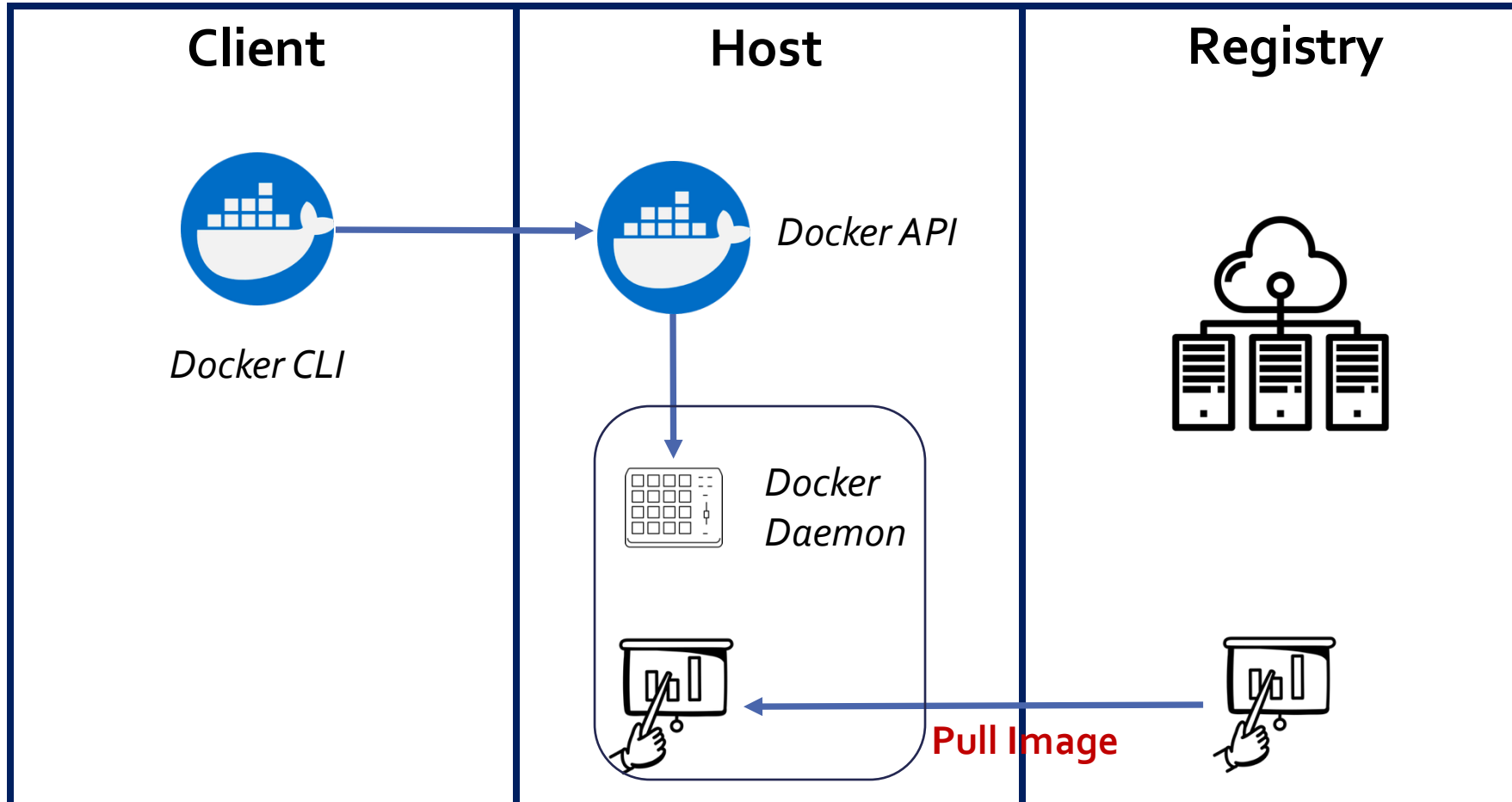
Kommandos zum Inspizieren von Images

```
$ docker images # Listet lokale Images auf
$ docker image inspect <<ImageName>>
$ docker image inspect <<ImageName>> --format "{{.Created}}"
# Weitere Beispiele:
# --format "{{.Size}}"
# --format "{{.Architecture}}"
# --format "{{.Os}}"
```

Docker Pull

- `docker pull` ist ein wichtiges Kommando zur Interaktion mit **Docker-Images** und **Remote-Registries** (z.B. Docker Hub).
- Mit `pull` werden Images oder spezifische Tags aus einer Registry **heruntergeladen**, um sie auf lokalen Systemen zu verwenden.
- Durch regelmäßige Nutzung von `pull` stellt man sicher, dass die **neuesten** oder gewünschten Versionen von Images und Anwendungen verfügbar sind.

Pull Workflow



Anatomie einer Image-URL



Verwirrend: Viele Namensbestandteile sind optional, wenn sie bestimmte Werte haben. Obiges Image z.B. kann auch einfach **python** genannt werden, da „docker.io“, „library“ und „latest“ jeweils die Standardwerte sind.

Weitere Details



- Der Host der Registry ist im Unternehmensumfeld in der Regel **nicht** „docker.io“. Firmen hosten meist ihre eigenen Registries.
- Die Bedeutung des „Namespace“-Teils der URL schwankt je nach Registry. Bei „docker.io“ ist es die Organisation, dem das Image gehört. Es kann sich aber auch um ein Verzeichnis in der Registry handeln.
- „Latest“ ist der Standard-Tag. Das macht ihn gefährlich: häufig machen sich Nutzer keine Gedanken darüber, was „latest“ für dieses Image genau bedeutet.

Docker Tags

- Docker-Tags sind **Labels**, die zur Unterscheidung verschiedener **Versionen** von Docker-Images innerhalb eines Repositorys dienen.
- Tags helfen Entwicklern, Images in Entwicklungs- und Bereitstellungsprozessen effizient zu **verwalten** und zu **organisieren**.
- Tags sind **mutable**: die Image-Version, auf die sie zeigen, kann sich ändern

Tagging

- Man kann mittels des `tag` einen Alias sowie ein neues Tag vergeben.

```
$ docker image tag <NAME:TAG> <NEW_NAME:NEW_TAG>
```

```
$ docker image tag ubuntu:latest my-ubuntu:latest # Namens-Alias, gleicher Tag
```

```
$ docker image tag python:3 python:my-3 # Gleicher Name, weiterer Tag
```

- Tags lassen sich über Image Inspect einsehen

```
$ docker image inspect -f "{{ .RepoTags }}" ubuntu
```

- Docker Pull kann direkt ein Image mit einem bestimmten Tag pullen

```
$ docker pull python:3
```

Images bzw. Tags löschen

- Man löscht Images nicht direkt, sondern erstmal nur Tags

```
$ docker image rm <IMAGE_NAME:TAG>
```

```
$ docker image rm python:3
```

- Erst wenn alle Tags eines Images entfernt wurden, wird das Image endgültig gelöscht
- Du kannst ein Image nur löschen, wenn bei dir kein Container mehr existiert, der darauf basiert

Praxisaufgabe

Image in Docker

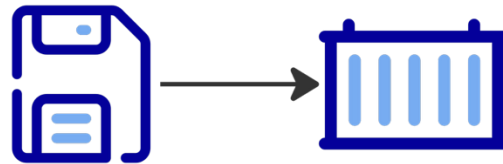
Mit Containern arbeiten

Container erzeugen

Wie kommt man nun von einem Image zu einem Container?

```
$ docker create <IMAGE>
```

Der Container ist aber noch nicht gestartet!



Container auflisten

```
$ docker container ls -a
```

Zeigt alle Container an, auch die, die noch nicht gestartet wurden.
Container haben eine ID, die wir brauchen, um sie zu starten/zu stoppen.
Der Name kann auch genutzt werden.

```
$ docker create nginx  
da69874e3135642ee
```

```
$ docker container ls -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	NAMES
da69874e3135	nginx	"..."	4 seconds ago	Created	happy_chatterjee

Container mit Namen erzeugen erzeugen

Man kann den Namen eines Containers selbst angeben.
Ein Name ist ein „Spitzname“ als Ersatz zur ID

```
$ docker create --name <MY-NAME> <IMAGE>
```

```
$ docker create --name my-nginx nginx
```

```
$ docker container ls -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	NAMES
da69874e3135	nginx	...	...	Created	my-nginx

Mehr zu Namen

Namen von Containern lassen sich ändern.
Sie müssen eindeutig sein.

```
$ docker rename <OLD NAME> <NEW NAME>  
$ docker rename ubuntu ubuntu-test
```

Container starten

Ein Container wird mit seiner ID oder seinem Namen gestartet.

```
$ docker start <ID/NAME>
```

```
$ docker start my-nginx  
da69874e3135642ee67f1fe8
```

```
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
da69874e3135	nginx	"..."	4 seconds ago	Up 4 seconds

Container stoppen

Ein Container wird mit seiner ID oder seinem Namen gestoppt

```
$ docker stop <ID/NAME>
```

Ein Container wird gestoppt, wenn das Hauptprogramm beendet wird.

Insbesondere ist dies der Fall, wenn das Hauptprogramm abstürzt.

Container löschen

Ein Container wird mit seiner ID oder seinem Namen gelöscht.

```
$ docker rm <ID/NAME>
```

Der Container muss vorher gestoppt werden!

Container pausieren

Statt einen Container zu stoppen, kann man ihn auch nur **temporär pausieren** und ihn später wieder starten.

Ein Container wird mit seiner ID oder seinem Namen pausiert.

```
$ docker pause <ID/NAME>  
$ docker unpause <ID/NAME>
```

Docker Run

`docker run == Pull + Create + Start`

```
$ docker run -it --rm --name my-python python:3
```

- `-t` öffnet ein Terminal im Container
- `-i` hält eine Verbindung zum Container offen
- `--rm` bewirkt, dass der Container nach dem Stopp direkt gelöscht wird.

Zusätzliche Programme starten

Mit Hilfe von folgendem Befehl kann man mehrere Programme innerhalb des gleichen Containers starten:

```
$ docker container exec [OPTIONS] <CONTAINER> <COMMAND>
```

Man kann damit z.B. Programme nachinstallieren oder Dateien im Container anlegen.

Argumente von Docker Exec

```
$ docker exec praxisaufgabe2 "apt-get" "install" "nano" "-y"
```

Enthält der Befehl Leerzeichen, so müssen die Wörter **einzeln** übergeben werden.

Praxisaufgabe

Container

Container Filesystem und Volumes

Docker Dateisystem

Images sind **read-only**.
Sie können nicht verändert werden

Dateien, die in einem Docker-Container erzeugt oder verändert werden, befinden sich im **Writable Layer** des Containers

Wenn ein Container gelöscht wird, dann gehen auch **alle** seine **Dateien** im Writable Layer **verloren**

Docker cp

Mit dem Befehl `cp` kannst du schnell Dateien oder Ordner in einen Container und aus einem Container kopieren:

```
$ docker cp <PATH_ON_HOST> <CONTAINER>:<PATH_ON_CONTAINER>  
$ docker cp ~/.bashrc my_container:/root/.bashrc
```

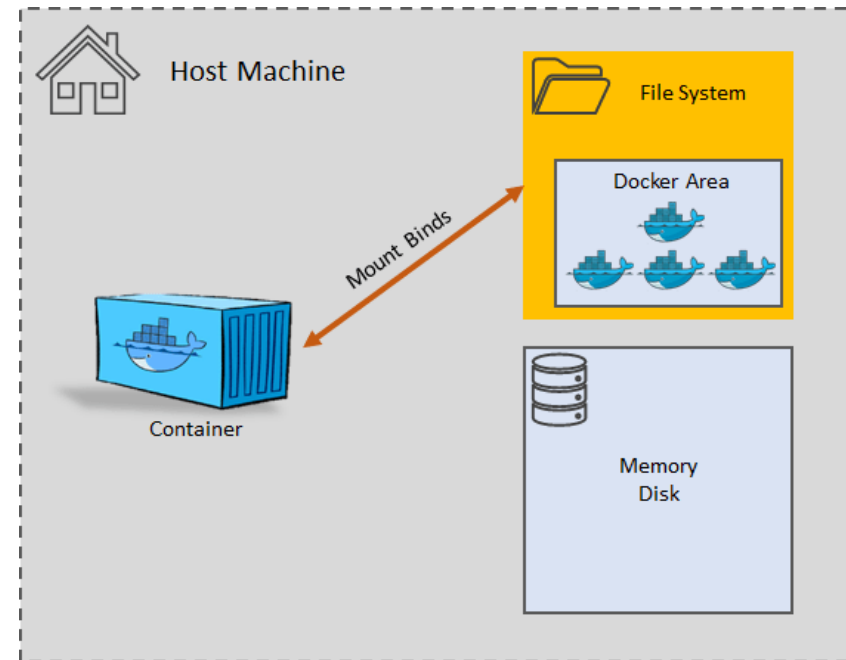
Element aus einem Container heraus in das Host-System kopieren:

```
$ docker cp <CONTAINER>:<PATH_ON_CONTAINER> <PATH_ON_HOST>  
$ docker cp my_container:/root/.bashrc ~/.bashrc
```

Funktioniert auch bei gestopptem Container

Dateisysteme mounten

- **Mounts** sind **Verknüpfung** von der Verzeichnisstruktur eines **Containers** mit **fremden Dateisystemen**
- Container können Daten so **persistent** auf dem **Host** sichern
- Dateien bleiben so auch nach Löschen des Containers bestehen



[Docker Volume vs Bind Mounts vs tmpfs mount - Digital Varys](#)

Arten von Mounts

Es gibt drei verschiedene Arten von Mounts

1. [bind](#)

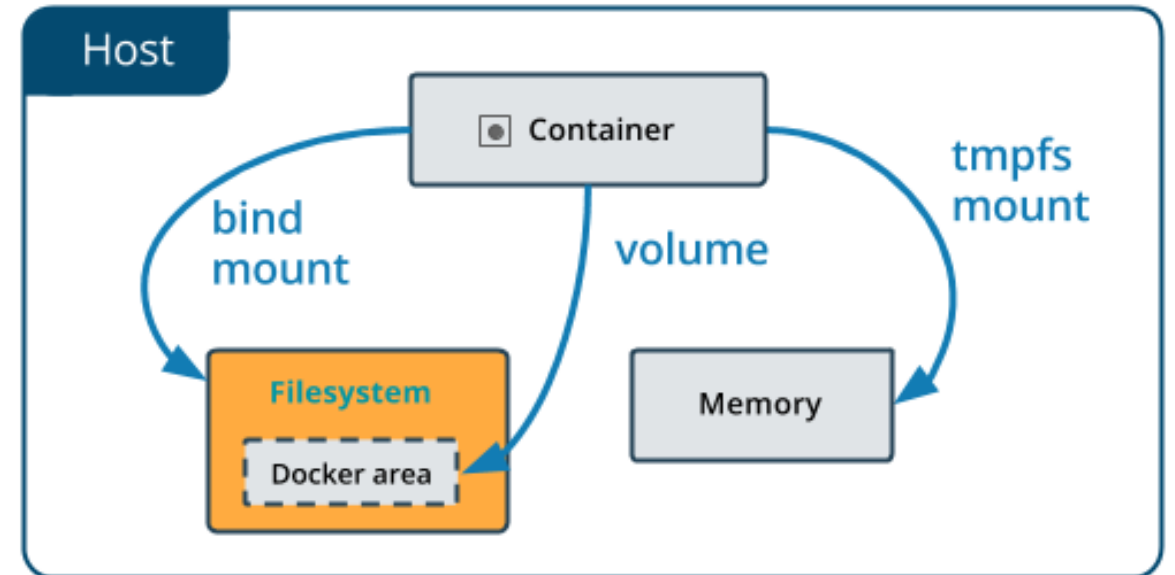
Container wird mit Host-Dateisystem verknüpft

2. [volume](#)

Container erhalten einen Mount, der von Docker selbst verwaltet wird

3. [tmpfs](#)

Daten werden in den RAM des Hosts geschrieben und nach dem Stopp gelöscht. Nützlich für sensible Daten. Nur unter Linux verfügbar!



[Bind mount](#) | Cloud Native Viet

Bind Mounts



https://as2.ftcdn.net/jpg/02/11/41/45/220_F_211414585_NdJFoIDP3BynbtFtBftwvzq6d4zudhRq.jpg

Bind Mounts

- 1.Integration:** Bind Mounts ermöglichen die Einbindung des Dateisystems eines Host-Computers in einen Docker-Container.
- 2.Wechselseitige Aktualisierung:** Änderungen an Dateien und Verzeichnissen im Bind Mount sind in Echtzeit für Container und Host sichtbar.
- 3.Entwicklung & Testen:** Bind Mounts sind nützlich, um Code während der Entwicklung direkt im Container zu testen und Daten flexibel zu speichern.



Bind Mount

Bind-Mounts werden mit dem `-v` Flag angelegt. Dabei wird zuerst der Pfad auf dem Host und dann der Pfad im Container angegeben.

```
-v <HOST_PATH>:<CONTAINER_PATH>
```

Read-Only Bind-Mounts

- Container kann keine Dateien des Hosts ändern
- Umgekehrt werden Änderungen auf dem Host aber übernommen

`-v <HOST_ABSOLUTE_PATH>:<CONTAINER_ABSOLUTE_PATH>:ro`

oder

```
--mount \
    type=bind,\
    source=<HOST_ABSOLUTE_PATH>, \
    destination=<CONTAINER_ABSOLUTE_PATH>,readonly
```

Volumes



<https://www.publicdomainpictures.net/pictures/20000/velka/pile-of-floppy-disks.jpg>

Volumes

- **Persistente Dateien** unter Verwaltung von Docker
- Werden auf dem Host gespeichert
- **Performanter als Bind Mounts**
- Zugriff sollte nur mit den **docker volume** Befehlen geschehen



Volume anlegen

- Volume anlegen:

```
$ docker volume create <NAME>
```

- Alle Volumes anzeigen:

```
$ docker volume ls
```

- Volume untersuchen:

```
$ docker volume inspect <NAME>
```

- Löschen

```
$ docker volume rm <NAME>
```

Volume mounten

Sehr ähnlich wie Bind Mount

`-v <VOLUME_NAME>:<CONTAINER_ABSOLUTE_PATH>`

tmpfs



<https://www.pngall.com/ram-png/download/48470>

tmpfs

- Daten nur im **RAM** des Hosts
- Gut geeignet für **Secrets, Credentials**
- Sicherer als der **writable layer**. Jener kann nämlich in ein neues Image gepackt werden.



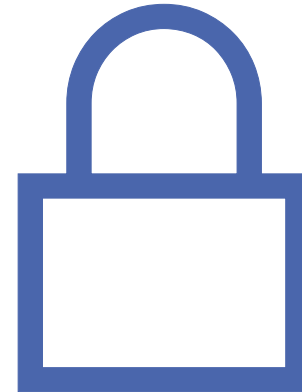
Praxisaufgabe

Bind Mounts

Docker Networking

Networking in Docker

- In Docker läuft jeder Dienst i.d.R. in einem eigenen Container
- Beispiel:
 - Ein Container für MySQL bzw. MariaDB
 - Ein Container für phpmyadmin
 - Ein Container für Wordpress
- Container werden über **Netzwerk** verbunden
- Essentiell für alle **realistischen Anwendungsszenarien**



Docker Networks

- Standardnetzwerke in Docker herausfinden:
`docker network ls`
- Die Standard-Netzwerke heißen:
 - `bridge`
 - `host`
 - `none`
- Später können noch weitere dazu kommen

Docker Networks

- Jedes Netzwerk hat einen **eigenen Namen** sowie eine Network **ID**
- Genauer gibt es verschiedene **Netzwerk-Treiber** (Driver) und für jeden davon ein Standard-Netzwerk
- Diese Treiber fungieren als Vorlagen, die das Verhalten von Containern spezifizieren
- Wie üblich können wir mit dem Kommando `docker inspect` ein Netzwerk genauer untersuchen

`docker network inspect bridge`

Bridge- und Host-Network

Bridge

- Ermöglicht Kommunikation zwischen Containern
- Container müssen auf demselben Host laufen
- Dies ist der Default
- Port-Weiterleitung von Host zum Container mit `-p` Parameter

Host

- Kein getrennter Netzwerkstack zwischen Host und Containern
- Wenn ein Container an Port 80 gebunden ist, dann ist er auf dem Host unter Port 80 erreichbar
- Container erreichen sich über Ports des Hosts
- Nur unter Linux verfügbar

Bridge-Netzwerke erstellen

- Bridge-Netzwerk anlegen über
`docker network create`
- Container anschließen mit `--network` Argument
`docker network create my-network`
`docker container run --network my-network`
- Warum ein eigenes Netzwerk?
 - Abschottung von anderen Containern
 - Gerade im produktiv-Betrieb sicherer und stabiler

Netzwerke löschen

- Mit `docker network rm` bzw. `docker network prune` löschen
- Die drei Standard-Netzwerke (`bridge`, `host` und `none`) kann man nicht löschen

Container nachträglich verbinden

- Container verbinden mit
`docker network connect <NETWORK_ID> <CONTAINER_ID>`
`docker network connect bridge my-mariadb`
- Per `docker network inspect bridge` kannst du dann prüfen, ob der Container dem Netzwerk angehört
- Mit `docker network disconnect` kannst du einen Container auch wieder aus einem Netzwerk entfernen

Dockerfiles

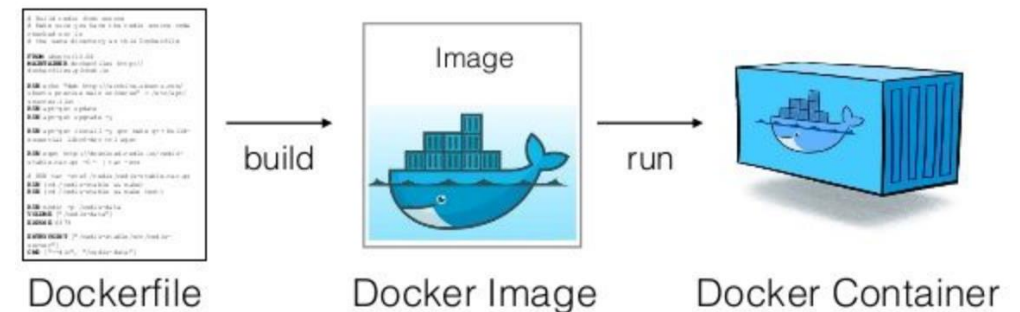
Dockerfiles

- Dockerfiles sind „**Rezepte**“, nach denen Images gebaut werden
- Sie sind **übersichtlicher** und weniger fehleranfällig als CLI-Kommandos
- **Versionierbar**
- Leichter zu **reviewen**



Dockerfiles

- In einem Dockerfile wird ein **neues Image** mit **Anweisungen** definiert
- Jede Zeile hat die Form **ANWEISUNG Parameter**
- Nach Konvention werden die Anweisungen immer **groß geschrieben**
- Kommentare beginnen mit **#** und werden beim Bauen ignoriert
- Ein Dockerfile hat **keine Endung** und heißt standardmäßig „**Dockerfile**“



[How to build application inside and outside Docker, Dockerfile structure and commands. | by Saurabh Singh | Medium](#)

Ein Beispiel

FROM python:3

WORKDIR /tmp

RUN apt-get update && apt-get install nano

COPY script.py script.py

CMD ["bash"]

Docker Build

- Docker-Builder erzeugt aus Dockerfile ein Image
- Build mit Dockerfile im aktuellen Verzeichnis:

```
$ docker build .
```

- Build mit explizit übergebenem Dockerfile:

```
$ docker build -f mydockerfile .
```

- Mit der Flag `-t` bzw. (`--tag`) kann man dem Image einen neuen Tag zuweisen

Build-Kontext

- Der Pfad, der `docker build` übergeben wird, wird zum **Kontext** für den Bau des Images
- Der Kontext ist die Gesamtheit der Daten, auf die der Docker Daemon zugreifen darf
- Best Practice: Dockerfile in Verzeichnis nur mit den benötigten Dateien speichern
- Mit **.dockerignore** die Dateien angeben, die ignoriert werden sollen

VORSICHT

- **Auf jeden Fall sensible Konfiguration und Credentials in die .dockerignore hinzufügen!**
- Sonst könnten aus Versehen sensible Daten in die Images kommen
- Das sollte das Erste sein, was ihr in einem neuen Projekt macht!



Wichtige Anweisungen

Im Folgenden werden wir einige wichtige Anweisungen für Dockerfiles kennen lernen:

- FROM
- RUN
- ADD und COPY
- CMD
- ARG und ENV
- EXPOSE

FROM-Anweisung

- Diese Anweisung gibt an, von welchem Image das neue Image abgeleitet werden soll
- Es ist empfehlenswert, ein offizielles Image zu nehmen
- Beliebte sind kompakte Images wie z.B. von Alpine-Linux

`FROM alpine:3.18`

RUN-Anweisung

- Führt **Befehle** aus, die das Image vorbereiten
- Klassisches Beispiel: Pakete herunterladen
`RUN apk update && apk add vim`
- **WICHTIG:** Jede **RUN**-Zeile erzeugt einen **Layer**!
- Daher mehrere Befehle mit **&&** zusammenfassen

ADD und COPY

- Den **COPY**-Befehl kennen wir schon aus dem CLI.
- Es **fügt Dateien** vom **Host** dem Image hinzu
- Der **ADD**-Befehl kann zusätzlich Dateien herunterladen und entpacken
- Es ist i.d.R. klarer verständlich, stattdessen im Image mit **curl** oder **wget** Dateien zu laden und zu entpacken
- In beiden Fällen wird ein **neuer Layer** erzeugt

CMD

- **CMD** gibt das **Standardkommando** an
- Hierbei ist die Schreibweise wie folgt:
- **Exec-Form** (offiziell empfohlen)
 - Das Programm wird direkt gestartet
 - `CMD ["<PROGRAM>", "<ARG1>", "<ARG_n>"]`
- **Shell-Form**
 - Hier wird eine Shell gestartet, die das Programm dann aufruft
 - Vorteil: Platzhalter wie z.B. "*", und Variablen können aufgelöst werden
 - `CMD <PROGRAM> <ARG1> <ARG_n>`
- **Es wird immer nur das letzte CMD-Kommando beachtet!**

WORKDIR

- Setzt das aktuelle Verzeichnis im Image
- Äquivalent zu `cd $DIR` im `RUN`

Variablen: ARG und ENV

- Mit **ARG** gesetzte Variablen stehen nur beim Build zur Verfügung
- Man kann sie auch **docker build** übergeben
`$ docker build --build-arg my_variable=neuer_wert`
- **ENV** setzt Umgebungsvariablen, die später im Container zur Verfügung stehen, aber auch im **CMD** oder **ENTRYPOINT**
- Sie können beim Start des Containers auch mit dem **-e** Argument gesetzt werden
- VORSICHT:
Sowohl **ENV als auch **ARG** stehen als Klartext in der History**

EXPOSE

- Mit `EXPOSE [PORT]` bzw. `EXPOSE [PORT]/[PROTOCOL]` können wir spezifizieren, dass dieser Container z.B. auf einem Port lauschen soll
- Wenn wir bei `docker run` alle Ports automatisch mappen möchten (Option: `-P`), dann werden nur Ports verwendet, die mit `EXPOSE` angegeben worden sind
- Beispiele:

`EXPOSE 80/tcp`

`EXPOSE 443/tcp`

`EXPOSE 80`

`EXPOSE 443`

Praxisaufgabe

Dockerfiles

Kubernetes





Was ist Kubernetes?

- Plattform zur Steuerung von Container-basierten Anwendungen
- "kubernetes" griechisch für "Steuermann" / "Pilot"

Was bietet es uns?

- Definition der Architektur unabhängig von den Maschinen
- Volumes, Netzwerke, Loadbalancer, Skalierung, Monitoring...
- Integration in Cloud-Provider
- Tool-Ökosystem (helm, istio, kustomize...)
- Mächtige eingebaute Werkzeuge

Unterschiedliche Konzepte

- Docker etc. arbeiten Befehls-basiert
starte dies, lege das an...
- Kubernetes ist anders: Wir definieren den gewünschten Zustand
- Danach kümmert sich Kubernetes darum, dass er dauerhaft eingehalten wird
- Das nennt man „Reconciliation“
- Mehr dazu später

Ein paar weitere Begriffe



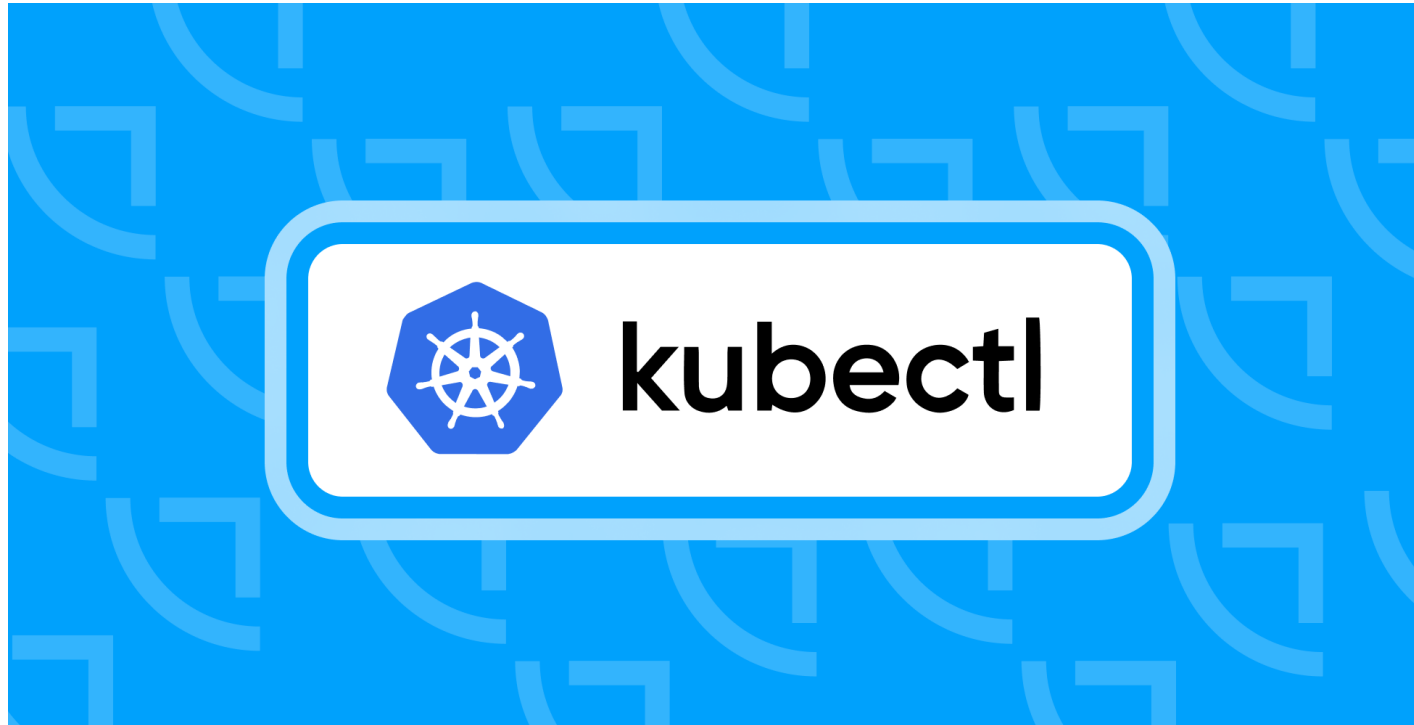
Pods

- Kleinste Einheit für Arbeitslast in k8s
- Ähnlich wie Service in docker compose
- Häufig ein Container
- ... können aber auch mehrere sein
- „Einheit“ → Container eines Pods werden zusammen deployt

Pods

- Ein Pod ist eine logische Gruppierung von Containern
- Sie sind „Wegwerf-Objekte“: Nach dem Stopp verschwinden sie
- Alle Container eines Pods laufen auf einem Knoten!

kubectl

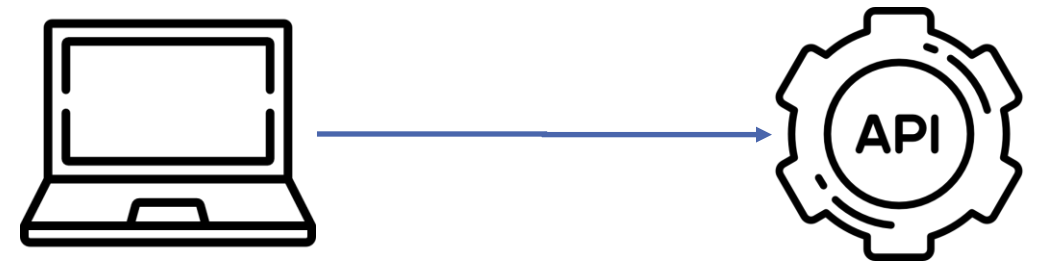


kubectl

- **Kubernetes-CLI** (Command-Line Interface)
- Kommunikation und Interaktion mit Kubernetes-Cluster
- Ausführung von Befehlen für Erstellung, Aktualisierung und Verwaltung von Ressourcen
- Zentrales Werkzeug für Administratoren und Entwickler

Kubectl und API-Server

- kubectl kommuniziert mit API-Server über REST-API
- Authentifizierung und Autorisierung der Anfragen
- JSON-/YAML-Dateien zur Beschreibung von Ressourcen



Kubectl Syntax

- Befehlssyntax:
`$ kubectl [command] [TYPE] [NAME] [flags]`
- Grundlegende Befehle:
`create, get, describe, delete`
- Ressourcentypen:
`Pods, services, deployments, configmaps`
- Optionale Flags zur Anpassung von Befehlen

kubectl Beispiel-Kommandos

```
$ kubectl get pods -n NAMESPACE
```

NAME	READY	STATUS	RESTARTS	AGE
my-pod-1	1/1	Running	0	120m
my-pod-2	1/1	Running	0	110m

kubectl Beispiel-Kommandos

```
$ kubectl describe pod POD_NAME -n NAMESPACE
```

Name: my-pod-1

Namespace: default

Node: node-1/1.2.3.4

Start Time: Tue, 08 Feb 2022 14:00:00 +0000 Labels:

app=my-pod

Annotations: <none>

Status: Running

IP: 10.244.1.2

..

Praxisaufgabe

kubectl

Komponenten eines k8s-Clusters

Zwei Haupt-Welten



Control Plane



Worker

Control Plane

kube-apiserver

etcd

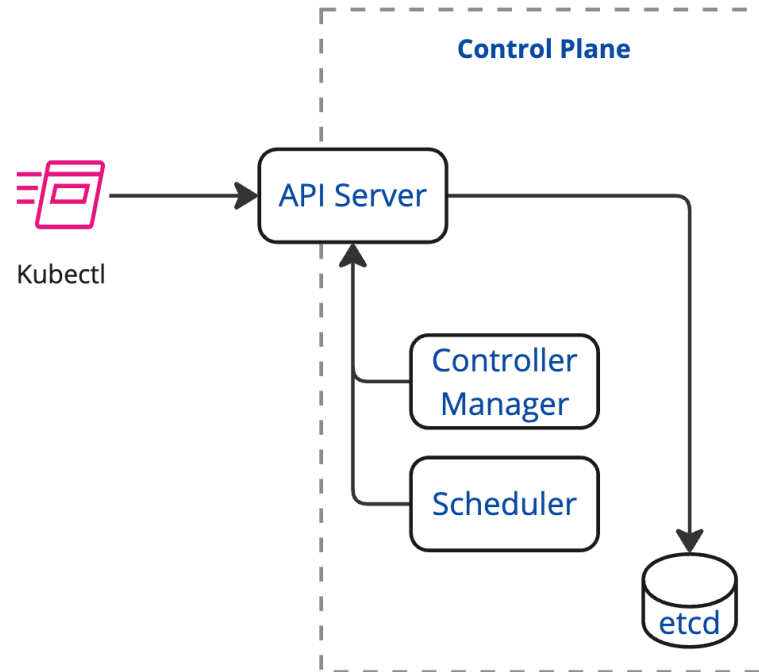
kube-controller-manager

kube-scheduler

cloud-controller-manager

kube-apiserver

Die Konfiguration von k8s wird zentral über den API-Server vorgenommen.



kube-apiserver

- Bietet REST-Interface für die Control Plane
- Steuerung von Kubernetes erfolgt in der Regel über apiserver
- Er ist der „Gatekeeper“ für jede Cluster-Änderung mit Zugriffskontrolle und Validierung

etcd

- Datenspeicher des Clusters
- ... aber NUR für die Control-Plane!
- Key-Value-Store
- Speichert Konfiguration und Zustands-Informationen



kube-controller-manager

- Beobachtet den **Zustand des Clusters**
- Hat mehrere Unterkomponenten
 - **Node Controller:** Beobachtet Zustand der Knoten/Hosts
 - **Replication Controller:** Beobachtet Zustand der Pods
 - **Service Account Controller:**
Legt Service Accounts an und stattet sie mit Token aus
 - **Endpoints Controller:** Verbindet Pods mit Services

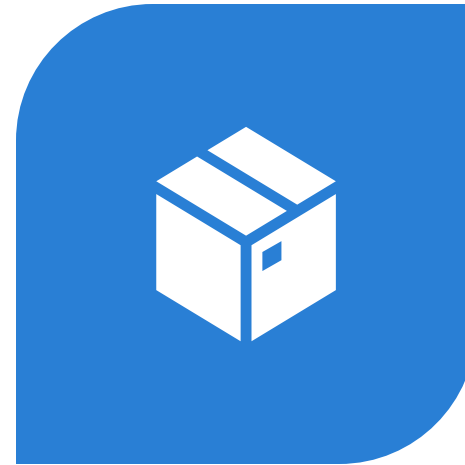
kube-scheduler

- Weist neu angelegte Pods einem Knoten zu
- Beachtet Node-Ressourcen
- Steuerbar über Affinität und Anti-Affinität von Pods

Worker



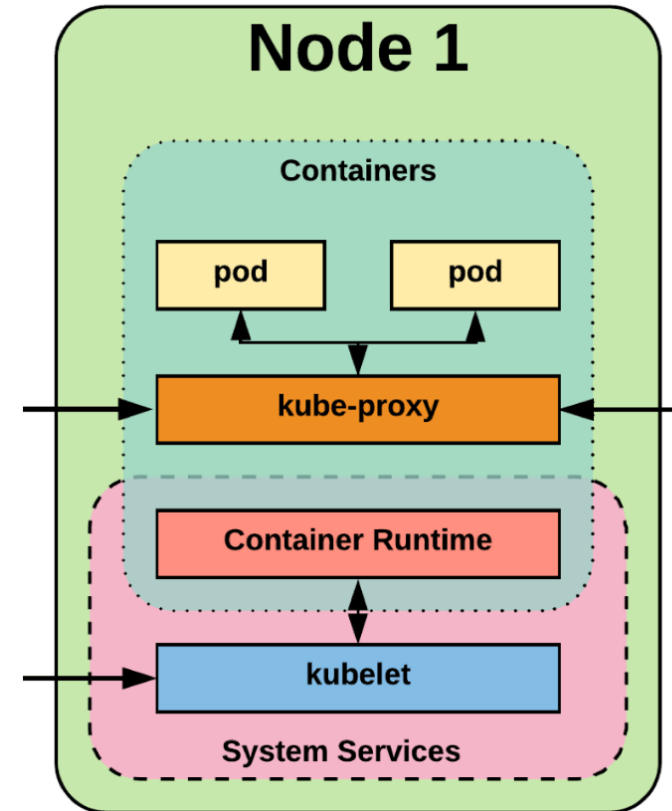
Übernehmen die
eigentliche Arbeitslast



Führen Anwendungs-
Container in Pods aus

Worker-Komponenten

- kubelet
- kube-proxy
- Container Runtime

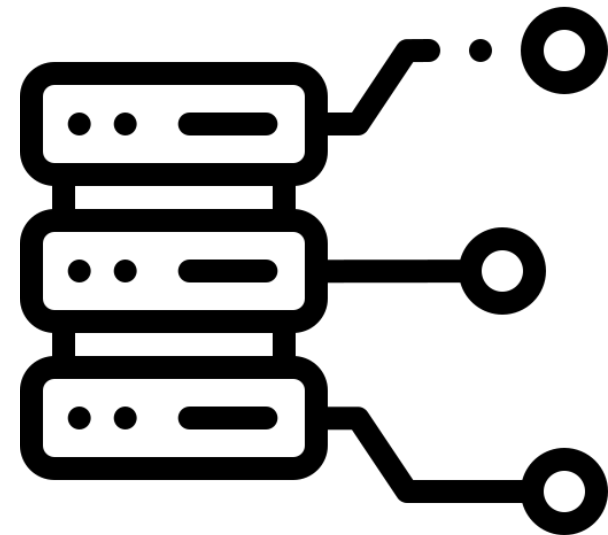


kubelet

- Programm, das auf jedem Worker läuft
- Nimmt vom Master die Spezifikationen der Pods entgegen
- Startet Pods

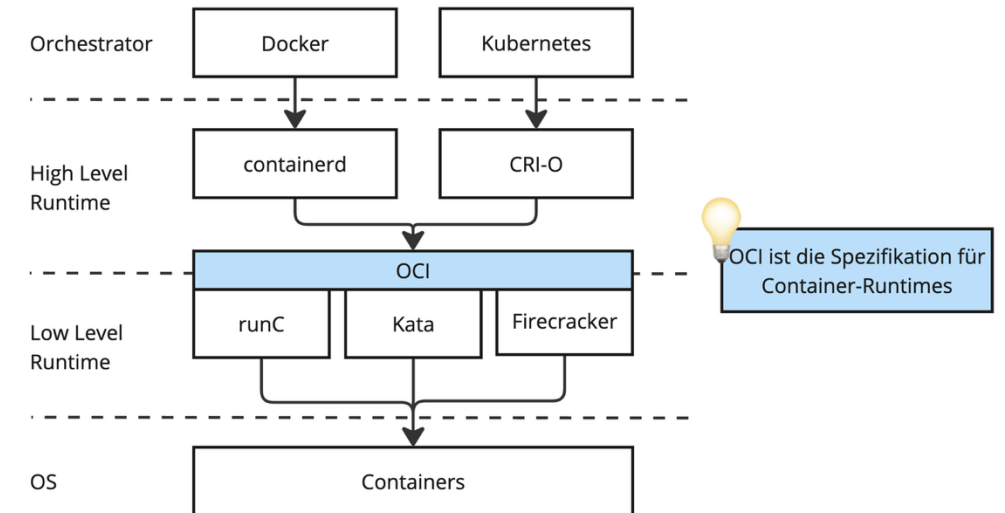
kube-proxy

- Netzwerk-Komponente
- Stellt sicher, dass Pods kommunizieren können



Container Runtime

- Die Basics von Container-Runtimes haben wir schon kennen gelernt
- Neu ist der Standard, mit dem Kubernetes auf OCI-Laufzeiten zugreift: **CRI**
- Eine CRI-Runtime wird von Kubernetes benutzt, um **Images und Container zu verwalten**
- Intern verwenden CRI-Runtimes eine low level runtime wie **runC**



Kubernetes Kern-Objekte

Namespace

- Logische Gruppierung von Ressourcen.
- Saubere Trennung von Berechtigungen und Zugriff.

```
apiVersion: v1
kind: Namespace
metadata:
  name: prod
  labels:
    app: MyBigWebApp
```

```
$ kubectl get ns --show-labels
NAME          STATUS    AGE      LABELS
default       Active    11h      <none>
kube-public   Active    11h      <none>
kube-system   Active    11h      <none>
prod          Active    6s       app=MyBigWebApp
```


Pods

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-example
spec:
  containers:
  - name: nginx
    image: nginx:stable-alpine
    ports:
    - containerPort: 80
```

Pod Metadaten

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-example
spec:
  containers:
  - name: nginx
    image: nginx:stable-alpine
    ports:
    - containerPort: 80
```

Pod Lifecycle

Pending

- Der Pod wurde von Kubernetes akzeptiert, aber mindestens ein Container ist noch nicht gestartet worden.

Running

- Der Pod wurde an einen Knoten gebunden und ist gestartet worden.

Succeeded

- Der Pod wurde erfolgreich beendet.

Failed

- Der Pod wurde unerwartet fehlerhaft beendet.

Unknown

- Der Zustand des Pods ist unbekannt, i.d.R. wegen eines Kommunikationsproblems mit der Control Plane.

Pod Container Configuration

- `name` – Name des Containers
- `image` – Image des Containers
- `ports` – Exponierte Ports mit einem Namen
- `env` – Liste von Umgebungsvariablen
- `command` – Wie Docker `ENTRYPOINT`
- `args` – Wie Docker `CMD` args

Container

```
name: nginx
image: nginx:stable-alpine
ports:
  - containerPort: 80
    name: http
    protocol: TCP
env:
  - name: MYVAR
    value: isAwesome
command: ["/bin/sh", "-c"]
args: ["echo ${MYVAR}"]
```

Pod Template

- Viele Ressourcen wie Deployments und ReplicaSets basieren auf Vorlagen (Templates) für Pods
- Sie enthalten nur die Infos, um Kopien des Pods zu erzeugen



```
template:  
  metadata:  
    labels:  
      app: nginx  
  spec:  
    containers:  
      - name: nginx  
        image: nginx
```

Label

- Schlüssel/Wert-Kombinationen
- NICHT eindeutig!
- Dient dem Filtern von Objekten

```
template:  
  metadata:  
    labels:  
      app: nginx  
  spec:  
    containers:  
      - name: nginx  
        image: nginx
```

Labels und Selectors

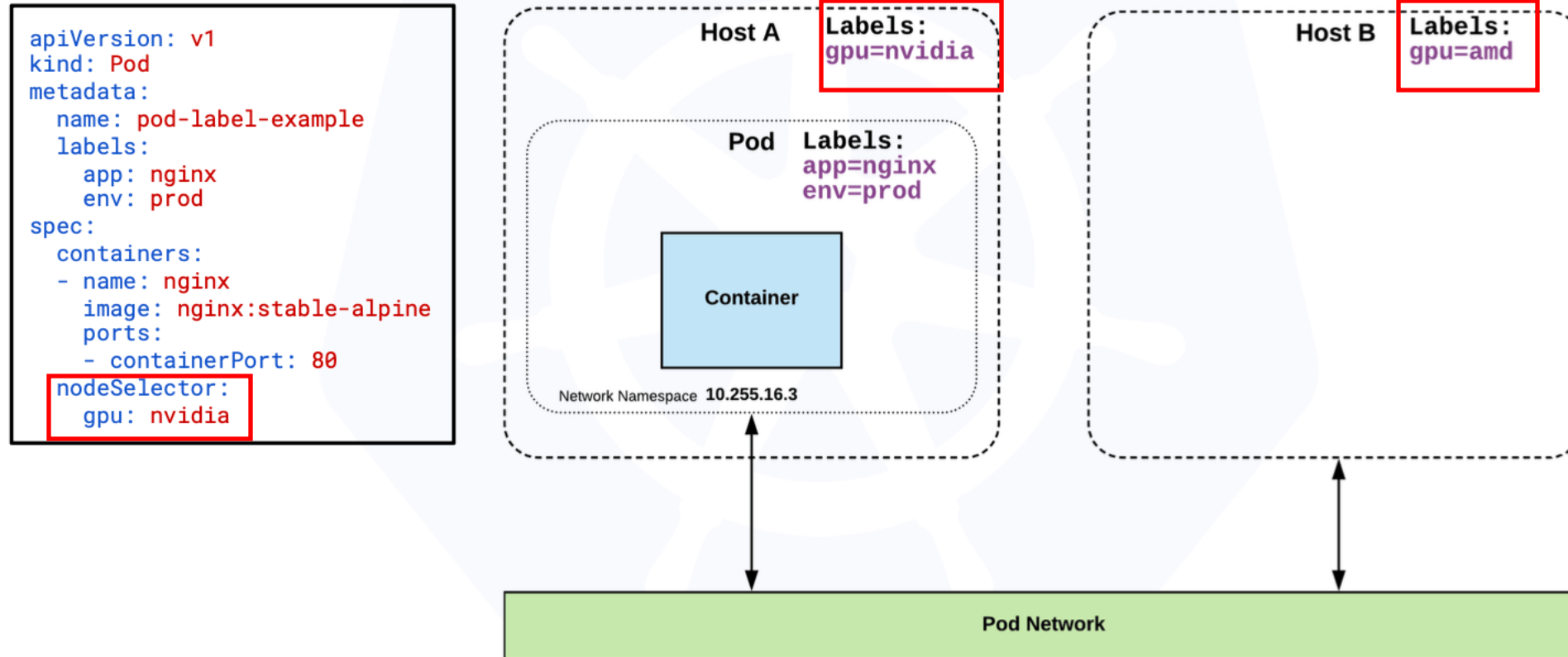


Selectors

- Selectors nutzen Labels
- Sie wählen Gruppen von Objekten aus
- Kernkonzept vieler Gruppen-Objekttypen
 - Service
 - ReplicaSet
 - Deployment

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-label-example
  labels:
    app: nginx
    env: prod
spec:
  containers:
    - name: nginx
      image: nginx:stable-alpine
      ports:
        - containerPort: 80
  nodeSelector:
    gpu: nvidia
```


Beispiel für Selectors



Typen von Selectors

Equality based

Einfacher (Un-)Gleichheits-Vergleich von Label und Werten

```
selector:  
  matchLabels:  
    gpu: nvidia
```

Set based

Neuerer Mechanismus, wird nicht überall unterstützt.

Ermöglicht Mengenoperationen (**in**, **notin**, **exists**)

```
selector:  
  matchExpressions:  
    - key: gpu  
      operator: in  
      values: ["nvidia"]
```

Services



Service

Services

- Bieten einheitlichen Zugang zu Pods
- Im Gegensatz zu Pods sind sie langlebig
 - Statische Cluster-IP
 - Statischer DNS-Name

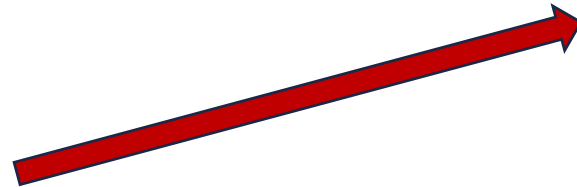
`<service name>.<namespace>.svc.cluster.local`

Services

- Pods werden über *equality based selectors* identifiziert
- kube-proxy wird genutzt für die Netzwerkkommunikation
- Der Proxy legt als Daemon auf jedem Host iptables-Routen an

Zuordnung via Labels

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-pod-demo-svc
labels:
  app: nginx-pod-demo
spec:
  selector:
    app: nginx-pod-demo
  ports:
    - port: 80
      name: http
```



```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod-1
labels:
  app: nginx-pod-demo
spec:
  containers:
    - name: nginx
      image: nginx:latest
      ports:
        - containerPort: 80
```

Service- Typen

ClusterIP

- Kommunikation innerhalb des Clusters

NodePort

- Ermöglicht externen Zugriff über statischen Port auf jedem Node

LoadBalancer

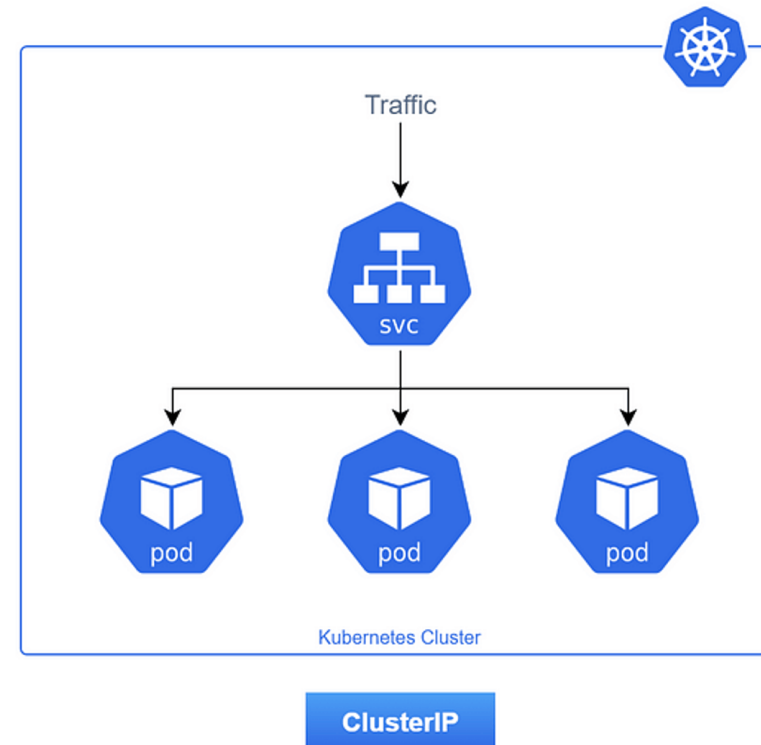
- Automatische Integration in Cloud-Anbieter für dynamisches Load Balancing

ExternalName

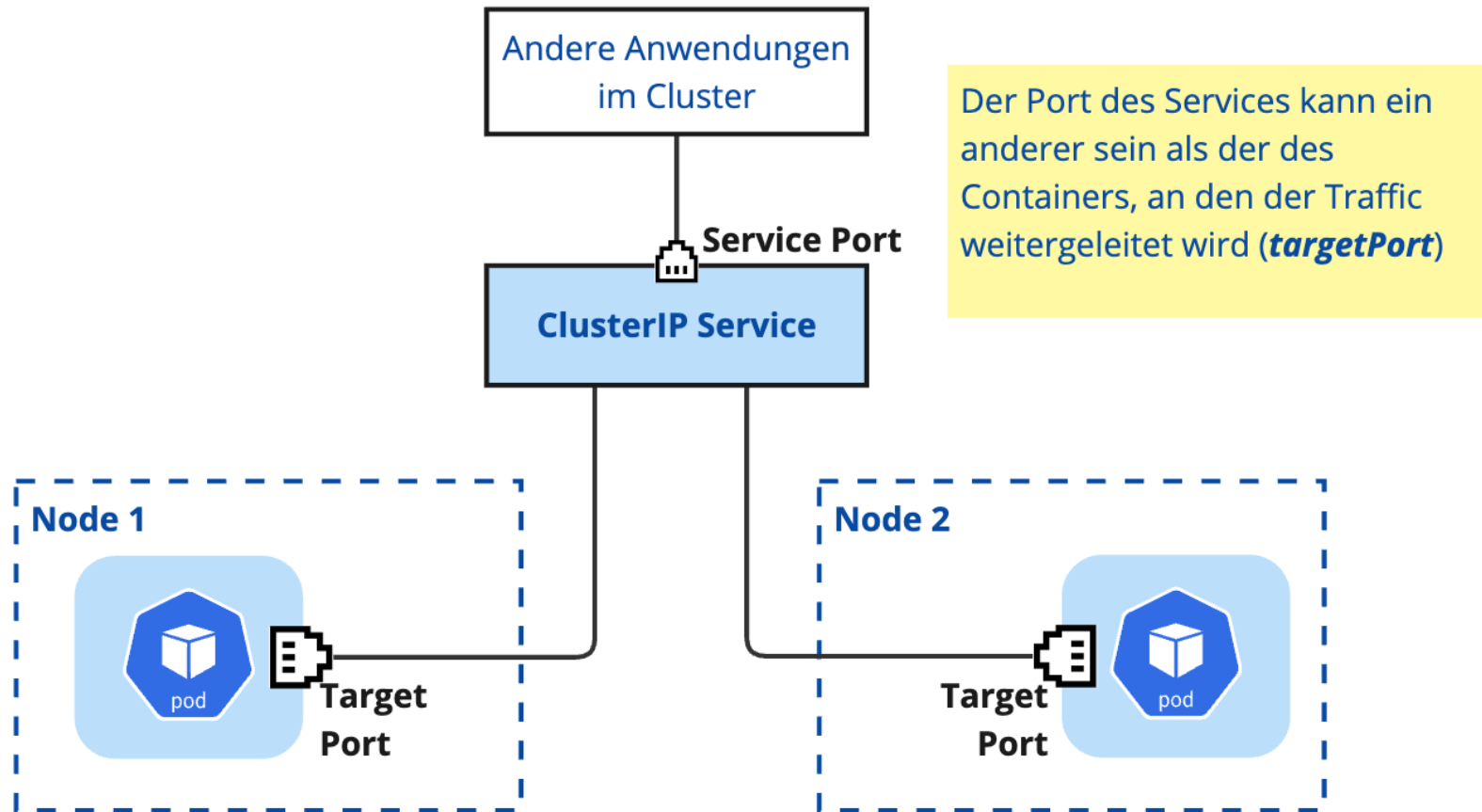
- Verweist auf externen DNS-Namen

ClusterIP

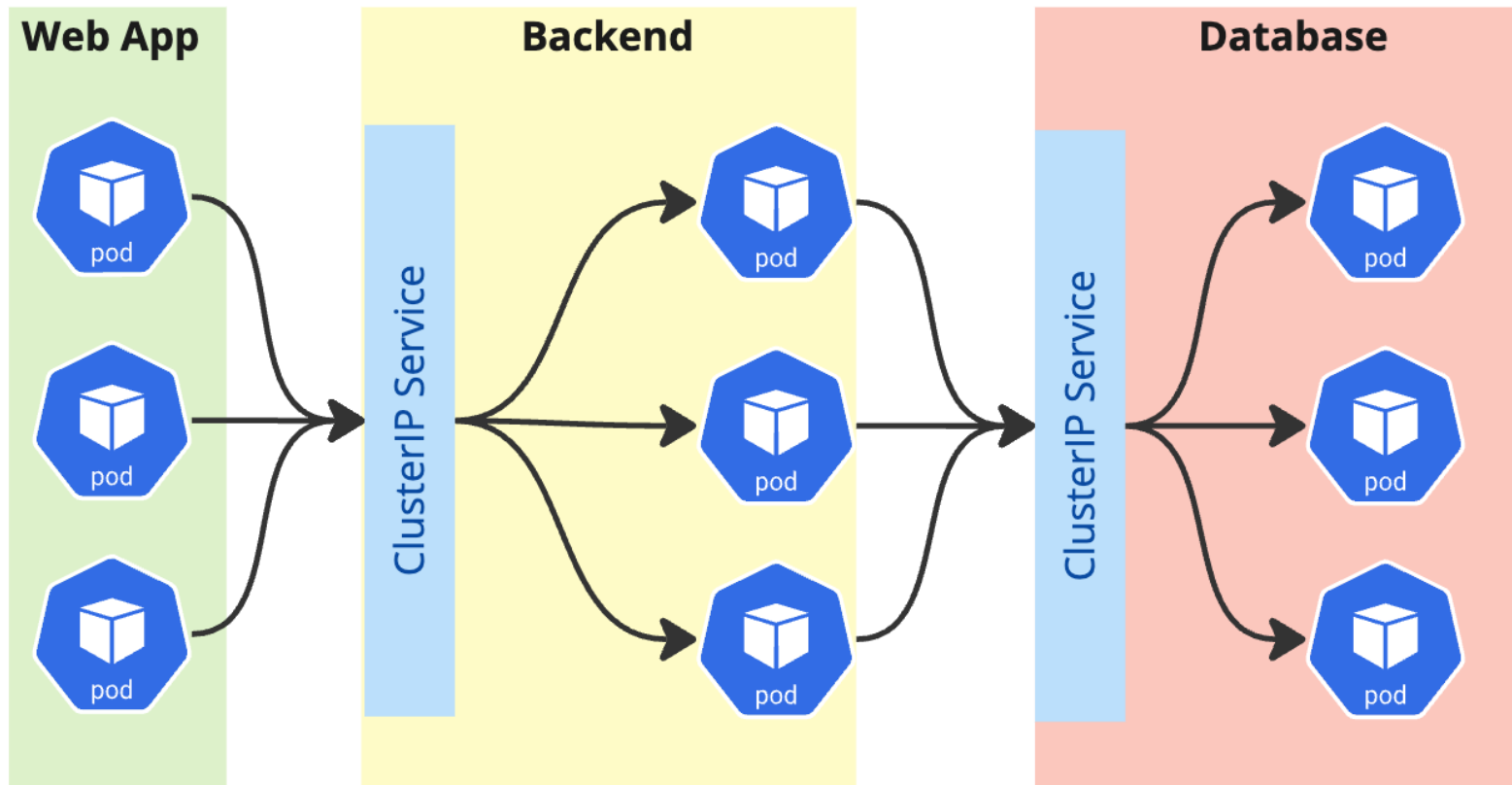
- Interne (!) statische IP für Service
- Ideal für intern kommunizierende Services, z.B. Datenbanken
- Beispiel: Anwendung mit separaten Backend- und Datenbank-Pods



ClusterIP Service

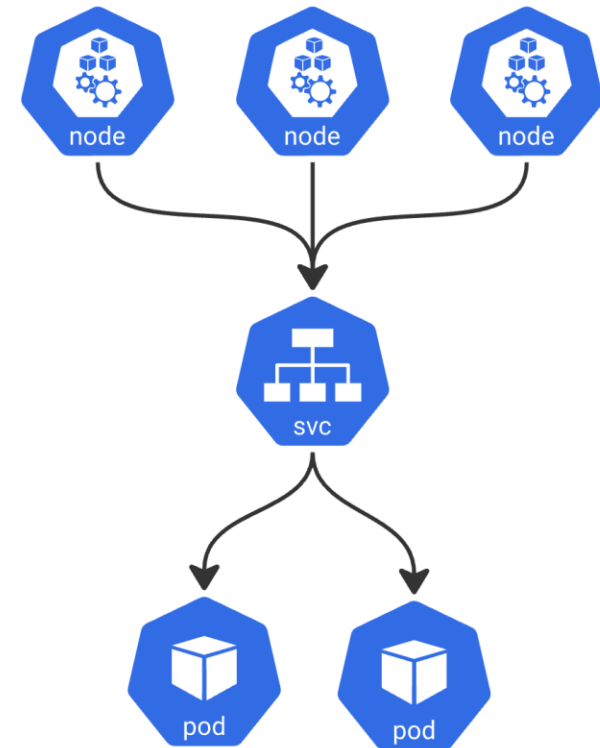


ClusterIP Service

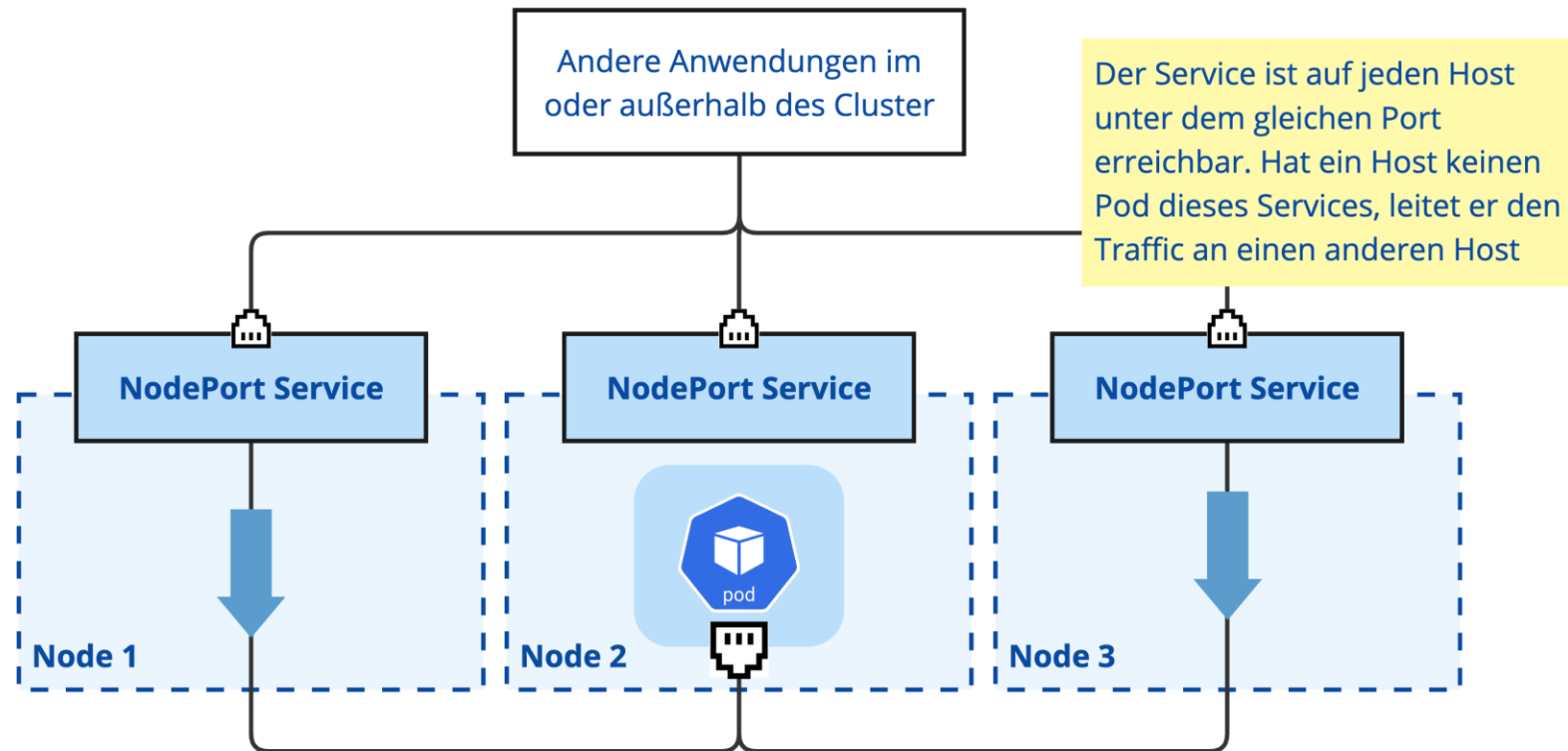


NodePort

- Exponiert Services extern über statische Ports auf Nodes
- Einfache Möglichkeit, Traffic "von außen" über Host zu einem Pod zu leiten
- Port kann explizit definiert oder automatisch alloziert werden

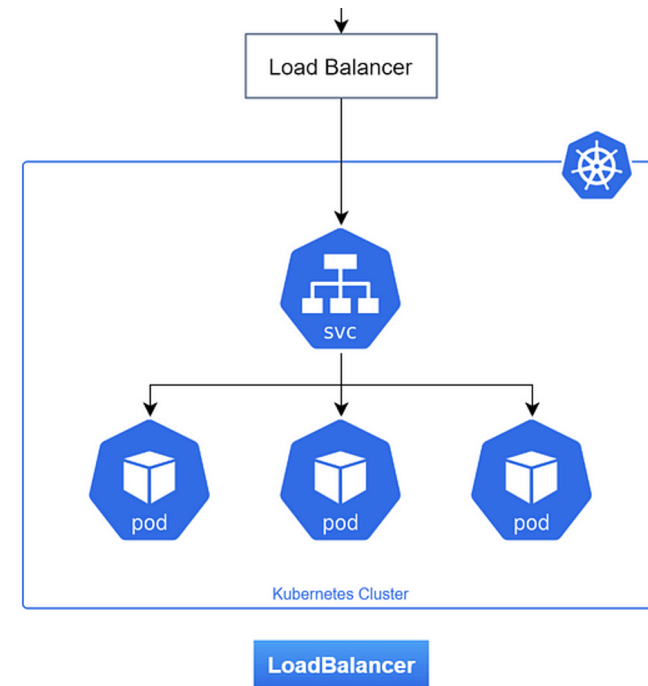


NodePort Service



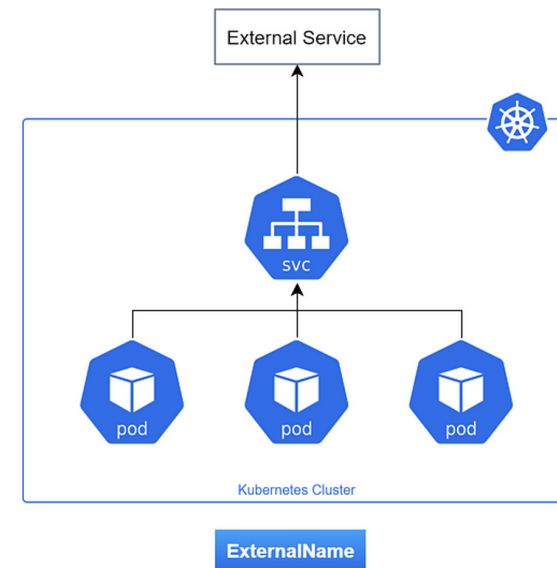
LoadBalancer

- Automatischer externer Zugriff durch Integration mit Cloud-Providern
- Cloud-Provider erstellt Load-Balancer und verteilt Last über alle Nodes
- Skalierung und Hochverfügbarkeit durch dynamische Lastverteilung



ExternalName

- Einziger Service, der für Zugriffe nach **außen** gedacht ist
- Stellt stabile interne DNS-Namen zu externen Diensten für Pods zur Verfügung



ExternalName Service

```
apiVersion: v1
kind: Service
metadata:
  name: my-database-svc
  namespace: prod
spec:
  type: ExternalName
  externalName: my.database.example.com
```

Dieser Service ist für Pods intern erreichbar unter
`my-database-svc.prod.svc.cluster.local`

ExternalName Service

- ExternalName-Services sind besonders praktisch bei **Migrationen**
- Pods können externen Dienst unter stabilem Namen erreichen
- Ändert sich die URL des externen Diensts, muss nur der Service angepasst werden, nicht jeder einzelne Pod

Praxisaufgabe

Pods und Services

Demo

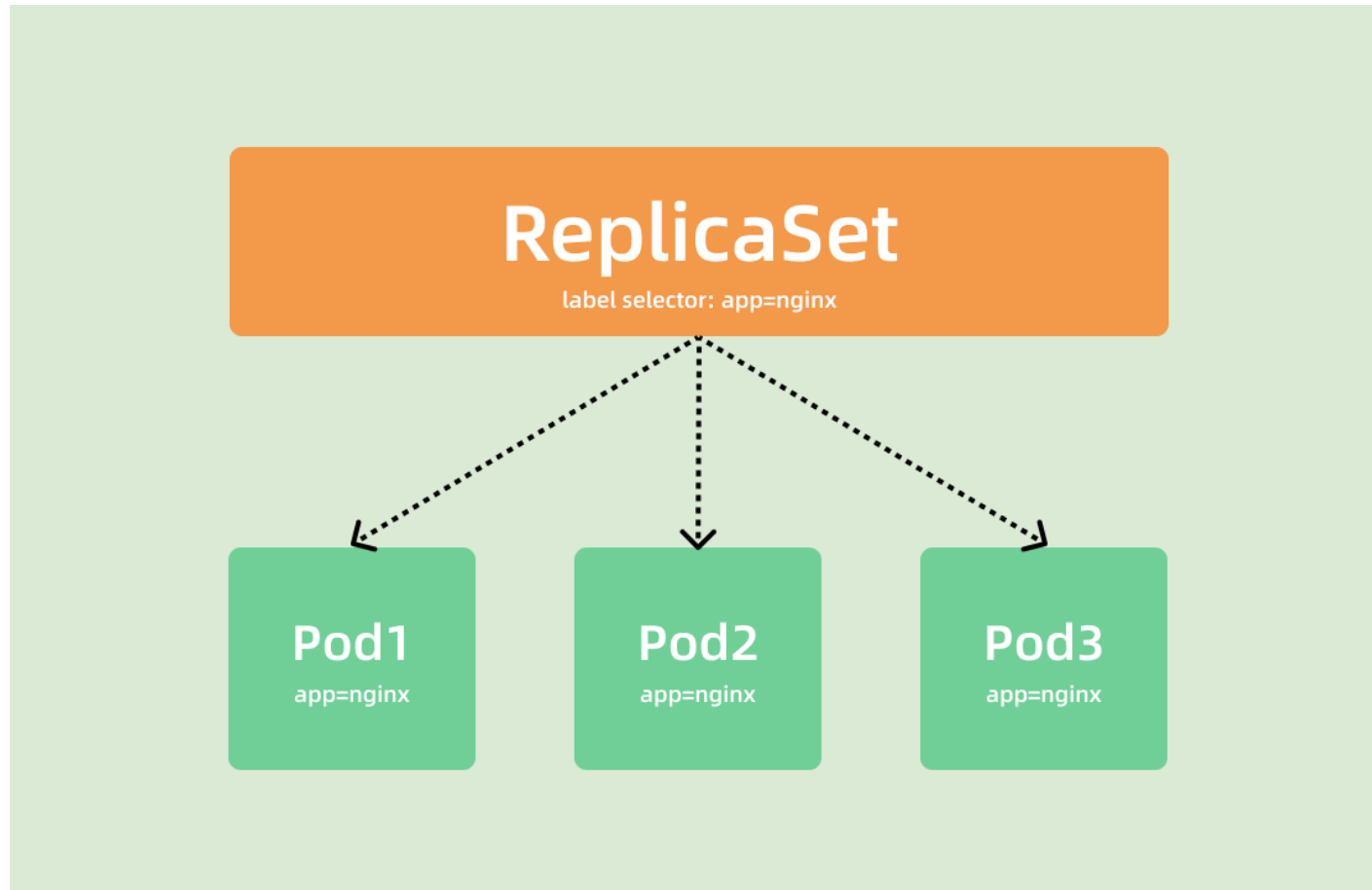
NodePort-Services auf EKS

Kubernetes Workloads

ReplicaSet

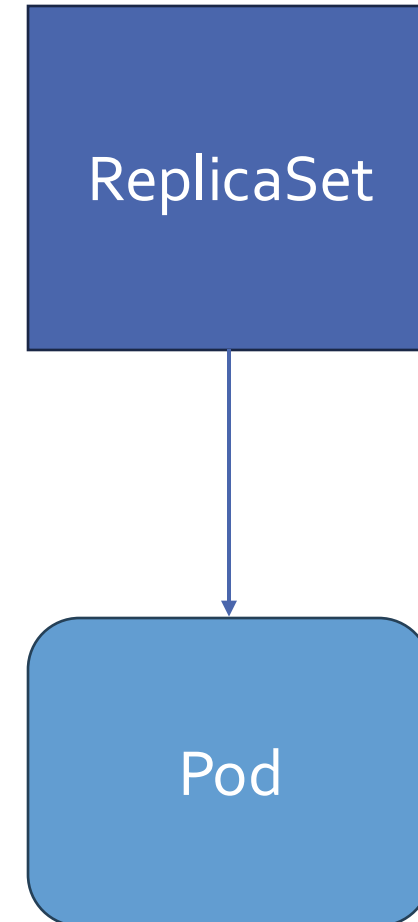


ReplicaSet



ReplicaSet

- Überwacht und verwaltet bereitgestellte Pods
- Ersetzt fehlgeschlagene oder fehlerhafte Pods automatisch
- ReplicaSets ersetzen den veralteten ReplicationController
- Normalerweise indirekt über Deployments erstellt und gesteuert



ReplicaSet

- **replicas**
Die gewünschte Anzahl an Pods
- **selector**
Wählt Pods aus, die das ReplicaSet verwaltet

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: rs-example
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
      env: prod
  template:
    <pod template>
```


Owner Reference

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: "2024-08-19T18:22:12Z"
  generateName: replicaset-demo-
  labels:
    app: replicaset-demo
  name: replicaset-demo-7hjdg
  namespace: default
  ownerReferences:
    - apiVersion: apps/v1
      blockOwnerDeletion: true
      controller: true
      kind: ReplicaSet
      name: replicaset-demo
      uid: 71484fc7-033d-4d17-abbf-59ebe7dd1333
  resourceVersion: "218834"
  uid: 79d9733e-5c09-4a6c-b123-9e5bb85888e1
```

blockOwnerDeletion führt dazu, dass der Garbage Collector beim Löschen des ReplicaSets wartet, bis alle Pods gelöscht sind.

controller zeigt an, dass das ReplicaSet den Pod kontrolliert. Wird das ReplicaSet gelöscht, werden auch die Pods gelöscht.

Praxisaufgabe

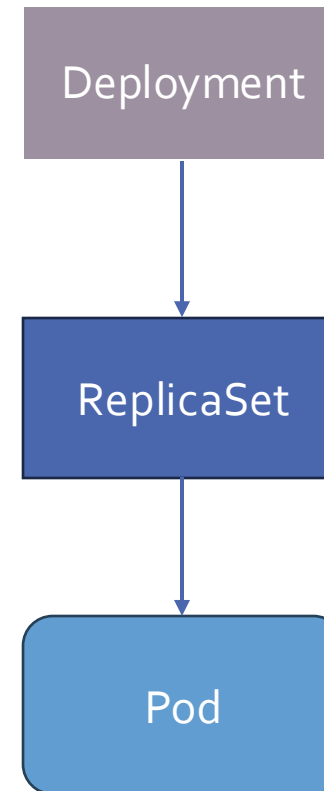
ReplicaSets

Deployment



Deployments

- Rolll ReplicaSets aus
- Bietet Rollback-Funktionalität
- Behält Verbindung zu Pods und ReplicaSet via Labels



Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: deploy-example
spec:
  replicas: 3
  revisionHistoryLimit: 3
  selector:
    matchLabels:
      app: nginx
      env: prod
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    <pod template>
```

- **replicas**
Wird für das erzeugte ReplicaSet genutzt
- **revisionHistoryLimit**
Wieviele vorherige Versionen werden behalten
- **strategy**
Rollout-Strategie

Deployment Rolling Updates

- Im folgenden Beispiel wurde bei einem Deployment die Pod Spec geändert.
- Das bedeutet, dass sich der Hash ändert, der über die Pod Spec generiert wird.
- Kubernetes weiß jetzt: Das Deployment muss neu ausgerollt werden.
- Dazu legt es ein neues ReplicaSet an (wir nennen es R2) und ersetzt das alte ReplicaSet (R1) damit.

RollingUpdate Deployment

Jedes Update erzeugt ein neues ReplicaSet, in diesem Fall „R2“. Das Pod Template hat sich geändert und dementsprechend auch der Hash.

R1 pod-template-hash:

676677fff

R2 pod-template-hash:

54f7ff7d6d

**Deployment
Revision 1**

ReplicaSet R1

ReplicaSet R2

Pod

Pod

Pod

R1

```
$ kubectl get replicaset
```

NAME	DESIRED	CURRENT	READY	AGE
mydep-676677fff	3	3	3	5h

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
mydep-676677fff-9r2zn	1/1	Running	0	5h
mydep-676677fff-hsfz9	1/1	Running	0	5h
mydep-676677fff-sjxhf	1/1	Running	0	5h

RollingUpdate Deployment

Der Deployment-Controller stellt das neue ReplicaSet mit DesiredCount == 1 ein, das alte skaliert er auf 2 zurück.

R1 pod-template-hash:
676677fff
R2 pod-template-hash:
54f7ff7d6d

**Deployment
Revision 2**

ReplicaSet R1

Pod

Pod

Pod

ReplicaSet R2

Pod

```
$ kubectl get replicaset
```

NAME	DESIRED	CURRENT	READY	AGE
mydep-54f7ff7d6d	1	1	1	5s
mydep-6766777fff	2	3	3	5h

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
mydep-54f7ff7d6d-9gv1l	1/1	Running	0	2s
mydep-6766777fff-9r2zn	1/1	Running	0	5h
mydep-6766777fff-hsfz9	1/1	Running	0	5h
mydep-6766777fff-sjxhf	1/1	Running	0	5h

R2
R1

RollingUpdate Deployment

Wir sind jetzt fast durch. Das neue ReplicaSet hat den Desired Count 3, das alte 0.

```
R1 pod-template-hash: 676677fff
R2 pod-template-hash: 54f7ff7d6d
```

Deployment Revision 2

ReplicaSet R1

ReplicaSet R2

Pod

~~Pod~~

~~Pod~~

Pod

Pod

Pod

	NAME	DESIRED	CURRENT	READY	AGE
R2	mydep-54f7ff7d6d	3	3	3	10s
R1	mydep-6766777fff	0	1	1	5h

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
mydep-54f7ff7d6d-9gvll	1/1	Running	0	7s
mydep-54f7ff7d6d-cqvlq	1/1	Running	0	5s
mydep-54f7ff7d6d-gccr6	1/1	Running	0	2s
mydep-6766777fff-9r2zn	1/1	Running	0	5h

RollingUpdate Deployment

Und fertig! Das alte ReplicaSet hat keine laufenden Pods mehr, das neue hat 3.

R1 pod-template-hash:
676677fff
R2 pod-template-hash:
54f7ff7d6d

Deployment
Revision 2

ReplicaSet R1

ReplicaSet R2

Pod

Pod

Pod

R2
R1

```
$ kubectl get replicaset
```

NAME	DESIRED	CURRENT	READY	AGE
mydep-54f7ff7d6d	3	3	3	15s
mydep-676677fff	0	0	0	5h

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
mydep-54f7ff7d6d-9gv1l	1/1	Running	0	12s
mydep-54f7ff7d6d-cqvlq	1/1	Running	0	10s
mydep-54f7ff7d6d-gccr6	1/1	Running	0	7s

Praxisaufgabe

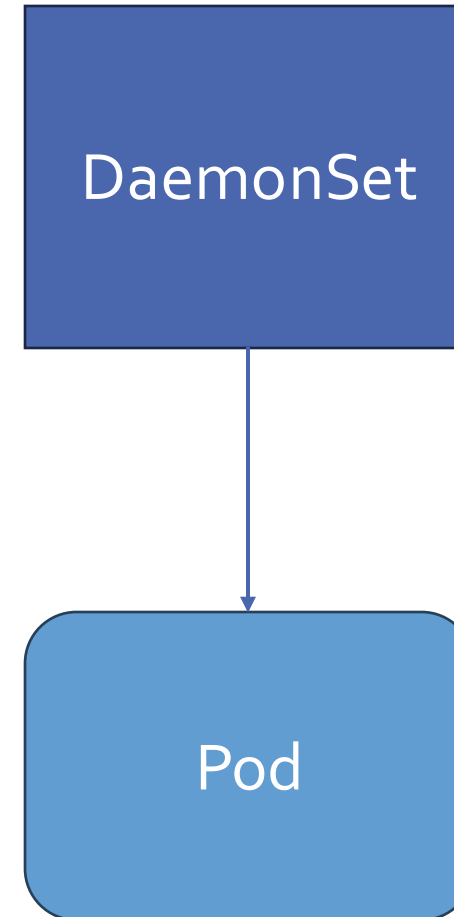
Deployment und RollingUpdate

DaemonSet

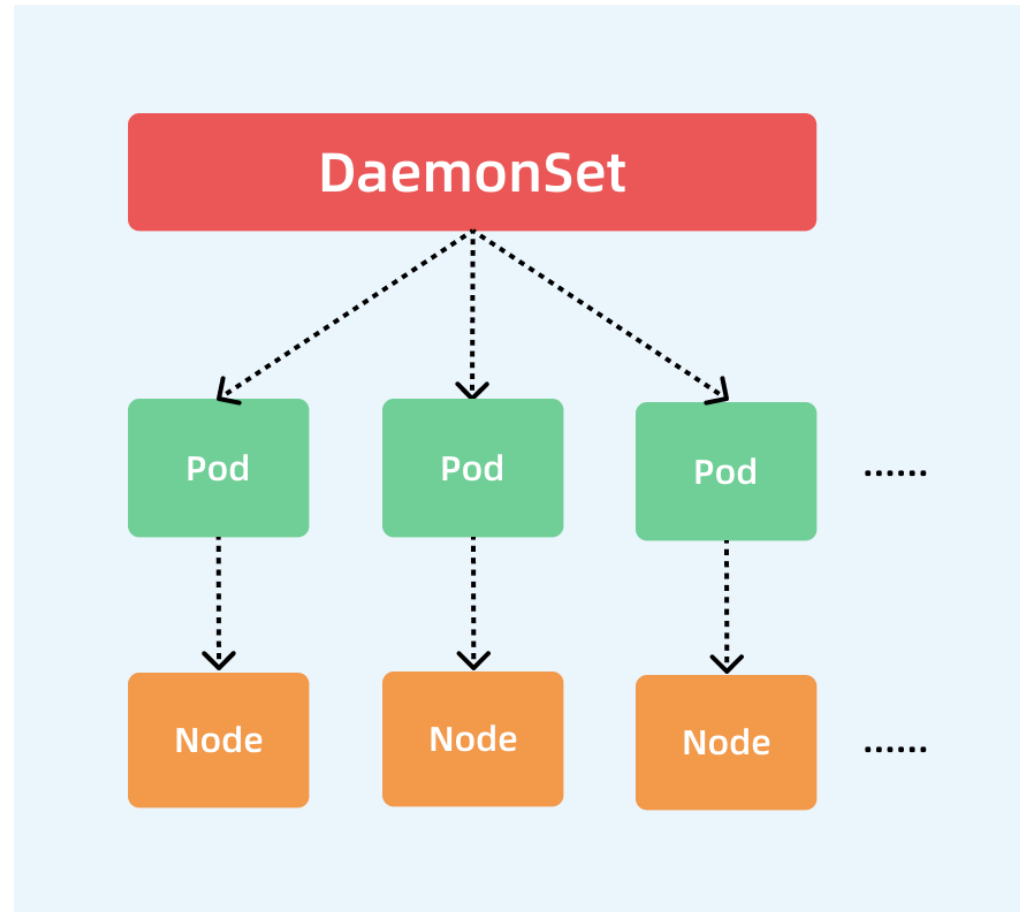


DaemonSet

- Stellt sicher, dass jeder Node eine Instanz des Pods hat
- Klassiker: Log Agents



DaemonSet



Demo

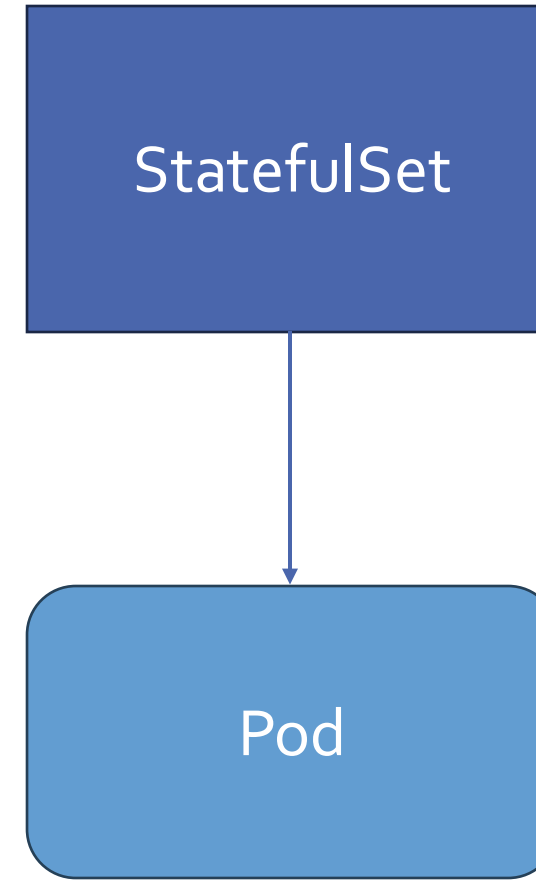
DaemonSets auf EKS

StatefulSet

- ReplicaSets sind für Arbeitslasten gedacht, bei denen Pods anonym sind
- Pods sind bei ReplicaSets sozusagen Vieh („Cattle“)
- Für zustandsbehaftete Anwendungen ist das möglicherweise nicht ausreichend
- StatefulSets bieten feste Nummerierung und Benennung von Pods sowie feste Reihenfolgen beim Deployment

StatefulSet

- Pods erhalten eindeutigen Namen
`<NameStatefulSet>-<Index>`
- Persistente Daten müssen auf einem persistenten Volume gespeichert werden
- Die Zuordnung von Volume zu Pod ist dank des StatefulSet aber fest



StatefulSet

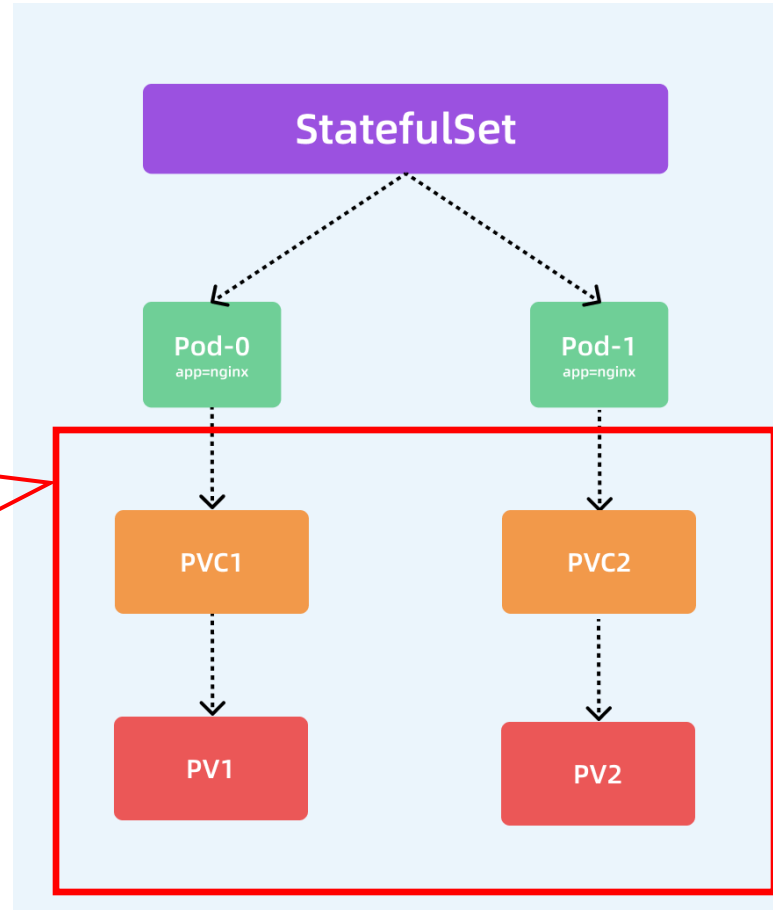
- **revisionHistoryLimit**
Wieviele vorherige Versionen werden behalten
- **serviceName**
Name des Services, der auf die Pods zeigen soll. Kann auch Headless sein.

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: sts-example
spec:
  replicas: 2
  revisionHistoryLimit: 3
  selector:
    matchLabels:
      app: stateful
  serviceName: app
  updateStrategy:
    type: RollingUpdate
    rollingUpdate:
      partition: 0
  template:
    <pod template>
```

StatefulSet

Über Persistent Volumes und Claims reden wir später.

Wichtig ist aber zu wissen:
StatefulSets sorgen dafür, dass Pods ihre Zuordnung zu Volumes behalten.



Headless Service

- StatefulSets werden fast immer zusammen mit sogenannten Headless Services eingesetzt
- Er gibt jedem Pod einen eindeutigen DNS-Namen
- Ein Headless Service hat aber **keine** ClusterIP
- Er ist **nicht** dafür gedacht, eine Adresse für alle Pods des Services zu bieten. Dafür ist der ClusterIP-Service gedacht.



Headless Service

```
apiVersion: v1
kind: Service
metadata:
  name: headless-svc
spec:
  clusterIP: None
  selector:
    app: web
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
```

Headless Service

Jeder Pod erhält eine eindeutige DNS-Adresse, die auch gleich bleibt, wenn der Pod neu gestartet werden muss.

```
<StatefulSet Name>-<Index>.<Service Name>.<Namespace>.svc.cluster.local
```

```
$ nslookup nginx-0.nginx-headless.default.svc.cluster.local
Server:          10.96.0.10
Address:         10.96.0.10:53
```

Headless Service

Außerdem können die IP-Adressen aller Pods auf einmal abgefragt werden.

```
<Service Name>.<Namespace>.svc.cluster.local
```

```
$ nslookup nginx-headless.default.svc.cluster.local  
Address: 10.244.0.65  
Address: 10.244.0.66  
Address: 10.244.0.64
```

Praxisaufgabe

StatefulSet und Headless Service

Jobs

- Der Job ist ein Ressourcentyp für einmalige Aufgaben
- Im Gegensatz zu Pods sind sie i.d.R. zeitlich begrenzt
- Bei Fehlern werden sie neu gestartet, bis das Limit erreicht ist
- Bietet Zusatzfeatures wie z.B. parallele Ausführung



Job-Spec

```
apiVersion: batch/v1
kind: Job
metadata:
  name: example-job
spec:
  template:
    spec:
      containers:
        - name: example-job-container
          image: busybox
          command: ["echo", "Hello, Kubernetes Job!"]
          restartPolicy: Never
      backoffLimit: 4
```

Beispiel: Parallele Jobs

```
apiVersion: batch/v1
kind: Job
spec:
  completions: 10
  parallelism: 3
  completionMode: Indexed # Stattet Job-Executions mit Indexen aus
  backoffLimitPerIndex: 1 # Wieviele Fehler sind pro Index erlaubt?
  maxFailedIndexes: 5      # Wieviele Indexe dürfen insgesamt fehlschlagen?
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: example
        image: python
        command:
        - python3
        - -c
        - |
          import os, sys
          if int(os.environ.get("JOB_COMPLETION_INDEX")) % 2 == 0:
            sys.exit(1)
```

Ergebnis

status:

completedIndexes: 1,3,5,7,9

failedIndexes: 0,2,4,6,8

succeeded: 5

failed: 10 # 2 *Versuche pro Pod*

CronJob

CronJobs bieten zeitliche Ausführung von Jobs. Sie werden vom kube-controller-manager verwaltet. Ansonsten verhalten sie sich wie Jobs.

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: hello
spec:
  schedule: "* * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: hello
              image: busybox:1.28
              imagePullPolicy: IfNotPresent
              command:
                - /bin/sh
                - -c
                - date; echo Hello from the cluster
          restartPolicy: OnFailure
```

Kubernetes Storage

Storage

- Pods sind schön und gut – aber in der Regel braucht man Persistenz
- Kubernetes bietet verschiedene Ressourcen zu dem Thema
 - Volumes innerhalb von Pods
 - StorageClasses
 - PersistentVolumes
 - PersistentVolumeClasses

StorageClass

- Bildet einen externen Speicher ab
- Arbeitet mit dem externen Speichersystem zusammen
- Erzeugt dynamisch Speicher
- Der Administrator muss keinen Speicher explizit anlegen



StorageClass

- **provisioner**
Der „Treiber“, der Speicher anlegt
- **parameter**
Konfigurationseinstellungen
- **reclaimPolicy**
Was passiert, wenn der Speicher nicht mehr gebraucht wird
 - **Retain**: Es bleibt
 - **Delete**: Es wird gelöscht

```
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: standard
provisioner: kubernetes.io/gce-pd
parameters:
  type: pd-standard
  zones: us-central1-a, us-central1-b
reclaimPolicy: Delete
```


Es gibt viele StorageClasses

- AWSElasticBlockStorage
- AzureFile
- AzureDisk
- CephFS
- iSCSI
- Local
- VsphereVolume
- ...

Volumes

- Volumes gehören zum **Lebenszyklus des Pods**
- Sind kein eigener Ressourcen-Typ: **Sie gehören zum Pod**
- Ein Pod kann auch mehrere Volumes haben
- Jeder Container des Pods hat Zugriff auf das Volume
- Überlebt einen Container-Neustart
- Wenn der Pod gelöscht wird, greift die **ReclaimPolicy**

Volumes in Pods

- **volumes**

Liste von Volume-Objekten, die in den Pod eingefügt werden. Muss einen Namen haben.

- **volumeMounts**

Für jeden Container. Gibt an, in welches Verzeichnis das Volume zu mounten ist.

```
apiVersion: v1
kind: Pod
metadata:
  name: volume-example
spec:
  containers:
    - name: nginx
      image: nginx:stable-alpine
      volumeMounts:
        - name: html
          mountPath: /usr/share/nginx/html
          ReadOnly: true
    - name: content
      image: alpine:latest
      command: ["/bin/sh", "-c"]
      args:
        - while true; do
            date >> /html/index.html;
            sleep 5;
          done
      volumeMounts:
        - name: html
          mountPath: /html
  volumes:
    - name: html
      emptyDir: {}
```

PersistentVolume

- Ein PersistentVolume (PV) ist ein langlebiger Speicher
- Hat einen vom Pod unabhängigen Lebenszyklus
- Kann NICHT direkt an einen Pod angehängt werden. Dafür braucht der Pod einen s.g. Claim.



PersistentVolume

- **capacity.storage**
Der Speicher, der im Volume verfügbar ist
- **volumeMode**
Je nach Art Filesystem oder Block
- **accessMode**
Gibt an, ob Lesen oder Schreiben möglich ist

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: nfsserver
spec:
  capacity:
    storage: 50Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Delete
  storageClassName: slow
  mountOptions:
    - hard
    - nfsvers=4.1
  nfs:
    path: /exports
    server: 172.22.0.42
```

PersistentVolume

- **persistentVolumeReclaimPolicy**

Wie beim Volume. Gibt an, was nach dem Lösen des Claims passiert.

- **storageClassName**

Wenn angegeben, dann können nur Claims verbunden werden, die die genannte Storage Class haben.

- **mountOptions**

Optionen, die beim Mount verwendet werden.

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: nfsserver
spec:
  capacity:
    storage: 50Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Delete
  storageClassName: slow
  mountOptions:
    - hard
    - nfsvers=4.1
  nfs:
    path: /exports
    server: 172.22.0.42
```

PersistentVolumeClaim

- Fordert für einen Pod ein PersistentVolume an
- Kann mit Regeln eingeschränkt werden, um die genauen Eigenschaften des Volumes zu spezifizieren
- Gedanklich: Ein Claim fordert Storage für einen Pod an.
- Der Storage Provisioner legt PersistentVolumes an, die den Claim erfüllen.

PersistentVolumeClaim

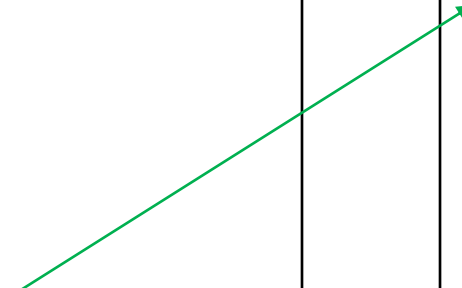
- **accessMode**
Gibt an, welche Zugriffsmodi verwendet werden. Muss vom PV zugelassen sein.
- **resources.requests.storage**
Angeforderte Menge an Speicher
- **storageClassName**
Aus welcher **StorageClass** das Volume stammen soll, das den Claim erfüllen soll.

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: pvc-sc-example
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: slow
```


Claim anhängen

```
apiVersion: v1
kind: Pod
metadata:
  name: task-pv-pod
spec:
  volumes:
    - name: task-pv-storage
      persistentVolumeClaim:
        claimName: task-pv-claim
  containers:
    - name: task-pv-container
      image: nginx
      ports:
        - containerPort: 80
      volumeMounts:
        - mountPath: "/usr/share/nginx/html"
          name: task-pv-storage
```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: task-pv-claim
spec:
  storageClassName: manual
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 3Gi
```



Claim sucht nach Volume

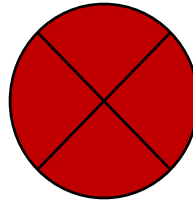
```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: task-pv-claim
spec:
  storageClassName: manual
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 3Gi
```



```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: task-pv-volume
spec:
  storageClassName: manual
  accessModes:
    - ReadWriteOnce
  capacity:
    storage: 10Gi
  hostPath:
    path: "/mnt/data"
```

Claim sucht nach Volume

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: task-pv-claim
spec:
  storageClassName: manual
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 3Gi
```



```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: task-pv-volume
spec:
  storageClassName: manual
  accessModes:
    - ReadWriteOnce
  capacity:
    storage: 2Gi
  hostPath:
    path: "/mnt/data"
```

Lebenszyklus eines PV

Available

- PV ist benutzbar
- Es kann geclaimt werden

Bound

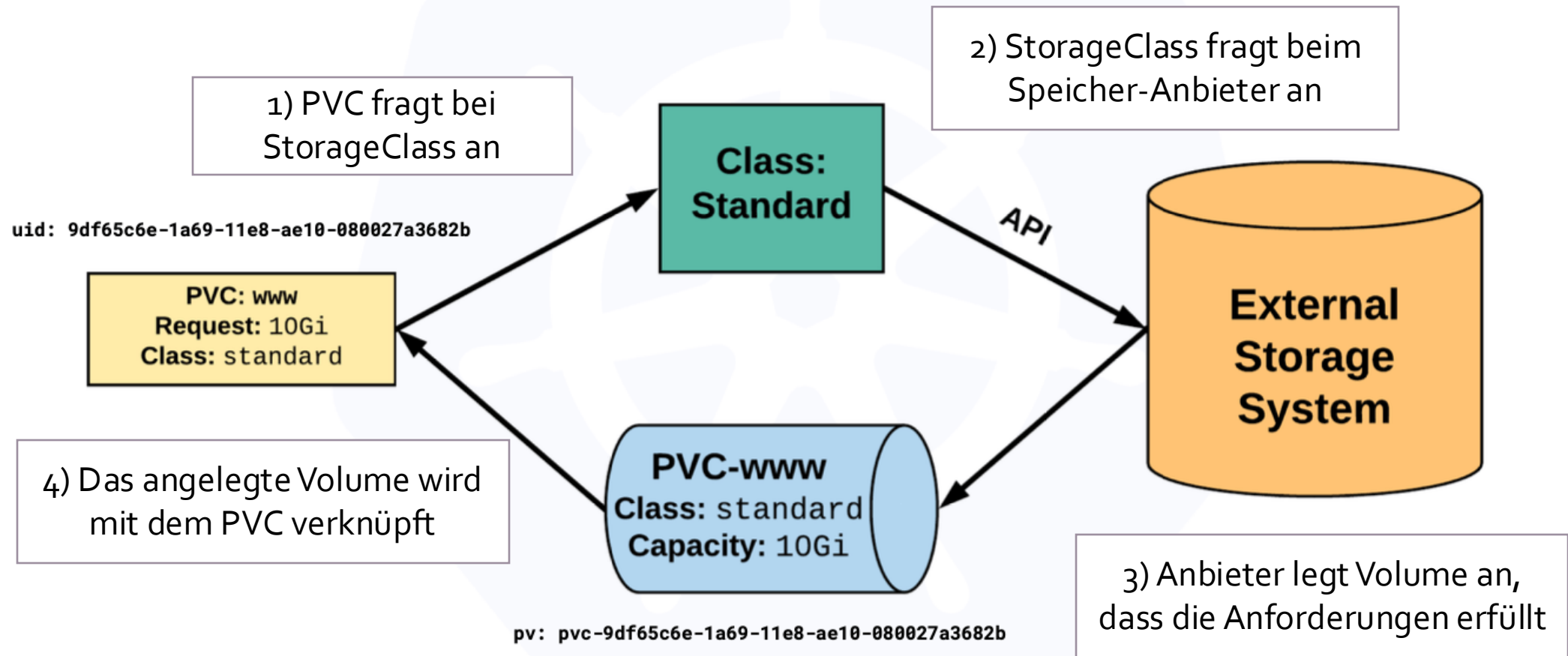
- PV ist gebunden
- Es kann nicht mehr geclaimt werden

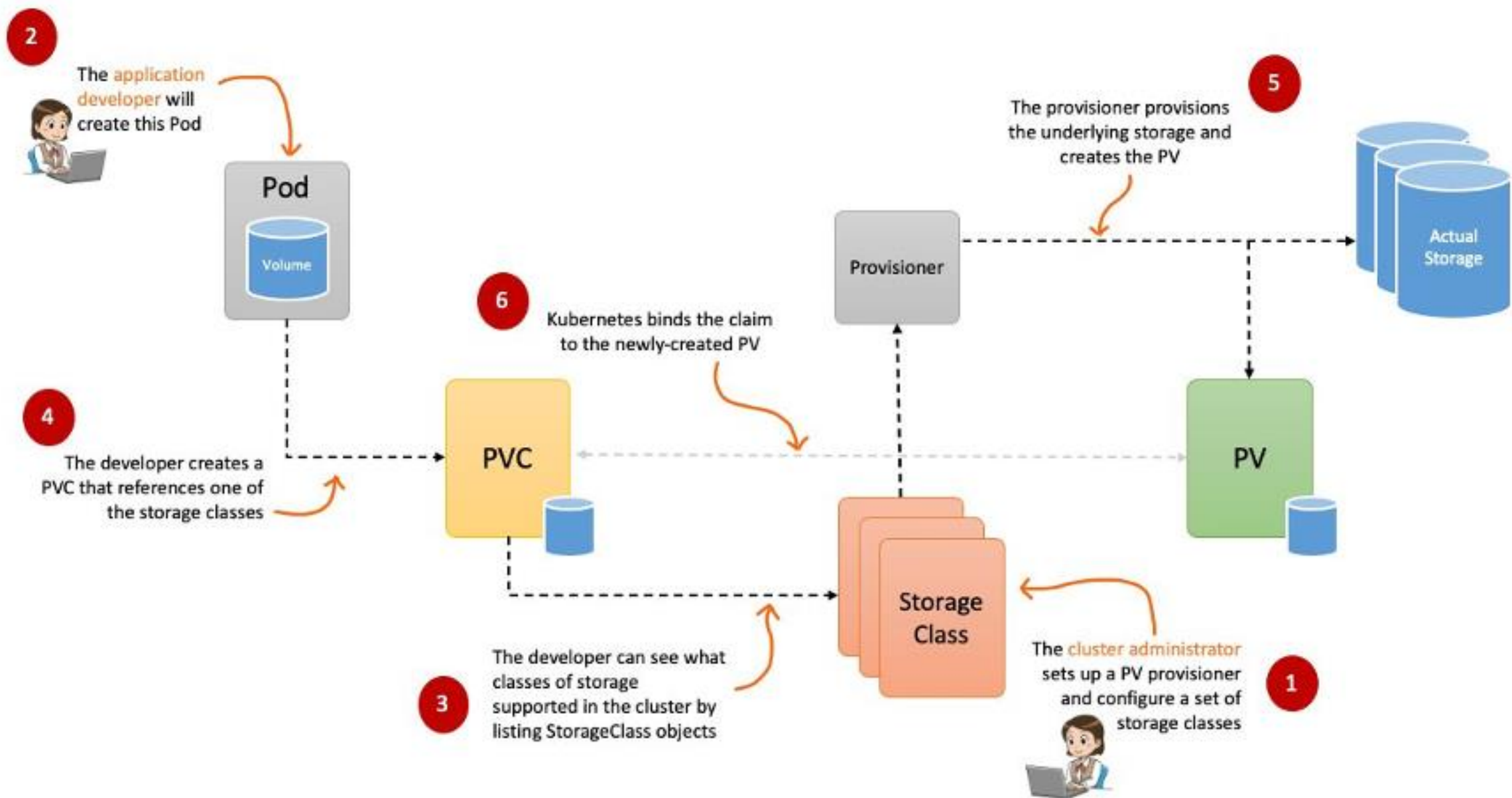
Released

- PV ist freigegeben
- Es kann neu geclaimt werden

Failed

- Ein Fehler ist aufgetreten





<https://aws.amazon.com/blogs/storage/persistent-storage-for-kubernetes/>

VolumeClaimTemplates

- PersistentVolumeClaims müssen nicht notwendigerweise manuell angelegt werden
- Man kann in Pods bzw. Pod Templates auch ein sogenannte **VolumeClaimTemplate** verwenden
- Sie dienen als Vorlage, nach denen Claims erzeugt werden für Pods

```
volumeMounts:  
- name: data  
  mountPath: /var/lib/mysql
```



```
volumeClaimTemplates:  
- metadata:  
    name: data  
  spec:  
    accessModes: ["ReadWriteOnce"]  
    resources:  
      requests:  
        storage: 10Gi
```

Praxisaufgabe

PersistentVolumes

ConfigMaps und Secrets

Warum ConfigMaps, warum Secrets?

```
spec:
  containers:
  - name: mariadb
    image: mariadb:latest
    env:
    - name: MYSQL_ROOT_PASSWORD
      value: "mysecretpassword"
    - name: MYSQL_DATABASE
      value: "nextcloud"
    - name: MYSQL_USER
      value: "nextcloud"
    - name: MYSQL_PASSWORD
      value: "nextcloudpassword"
```

Warum ConfigMaps, warum Secrets?

```
spec:
  containers:
  - name: mariadb
    image: mariadb:latest
    env:
      - name: MYSQL_ROOT_PASSWORD
        value: "mysecretpassword"
      - name: MYSQL_DATABASE
        value: "nextcloud"
      - name: MYSQL_USER
        value: "nextcloud"
      - name: MYSQL_PASSWORD
        value: "nextcloudpassword"
```



ConfigMaps und Secrets

- Sowohl **ConfigMaps** (CMs) als auch **Secrets** bieten wiederverwendbare **Konfiguration**
- Verhindert Duplizierung
- Können sogar auch **extern gespeichert** werden („ExternalSecrets“)

ConfigMaps und Secrets

- Haben einiges gemeinsam
- Wir betrachten zunächst die Unterschiede
- Danach wird an der Verwendung in Containern klar, wo die Gemeinsamkeiten sind

ConfigMaps

- **Konfigurationsdaten** in Kubernetes gespeichert
- Kann auf verschiedenen Wegen eingebunden werden
 - **Umgebungsvariable**
 - **CLI-Argument**
 - Datei als **Volume-Mount**
- Kann aus verschiedenen **Quellen** erzeugt werden

ConfigMap

data: Enthält die Daten als Schlüssel-Wert-Kombinationen

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: manifest-example
data:
  state: Michigan
  city: Ann Arbor
  content: |
    Look at this,
    its multiline!
```

Joe Beda CC 4.0

Alternativen bei der Erstellung

- Es gibt vier Alternativen, wie CMs erstellt werden können
 - Von einem **Manifest**
 - Von einem **Literal**
 - Von einem **Verzeichnis**
 - Von **Einzeldateien**
- Alle folgenden Varianten führen zum gleichen Ergebnis!

Alternativen bei der Erstellung

Von einem **Manifest**

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: manifest-example
data:
  city: Köln
  state: NRW
```

Alternativen bei der Erstellung

Als Literal

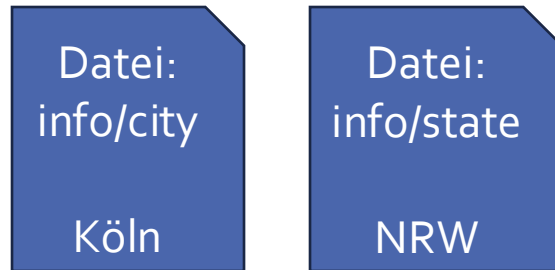
```
$ kubectl create configmap literal-example \
  --from-literal="city=Köln" \
  --from-literal="state=NRW"
```

```
# configmap "literal-example" created
```

Alternativen bei der Erstellung

Von einem **Verzeichnis**. Die Dateinamen werden dann die Schlüssel

```
$ cat info/city  
Köln  
$ cat info/state  
NRW
```



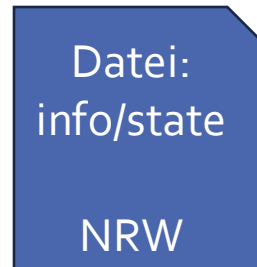
```
$ kubectl create configmap dir-example --from-file=info/  
# configmap "dir-example" created
```

Alternativen bei der Erstellung

Direkt aus Einzel-Dateien.

```
$ cat info/city  
Köln
```

```
$ cat info/state  
NRW
```



```
$ kubectl create configmap file-example \  
  --from-file=info/city \  
  --from-file=info/state  
  
# configmap "file-example" created
```

Secrets

- Sind sehr ähnlich wie ConfigMaps
- Werden intern **base64-encoded** abgespeichert
- In der Regel **Encryption at Rest**
- Sind auf jedem **Worker** in **tmpfs** abgespeichert
- Sinnvoll für **Benutzernamen, Passwörter** etc.

Secrets

- **type**
 - **docker-registry**: Credentials, um sich bei einer Registry anzumelden
 - **generic/opaque**: literale Secrets
 - **tls**: Zertifikate
- **data**: Base64-encodierte Schlüssel-Wert-Kombinationen

```
apiVersion: v1
kind: Secret
metadata:
  name: manifest-secret
type: Opaque
data:
  username: ZXhhbXBsZQ==
  password: bXlwYXNzd29yZA==
```

Mehrere Wege

Die folgenden Wege erzeugen
alle das gleiche Secret

Manifest

```
apiVersion: v1
kind: Secret
metadata:
  name: manifest-example
  type: Opaque
data:
  username: ZXhhbXBsZQ==
  password: bXlwYXNzd29yZA==
```


Literal

```
$ kubectl create secret generic literal-secret \  
  --from-literal=username=example \  
  --from-literal=password=mypassword
```

Verzeichnis

Datei:
secret/username

example

Datei:
secret/password

mypassword

```
$ kubectl create secret generic dir-secret \
  --from-file=secret
```

Einzelne Dateien

Datei:
secret/username

example

Datei:
secret/password

mypassword

```
$ kubectl create secret generic file-secret \  
  --from-file=secret/username \  
  --from-file=secret/password
```

ConfigMap und Secrets nutzen: Env

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-1
spec:
  containers:
    - name: mypod
      image: alpine:latest
      env:
        - name: CITY
          valueFrom:
            configMapKeyRef:
              name: manifest-example
              key: city
```

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-1
spec:
  containers:
    - name: mypod
      image: alpine:latest
      env:
        - name: USERNAME
          valueFrom:
            secretKeyRef:
              name: manifest-example
              key: username
```

ConfigMap und Secrets nutzen: Command

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-1
spec:
  containers:
  - name: mypod
    image: alpine:latest
    command: ["/bin/sh", "-c"]
    args: ["echo Hello ${CITY}!"]
    env:
    - name: CITY
      valueFrom:
        configMapKeyRef:
          name: manifest-example
          key: city
```

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-1
spec:
  containers:
  - name: mypod
    image: alpine:latest
    command: ["/bin/sh", "-c"]
    args: ["echo Hello ${USERNAME}!"]
    env:
    - name: USERNAME
      valueFrom:
        secretKeyRef:
          name: manifest-example
          key: username
```

ConfigMap und Secrets nutzen: Volume

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-1
spec:
  containers:
  - name: mypod
    image: alpine:latest
    command: ["/bin/sh", "-c"]
    args: ["cat /myconfig/city"]
    volumeMounts:
    - name: config-volume
      mountPath: /myconfig
  volumes:
  - name: config-volume
    configMap:
      name: manifest-example
```

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-1
spec:
  containers:
  - name: mypod
    image: alpine:latest
    command: ["/bin/sh", "-c"]
    args: ["cat /myconfig/username"]
    volumeMounts:
    - name: config-volume
      mountPath: /myconfig
  volumes:
  - name: config-volume
    secret:
      name: manifest-example
```

Praxisaufgabe

ConfigMaps und Secrets

Case Study: Nextcloud

Nextcloud

- Privater Cloud-Speicher
- In PHP entwickelt und verwaltet Daten in Form von Dateien und Datenbanken.
- Unterstützt werden die Datenbankmanagementsysteme SQLite-, MariaDB-, MySQL- oder PostgreSQL.

Was brauchen wir?



Was brauchen wir?



MariaDB



Wie realisieren wir MariaDB

- MariaDB braucht Persistenz
- Dazu brauchen wir ein PersistentVolume und demgemäß auch ein PersistentVolumeClaim
- Außerdem muss die Zuordnung von Datenbank zu Speicher fest sein
- Nur ein Pod sollte laufen mit genau dem passenden Volume



StatefulSet

- Es garantiert, dass der Pod für die Datenbank genau das passende Volume erhält
- Im Fehlerfall wird der Pod neu gestartet und neu verbunden
- Wir erhalten einen festen Namen für die Datenbank mit einem Headless Service

Wie realisieren wir Nextcloud?



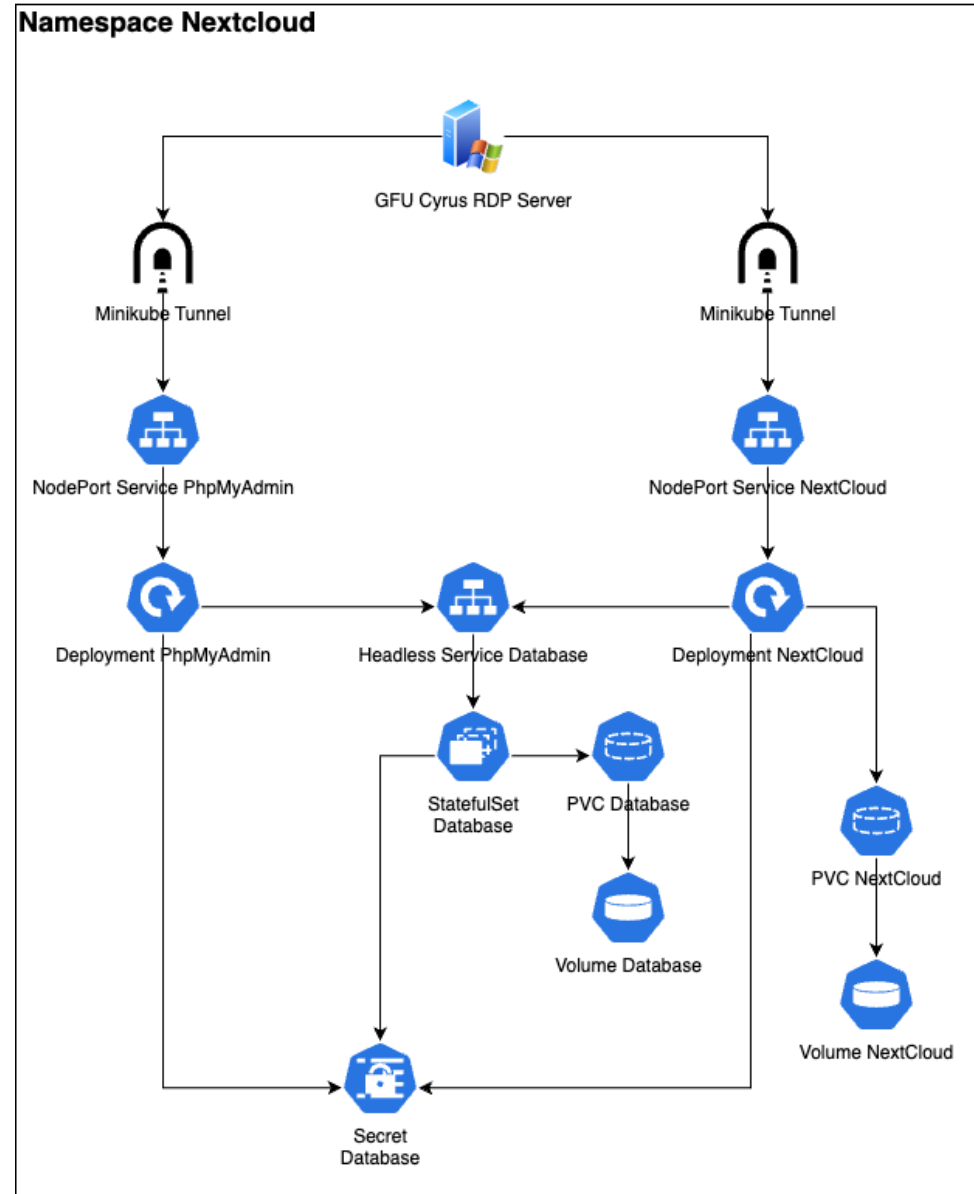
- Auch Nextcloud braucht persistenten Speicher
- Es ist aber akzeptabel, mehrere Instanzen gegen den gleichen Speicher laufen zu lassen
- Nextcloud benutzt die Datenbank und braucht Zugriff darauf

Deployment

- Sowohl Deployment als auch StatefulSet sind denkbar
- Deployments haben aber den Vorteil, dass wir Nextcloud besser skalieren können
- Es ist möglich, mehrere Nextcloud-Replikas gegen das gleiche Volume laufen zu lassen

Was brauchen wir noch?

- Wir brauchen einen **Headless Service** für die Datenbank, damit Nextcloud intern zugreifen kann
- Wir brauchen einen **NodePort-Service** für Nextcloud, damit wir uns mit Minikube „reintunneln“ können
- Wir sollten die **Datenbank-Zugangsdaten** in ein Secret speichern
- Wenn wir PhpMyAdmin wollen, brauchen wir auch dafür einen NodePort-Service



Projekt

Nextcloud auf Kubernetes

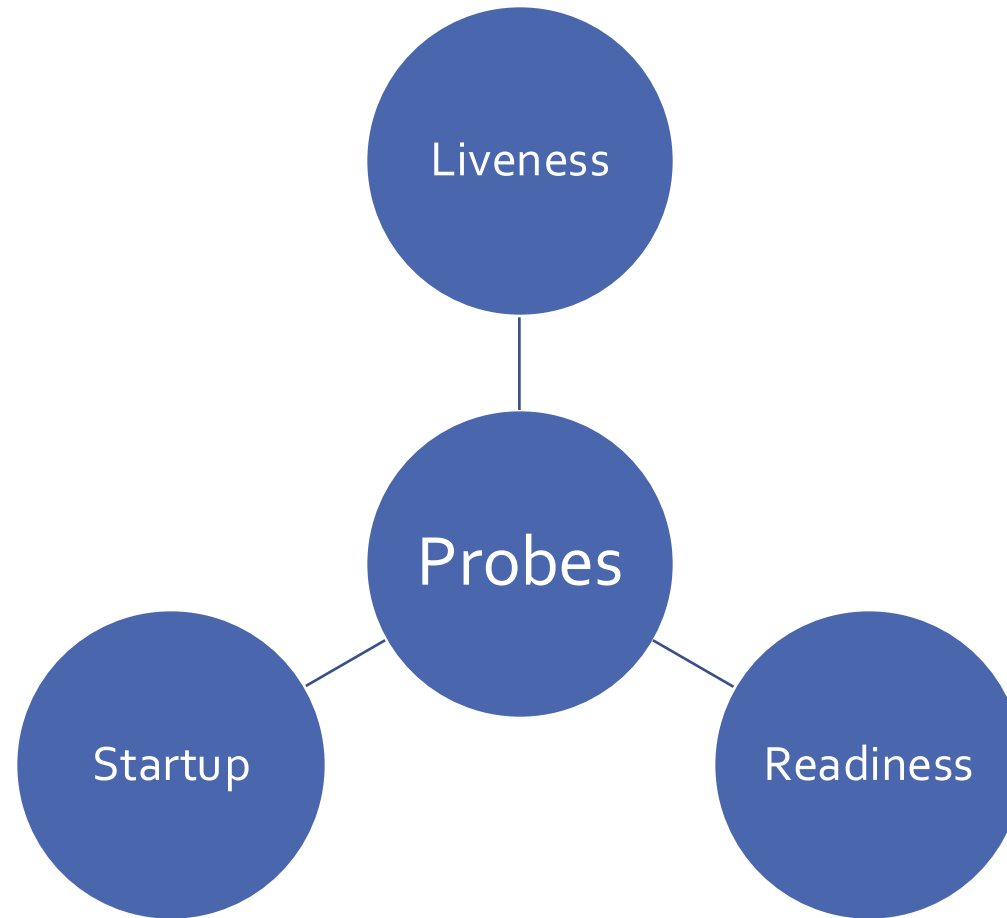
Pod Lifecycle und Health Checks

Hochverfügbarkeit von Pods

- Kubernetes kann erkennen, ob Pods auf einem Node abgestürzt sind
- Es kann aber nicht erkennen, ob Pods noch „gesund“ sind
- Dafür bietet Kubernetes unterschiedliche Typen von Health Checks



Health Checks



Liveness Probe

- Prüft, ob ein Pod überhaupt noch funktionsfähig ist
- Schlägt zum Beispiel fehl, wenn die Anwendung sich komplett aufgehängt hat
- Führt dazu, dass Kubernetes den Container im Pod ersetzt
- Mit **gleichem** Image.

Startup Probe

- Liveness-Probe mit Verzögerung
- Dient dazu, dem Pod Zeit zur Initialisierung zu lassen
- Diese Zeit wird auch Grace Period genannt



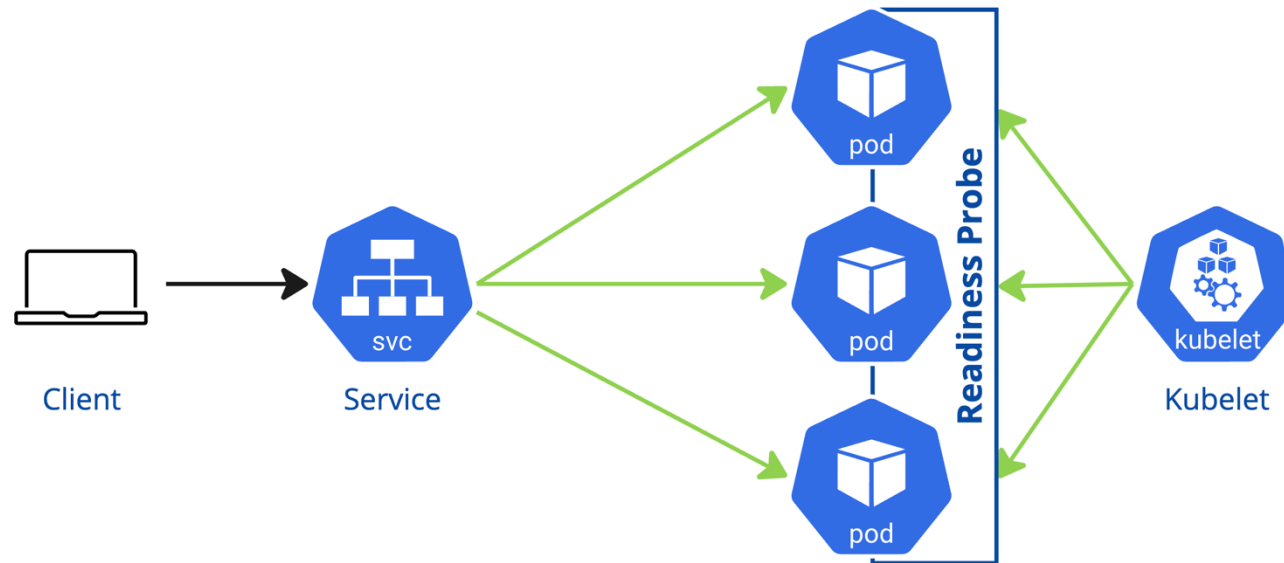
Liveness und Startup Probes – Eher zweifelhaft

- Fehlschlagende Liveness- und Startup-Probes führen zu Container-Replacements
- In aller Regel hilft dies nicht, weil das Problem das Image ist
- Schlägt der Start wiederholt fehl, kommt es zum **CrashLoopBackoff**.
- In 99,9% der Fälle muss durch eine neue Image-Version das Problem behoben werden
- Besser ist es, wenn die Anwendung so gebaut ist, dass sie direkt abstürzt, wenn sie nicht lauffähig ist

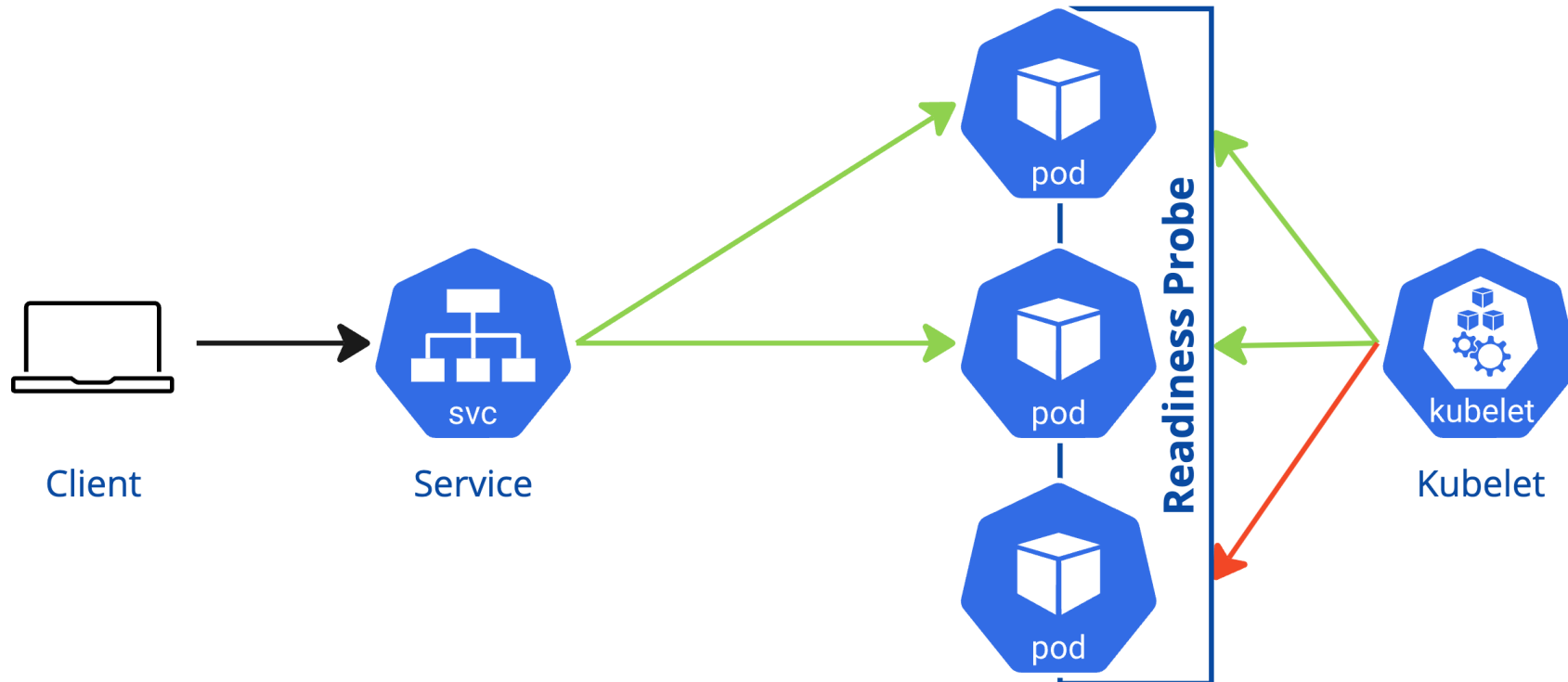
Readiness Probe

- Prüft die korrekte Funktion der Anwendung
- Beispielsweise wird bei einem Webserver geprüft, ob auf Port 80 die Standardseite erscheint
- Schlagen sie fehl, erhält der Pod keinen Traffic mehr von Services
- Es kommt nicht zu Container-Recreation bei Fehlschlag.

Readiness Probe



Readiness Probe



Thresholds – Beispiele

livenessProbe:

failureThreshold: 3
periodSeconds: 10
successThreshold: 1

readinessProbe:

failureThreshold: 3
periodSeconds: 10
successThreshold: 1

Probe-Verfahren

Health Checks können über vier Verfahren durchgeführt werden:

- **httpGet**
Ruft eine URL via GET auf. Codes von 200 bis 399 werden als Erfolg gewertet.
- **tcpSocket**
Baut eine TCP-Verbindung auf. Ist so lange healthy, wie auf den Socket zugegriffen werden kann.
- **gRPC**
Für Container, die ein gRPC-Interface haben
- **exec**
Führt beliebige Skripte in Containern aus. Der Container gilt als healthy, wenn das Skript den Exit Code „0“ zurückgibt.

Beispiel Exec-Probe

```
spec:
  containers:
  - name: busybox-container
    image: busybox
    command: [ "...", ]
    livenessProbe:
      exec:
        command:
        - sh
        - -c
        - test -f /tmp/health
      initialDelaySeconds: 5
      periodSeconds: 10
```

Beispiel httpGet-Probe

```
spec:
  containers:
  - name: nginx
    image: nginx:latest
    ports:
    - containerPort: 80
  livenessProbe:
    httpGet:
      path: /
      port: 80
    initialDelaySeconds: 10
    periodSeconds: 5
```

Praxisaufgabe

Readiness Probes

Pod Lifecycle & Scheduling

Pod-Lifecycle: Hooks

- Container in Pods lassen sich mit Lifecycle Hooks ausstatten
- Die werden nach Start und vor dem Stopp eines Containers ausgelöst
- Sie lassen sich mit httpGet und exec verwenden

Beispiel: Pod Lifecycle Hooks

```
spec:
  containers:
  - name: example-container
    image: busybox
    command: ["/bin/sh", "-c", "sleep 3600"] # Container bleibt für 1 Stunde aktiv
    lifecycle:
      postStart:
        exec:
          command:
            - /bin/sh
            - -c
            - "echo 'Container has started' > /tmp/postStart.txt"
      preStop:
        exec:
          command:
            - /bin/sh
            - -c
            - "echo 'Container is stopping' > /tmp/preStop.txt"
```

Scheduling

Node Selectors

Scheduling

- Kubernetes-Scheduling ist ein komplexes Regelsystem
- Basis sind Predicates und Priorities
- **Predicates** sind „harte“ Regeln die bestimmen, ob ein Pod überhaupt auf einen Node passt
- **Priorities** bestimmen, auf welchen Node ein Pod bestmöglich passt.

Node Selectors

- Einfachstes Constraint zum Steuern des Scheduling.
- Verwendet node-Labels, um die Platzierung zu steuern.
- Beispiel für das Labelling von Nodes:

```
$ kubectl label node first-node second-node "tier=frontend"  
$ kubectl label node third-node "tier=backend"
```

first-node
tier=frontend

second-node
tier=frontend

third-node
tier=backend

Deployment mit NodeSelector

```
apiVersion: apps/v1
kind: Deployment
...
spec:
  replicas: 4
  selector:
    matchLabels:
      app: selector-demo
  template:
    metadata:
      labels:
        app: selector-demo
    spec:
      nodeSelector:
      tier: frontend
      containers:
      - name: nginx
        image: nginx
```

Abgleich der Labels

```
apiVersion: apps/v1
kind: Deployment
...
spec:
  replicas: 4
  selector:
    matchLabels:
      app: selector-demo
  template:
    metadata:
      labels:
        app: selector-demo
    spec:
      nodeSelector:
        tier: frontend
      containers:
        - name: nginx
          image: nginx
```

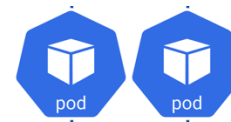
first-node
tier=frontend

second-node
tier=frontend

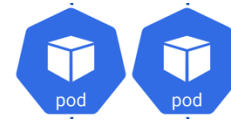
third-node
tier=backend

Abgleich der Labels

```
apiVersion: apps/v1
kind: Deployment
...
spec:
  replicas: 4
  selector:
    matchLabels:
      app: selector-demo
  template:
    metadata:
      labels:
        app: selector-demo
    spec:
      nodeSelector:
        tier: frontend
      containers:
        - name: nginx
          image: nginx
```



first-node
tier=frontend



second-node
tier=frontend

third-node
tier=backend

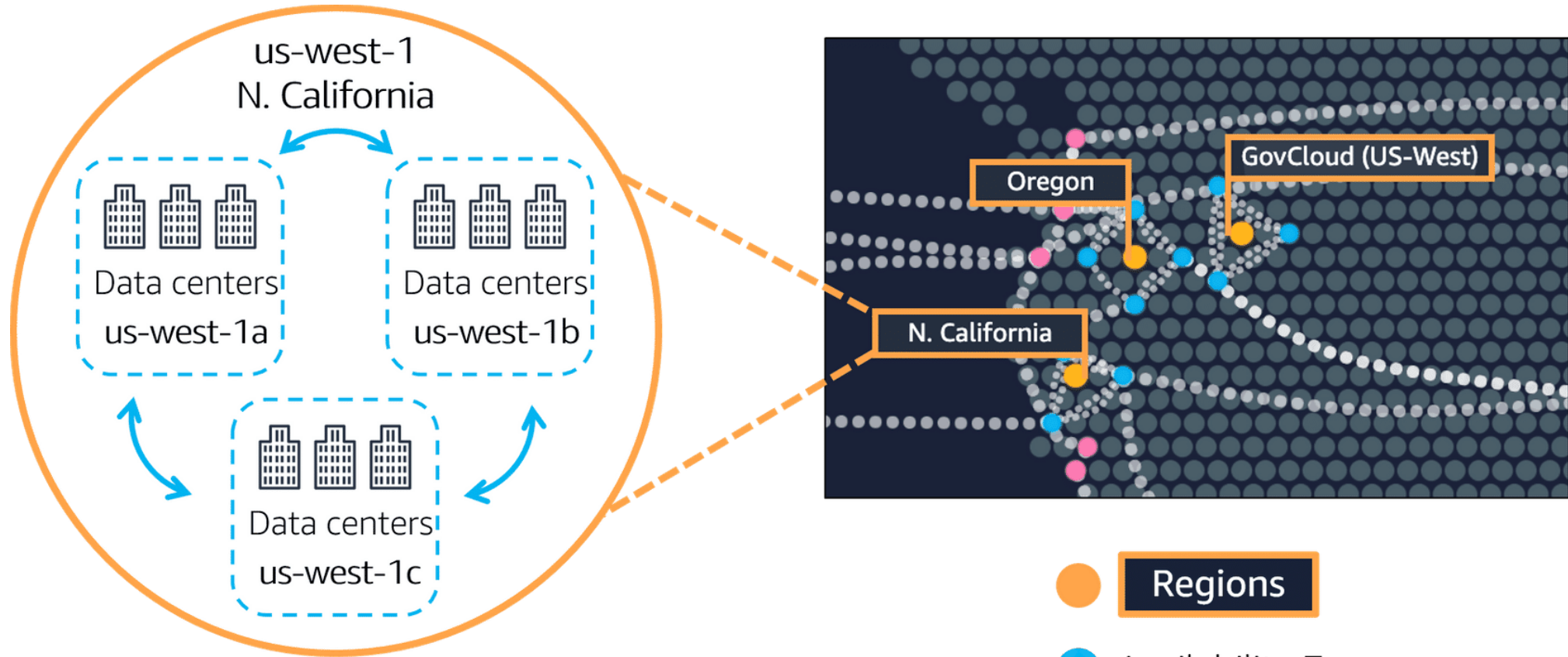
Scheduling

Pod Topologies

Pod Topology Spread Constraints

- Pods werden standardmäßig dorthin verteilt, wo Platz ist
- Dabei wird zunächst nicht auf **Verfügbarkeitszonen** geachtet
- Alle Pods könnten im gleichen Rechenzentrum landen
- Das ist nicht ideal für **Hochverfügbarkeit**

Exkurs: Availability Zones

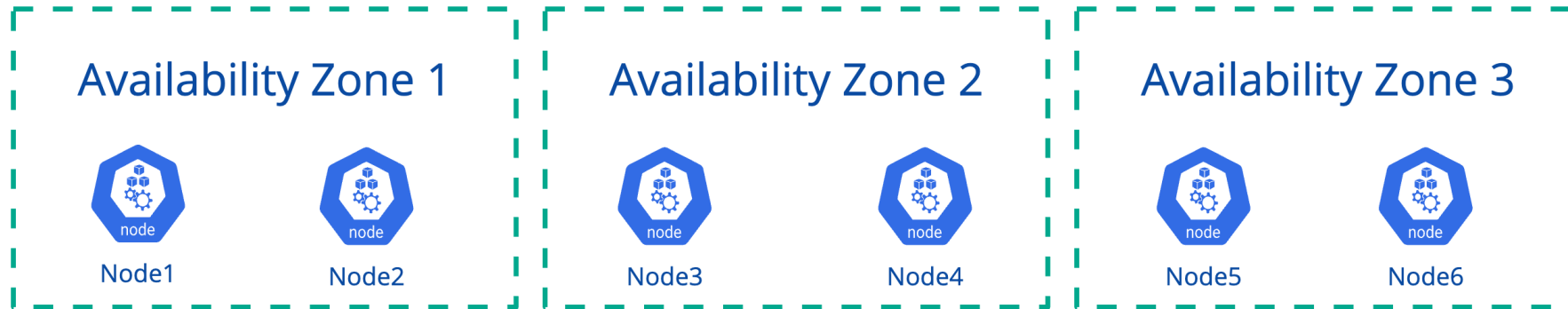


Regions

Availability Zones

Quelle: Amazon Web Services

Topologien



LABELS

node=node1, zone=AZ1
node=node2, zone=AZ1
node=node3, zone=AZ2
node=node4, zone=AZ2
node=node5, zone=AZ3
node=node6, zone=AZ3

zone ist ein eingebauter
Topologie-Schlüssel.
Daneben gibt es noch
hostname und **region**.

Topology Spread

spec:

topologySpreadConstraints:

- **maxSkew:** 1

whenUnsatisfiable: *ScheduleAnyway*

topologyKey: *hostname*

labelSelector:

matchLabels:

app: tracking-backend

Topology Spread

- **maxSkew** bestimmt, wie viele Pods höchstens ungleichmäßig verteilt sein dürfen
- **whenUnsatisfiable** bestimmt, ob der Spread ein Pflichtkriterium ist (DoNotSchedule) oder nur eine Priorität beim Scheduling (ScheduleAnyway)
- **topologyKey** gibt ab, auf welche Art von Topologie sich der Constraint bezieht. Es kann sich um die Region, die Zone oder den Hostname handeln.
- **labelSelector** selektiert die Pods, für die der Constraint gilt

Beispiel

```
spec:  
  topologySpreadConstraints:  
    - maxSkew: 1  
      whenUnsatisfiable: DoNotSchedule  
      topologyKey: zone  
      labelSelector:  
        matchLabels:  
          app:nginx
```


Beispiel: DoNotSchedule



Bei $\text{maxSkew} = 2$ kann ein weiterer Pod sowohl in AZ 2 als auch AZ 3 gestartet werden

Bei $\text{maxSkew} = 1$ geht dies nur in AZ 2.

Scheduling

Taints und Tolerations

Affinities

- Zur Platzierung von Nodes kennen wir bereits **NodeSelectors**
- Es gibt weitere Möglichkeiten, das Scheduling zu beeinflussen
- Affinities sind flexibler als NodeSelectors



Node Affinities

- **requiredDuringSchedulingIgnoredDuringExecution**
ist eine harte Regel. Das Scheduling eines Pods ist nur möglich, wenn die Prüfung als Ganzes positiv ausfällt.
- **preferredDuringSchedulingIgnoredDuringExecution**
drückt gegenüber dem Scheduler nur den Wunsch aus, dass die Regel eingehalten wird. Sie ist kein hartes Kriterium.

Node Affinities

```
apiVersion: v1
kind: Pod
metadata:
  name: with-node-affinity
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: topology.kubernetes.io/zone
                operator: In
                values:
                  - antarctica-east1
                  - antarctica-west1
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 1
          preference:
            matchExpressions:
              - key: another-node-label-key
                operator: In
                values:
                  - another-node-label-value
  containers:
    - name: with-node-affinity
      image: registry.k8s.io/pause:2.0
```

Required Selection

requiredDuringSchedulingIgnoredDuringExecution:

nodeSelectorTerms:

- **matchExpressions:**

- **key:** topology.kubernetes.io/zone

- operator:** In

- values:**

- antarctica-east1

- antarctica-west1

Preference

preferredDuringSchedulingIgnoredDuringExecution:

- **weight: 1**

- preference:**

- matchExpressions:**

- **key: another-node-label-key**

- operator: In**

- values:**

- **another-node-label-value**

Taints und Tolerations

- Taints erlauben es Nodes, Pods von sich fernzuhalten
- Dieses Feature wird von der Control Plane genutzt, um keine Arbeitslasten auszuführen
- Taints können selektiv über Tolerations umgangen werden
- Ein Taint ist wie auch ein Label ein Schlüssel-Wert-Paar

Effects

Taints und Tolerations können mit drei verschiedenen Effekten konfiguriert werden:

- **NoSchedule:** Pods ohne Tolerations können einem Node nicht mehr zugewiesen. Bestehende Pods verbleiben.
- **PreferNoSchedule:** Pods ohne Tolerations werden nur, wenn es gar nicht anders geht, dem Node zugewiesen.
- **NoExecute:** Pods ohne Tolerations können einem Node nicht mehr zugewiesen. Außerdem werden bestehende Pods vom Node entfernt, wenn sie den Taint nicht tolerieren.

Taints auf Nodes

```
$ kubectl taint nodes node1 mytaint=mytaintvalue:NoSchedule
```

Node1 kann nun kein Pod mehr zugewiesen, der keine Tolerant für den Taint **mytaint=mytaintvalue** mit Effect **NoSchedule** hat.

Bekanntes Beispiel-Taint:
node-role.kubernetes.io/control-plane

Toleration in einem Pod

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    env: test
spec:
  containers:
  - name: nginx
    image: nginx
    imagePullPolicy: IfNotPresent
  tolerations:
  - key: "mytaint"
    operator: "Exists"
    effect: "NoSchedule"
```

Scheduling

Requests und Limits

Requests und Limits

- Für jeden Container können Ressourcen definiert werden, die er benötigt
- Dies geht mit sogenannten Requests und Limits
- Requests werden vom Scheduler genutzt um zu beurteilen, wo Container passen
- Limits sind das Maximum des Ressourcenverbrauchs, bevor ein Container gekillt wird
- Kann für CPU und für Memory angegeben werden



Resources in der Spec

```
spec:  
  containers:  
  - name: nginx  
    image: nginx:latest  
    ports:  
    - containerPort: 80  
  resources:  
    requests:  
      memory: "256Mi"  
      cpu: "500m"  
    limits:  
      memory: "512Mi"  
      cpu: "1"
```

Default Limit Ranges

- Sowohl für Requests als auch für Limits kann ein Default pro Namespace gesetzt werden
- Jeder Pod kann die Defaults beliebig überschreiben
- Man muss damit aber vorsichtig sein

Default Limit Ranges

```
apiVersion: v1
kind: LimitRange
metadata:
  name: cpu-resource-constraint
spec:
  limits:
  - default:
      cpu: 500m
    defaultRequest:
      cpu: 500
    max:
      cpu: "1"
    min:
      cpu: 100m
  type: Container
```


Fehlkonfiguration möglich

```
apiVersion: v1
kind: Pod
metadata:
  name: conflict-example
spec:
  containers:
  - name: demo
    image: registry.k8s.io/pause:2.0
    resources:
      requests:
        cpu: 700m
```

Fehlkonfiguration möglich

```
apiVersion: v1
kind: Pod
metadata:
  name: conflict-example
spec:
  containers:
  - name: demo
    image: registry.k8s.io/pause:2.0
    resources:
      requests:
        cpu: 700m
```



```
limits:
- default:
  cpu: 500m
```

Fehlkonfiguration möglich

```
apiVersion: v1
kind: Pod
metadata:
  name: conflict-example
spec:
  containers:
  - name: demo
    image: registry.k8s.io/pause:2.0
    resources:
      requests:
        cpu: 700m
```



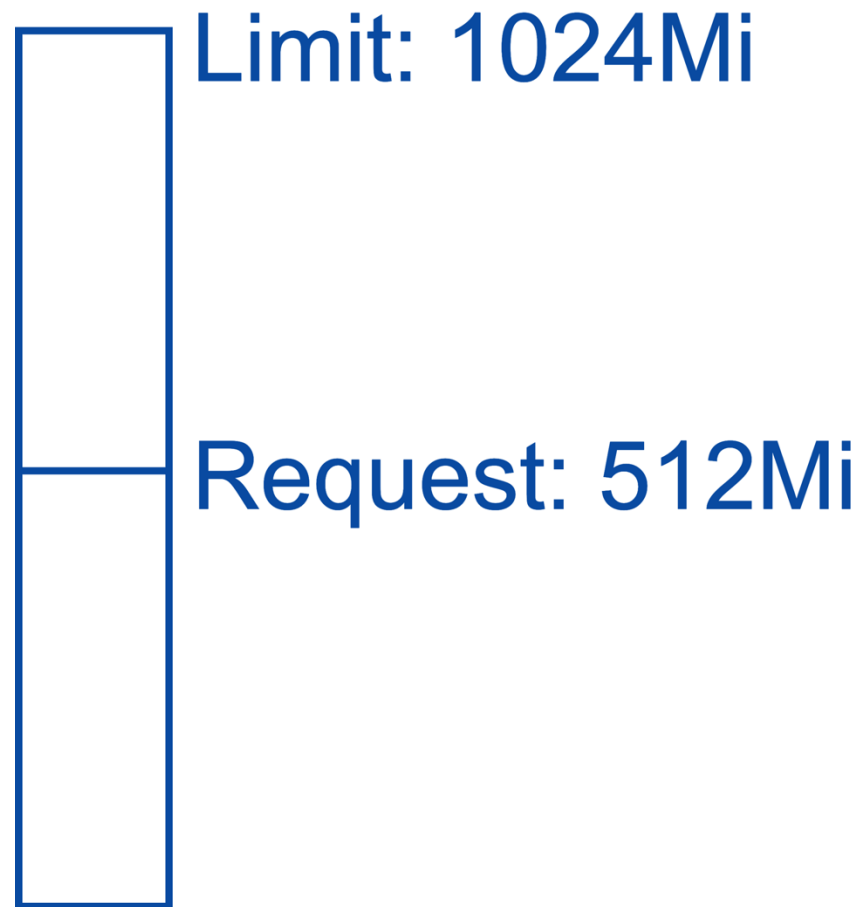
```
limits:
- default:
  cpu: 500m
```

```
Pod is invalid:
spec.containers[0].resources.requests:
Invalid value: "700m": must be less than
or equal to cpu limit
```

Resources beim Scheduling

- Beim Scheduling werden nur die **Requests** beachtet
- Die **Limits** spielen beim Scheduling keine Rolle
- Das kann in der Praxis gravierende Konsequenzen haben

Limits und Requests



Limits und Requests

Node: 2048Mi

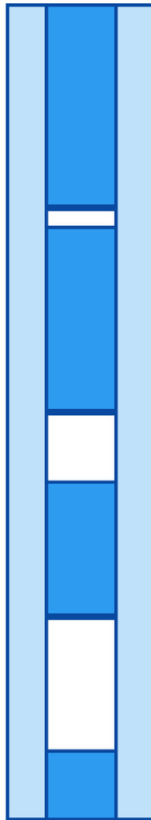


Node: 2048Mi

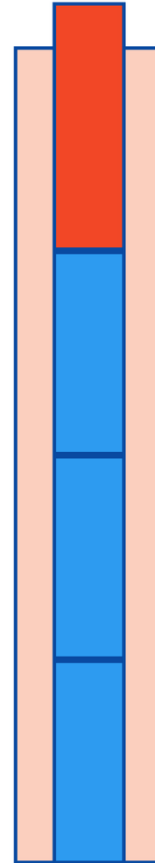


Limits und Requests

Node: 2048Mi



Node: 2048Mi



Overcommitment

- Ist die Summe der Limits höher als die verfügbaren Ressourcen des Hosts, nennt man das **Overcommitment**
- Dies kann zu Swap und schlimmstenfalls zu OOMs führen
- Es kann deswegen sinnvoll sein, die Limits immer gleich den Requests zu setzen

Scheduling

PriorityClasses und PDBs

PriorityClasses

- Man kann Pods den Vorzug vor anderen geben beim Scheduling
- Interessant z.B. bei Pods für HA-Anwendungen
- Kann auch dazu benutzt werden, Pods in Lastszenarien zu verdrängen

PriorityClasses

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: high-priority
value: 1000000
preemptionPolicy: PreemptLowerPriority
globalDefault: false
description: ""
```

PriorityClass in Pods

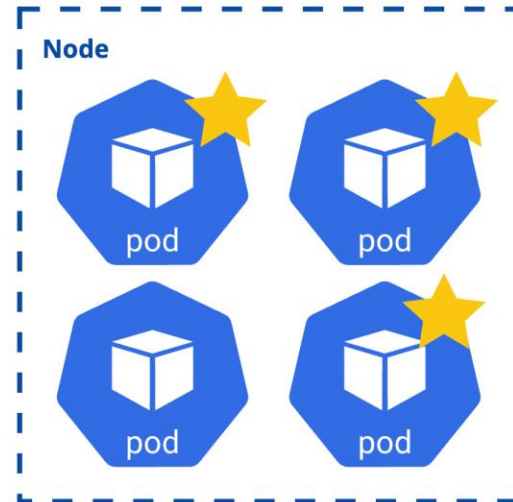
```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    env: test
spec:
  containers:
  - name: nginx
    image: nginx
  priorityClassName: high-priority
```

Preemption

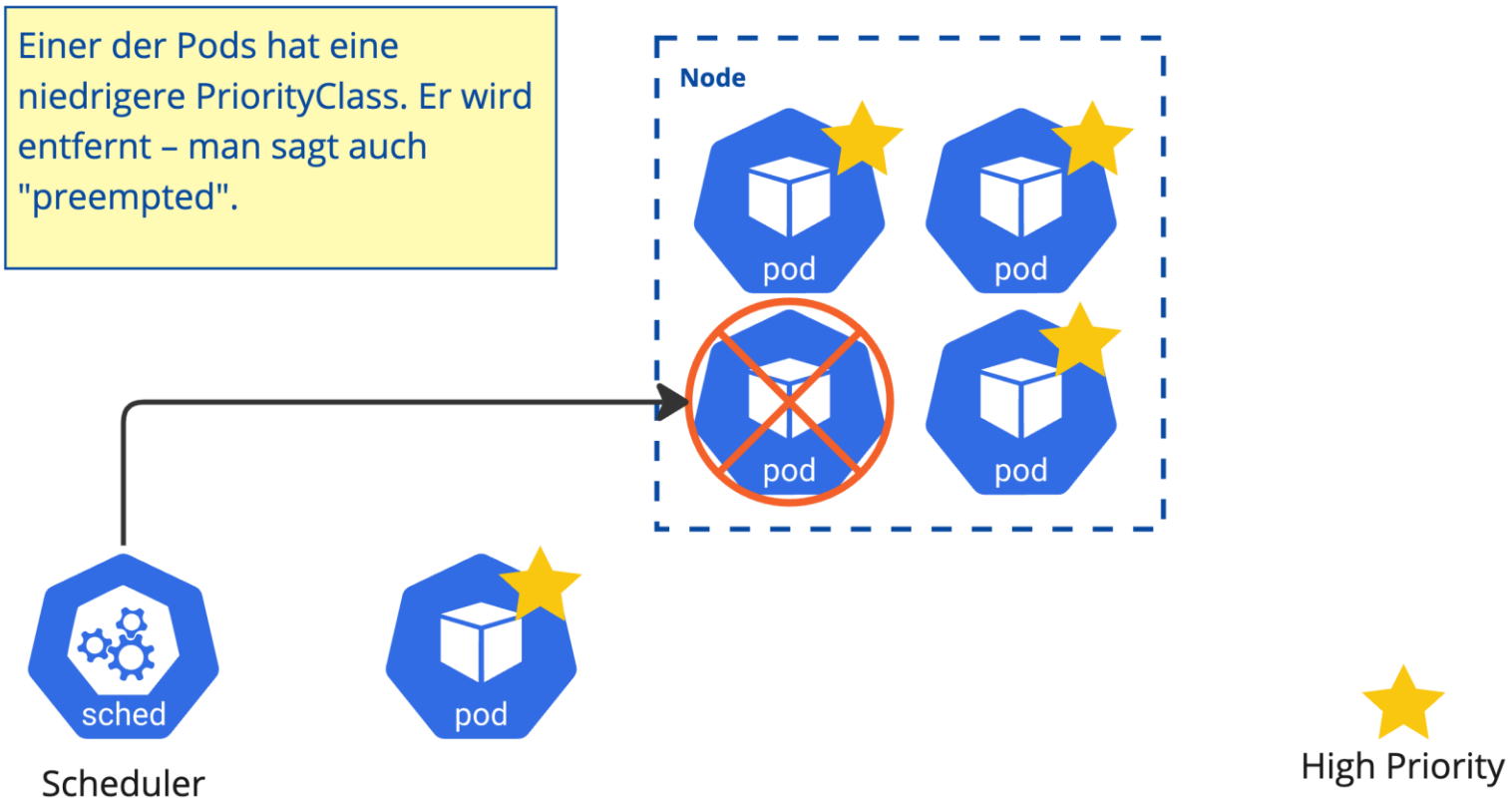
Der Scheduler muss einen neuen Pod platzieren mit einer hohen PriorityClass.
Auf keinem Node ist Platz übrig.



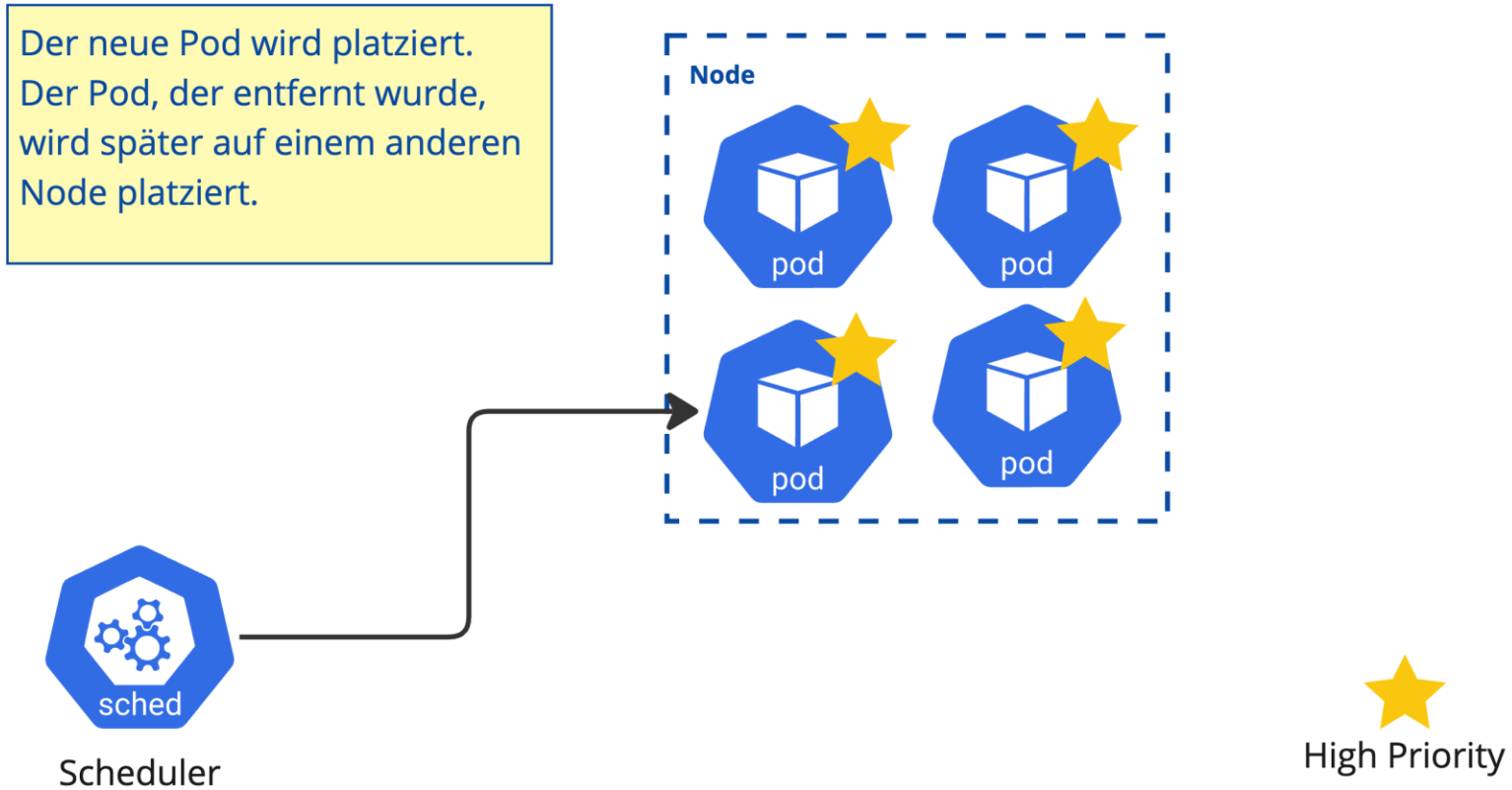
Scheduler



Preemption



Preemption



Preemption unterbinden

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: high-priority
value: 1000000
preemptionPolicy: Never
globalDefault: false
description: ""
```

Hier dient die PriorityClass nur der Priorisierung beim Scheduling.

PodDisruptionBudget

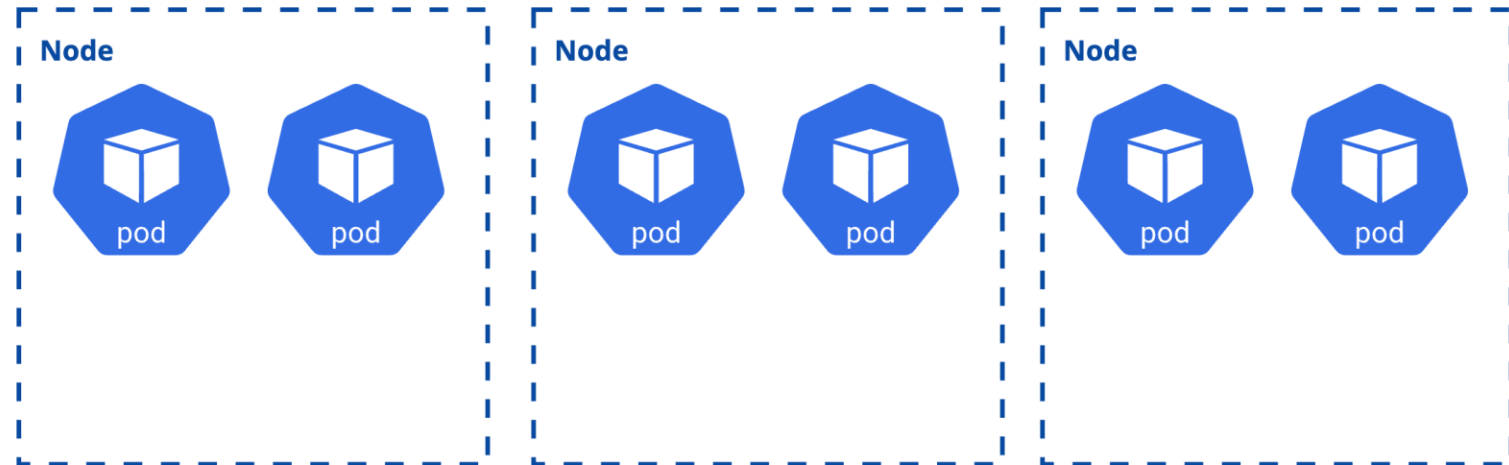
- Disruptionen können durch unterschiedliche Situation entstehen
- Wir haben schon eine kennen gelernt: Rolling Update
- Aber es können z.B. auch Nodes ausfallen oder geplant heruntergefahren werden

PDBs

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: my-pdb
spec:
  maxUnavailable: 1
  selector:
    matchLabels:
      app: nginx
```

PDBs

```
spec:  
  maxUnavailable: 1  
  selector:  
    matchLabels:  
      app: nginx
```



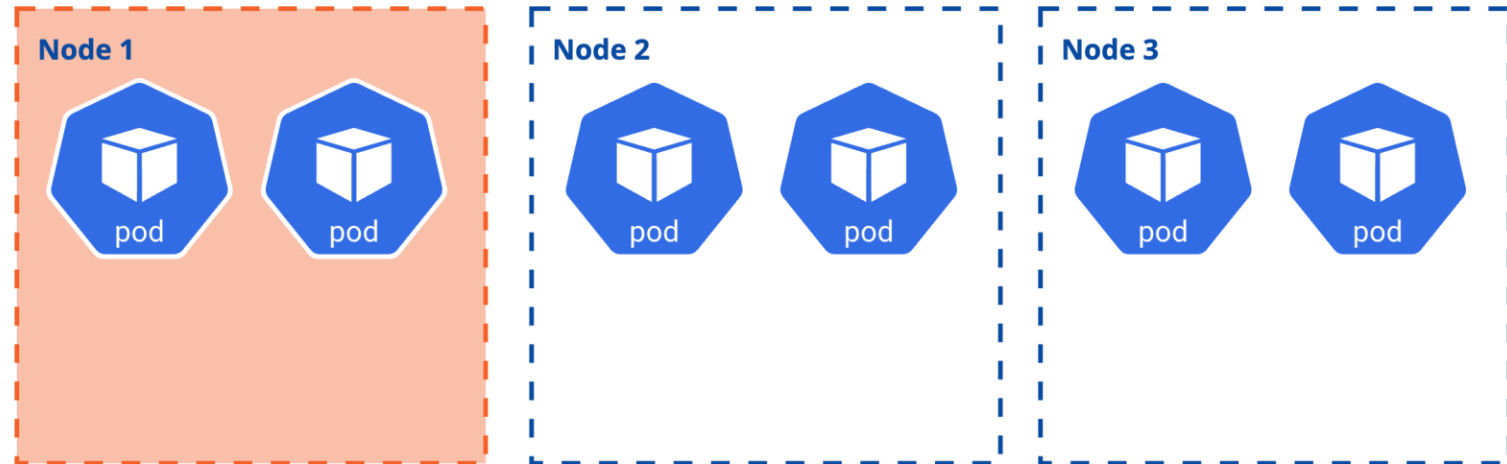
Scheduler

PDBs

```
spec:  
  maxUnavailable: 1  
  selector:  
    matchLabels:  
      app: nginx
```



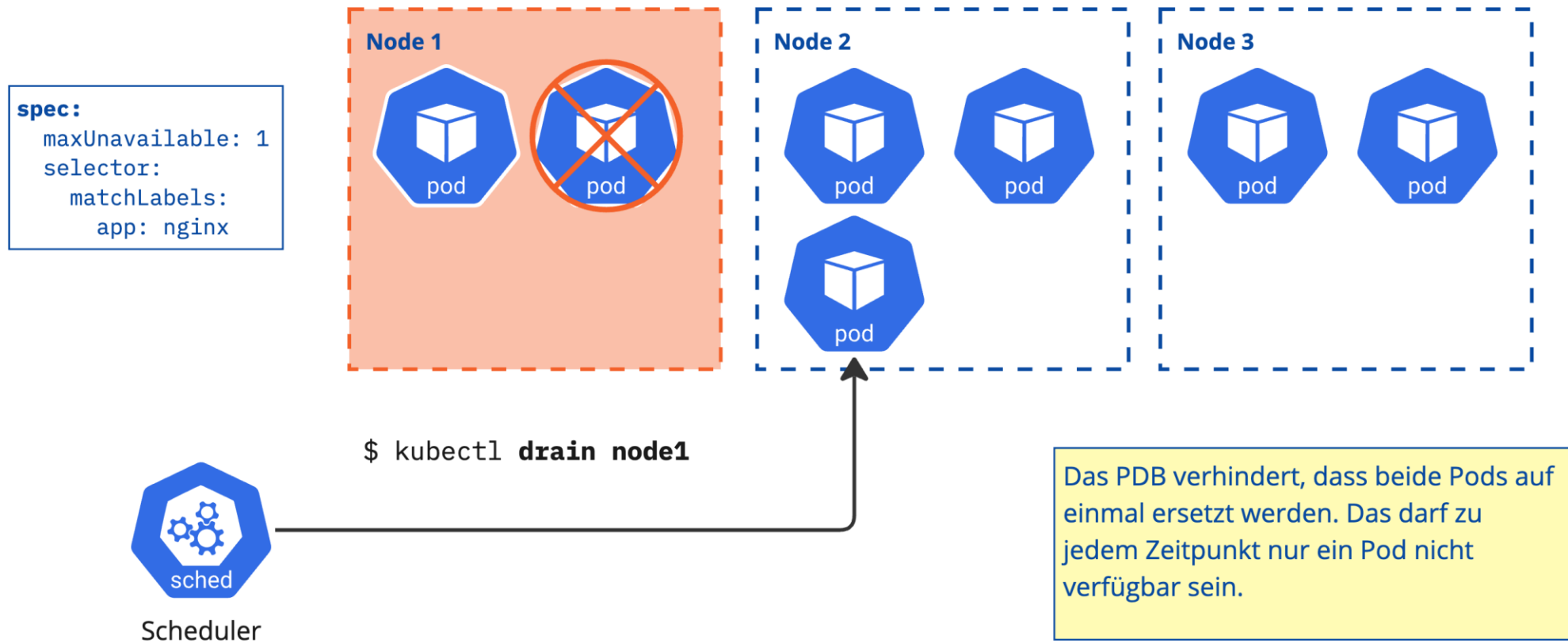
Scheduler



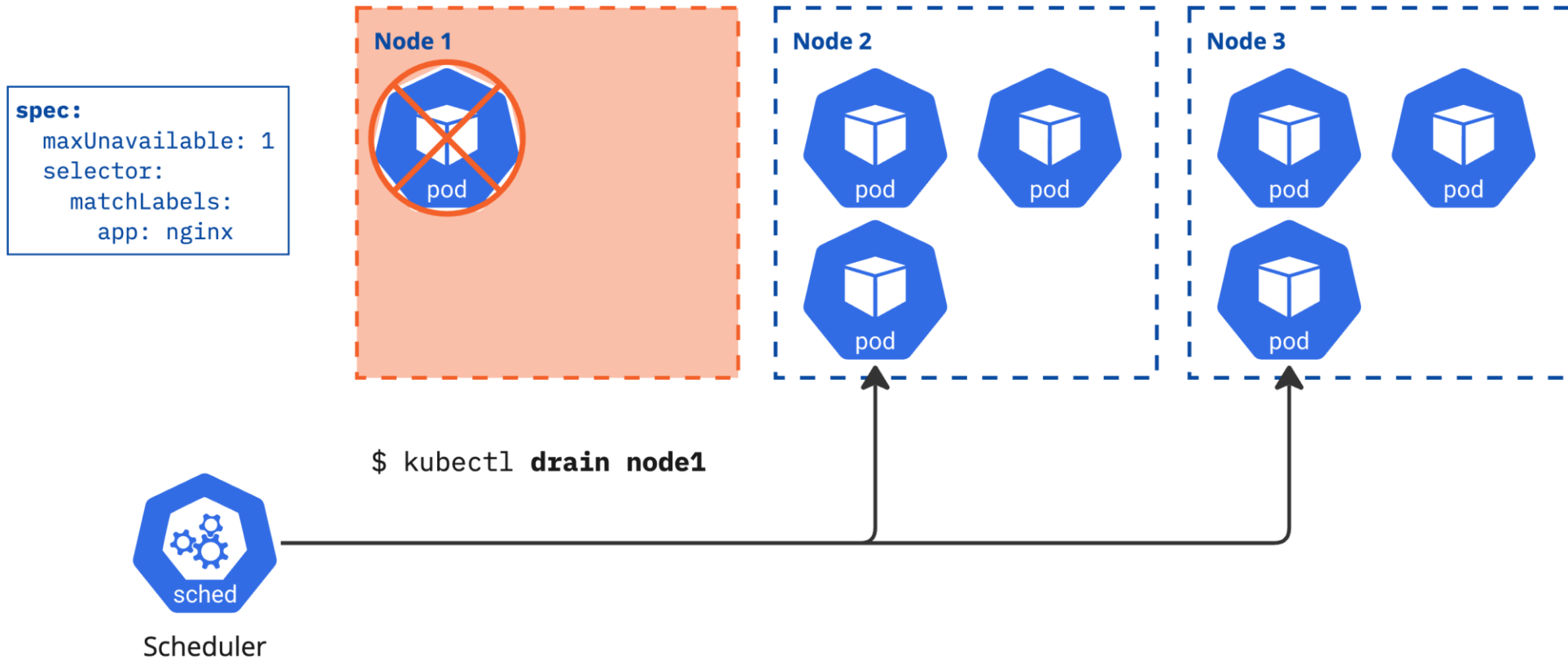
```
$ kubectl drain node1
```

Das Draining des Nodes startet. Alle Pods werden entfernt, da der Node außer Dienst genommen wird.

PDBs

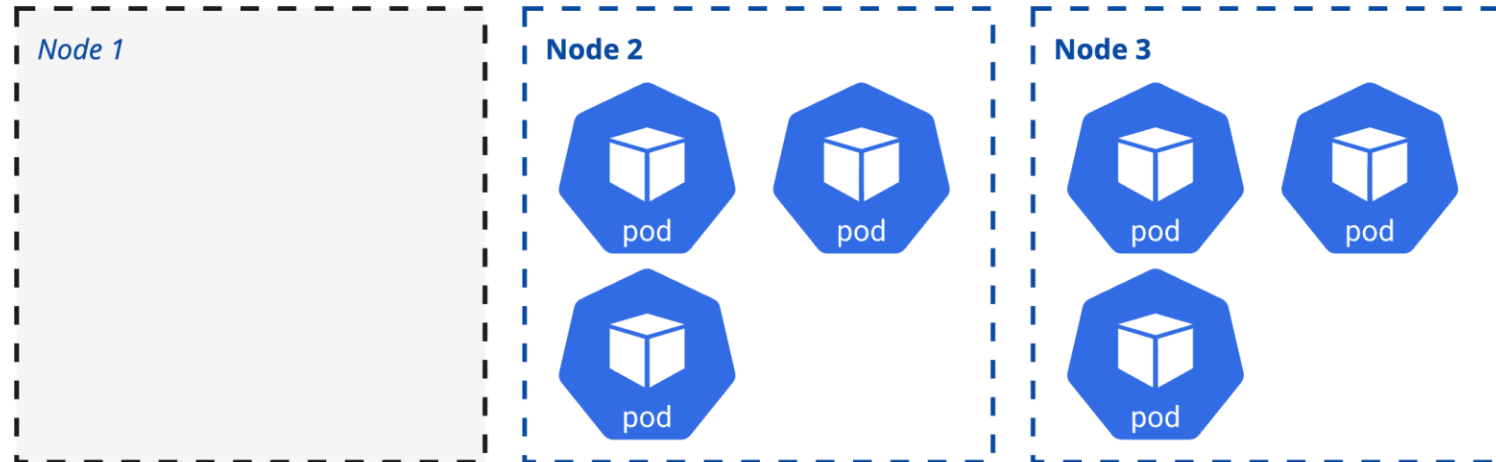


PDBs



PDBs

```
spec:  
  maxUnavailable: 1  
  selector:  
    matchLabels:  
      app: nginx
```



\$ **kubectl delete node node1**



Scheduler

Cluster-Automatisierung und Monitoring

Autoscaling mit Kubernetes

- **Scaling** ist wichtig in Produktion
- Klassisches Beispiel: **Black Friday**
- E-Commerce-Systeme müssen massenweise Requests aushalten



Scaling und Metriken



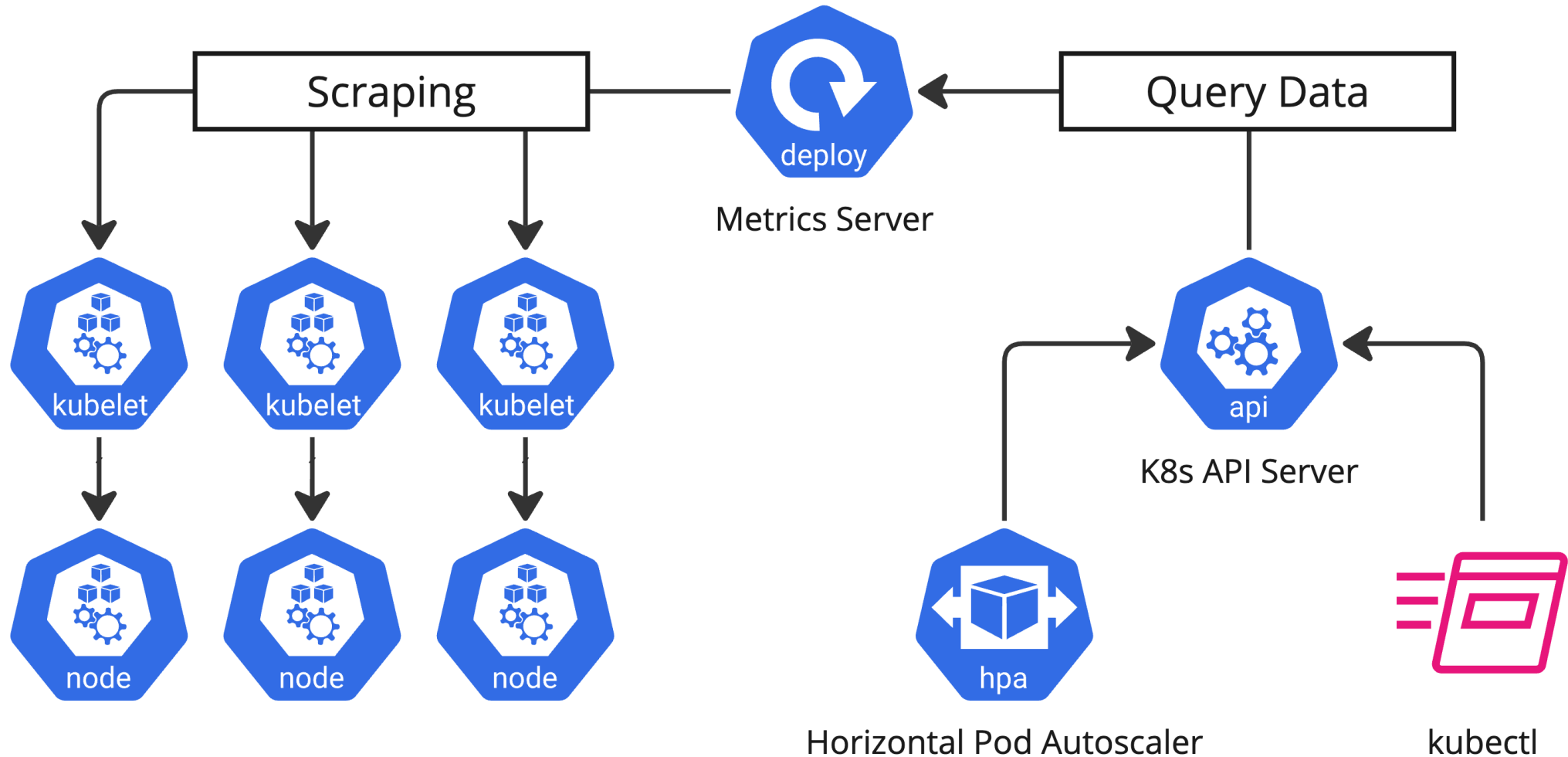
Metrics Server

- Um zu skalieren, brauchen wir **Metriken!**
- Kubernetes muss die CPU-Auslastung der Pods kennen
- Dafür brauchen wir den **Metrics-Server**
- Kubelet sammelt die Informationen, der Metrics-Server macht sie verfügbar

Metrics Server

- Der Metrics-Server ist eine eigene Komponente
- Er ist **nur** für das **interner** Monitoring gedacht
- Kubernetes benutzt diese Metriken für Autoscaling
- Er wird **nicht für Alerts oder Dashboards genutzt**

Metrics Server – Aufbau



Metrics-Server in Minikube aktivieren

```
$ kubectl get pods -n kube-system
```

NAME		READY	STATUS	RESTARTS	AGE
coredns-5dd5756b68-ntktw	1/1	Running	0	37s	
coredns-5dd5756b68-sdvk4	1/1	Running	0	37s	
etcd-minikube	1/1	Running	0	50s	
kube-apiserver-minikube	1/1	Running	0	50s	
kube-controller-manager	1/1	Running	0	50s	
kube-proxy-mvmfv	1/1	Running	0	37s	
kube-scheduler-minikube	1/1	Running	0	50s	
storage-provisioner		1/1	Running 0		49s

Metrics-Server bereitstellen

\$ minikube addons enable metrics-server

NAME		READY	STATUS	RESTARTS	AGE
coredns-5dd5756b68-ntktw	1/1	Running	0	37s	
coredns-5dd5756b68-sdvk4	1/1	Running	0	37s	
etcd-minikube	1/1	Running	0	50s	
kube-apiserver-minikube	1/1	Running	0	50s	
kube-controller-manager	1/1	Running	0	50s	
kube-proxy-mvmfv	1/1	Running	0	37s	
kube-scheduler-minikube	1/1	Running	0	50s	
metrics-server-75d6c48ddd	1/1	Running	0	106s	
storage-provisioner		1/1	Running 0		49s

Metrics-Server bereitstellen

```
$ kubectl get deploy -n kube-system
```

NAME			READY	UP-TO-DATE	AVAILABLE	AGE
coredns	2/2	2	2		8m39s	
metrics-server	1/1	1	1		6m17s	

Der Metrics-Server wird als Deployment im Cluster installiert.

Das Projekt findet sich hier:

<https://github.com/kubernetes-sigs/metrics-server>

Metriken abfragen

```
$ kubectl get --raw "/api/v1/nodes/minikube/proxy/stats/summary"
```

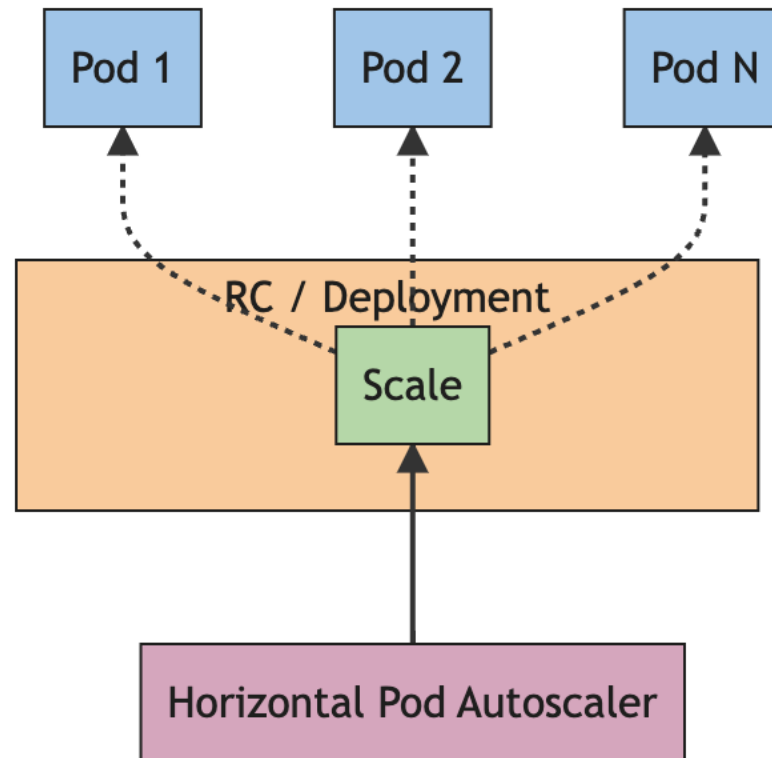
```
{  
  "node": {  
    "nodeName": "minikube",  
    "systemContainers": [  
      {  
        "name": "pods",  
        "startTime": "2024-08-12T09:12:14Z",  
        "cpu": {  
          "time": "2024-08-12T09:24:11Z",  
          "usageNanoCores": 123322656,  
          "usageCoreNanoSeconds": 88769759000  
        }, ...  
      }  
    ]  
  }  
}
```

Diese Metriken können von anderen k8s-Komponenten genutzt werden. Insbesondere der **HorizontalPodAutoscaler** hängt von ihnen ab.

HorizontalPodAutoscaler

- Ressource, die die **Kontrolle** über den **Replica-Count** übernimmt
- Liest den Metrics-Server aus
- Bestimmt die **aktuelle Auslastung** aus Metriken
- Gleicht es gegen Wunsch-Auslastung ab
- Stellt Replika-Count ggf. neu ein

HorizontalPodAutoscaler



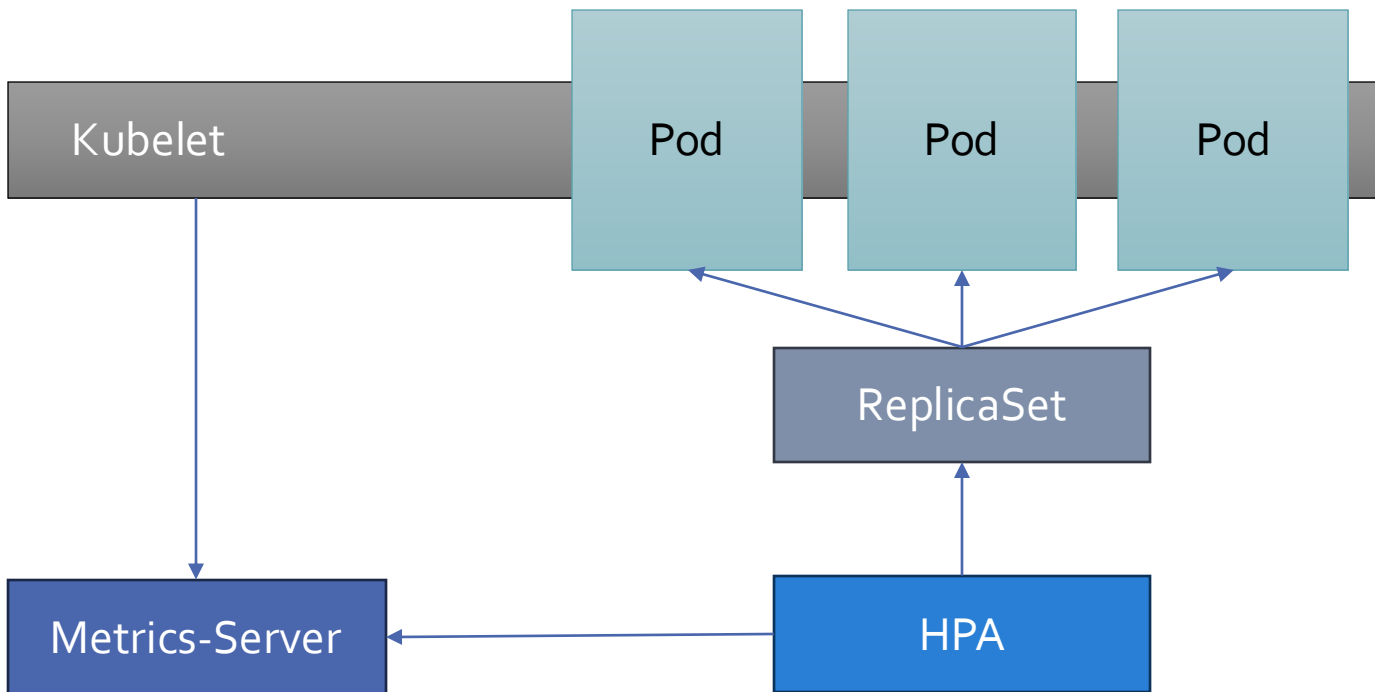
<https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>

HorizontalPodAutoscaler

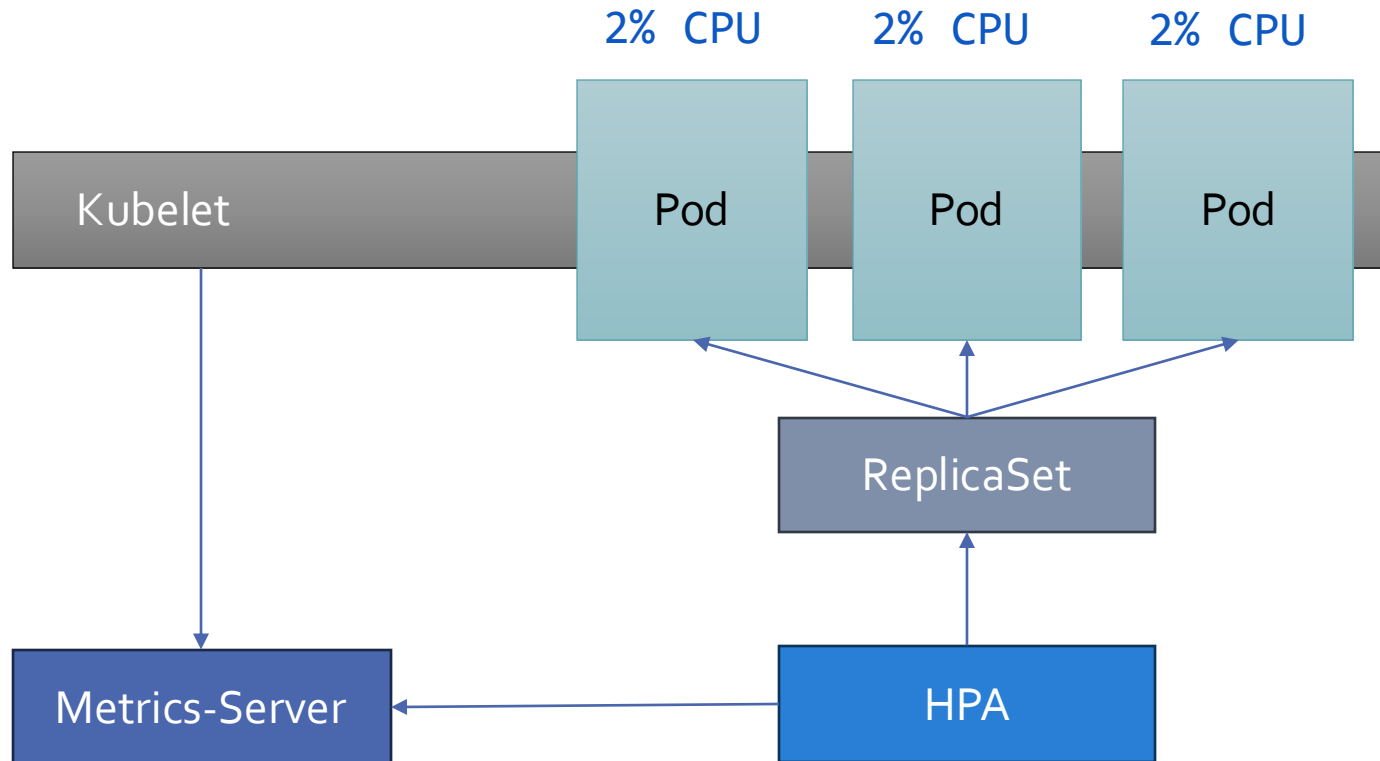
```
apiVersion: autoscaling/v1
kind: HorizontalPodAutoscaler
metadata:
  name: nginx-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: ReplicaSet
    name: nginx-replicaset
  minReplicas: 3
  maxReplicas: 10
  targetCPUUtilizationPercentage: 50
```

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-replicaset
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.21
          resources:
            requests:
              cpu: "100m"
          ports:
            - containerPort: 80
```

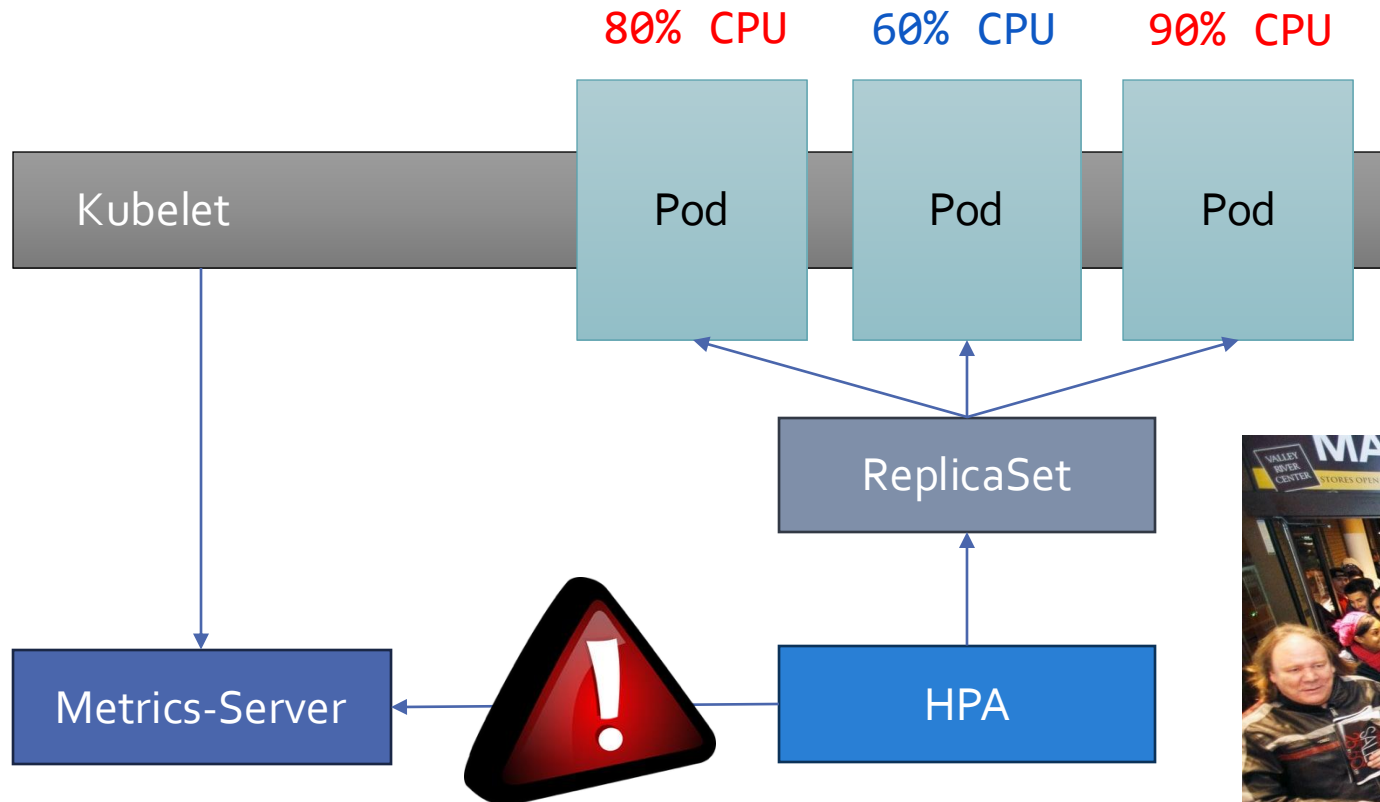
Wie funktioniert der HPA?



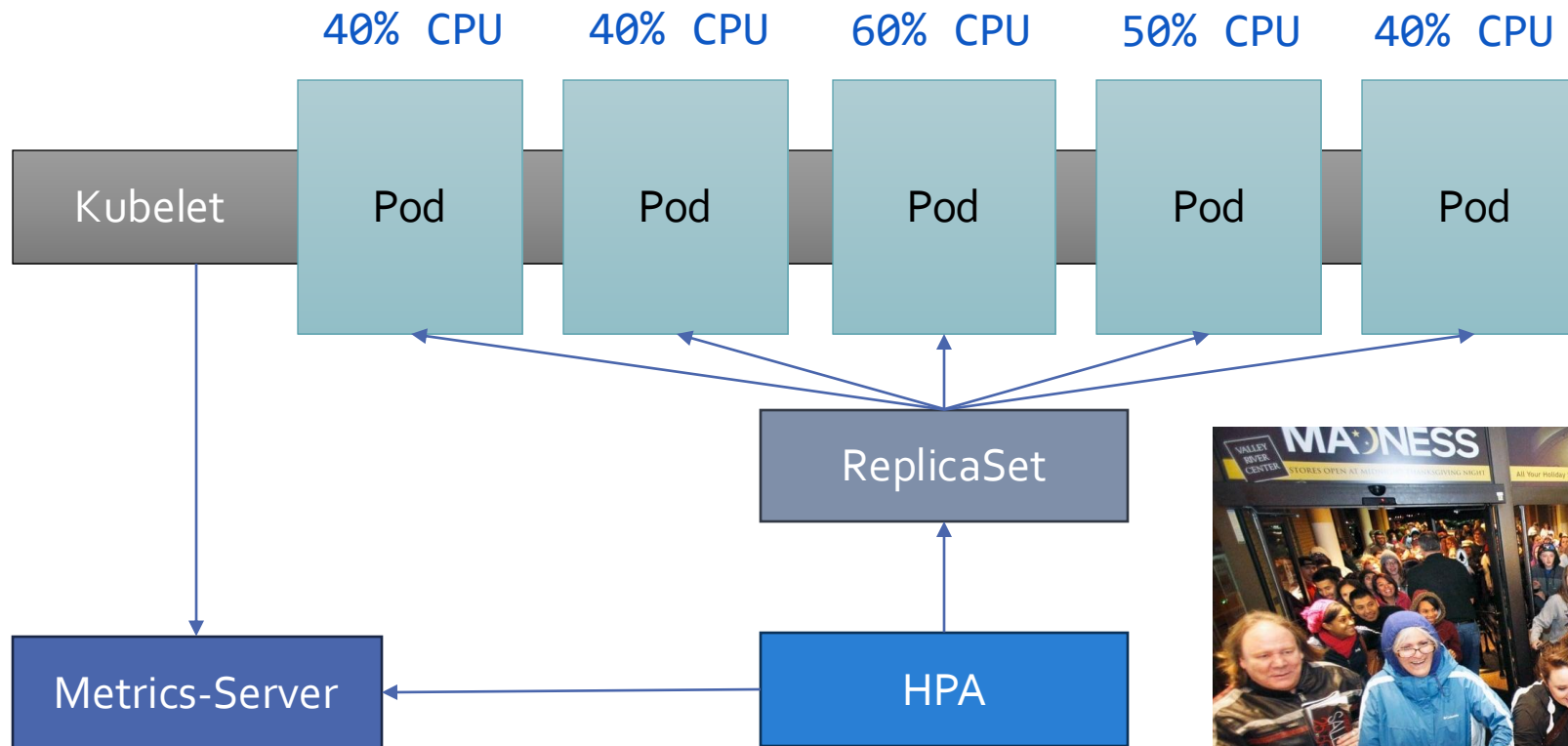
Wie funktioniert der HPA?



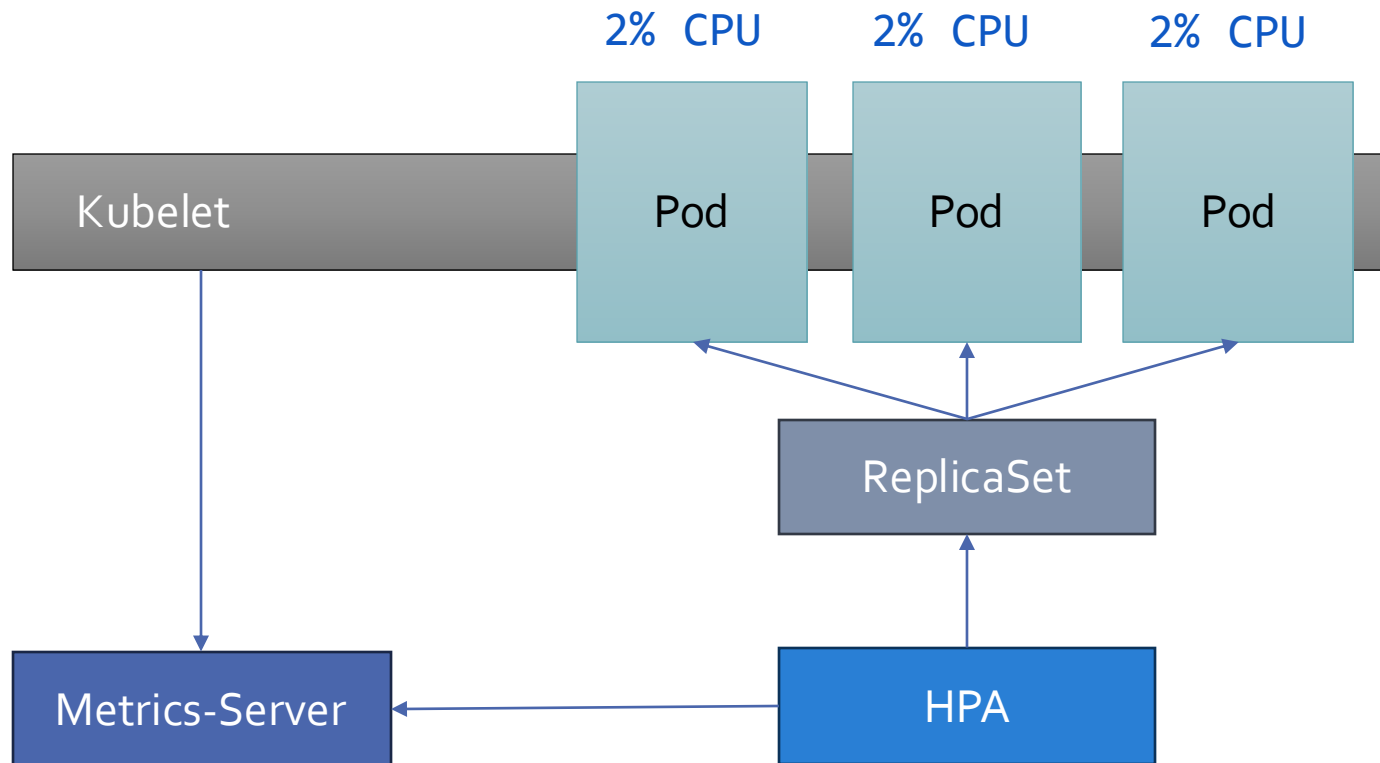
Wie funktioniert der HPA?



Wie funktioniert der HPA?



Wie funktioniert der HPA?



Praxisaufgabe

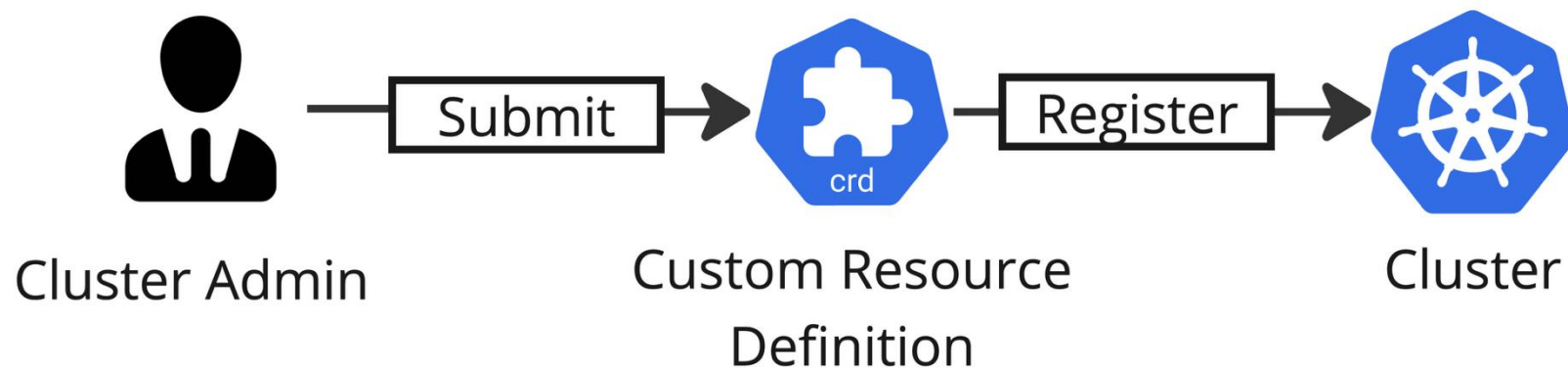
Autoscaling

Metriken für externe Zwecke

- Der Metrics-Server ist ausdrücklich für interne Zwecke in k8s vorgesehen
- Für Alert-Regeln und externe Zwecke wie Dashboards etc. sollte man ihn nicht nutzen
- Dafür gibt es sogenannte Operatoren, die Metriken exportieren

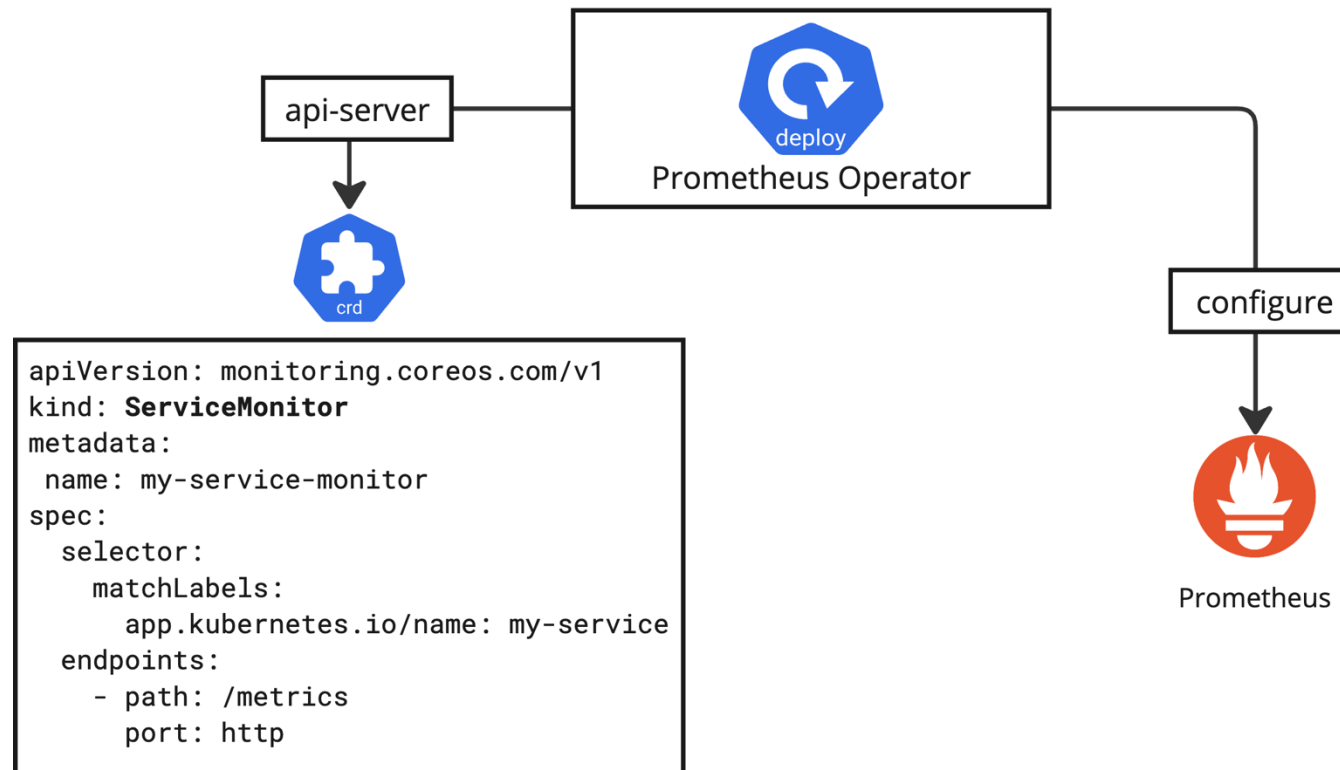
Exkurs: Custom Resources in Kubernetes

- Kubernetes ist erweiterbar um eigene Ressourcentypen
- Diese werden *Custom Resources* genannt
- Sie werden im Cluster via *Custom Resource Definitions* registriert und sind dann über den api-server abfragbar, z.B. mit `kubectl`



Operatoren in Kubernetes

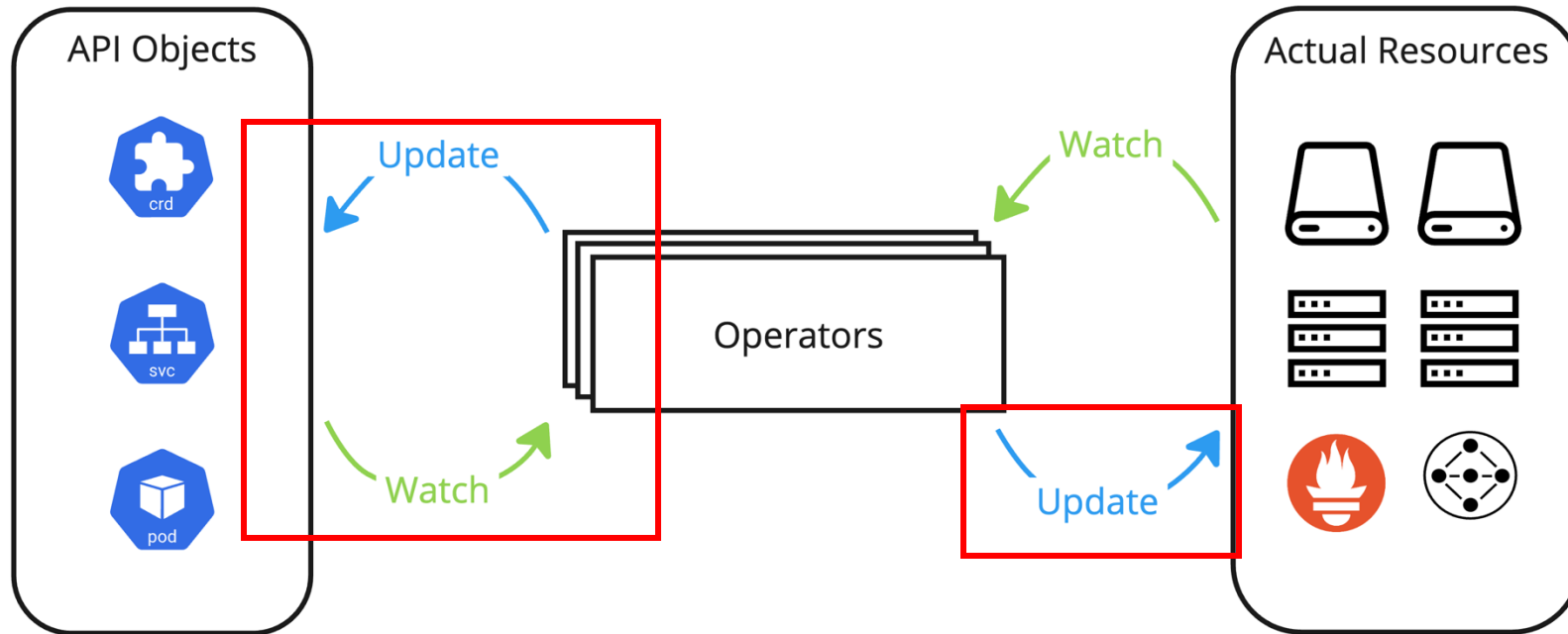
- Operatoren sind Plugins, die „ihre“ Custom Resources überwachen
- Sie treten in Aktion, wenn eine neue CR erscheint oder sich ändert



Operatoren in Kubernetes

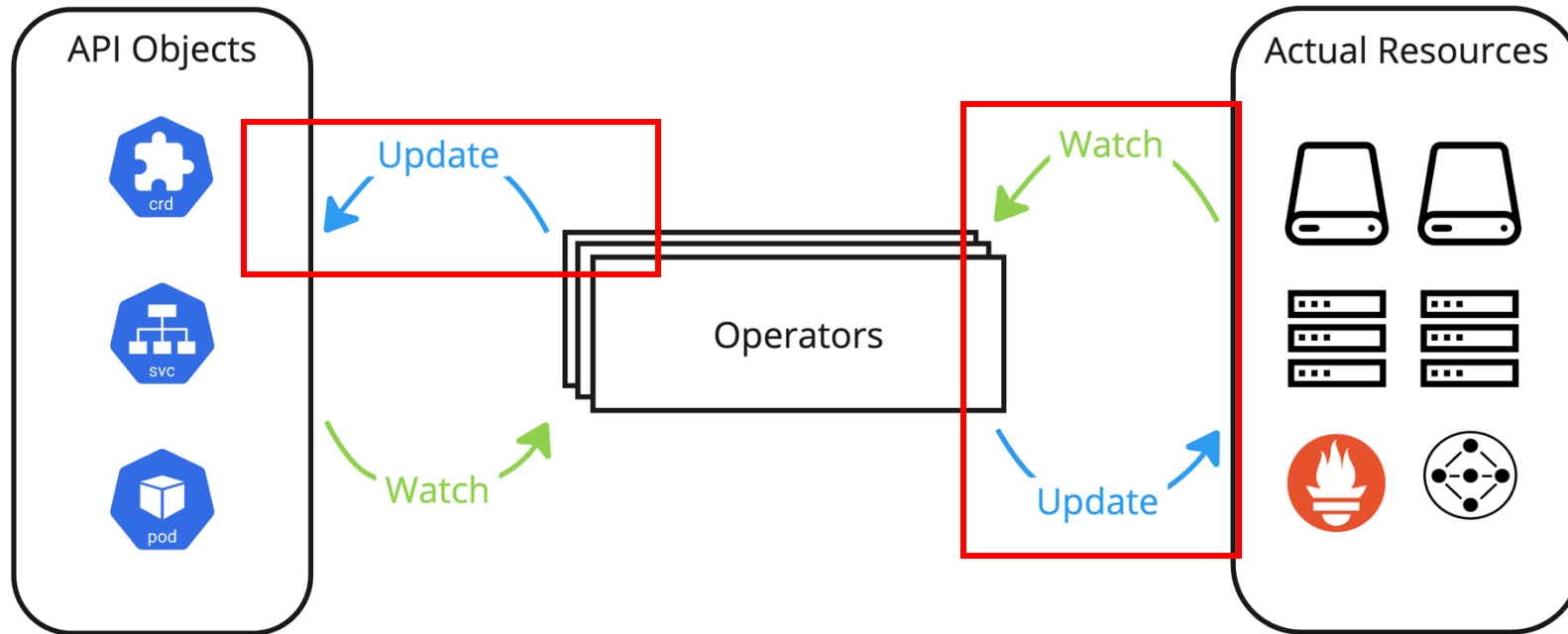
- Sobald eine CRD im Cluster registriert ist, kann sie über die API abgefragt werden
- Das geht über kubectl genauso wie intern
- Ein Operator fragt also den Zustand über den api-server ab
- Er führt dann ein Konzept durch, das sich *Reconciliation* nennt.

Reconciliation



Operatoren überwachen API-Objekte. Wenn sie sich ändern, passen sie das System an und aktualisieren die Objekte.

Reconciliation

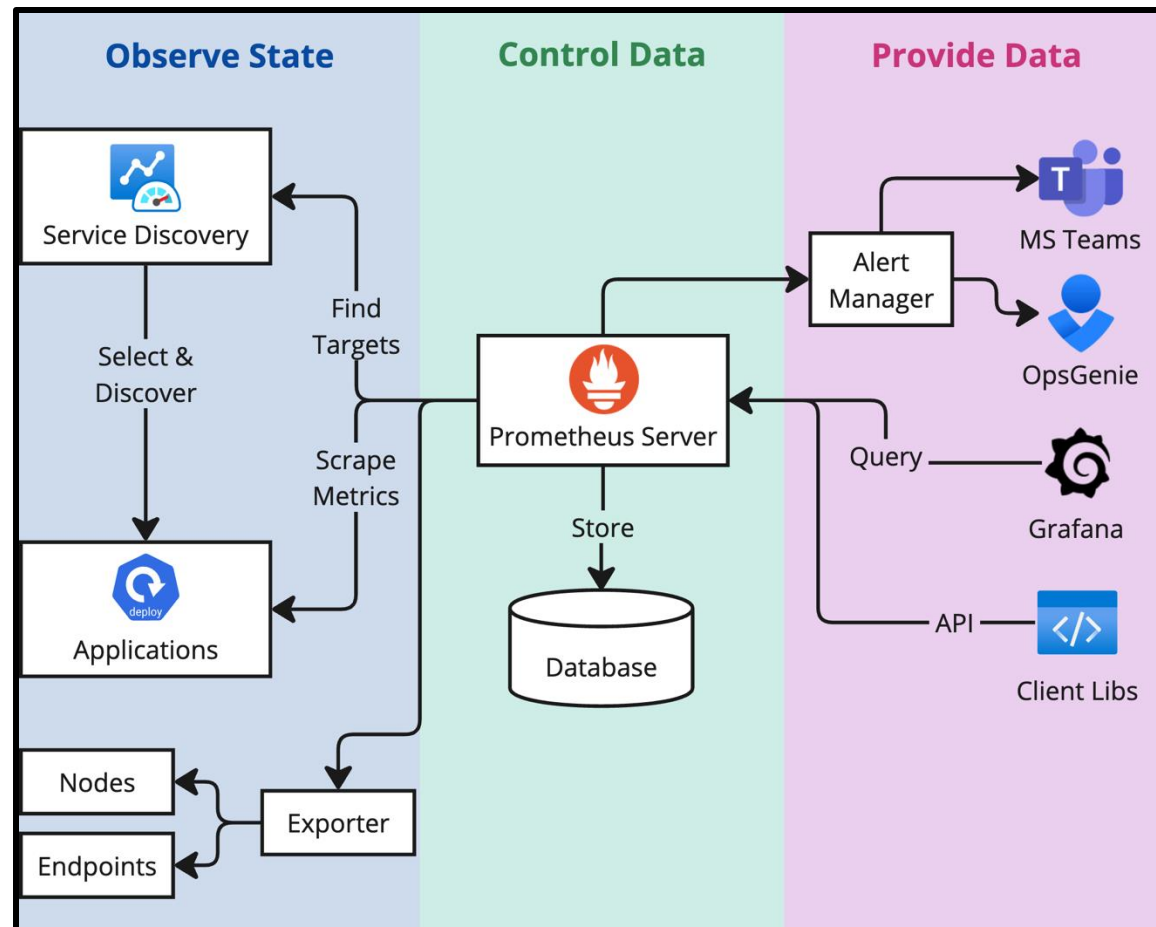


Operatoren überwachen aber auch das System und korrigieren es bei Bedarf, z.B. wenn eine Komponente ausfällt.

Metriken mit Prometheus

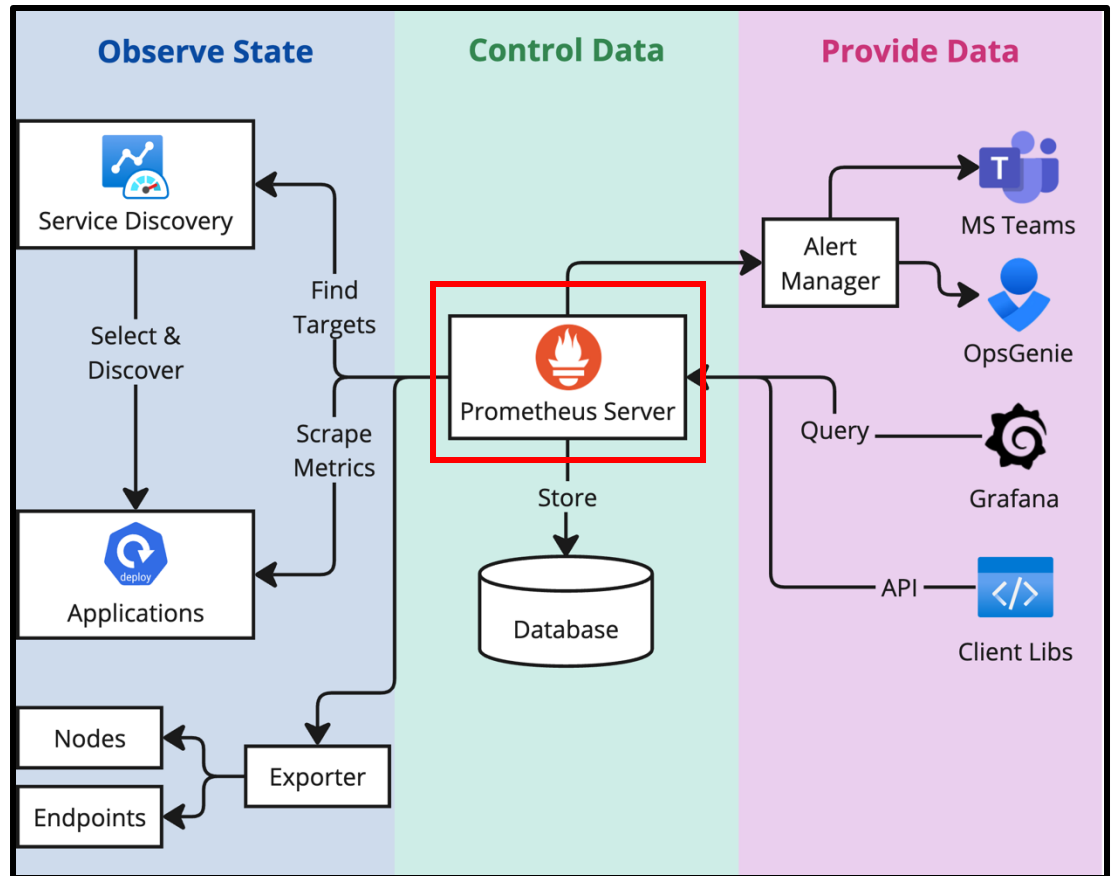
- Für externe Abfrage von Metriken wird oft Prometheus eingesetzt
- Prometheus ist ein Paket aus mehreren Komponenten
 - Einem Server mit Datenbank und Scraper
 - Client-Bibliotheken
 - Push-Gateway
 - Exporter
 - Alertmanager
- Prometheus ist Scraping-basiert. Es holt sich die Metriken von Exportern per HTTP regelmäßig ab.

Komponenten von Prometheus



Prometheus Server

- Der Server enthält die Hauptkomponenten von Prometheus
- Er wird vom Operator installiert und verwaltet
- Realisiert als StatefulSet



Prometheus Frontend

Prometheus Alerts Graph Status ▾ Help

☐ Enable query history

[Try experimental React UI](#)

up

Load time: 23ms
Resolution: 14s
Total time series: 3

Execute

- insert metric at cursor - ▾

Graph

Console



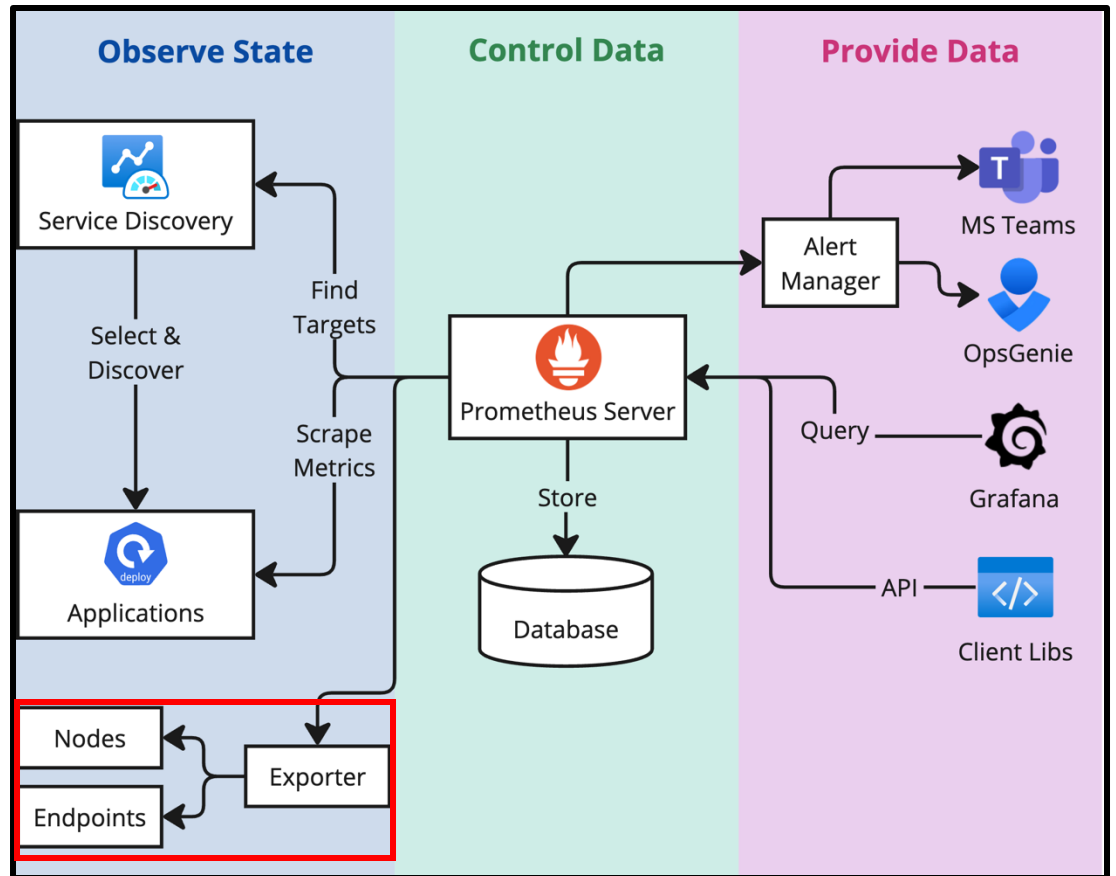
Moment



Element	Value
up{instance="localhost:9090",job="prometheus"}	1
up{instance="localhost:9091",job="prom_middleware"}	0
up{instance="localhost:9100",job="node_exporter"}	0

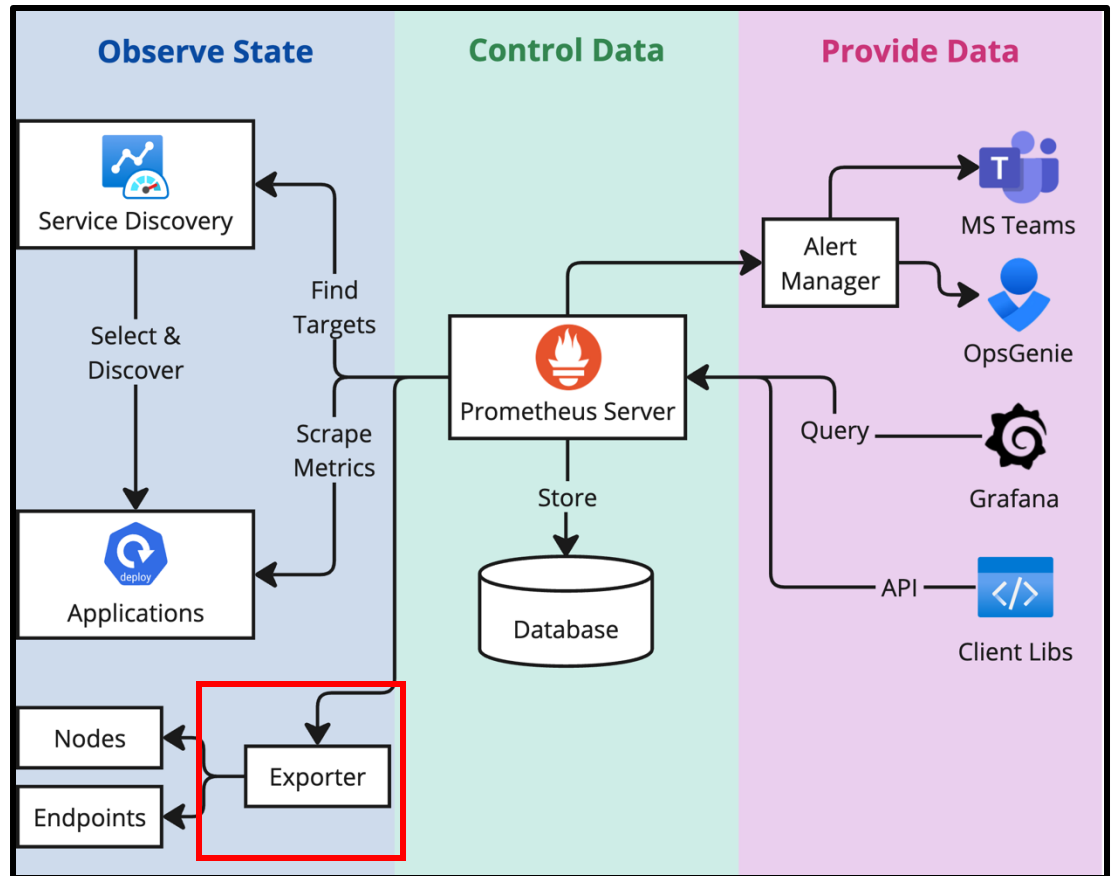
Node Exporter

- Sammelt Daten über alle Cluster Nodes
 - CPU-Auslastung
 - RAM-Verbrauch
 - Netzwerk-Traffic
- Realisiert als DaemonSet für einen Pod pro Node
- Stellt eine REST-API bereit für den Prometheus Server
- Der Scraper liest dann die API aus



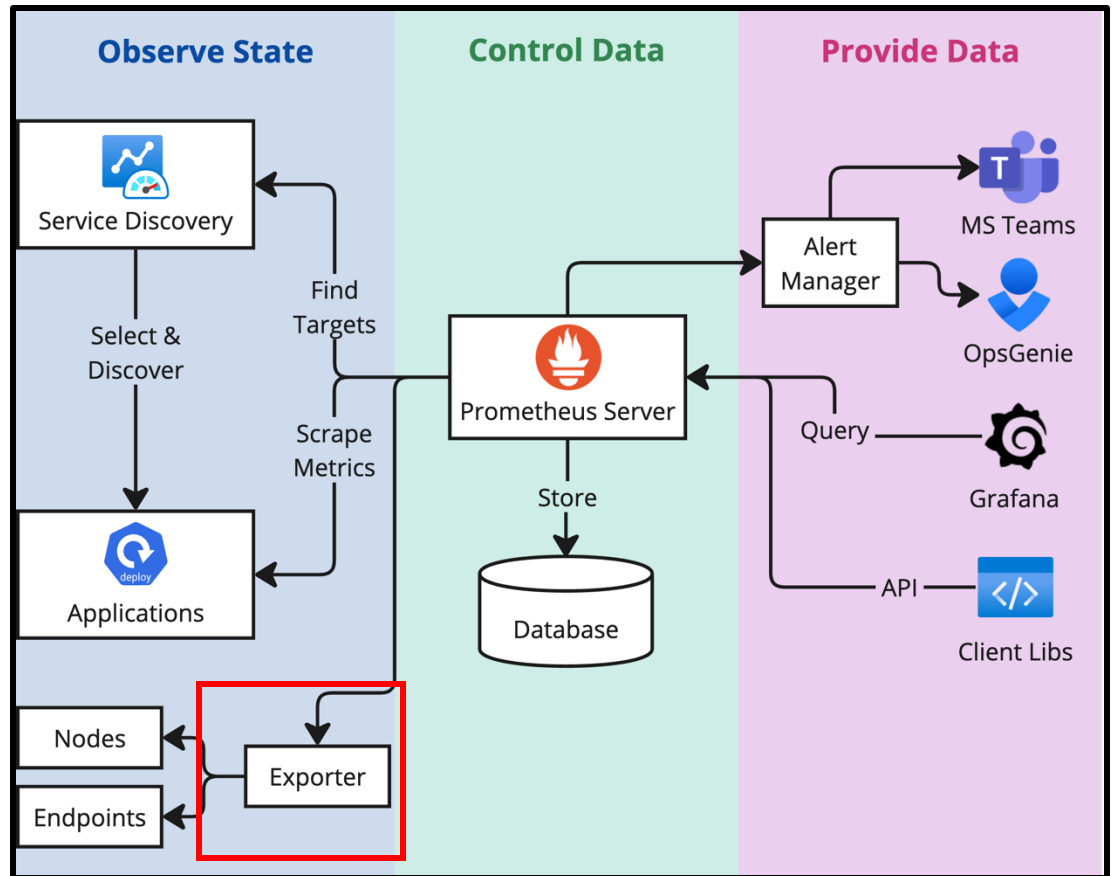
Kube-State-Metrics

- kube-state-metrics (KSM) liest Zustand von k8s-Objekten über den API-Server aus
- Wird installiert als Deployment
- KSM kann ebenfalls zu den Exportern gezählt werden
- Sie bieten eine HTTP-Schnittstelle für den Prometheus-Scraper



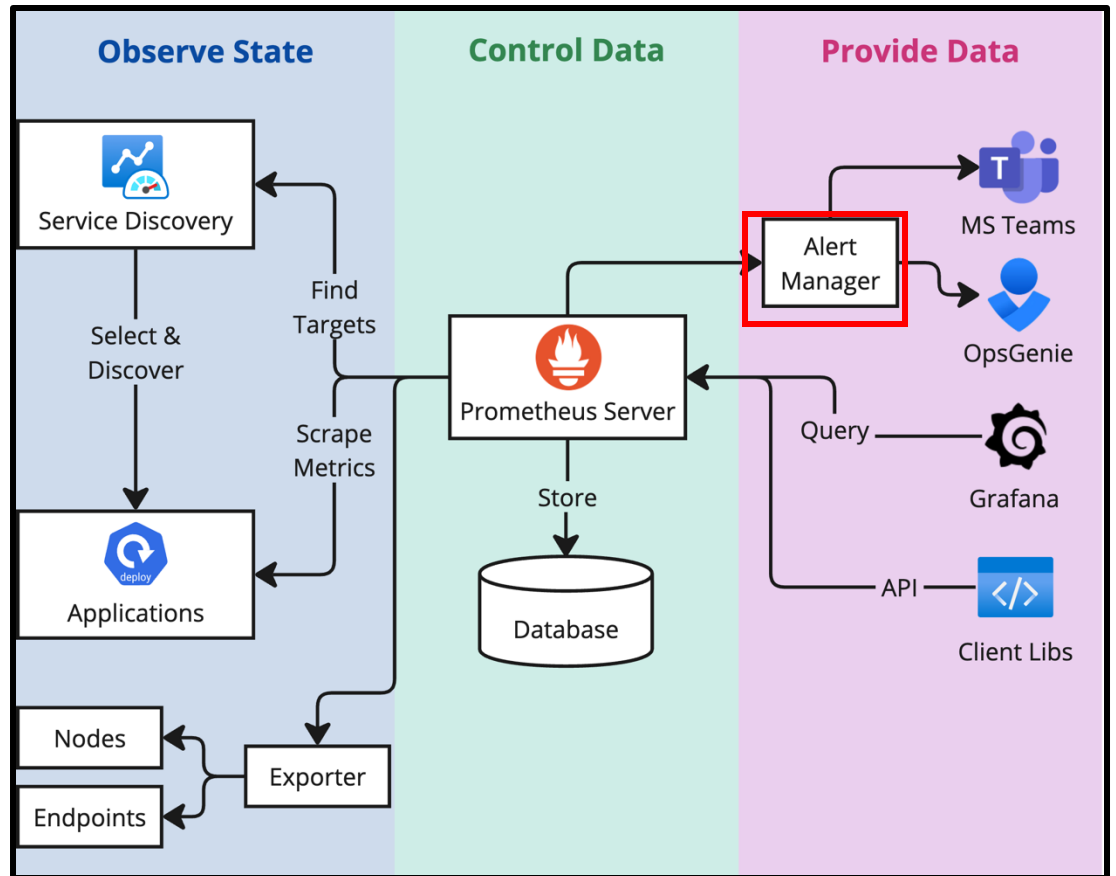
Blackbox Exporter

- Der Blackbox Exporter bietet Überwachung von beliebigen Endpunkten
- Überwacht HTTP(S), DNS, TCP, ICMP oder gRPC-Endpunkte
- Ergebnis der Proben stehen via HTTP für den Scraper von Prometheus zur Verfügung

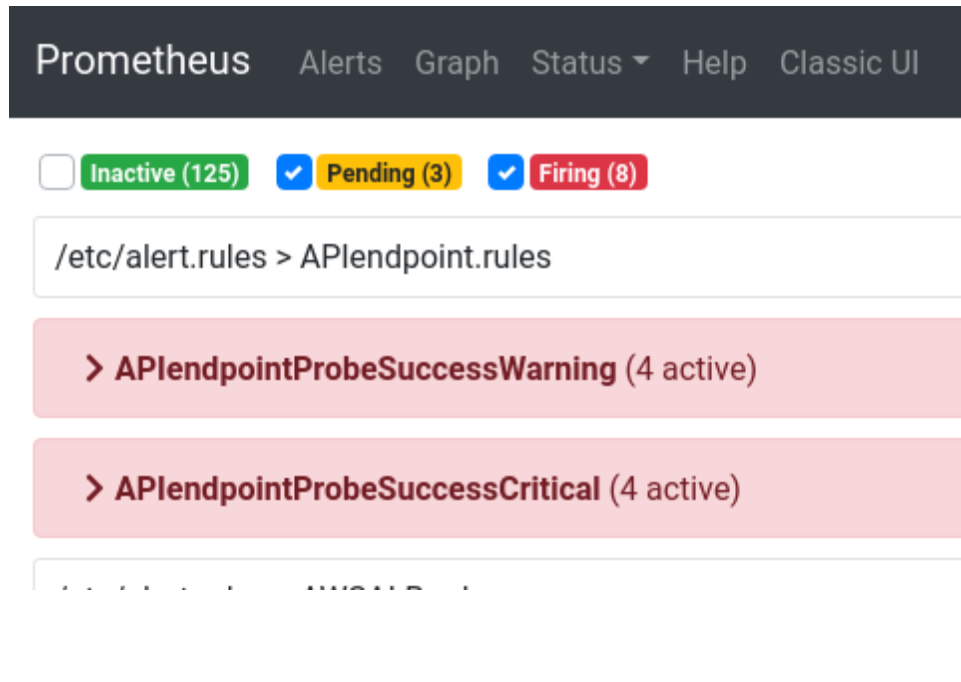


Alertmanager

- Ermittelt Fehlerzustände aus den Metriken
- Informiert externe Dienste wie z.B. OpsGenie
- Als StatefulSet realisiert

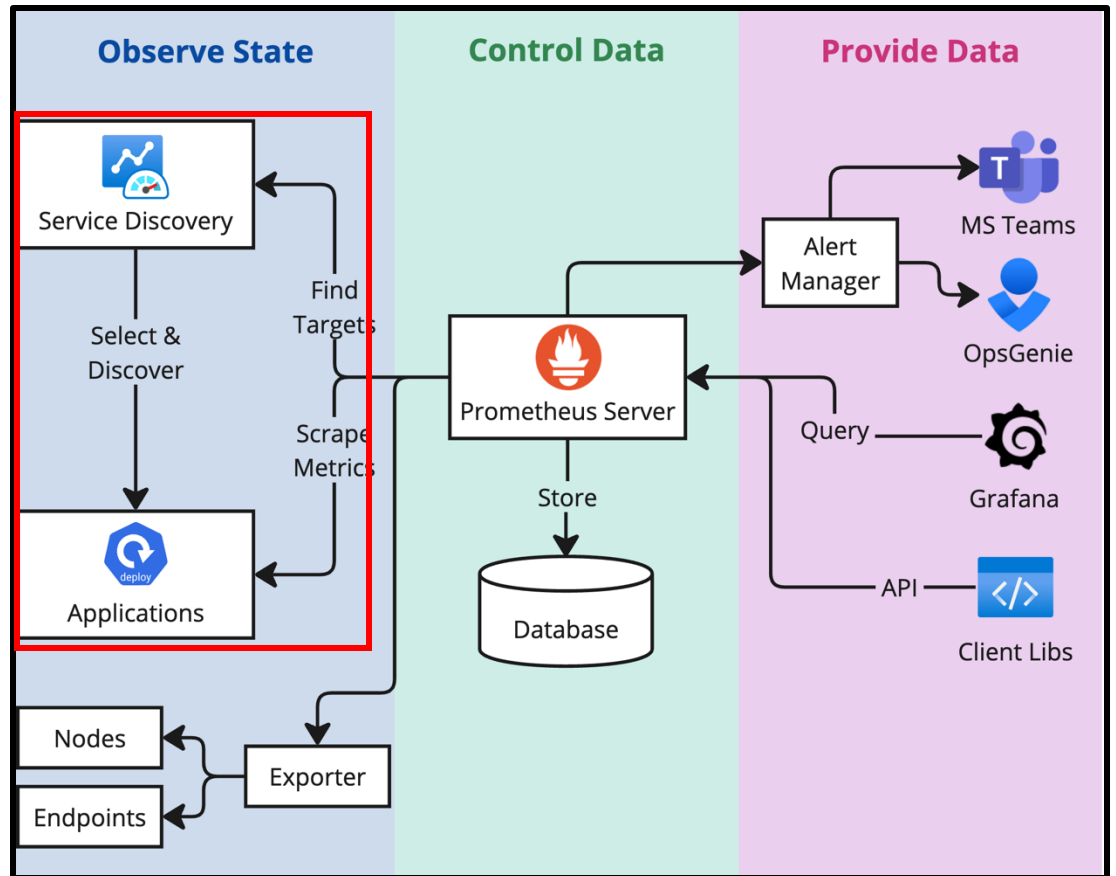


Alertmanager Frontend

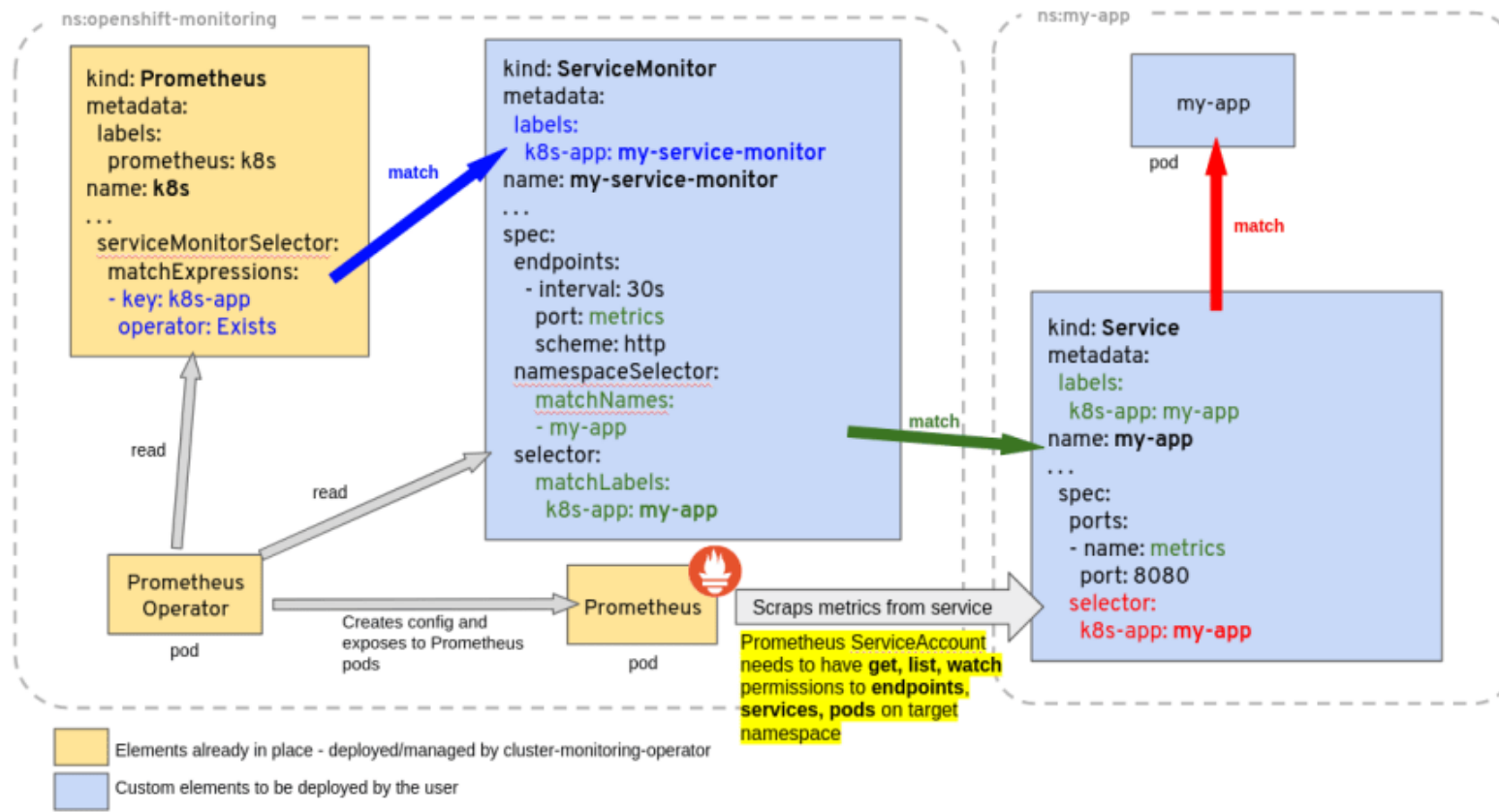


Service- und Pod-Monitore

- Prometheus kann auch unsere eigenen Anwendungen überwachen
- Dafür müssen die Anwendungen die Metriken via HTTP für den Scraper bereitstellen
- Wir konfigurieren den Metrics-Endpoint mit einem sogenannten **ServiceMonitor**

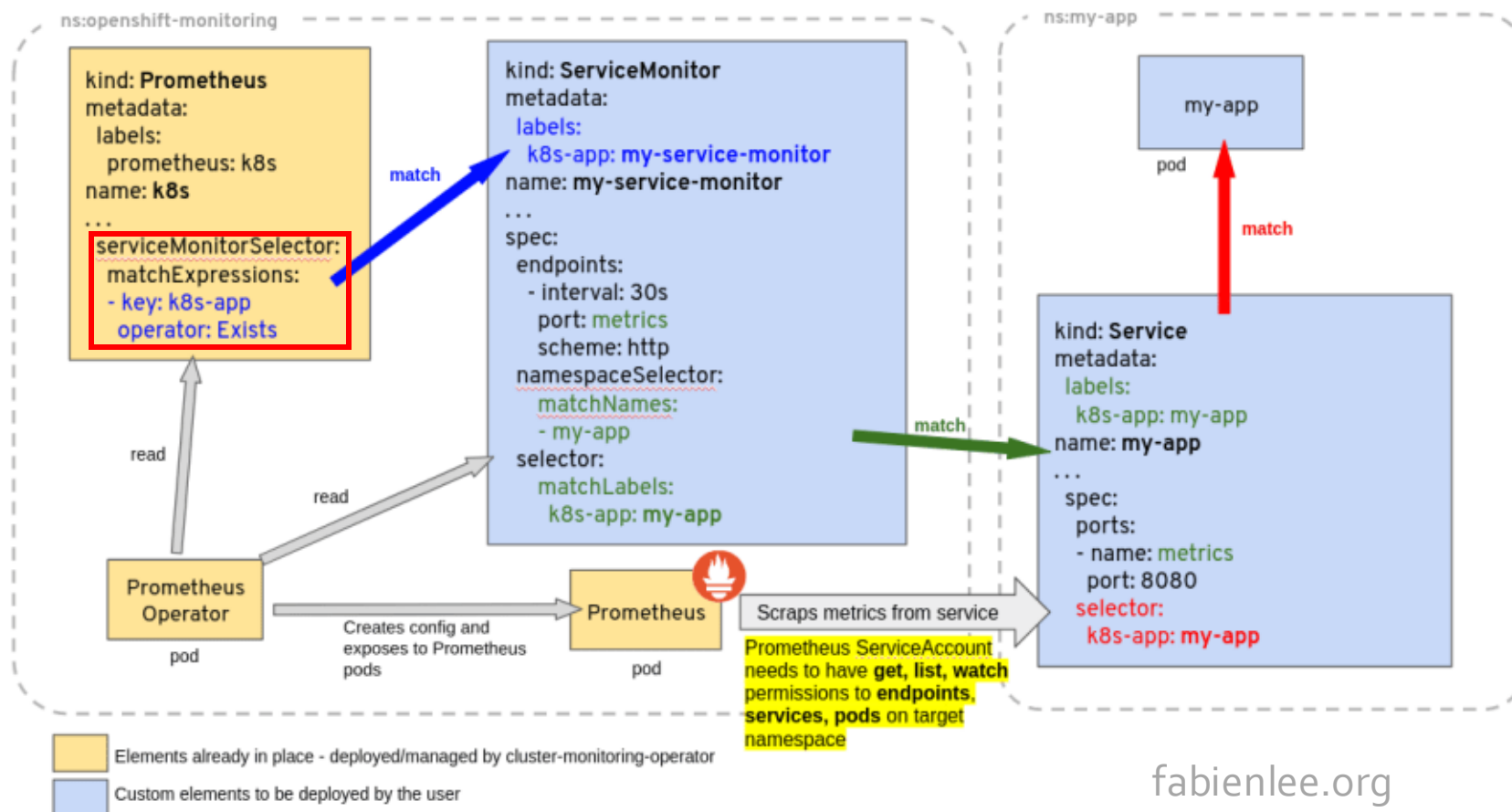


Service Monitor Overview



Service Monitors

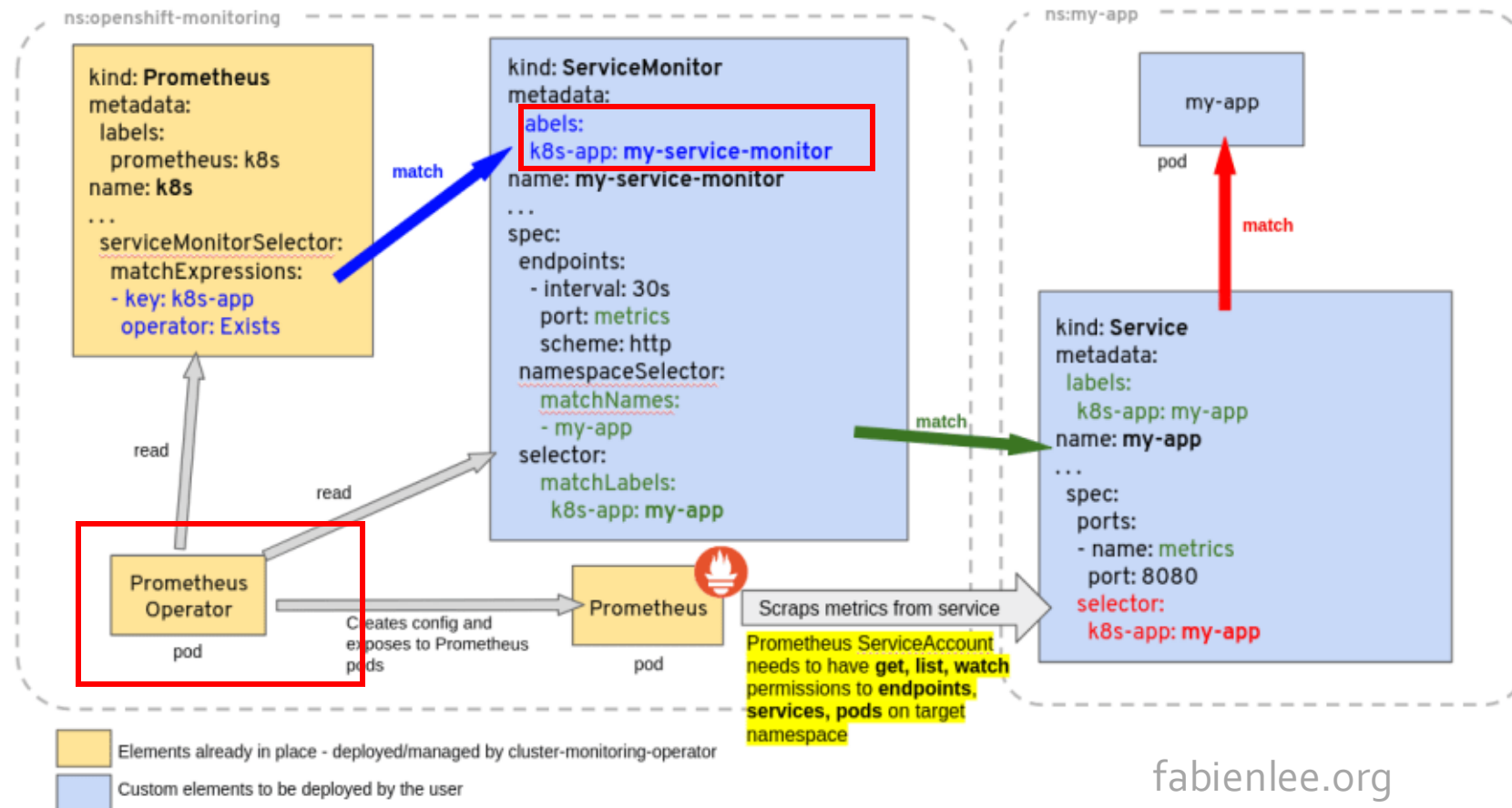
`serviceMonitorSelector` zeigt an, welches Label die Monitors tragen, die diese Prometheus-Instanz berücksichtigen soll.



fabienlee.org

Service Monitors

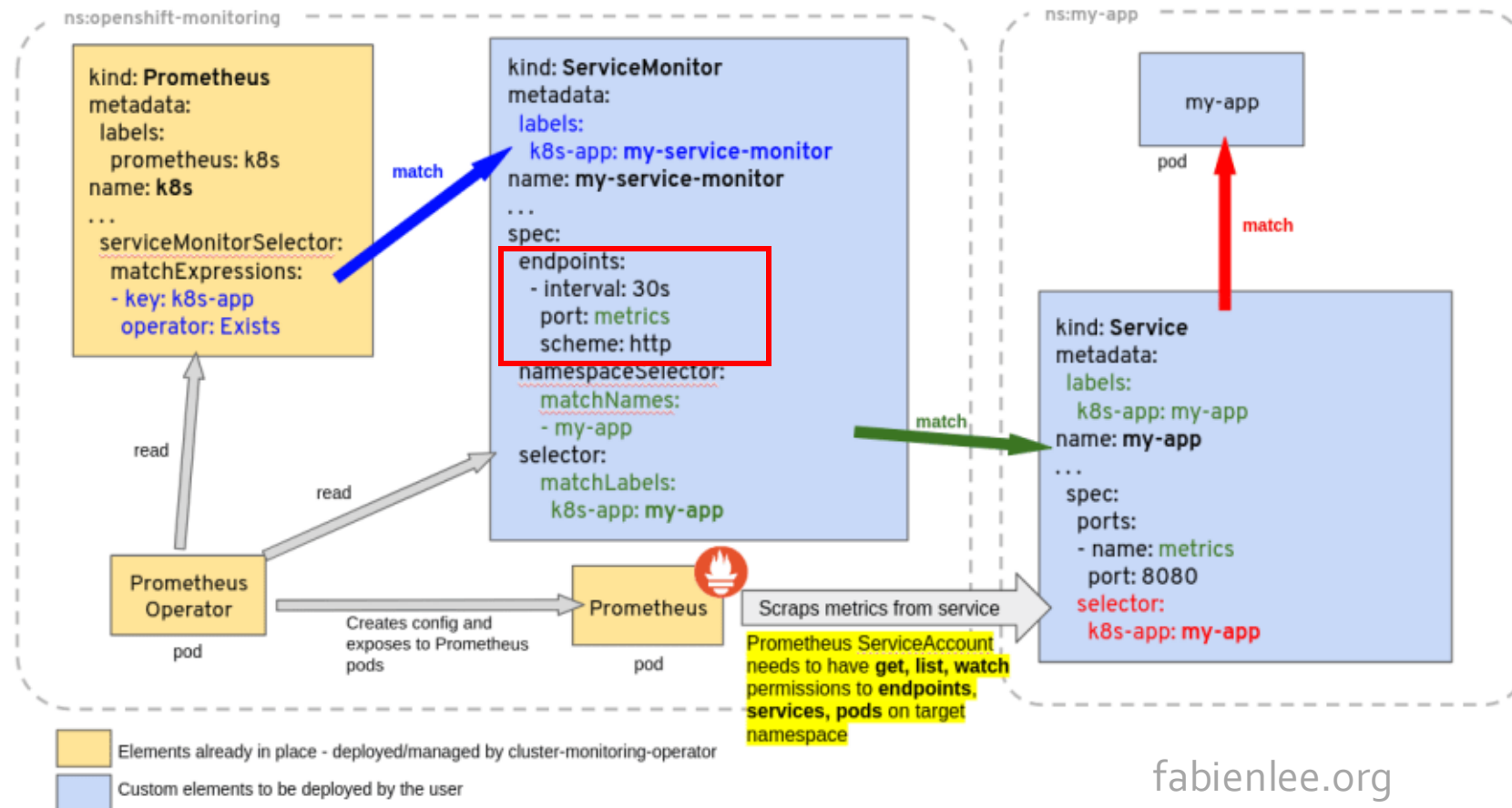
Der Prometheus Operator
liest die Spezifikation aus und
sucht nach ServiceMonitors,
die passende Labels haben.



fabienlee.org

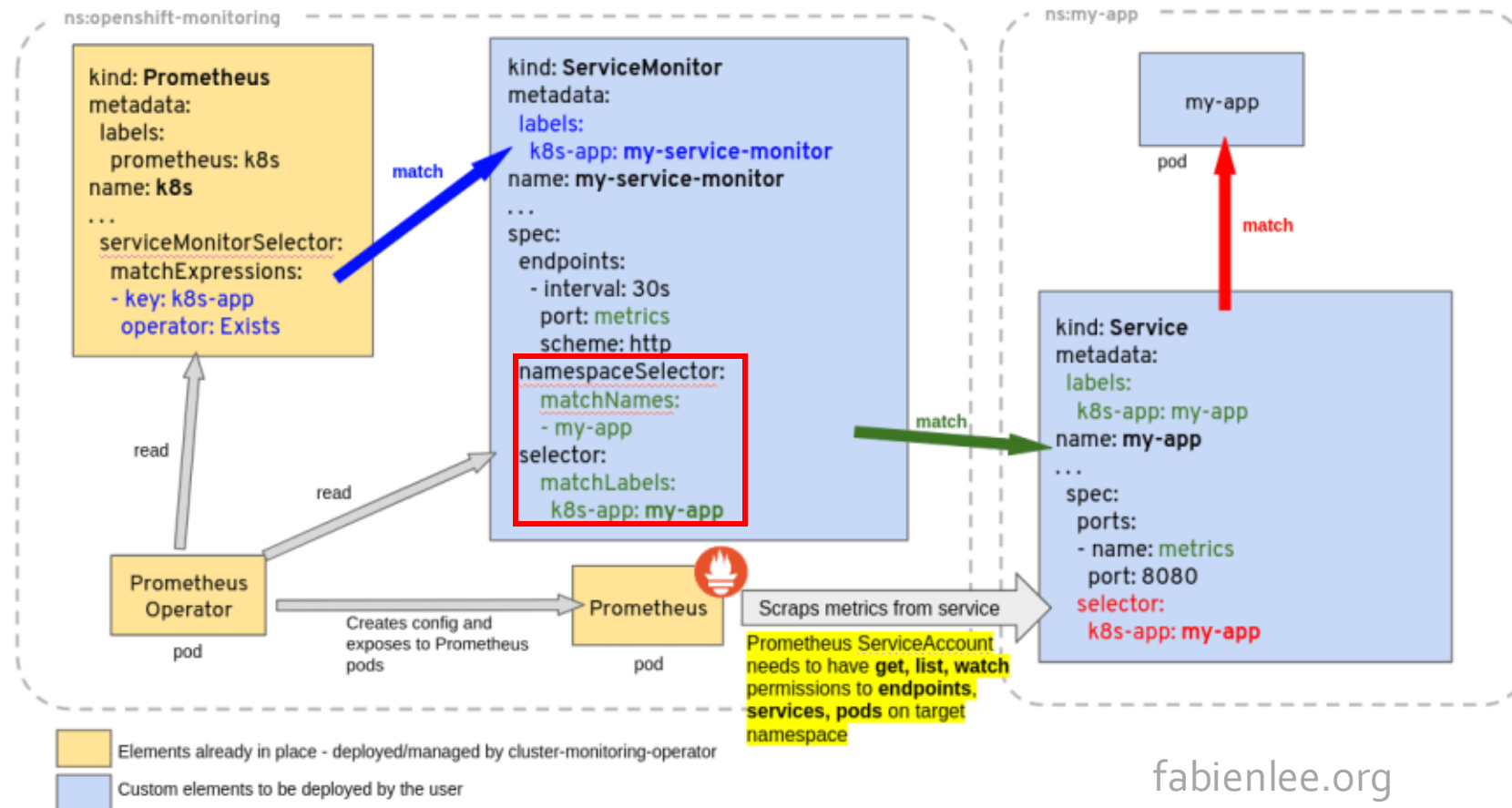
Service Monitors

Der ServiceMonitor zeigt mit dem **endpoints**-Feld an, wie Prometheus über HTTP die Metriken abholen kann.



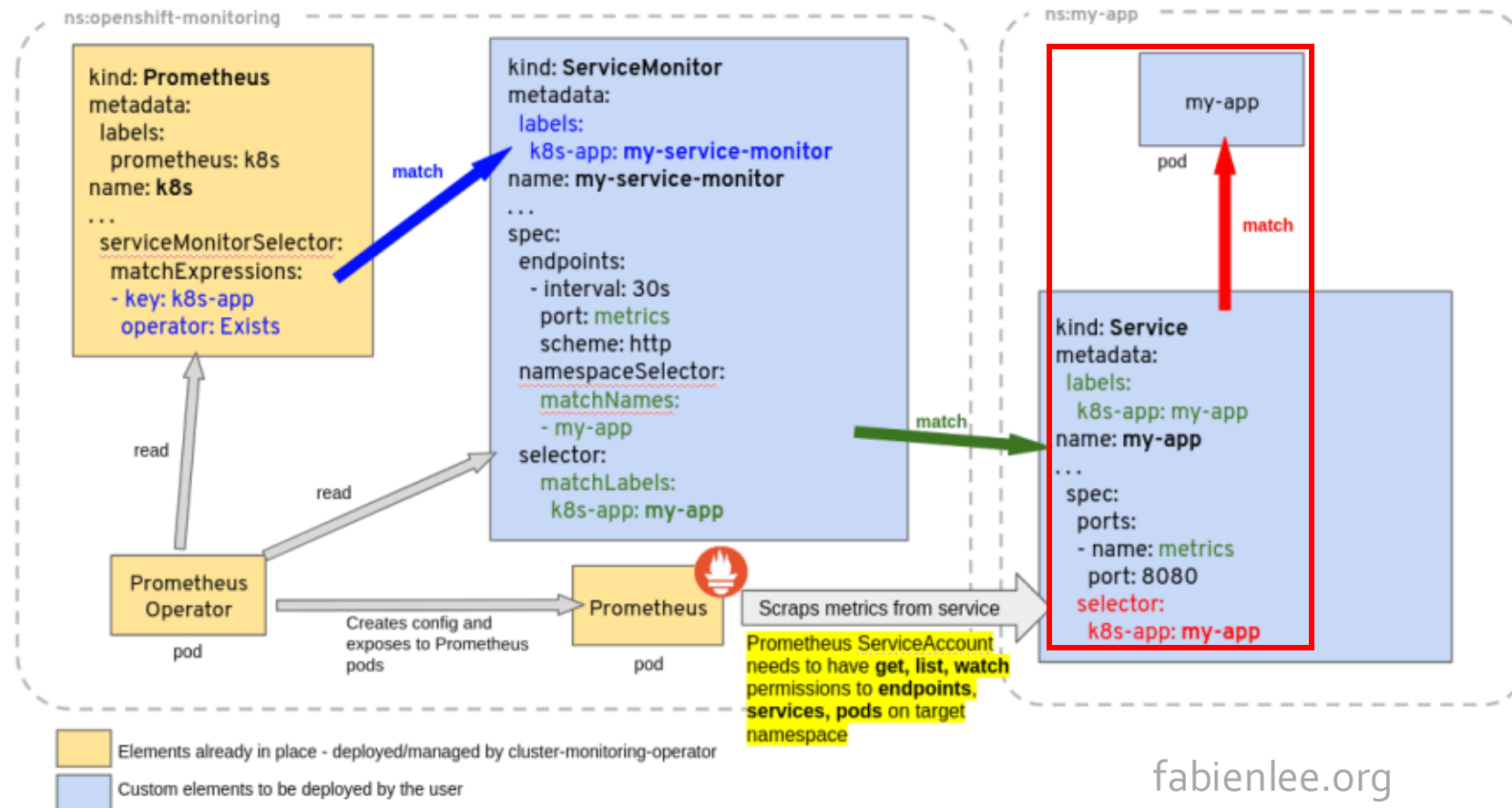
Service Monitors

Die passenden Services, die die Endpunkte bereitstellen, werden per Namespace und Label selektiert.



Service Monitors

Der Service wiederum muss per Labels den richtigen Pods zugeordnet sein. Die Pods müssen eine URL zur Verfügung stellen, über die Prometheus die Metriken scrapen kann.



fabienlee.org

Projekt

Prometheus auf Kubernetes

Logging-Stacks für Kubernetes

- Logs sind für die tägliche mit einem Cluster unerlässlich
- In der Regel werden Logging-Systeme nicht nur für einen Cluster eingerichtet
- Es gibt oft dedizierte Logging-Cluster für die Logs eines ganzen Bereichs oder Unternehmens

```
01
03/22 08:51:01 INFO  :.main: ***** RSVP Agent started *****
02
03/22 08:51:01 INFO  :...locate_configFile: Specified configuration file:
/u/user10/rsvpd1.conf
03/22 08:51:01 INFO  :.main: Using log level 511
03/22 08:51:01 INFO  :..settcpimage: Get TCP images rc - EDC8112I Operation not supported on
socket.
03
03/22 08:51:01 INFO  :..settcpimage: Associate with TCP/IP image name = TCPCS
03/22 08:51:02 INFO  :..reg_process: registering process with the system
03/22 08:51:02 INFO  :..reg_process: attempt OS/390 registration
03/22 08:51:02 INFO  :..reg_process: return from registration rc=0
04
03/22 08:51:06 TRACE :...read_physical_netif: Home list entries returned = 7
03/22 08:51:06 INFO  :...read_physical_netif: index #0, interface VLINK1 has address
129.1.1.1, ifidx 0
03/22 08:51:06 INFO  :...read_physical_netif: index #1, interface TR1 has address
9.37.65.139, ifidx 1
03/22 08:51:06 INFO  :...read_physical_netif: index #2, interface LINK11 has address
9.67.100.1, ifidx 2
03/22 08:51:06 INFO  :...read_physical_netif: index #3, interface LINK12 has address
9.67.101.1, ifidx 3
03/22 08:51:06 INFO  :...read_physical_netif: index #4, interface CTCDO has address
9.67.116.98, ifidx 4
03/22 08:51:06 INFO  :...read_physical_netif: index #5, interface CTCDO2 has address
9.67.117.98, ifidx 5
03/22 08:51:06 INFO  :...read_physical_netif: index #6, interface LOOPBACK has address
127.0.0.1, ifidx 0
03/22 08:51:06 INFO  :...mailslot_create: creating mailslot for timer
03/22 08:51:06 INFO  :...mailbox_register: mailbox allocated for timer
```

Logging-Systeme

- Lange Zeit war der Elastic-Stack (Elasticsearch + Logstash + Kibana) der Platzhirsch
- Wegen Lizenzproblemen kam es zu einem Fork: Opensearch
- Weitere bekannte Systeme sind Graylog und Loki



Elastic Stack

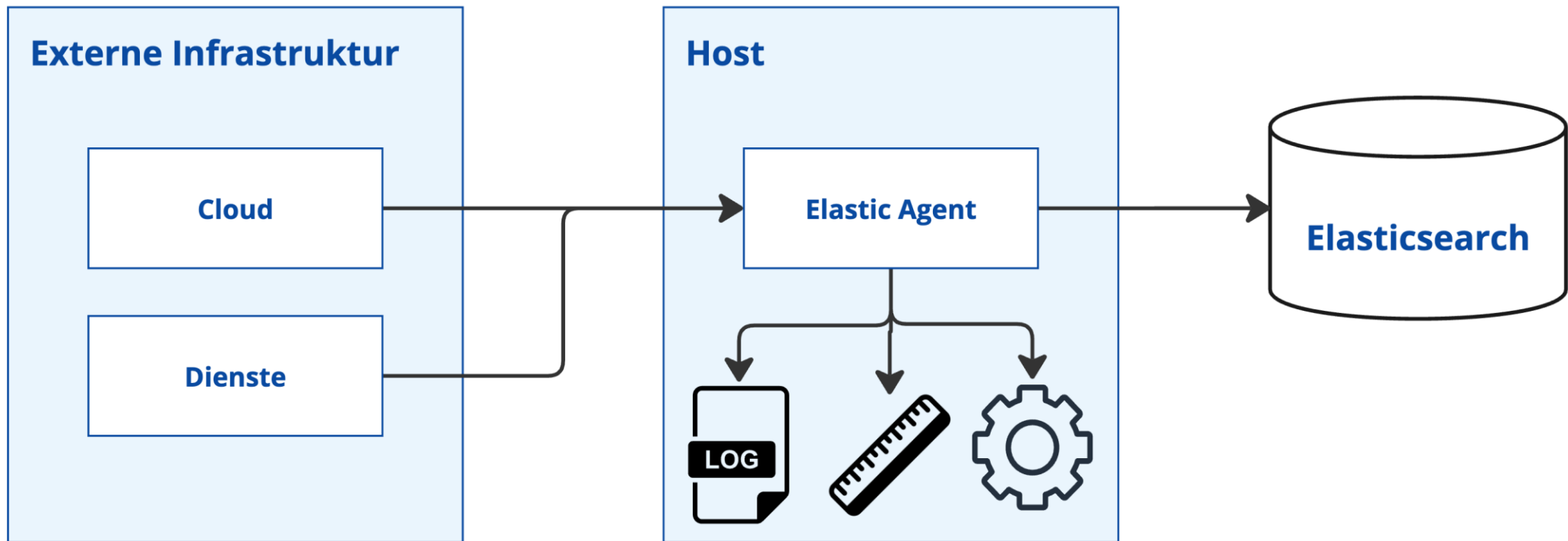
- Elasticsearch war zu Beginn eine Open Source-Suchmaschine
- Sie basiert auf Apache Lucene
- Ermöglicht effiziente Abfragen aus Textbeständen, Geodaten, numerischen Daten uvm.
- Existiert bereits seit 2010 und ist stark verbreitet

Bestandteile des Elastic Stacks

Der Stack besteht aus drei Teilen:

- **Elasticsearch** als Suchmaschine
- Ein **Logshipper** wie Logstash, Fluentd oder Elastic Agent
- **Kibana** als Frontend

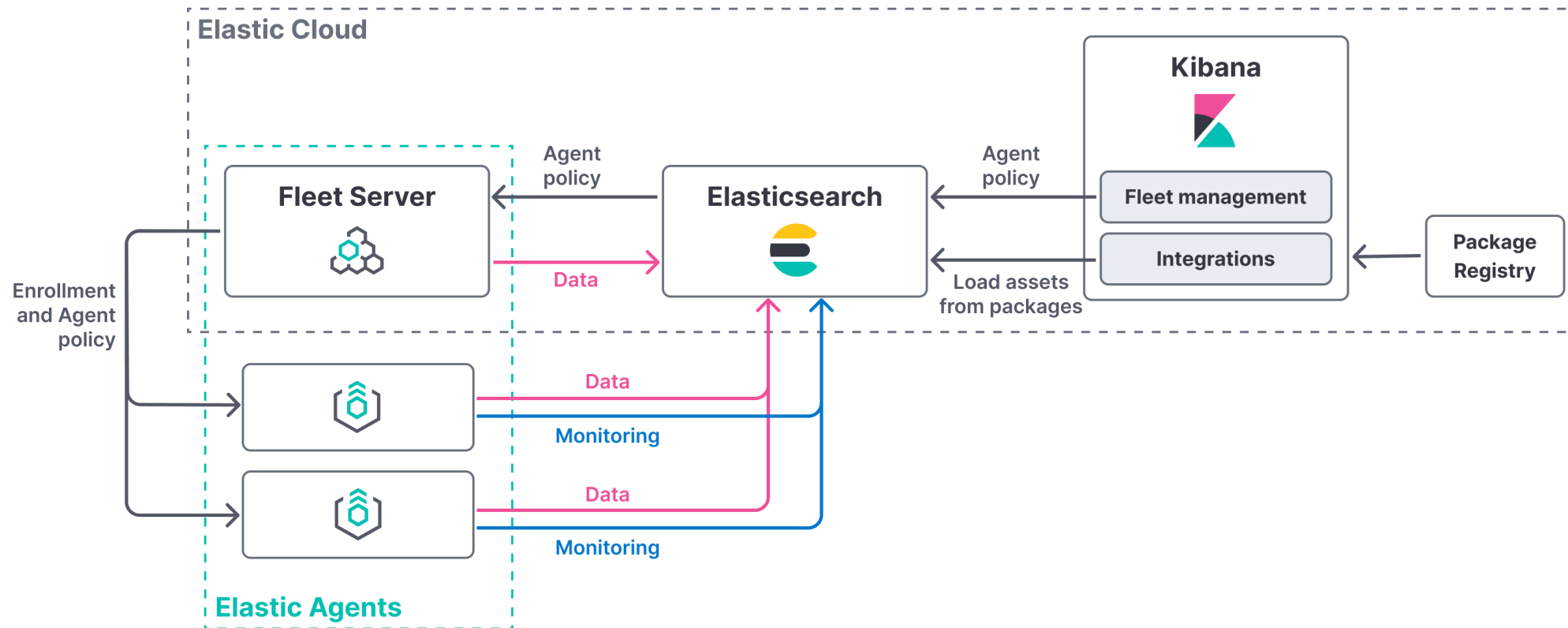
Elastic Agent



Elastic Agent-Verwaltung

- Für Kubernetes steht ein Operator zur Verfügung, der im Cluster einen Elastic Agent installiert
- Agents werden nicht nur in Kubernetes eingesetzt
- Im Gegenteil: Sie werden idealerweise in jeder Infrastruktur im Unternehmen eingesetzt
- Dabei stellt sich schnell die Frage, wie Agents verwaltet und aktualisiert werden

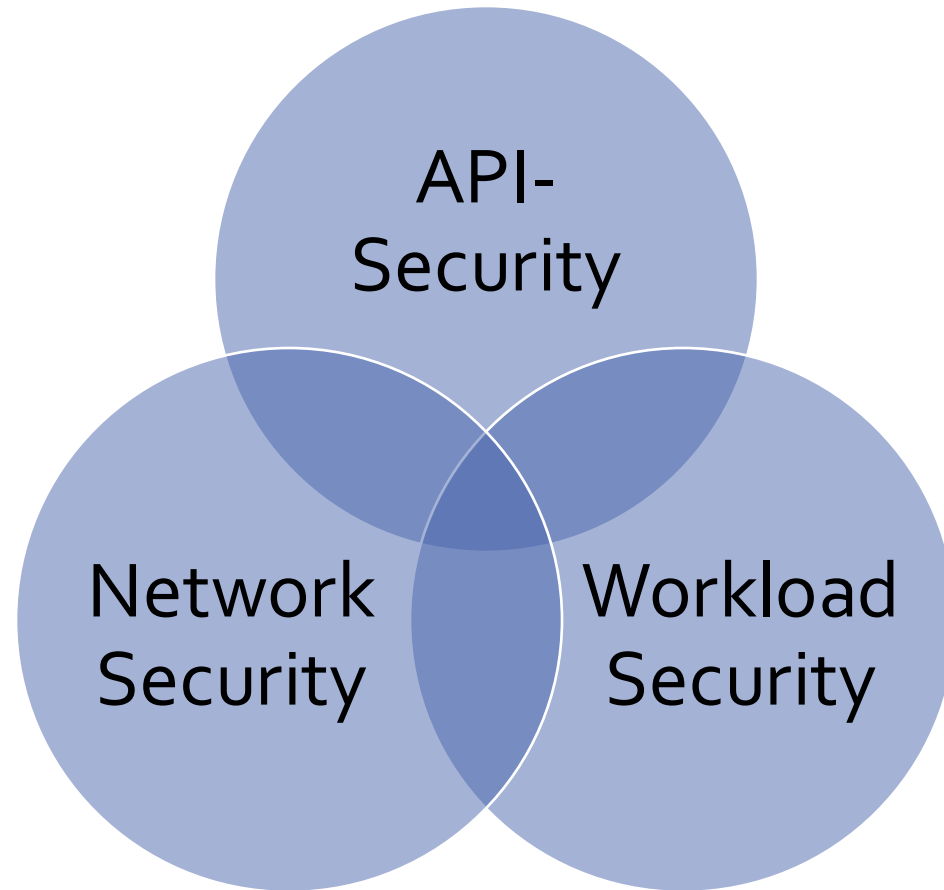
Elastic Fleet



Kubernetes Security

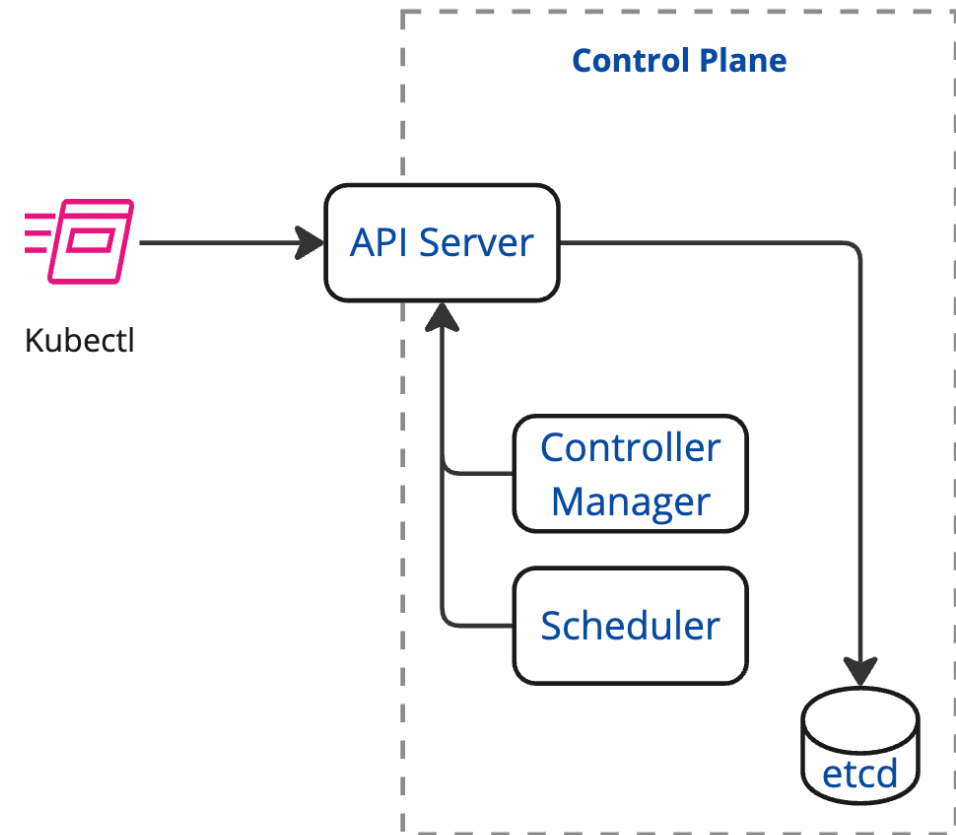
API-, Container- und Network Security

Kubernetes Security

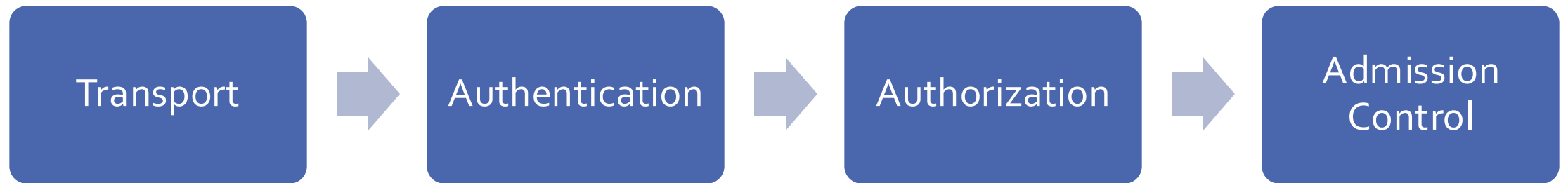


API Security

- API Security bezeichnet die Absicherung des Clusters gegen unzulässige Bedienung der API-Objekte von Kubernetes, z.B. via kubectl
- Zur Erinnerung: Kubernetes wird über den API Server bedient

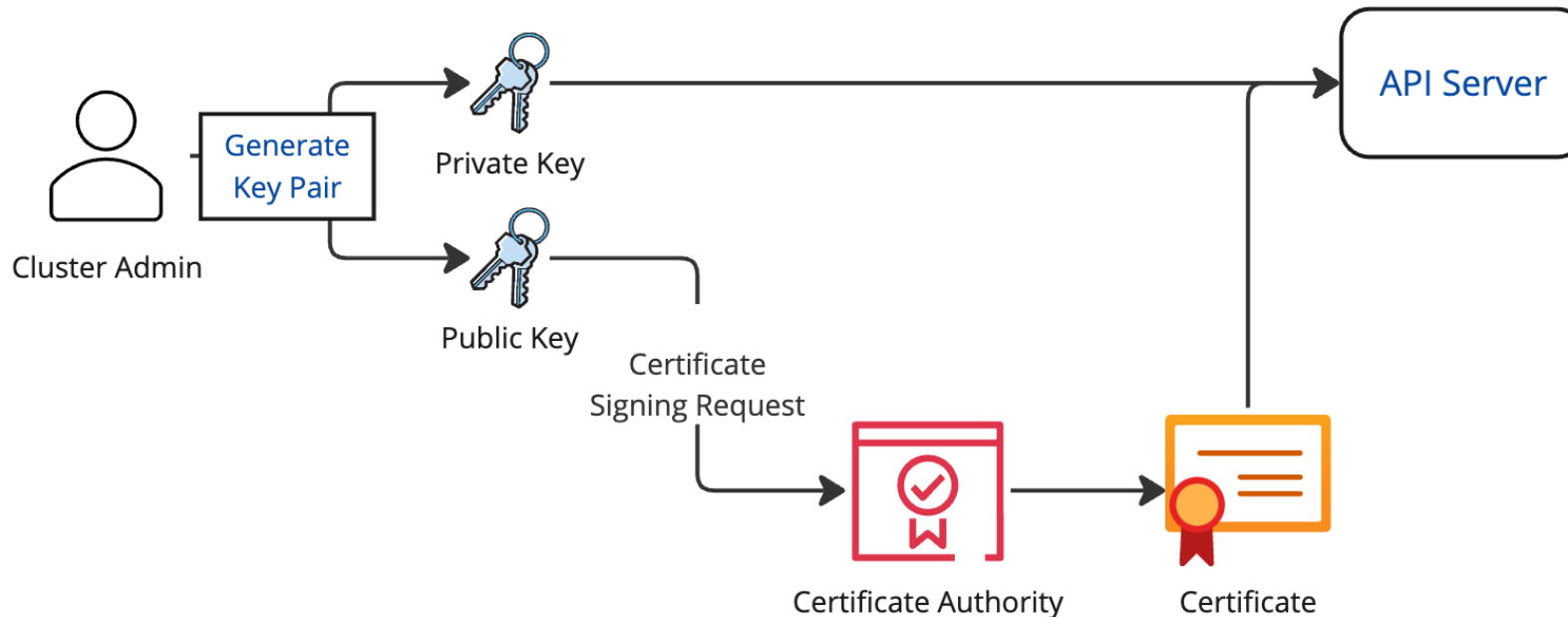


API Security – Ablauf



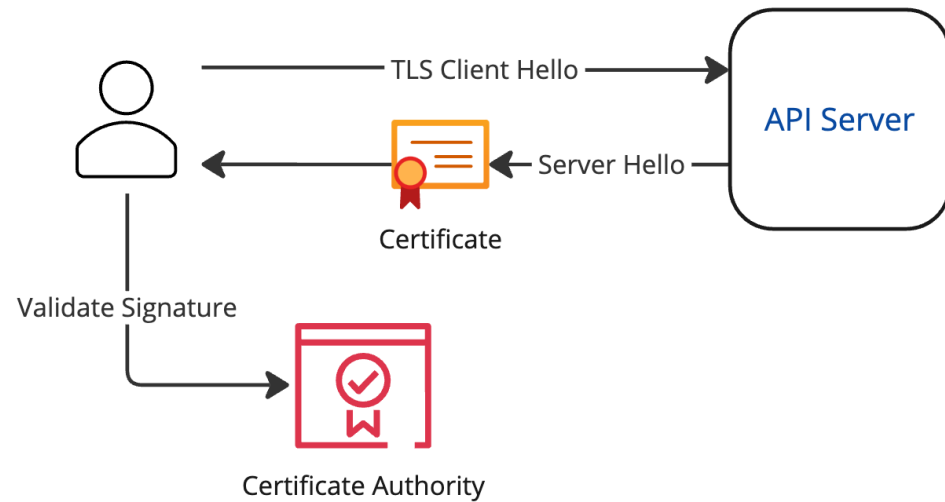
Transport Security

- Der Cluster Admin generiert beim Setup des API-Servers ein Key Pair
- Der Public Key wird mit einer Public oder Private CA signiert und das entstehende Zertifikat gespeichert



Transport Security

- Der API-Server präsentiert das Zertifikat beim TLS-Handshake.
- Der Client prüft es.
- Wurde eine private CA verwendet beim Signieren, muss der Client sie haben (in der Kubeconfig)



Authentication

- Es gibt unterschiedliche Module für Authentifizierung in k8s
- Eingabe ist der komplette HTTP-Request inklusive Header und (Client-)Zertifikat
- Beispielmechanismen:
 - Clientzertifikat
 - Passwort
 - Plain Token
 - JSON Web Token
- Kubernetes kennt Benutzernamen, besitzt aber keine Benutzer-Objekte in der API.

OIDC in Kubernetes

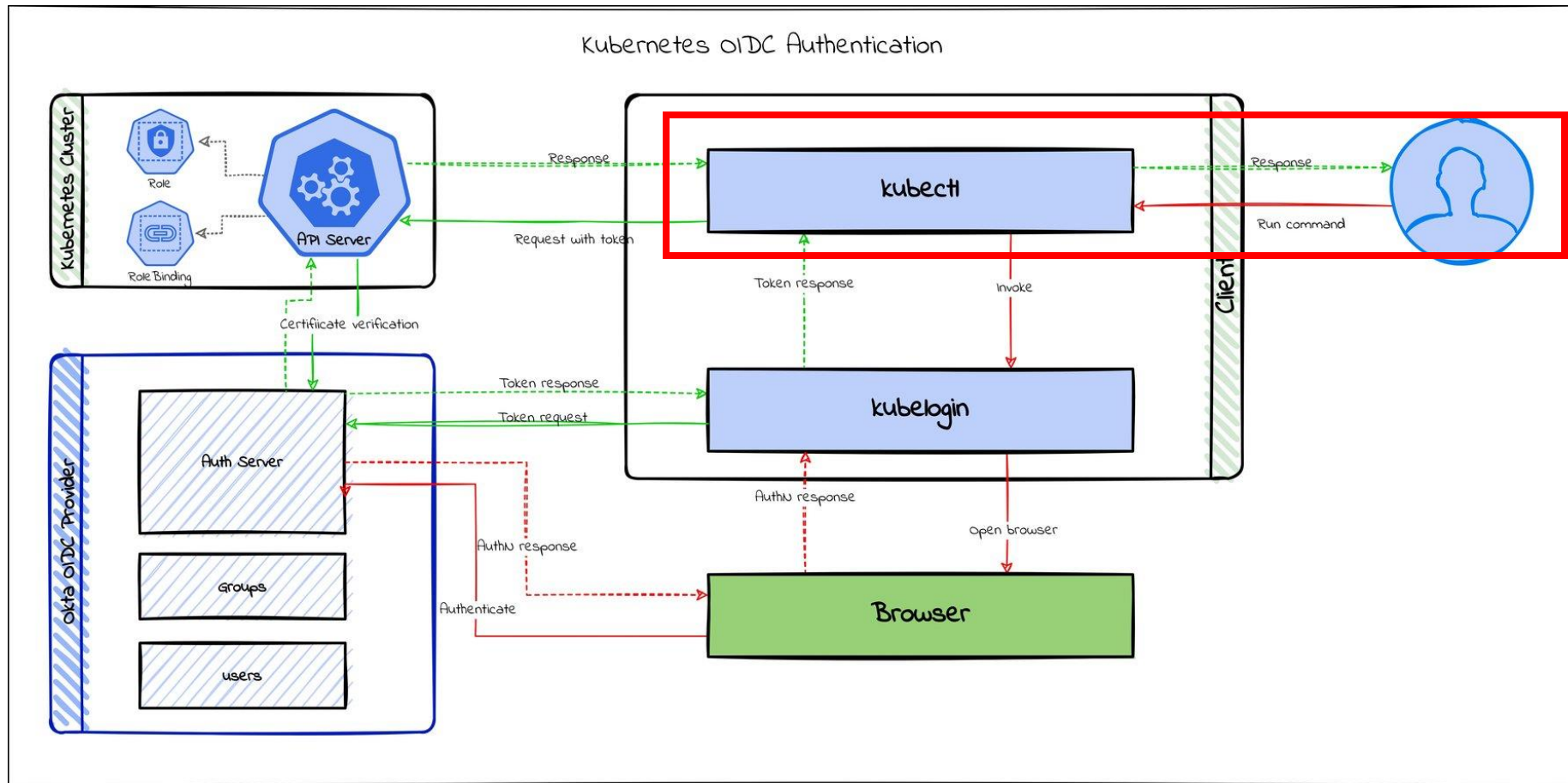
- In Enterprise-Szenarien wird Kubernetes in der Regel mit IDPs verbunden
- Ein häufiges Szenario ist der Einsatz von OpenID Connect, beispielsweise über Azure Entra ID oder Okta
- In diesem Fall kann die Identität über einen IDP ermittelt werden und Gruppen für RBAC im Cluster genutzt werden

Api-server mit OIDC

Um in Kubernetes OIDC für Microsoft Entra ID in Azure zu aktivieren, müssen dem API-Server beim Start einige Parameter mitgegeben werden

```
--oidc-issuer-url=https://login.microsoftonline.com/{tenant_id}/v2.0  
--oidc-client-id={client_id}  
--oidc-username-claim=email  
--oidc-groups-claim=groups
```

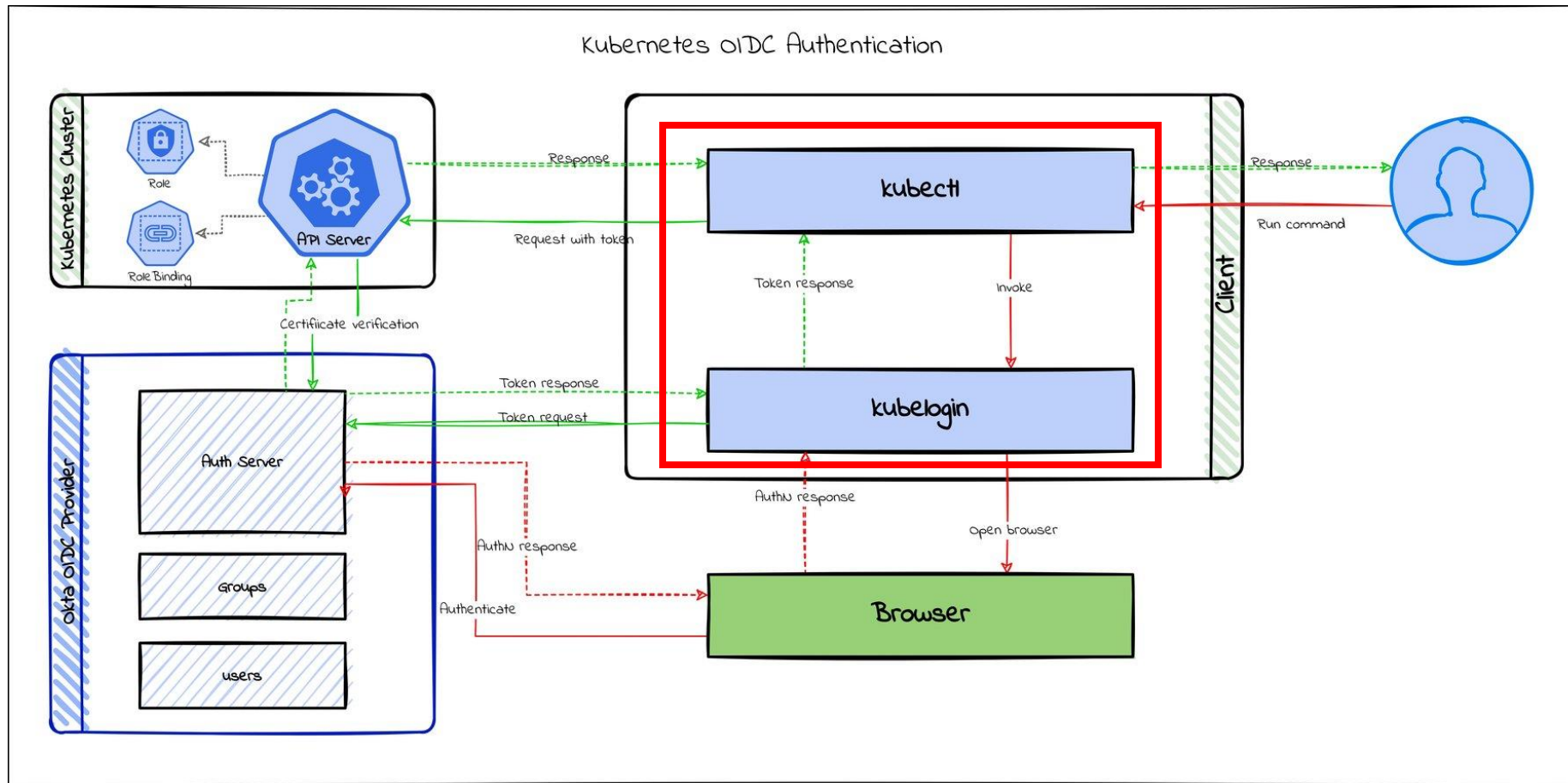
OIDC Flow



Zu Beginn führt der Benutzer kubectl aus. Es ist noch keine gültige Authentifizierung vorhanden.

<https://developer.okta.com/blog/2021/11/08/k8s-api-server-oidc>

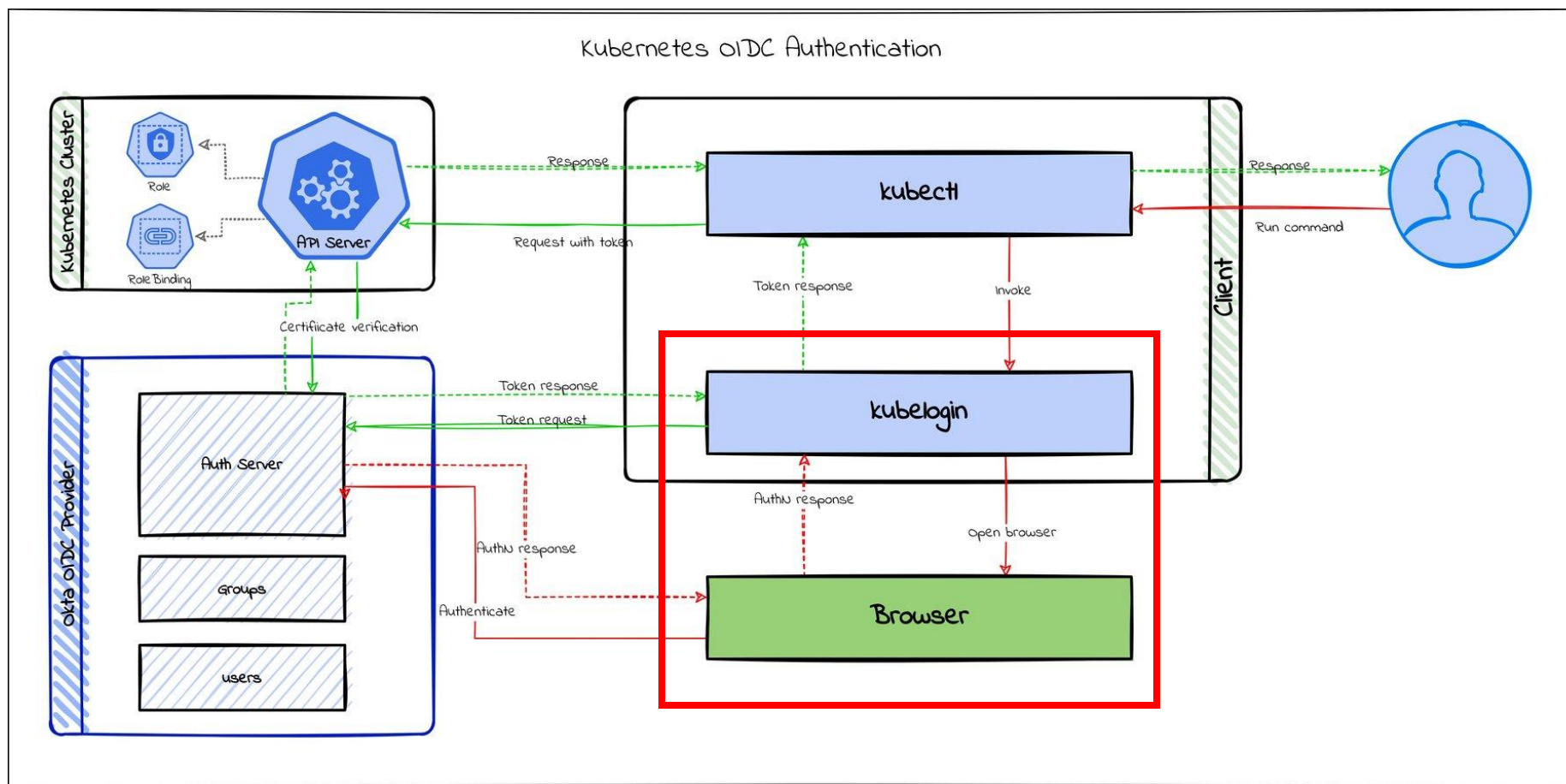
OIDC Flow



kubectl besitzt kein gültiges Anmeldetoken, um den api-server anzufragen. Es verwendet kubelogin um eines zu erwerben. Dies passiert noch im Client!

<https://developer.okta.com/blog/2021/11/08/k8s-api-server-oidc>

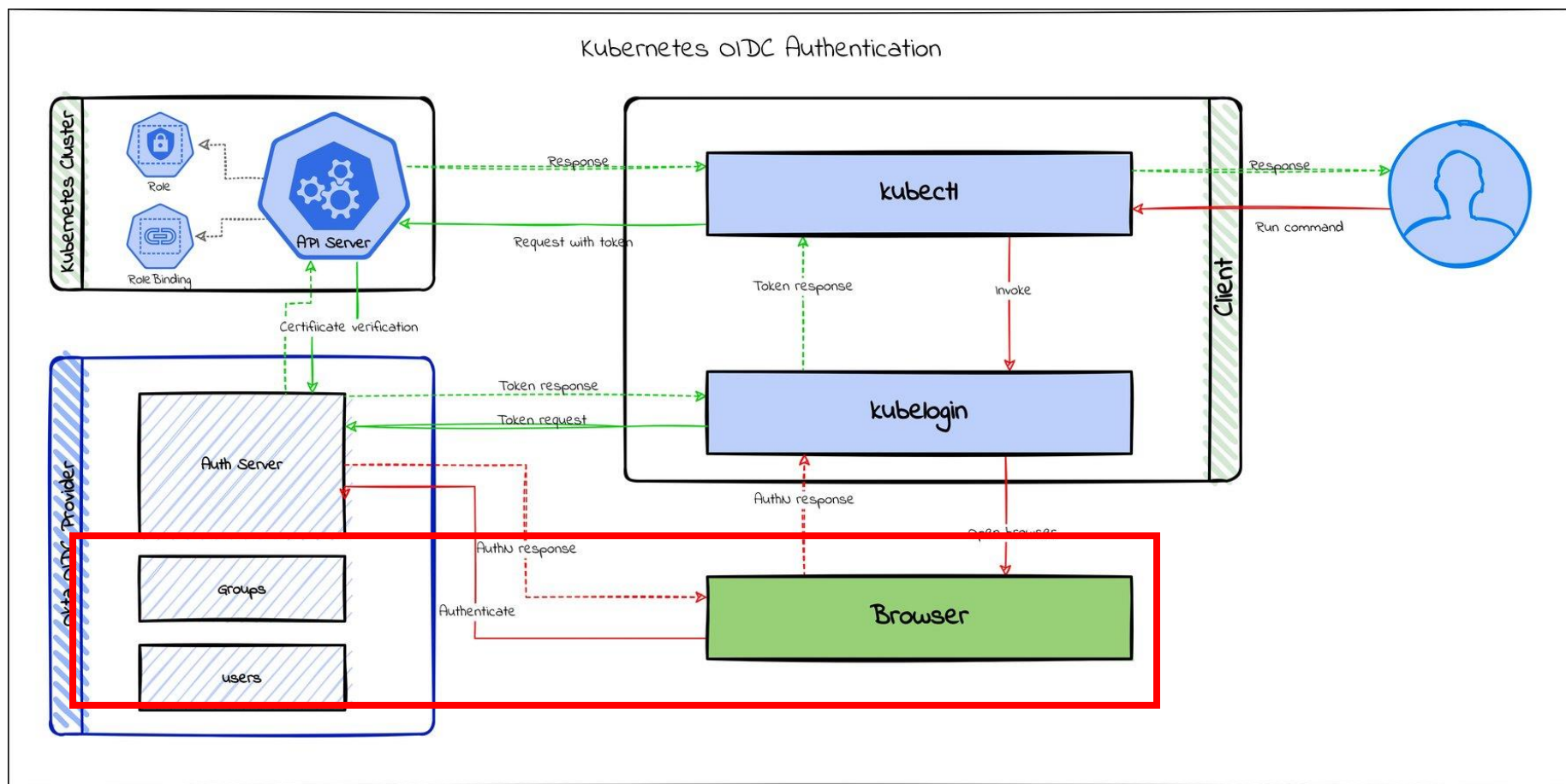
OIDC Flow



Kubelogin öffnet eine Browsersitzung und wartet darauf, dass der Benutzer den Login mit dem Identity Provider durchführt.

<https://developer.okta.com/blog/2021/11/08/k8s-api-server-oidc>

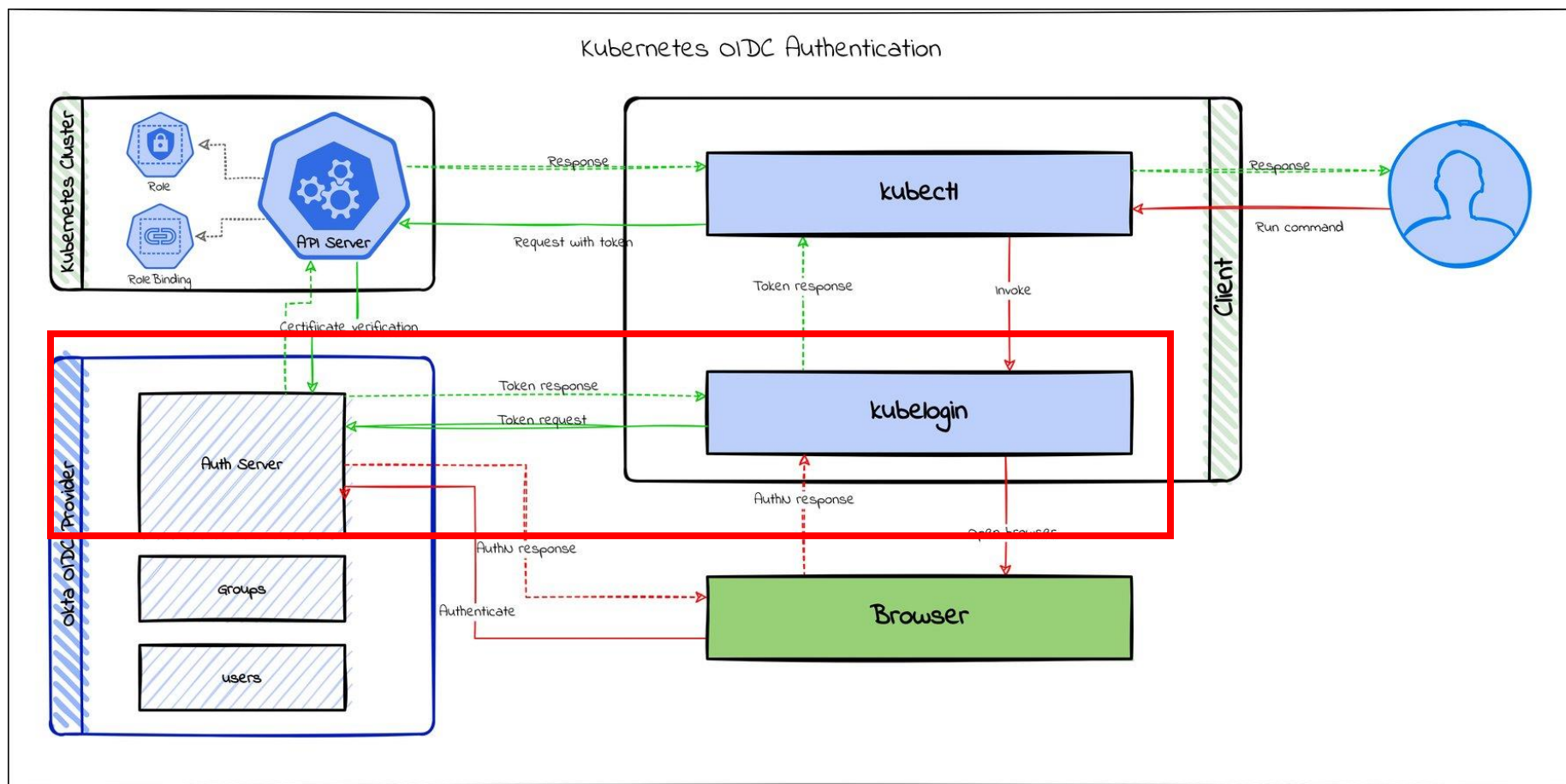
OIDC Flow



In der Browsersitzung wird der Login durchgeführt – ohne jede Beteiligung von kubelogin. Bei Erfolg erhält der Browser einen kurzlebigen Auth Code, den kubelogin verwenden kann.

<https://developer.okta.com/blog/2021/11/08/k8s-api-server-oidc>

OIDC Flow

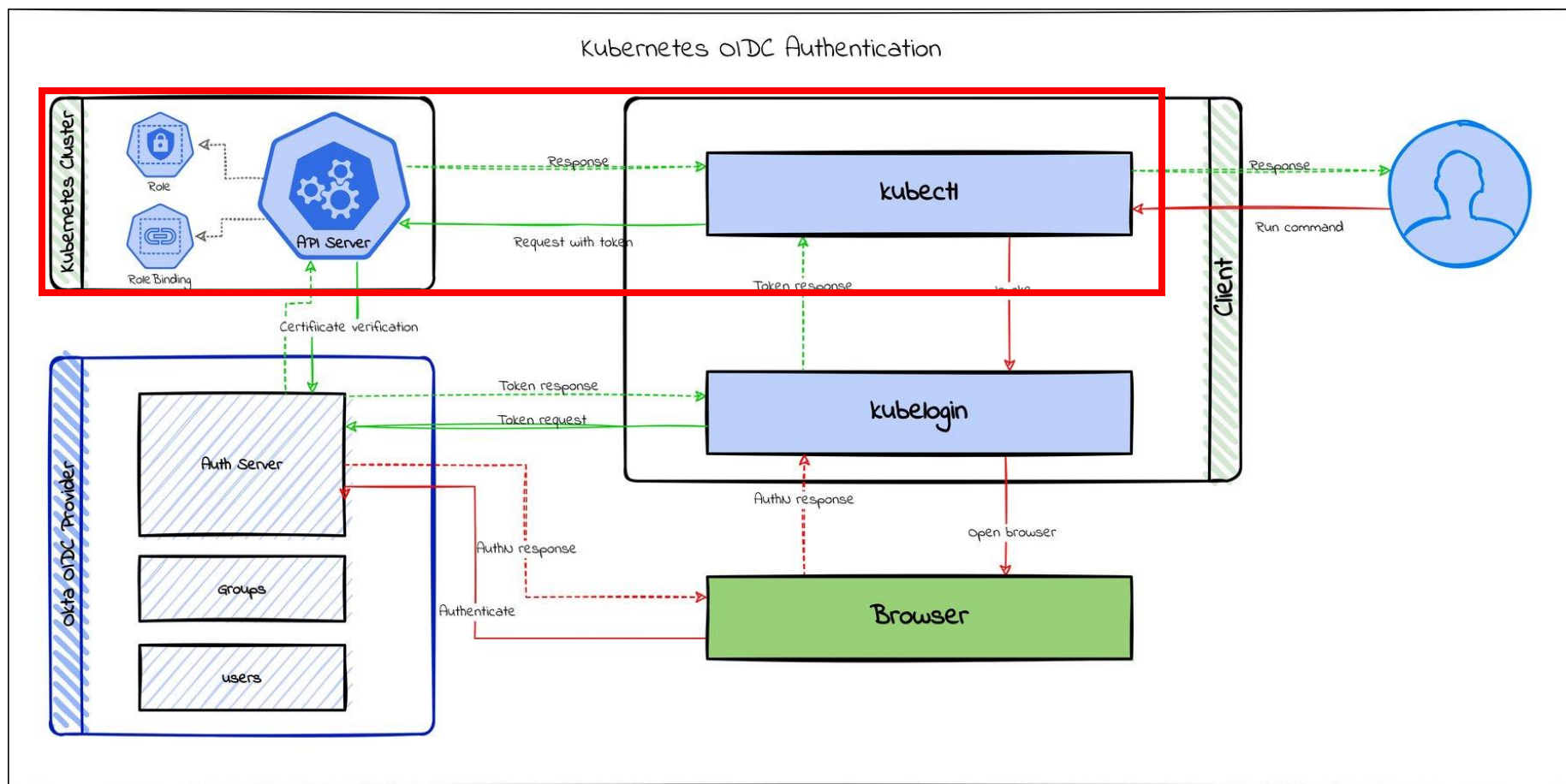


kubelogin verwendet den Auth Code, um ein ID-Token und ggf. ein Refresh Token vom IDP anzufragen.

Dieses ID Token wird vom api-server als Anmelde token akzeptiert. Läuft es ab, kann es per Refresh Token aktualisiert werden.

<https://developer.okta.com/blog/2021/11/08/k8s-api-server-oidc>

OIDC Flow



Kubectl sendet bei jedem Request das ID Token im Authorization-HTTP-header mit. Der Api-Server kann die Echtheit und Integrität des Tokens mittels der Signatur prüfen. Dafür nutzt er den öffentlichen Schlüssel des IDPs.

<https://developer.okta.com/blog/2021/11/08/k8s-api-server-oidc>

Group Claims

- Die Benutzergruppe muss Teil der Claims sein, die im ID-Token hinterlegt sind
- Mit dem Gruppen-Claim kann hinterher eine oder mehrere Rollen gebunden werden an die AD-Gruppe
- Rollen können sowohl in Namespaces als auch clusterweit angelegt werden (mehr dazu später)

Beispiel-Token

```
{
  "aud": "my-client-id",
  "iss": "https://login.microsoftonline.com/{tenant-id}/v2.0",
  "exp": 1692616060,
  "sub": "b275d4e9-0f21-4b55-89a8-7095d0983a7e",
  "name": "Peter Peters",
  "preferred_username": "peters@gfu.de",
  "email": "peters@gfu.de",
  "groups": [
    "92a1b0d9-3c4e-4b5f-8c2d-a77fada8d030",
    "b29cd847-d94d-4a3d-923a-4588d2e95b43",
    "3e58b8a6-1c8d-473b-9e32-5179b1234c1e"
  ],
  ...
}
```

- Token enthalten die Gruppen als Claims
- Als Prüfmechanismus wird eine Signatur verwendet
- Der IDP signiert das Token mit seinem Private Key
- Der API-Server kann die Signatur mit dem öffentlichen Schlüssel des IDP verifizieren
- Dadurch ist sichergestellt, dass die Gruppenliste nicht manipuliert wurde

Kubernetes Security

Authorization: RBAC und ABAC

Authorization

- Ist der Benutzer authentifiziert, wird geprüft, ob er Berechtigung für die Anfrage hat
- Es gibt zwei grundlegende Mechanismen
 - ABAC: Attribute Based Access Control
 - RBAC: Role Based Access Control
- ABAC basiert auf Policies, welche den Benutzern einzelne Berechtigungen auf Objekte geben
- RBAC ordnet Benutzern Rollen zu, die dann jeweils unterschiedliche Berechtigungen besitzen

Beispiel ABAC

```
{  
  "apiVersion": "abac.authorization.kubernetes.io/v1beta1",  
  "kind": "Policy",  
  "spec": {  
    "user": "Peter",  
    "namespace": "project-falcon",  
    "resource": "pods",  
    "verb": "get",  
    "effect": "allow"  
  }  
}
```

Nachteile ABAC

- Die Policies müssen beim Start des API-Servers konfiguriert werden
- Ein Nachladen zur Laufzeit ist **nicht** möglich
- RBAC wird daher viel häufiger verwendet

```
$ kube-apiserver \  
    --authorization-policy-file=policy.json \  
    --authorization-mode=ABAC
```

RBAC – Role Based Access Control

- In RBAC werden Benutzern zu Rollen zugewiesen
- Die Rollen besitzen Zugriff auf Objekte (oder eben nicht)
- Die notwendigen k8s-Objekte sind in der Gruppe `rbac.authorization.k8s.io`
- Der RBAC-Modus wird beim Start des API-Servers aktiviert

```
$ kube-apiserver --authorization-mode=RBAC
```

Bei den meisten k8s-Setups ist RBAC
bereits aktiv.

RBAC in Minikube

```
$ kubectl describe pod kube-apiserver-minikube -n kube-system
```

Containers:

kube-apiserver:

Container ID: docker://c91a6b2c430c8d7f44207d8718
Image: registry.k8s.io/kube-apiserver:v1.28.3
Image ID: docker-pullable://registry.k8s.io/kube-apiserver
Port: <none>
Host Port: <none>

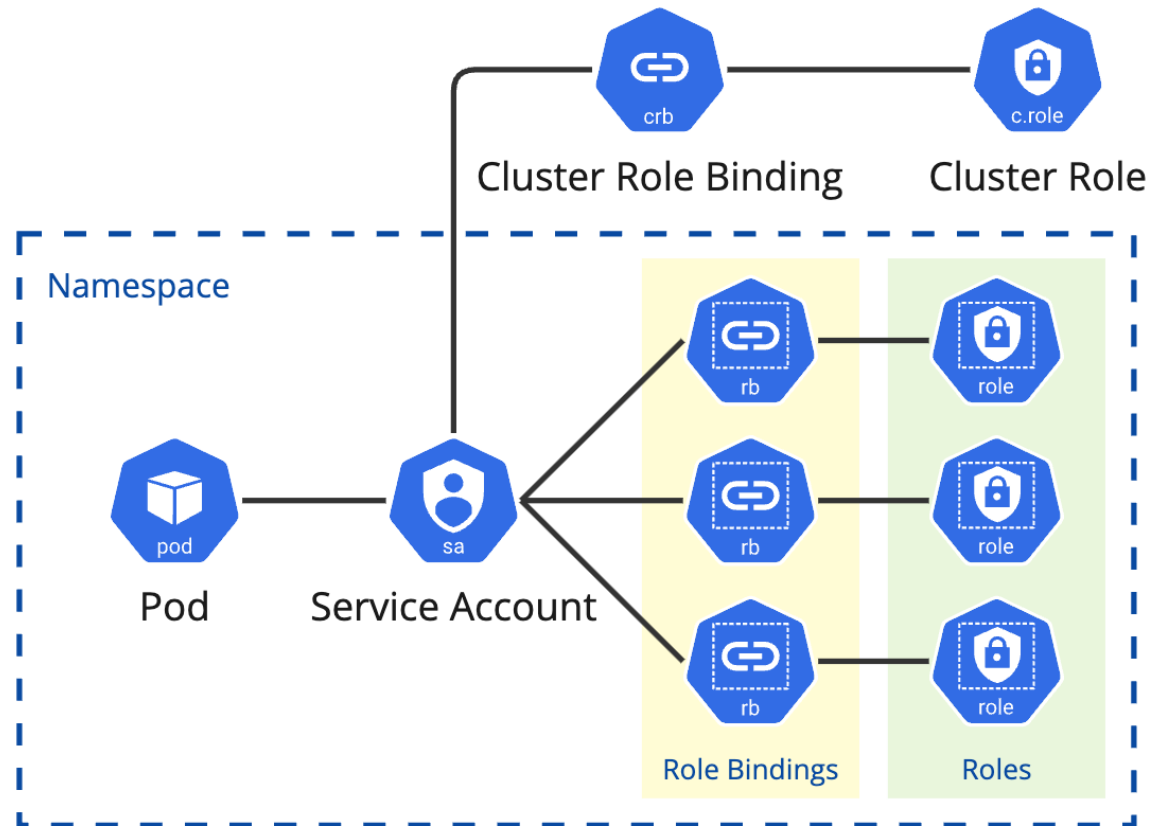
Command:

kube-apiserver
--advertise-address=192.168.58.2
--allow-privileged=true
--**authorization-mode=Node,RBAC**

RBAC ist bei Minikube bereits aktiv!

Der Node-AuthORIZER erlaubt es den Kubelets, Pods zu bearbeiten auf dem Knoten, für den sie zuständig sind.

RBAC – High Level



ServiceAccount

- Account, der nicht für Menschen gedacht ist
- Genutzt von Pods und Systemkomponenten von k8s
- Befindet sich in Namespace
- Sehr leichtgewichtig: Schnell angelegt und ohne kompliziertes Setup



Service Account anlegen

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: my-serviceaccount
  namespace: my-namespace
```



Service Account für Pod konfigurieren

```
apiVersion: v1
kind: Pod
metadata:
  name: nginxpod
  namespace: my-namespace
spec:
  containers:
  - image: nginx:latest
    name: nginx
  serviceAccountName: my-serviceaccount
```



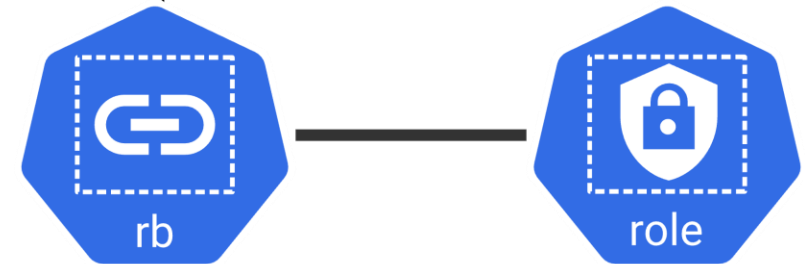
Rollen anlegen

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-creator
  namespace: my-namespace
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["create", "get", "watch", "list"]
```



Rolle an Service Account binden

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: pod-creator
  namespace: my-namespace
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: pod-creator
subjects:
- kind: ServiceAccount
  name: my-serviceaccount
  namespace: my-namespace
```



Role Bindings über (OIDC-)Gruppen

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-only-binding
  namespace: my-namespace
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: read-only
subjects:
- kind: Group
  name: <Azure_Entra_ID_group_id>
  apiGroup: rbac.authorization.k8s.io
```

Nutzer erhalten bei OIDC die Gruppen via Claims. Die Gruppe kann als Subject beim RoleBinding genutzt werden.

ClusterRoleBinding mit OIDC

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: developer
rules:
- apiGroups: [""]
  resources: ["pods", "services", "configmaps", "secrets"]
  verbs: ["get", "list", "watch"]
- apiGroups: ["apps"]
  resources: ["deployments", "replicasets", "statefulsets"]
  verbs: ["get", "list", "watch"]
- apiGroups: ["batch"]
  resources: ["jobs", "cronjobs"]
  verbs: ["get", "list", "watch"]
```

Hier bauen wir eine clusterweite Read-Only-Rolle für Entwickler.

ClusterRoleBinding mit OIDC

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: developer-binding
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: developer
subjects:
- apiGroup: rbac.authorization.k8s.io
  kind: Group
  name: oidc-developers
```

Die Read-Only-Rolle wird per Role Binding an die OIDC-Gruppe gebunden.

Praxisaufgabe

RBAC

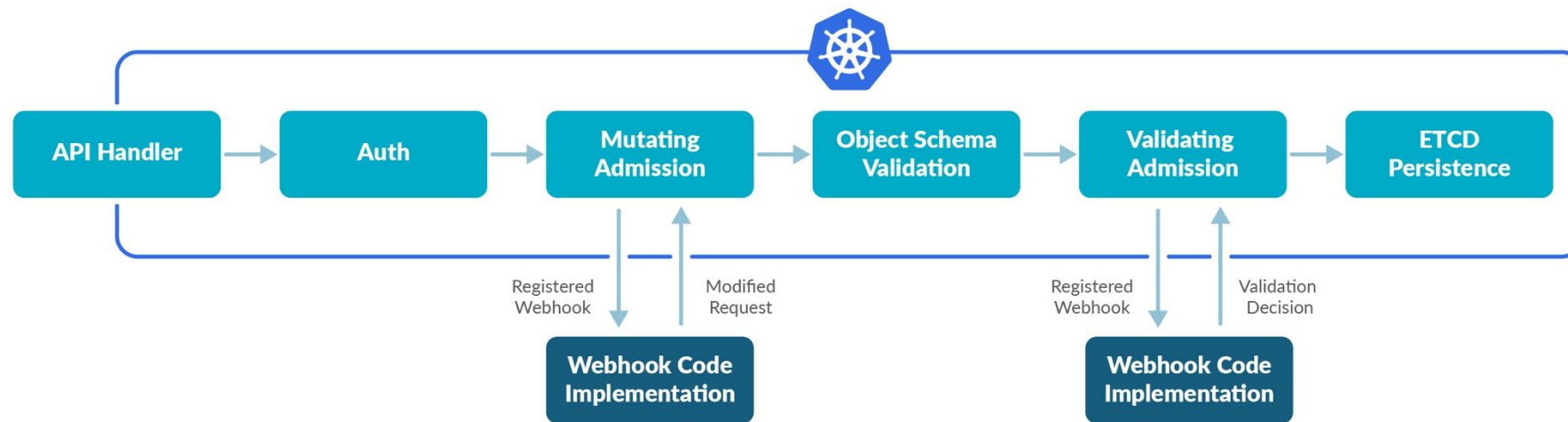
Kubernetes Security

Admission Control

Admission Control

- Nach der Authorisierung folgt als letzter Schritt die Admission Control
- Es gibt im Wesentlichen drei Schritte in diesen Prozess:
 - **Mutating Admission:** Verändern von eingehenden Objekten
 - **Schema Validation**
 - **Validating Admission:** Prüfungen, ob Operation gültig ist in diesem Cluster
- Schlägt eine der Schritte fehl, lehnt der apiserver den Request ab

Admission

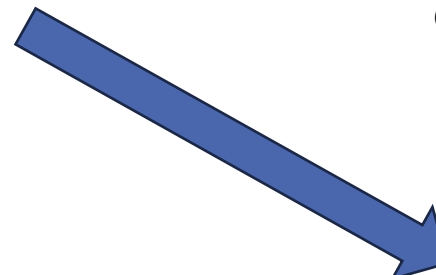


Beispiele für Mutating Admission

- **NamespaceAutoProvision:** Legt Namespace eines Objekts an, falls es fehlt
- **AlwaysPullImages:** Setzt automatisch die Image Pull Policy von Pods auf *always*
- **DefaultStorageClass:** Wenn für ein PVC keine StorageClass gesetzt ist, wird ein default gesetzt

Beispiele für Validating Admission

- **LimitRanger:** Erzwingt eine Limit Range für Workloads
- **NamespaceExists:** Lehnt Objekte ab, wenn ihr Namespace nicht existiert
- **ServiceAccount:** Überprüft Regeln im Umgang mit ServiceAccounts



```
containers:
- image: nginx
  name: default-mem-demo-ctr
  resources:
    limits:
      memory: 512Mi
    requests:
      memory: 256Mi
```

Kubernetes Security

Workload Security

Motivation

- Container teilen sich auf ihrem Host den gleichen Linux-Kernel
- Es ist wichtig, ihren Zugriff einzuschränken
- Kubernetes bietet verschiedene Optionen dazu
- Wir beschränken uns auf eine moderne Variante:
Pod Security Standards

Pod Security Standards

Der Pod Security Standard definiert drei Stufen:

Profil	Beschreibung
Privileged	Erlaubt Vollzugriff auf den Kernel Unter anderem haben Pods Vollzugriff auf das Hostnetzwerk und das Hostdateisystem. Nur einzusetzen bei Pods, die den Host vorbereiten, z.B. indem sie Netzwerkinterfaces konfigurieren.
Baseline	Schränkt kritischste Zugriffsmuster ein Host-Prozesse sind nicht sichtbar, das Host-Dateisystem kann nicht gemountet werden, auf Host Ports kann nicht zugegriffen werden, /proc ist maskiert ... und noch mehr.
Restricted	Stark eingeschränktes Zugriffsniveau für maximale Sicherheit Container müssen als non-root laufen, nur eine beschränkte Menge an Volumes mit geringer Angriffsfläche ist verfügbar.

Pod Security Standards definieren

Standards werden auf Namespace-Ebene gesetzt:

```
apiVersion: v1
kind: Namespace
metadata:
  name: my-secure-namespace
  annotations:
    pod-security.kubernetes.io/enforce: "baseline"
    pod-security.kubernetes.io/enforce-version: "latest"
    pod-security.kubernetes.io/warn: "restricted"
    pod-security.kubernetes.io/warn-version: "latest"
```

SecurityContext – Pod mit Warnung

```
apiVersion: v1
kind: Pod
metadata:
  name: warning-pod
  namespace: my-secure-namespace
spec:
  containers:
    - name: nginx
      image: nginx:latest
      securityContext:
        runAsUser: 0 # Läuft als root
        allowPrivilegeEscalation: false
```

SecurityContext – Pod mit Blockade

```
apiVersion: v1
  kind: Pod
metadata:
  name: blocked-pod
  namespace: my-secure-namespace
spec:
  containers:
    - name: privileged-container
      image: alpine:latest
      securityContext:
        privileged: true # Privilegierter Container
```

Praxisaufgabe

Pod Security Standards

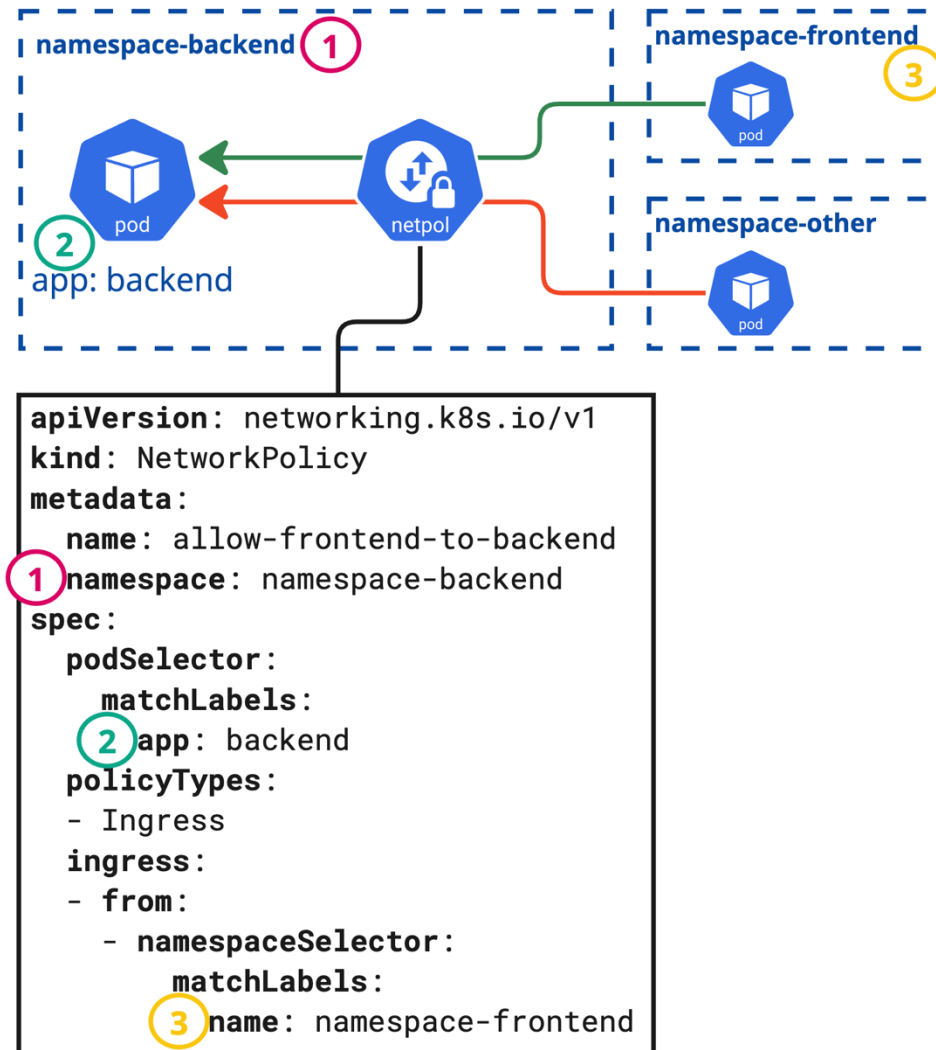
Kubernetes Security

Network Security

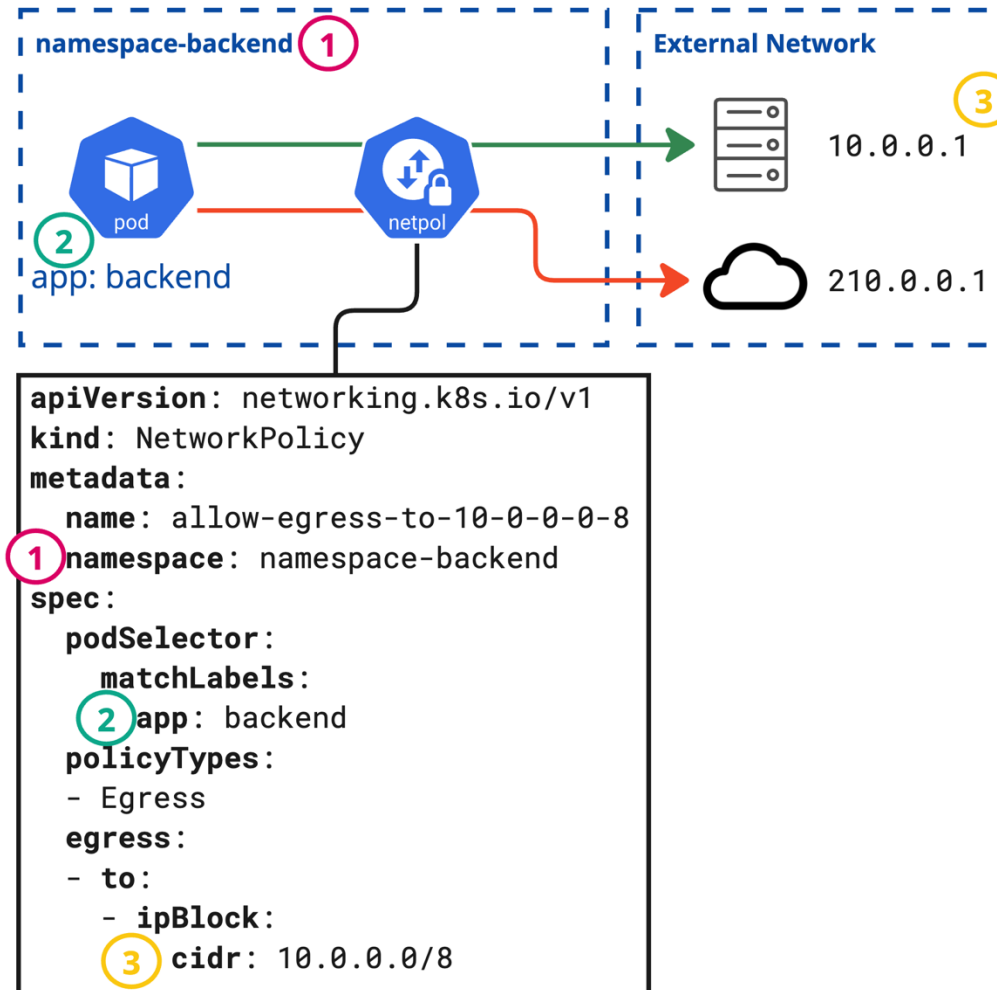
Network Policies

- NetworkPolicies sind Regeln zur Steuerung von Netzwerkverkehr
- Sie unterstützen Ingress und Egress-Regeln
- Beziehen sich auf einen Namespace
- Unterschiedliche Ziele und Quellen sind verfügbar
 - Pods via Label-Selectors
 - Namespaces
 - IP-Adress-Blöcke

NetworkPolicies: Ingress



NetworkPolicies: Egress

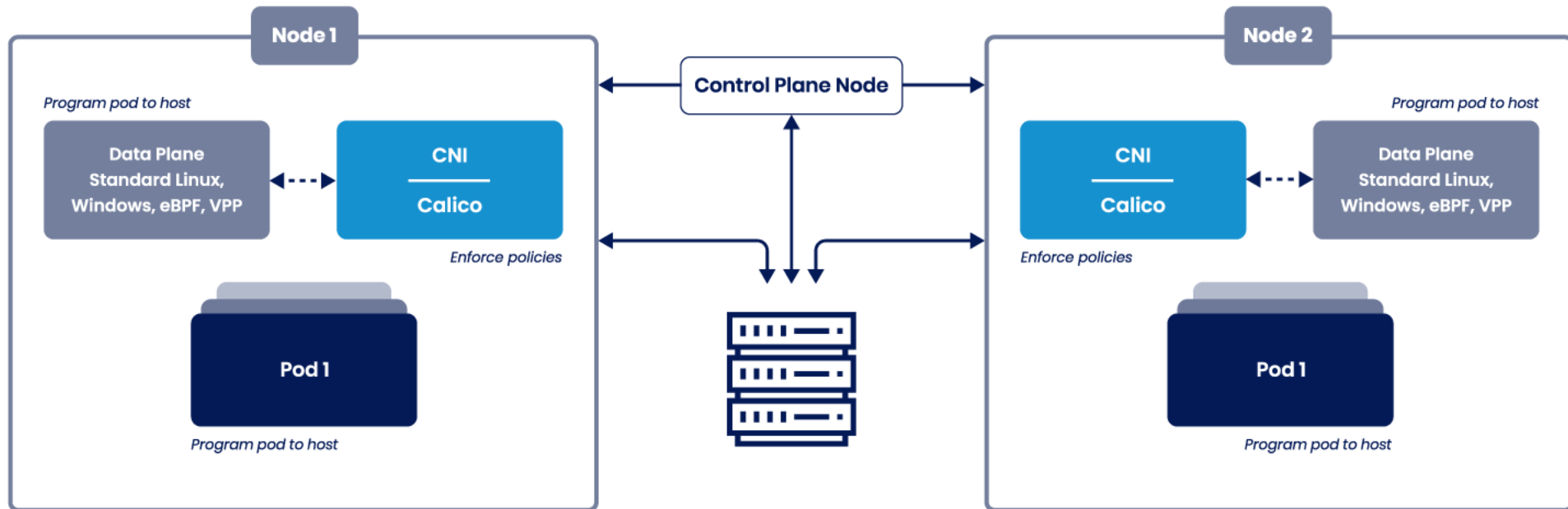


Mehr Features: Calico

- **Netzwerk-Sicherheitslösung:** Calico bietet nicht nur einfache Netzwerkverbindungen, sondern auch erweiterte Sicherheitsfunktionen wie Richtliniensteuerung und Firewalling.
- **Kubernetes-Ingress/Egress:** Im Gegensatz z.B. zu Flannel, das sich auf einfache Netzwerkbereitstellung konzentriert, ermöglicht Calico detaillierte Kontrolle über den ein- und ausgehenden Datenverkehr



Calico-Architektur



Calico muss als Netzwerkplugin auf jedem Host installiert werden. Dies geschieht i.d.R. über ein DaemonSet.

GlobalNetworkPolicy

GlobalNetworkPolicies erlauben es, bestimmte Zugriffe anhand von IP-Adress-Blöcken **clusterweit** komplett zu unterbinden. Das können NetworkPolicies allein nicht.

```
apiVersion: projectcalico.org/v3
kind: GlobalNetworkPolicy
metadata:
  name: deny-external-ingress
spec:
  order: 10
  ingress:
    - action: Deny
      source:
        nets:
          - 0.0.0.0/0
  types:
    - Ingress
```

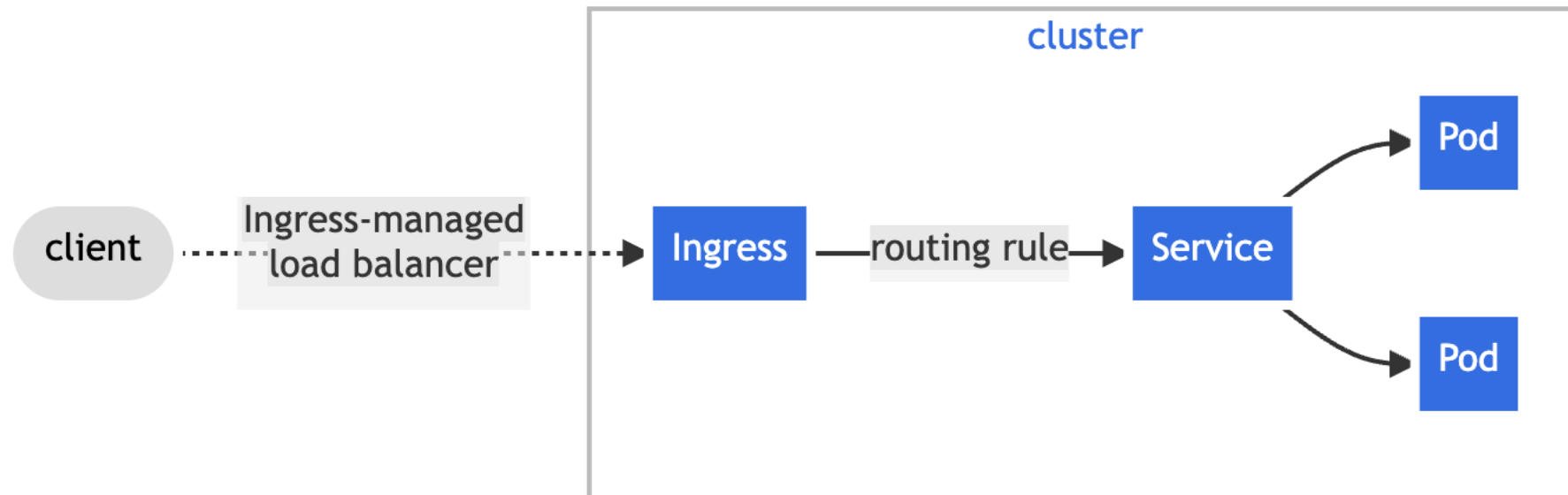
Advanced Networking

Ingress

Ingress

- **Routen von außen nach innen:** Ingress ist ein Kubernetes-Objekt, das eingehenden HTTP/HTTPS-Traffic über eine öffentliche IP oder Domain zu internen Services leitet.
- **Regelbasiert & flexibel:** Pfad- und Host-basierte Routingregeln, TLS-Termination, optional Load-Balancing und Virtual Hosts.
- **Benötigt Controller:** Funktioniert nur in Kombination mit einem Ingress Controller (z. B. NGINX, Traefik, Cloud-Load-Balancer).

Ingress



Einfacher Ingress

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: minimal-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx-example
  rules:
  - http:
      paths:
      - path: /testpath
        pathType: Prefix
        backend:
          service:
            name: test
            port:
              number: 80
```

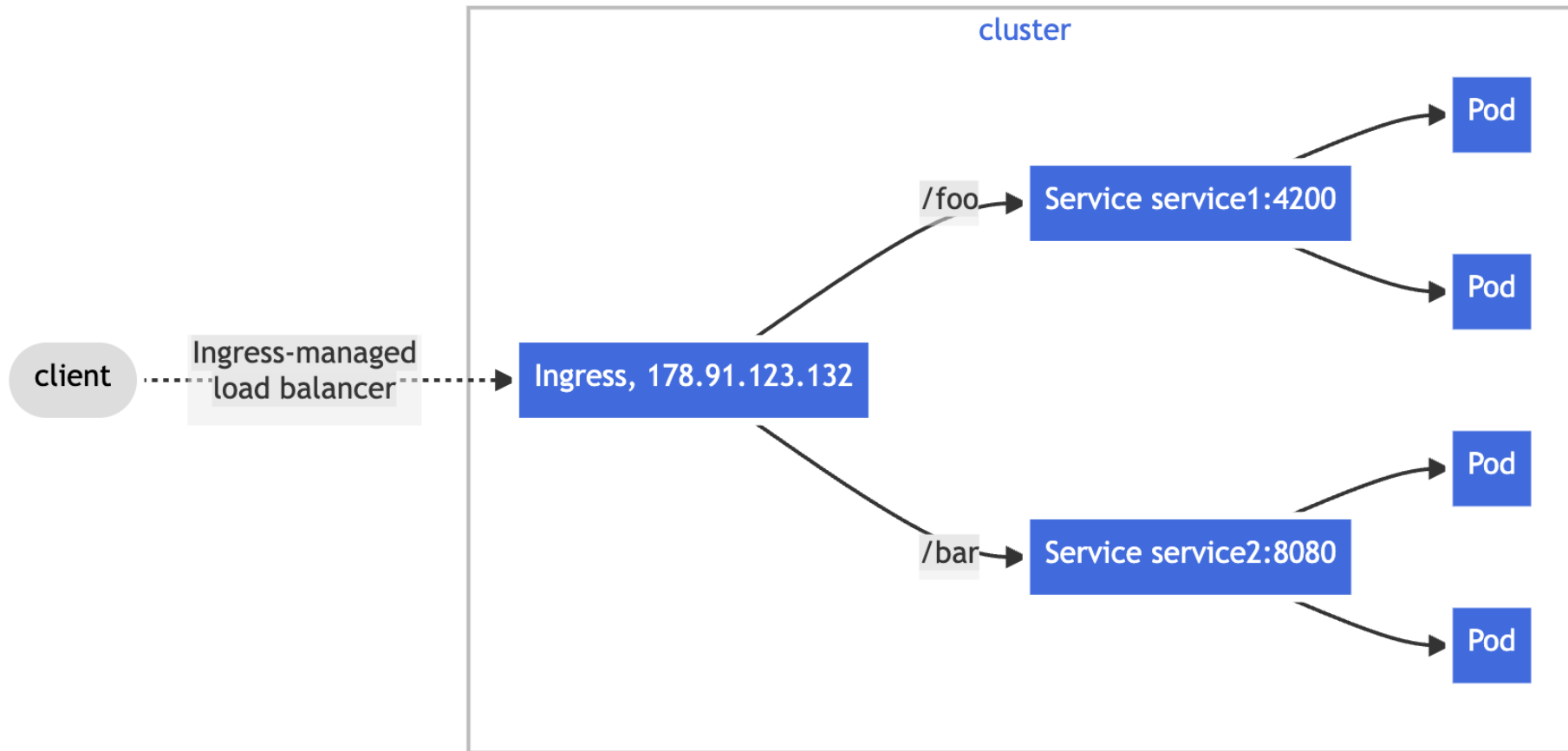
IngressClass wird vom Cluster Admin bereitgestellt. Er „realisiert“ den Ingress, wie eine StorageClass den Storage „realisiert“.

Einfacher Ingress

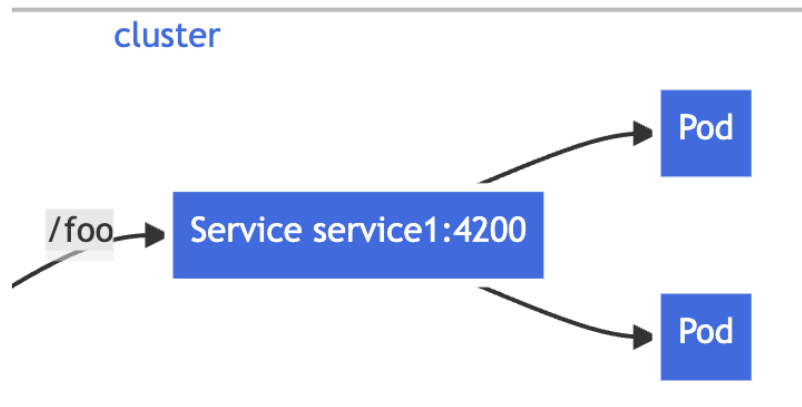
```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: minimal-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx-example
  rules:
  - http:
      paths:
      - path: /testpath
        pathType: Prefix
        backend:
          service:
            name: test
            port:
              number: 80
```

Eine **http-Rule** gibt ab, welche Pfade auf welchen Service leiten sollen.

Ingress Fan-Out

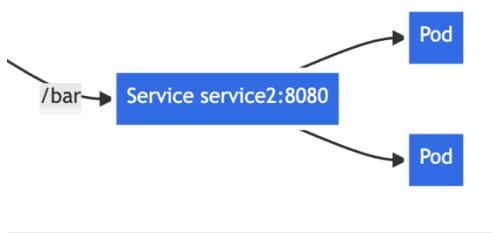


Fan-Out



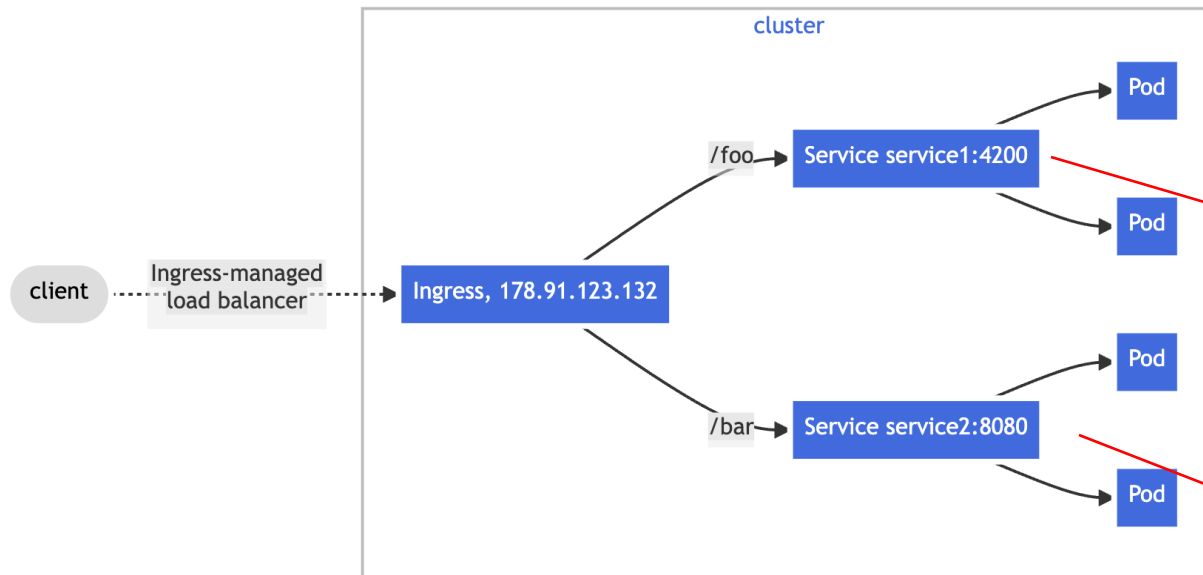
```
rules:  
- host: foo.bar.com  
  http:  
    paths:  
    - path: /foo  
      pathType: Prefix  
      backend:  
        service:  
          name: service1  
          port:  
            number: 4200
```

Fan-Out



```
- path: /bar
  pathType: Prefix
  backend:
    service:
      name: service2
      port:
        number: 8080
```

Fan-Out



```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: simple-fanout-example
spec:
  rules:
  - host: foo.bar.com
    http:
      paths:
      - path: /foo
        pathType: Prefix
        backend:
          service:
            name: service1
            port:
              number: 4200
      - path: /bar
        pathType: Prefix
        backend:
          service:
            name: service2
            port:
              number: 8080
```

Advanced Networking

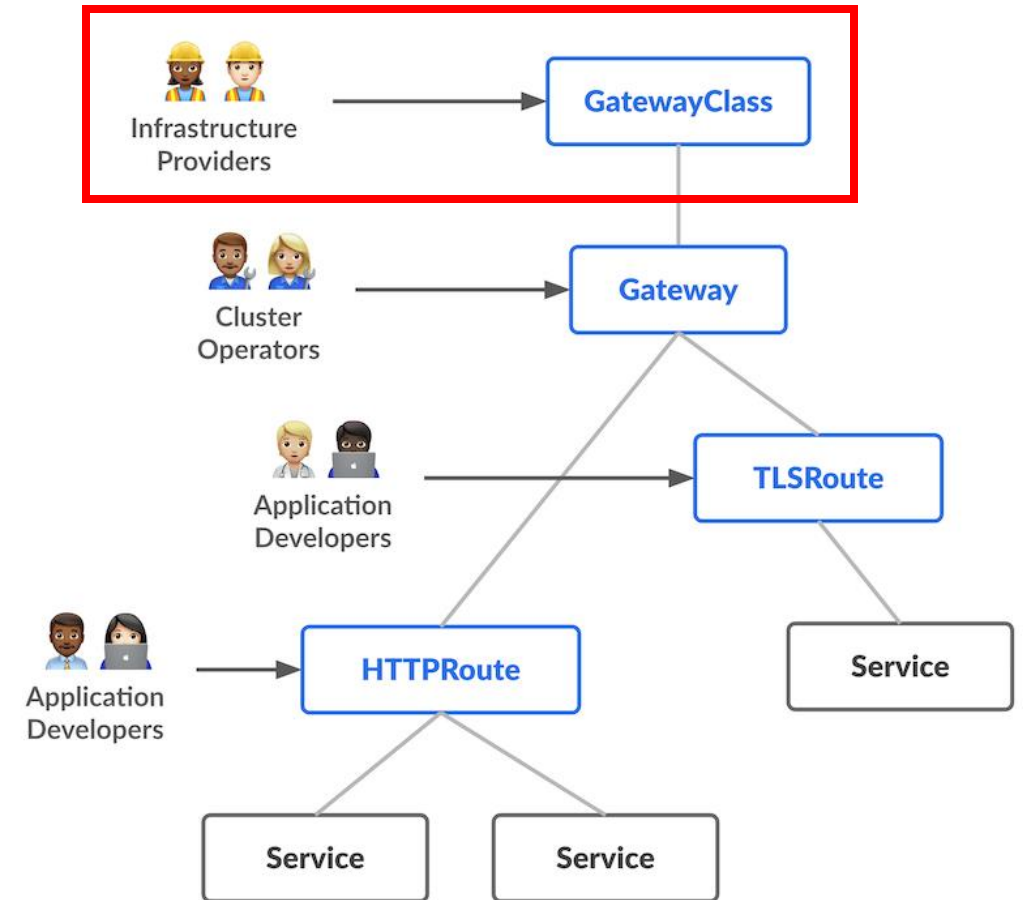
Gateways

Gateways

- Kubernetes bietet zwei Ressourcen zum Routing von externem Traffic
- Ingress ist der ältere Ressourcentyp. Er ist mittlerweile im Maintenance Mode.
- Die moderne Variante ist das **Gateway**

Gateways

Ähnlich wie bei Storage konfiguriert der Administrator einen GatewayClass.



GatewayClasses

- GatewayClasses werden vom Infrastruktur-Provider verwaltet
- Unterschiedliche GatewayClasses werden zur Unterscheidung von Einstiegspunkten in den Cluster genutzt
- Denkbar ist zum Beispiel eine GatewayClass für das Internet und eine für ein internes Netzwerk

```
kind: GatewayClass
```

```
metadata:
```

```
  name: internet
```

```
  ...
```

```
---
```

```
kind: GatewayClass
```

```
metadata:
```

```
  name: private
```

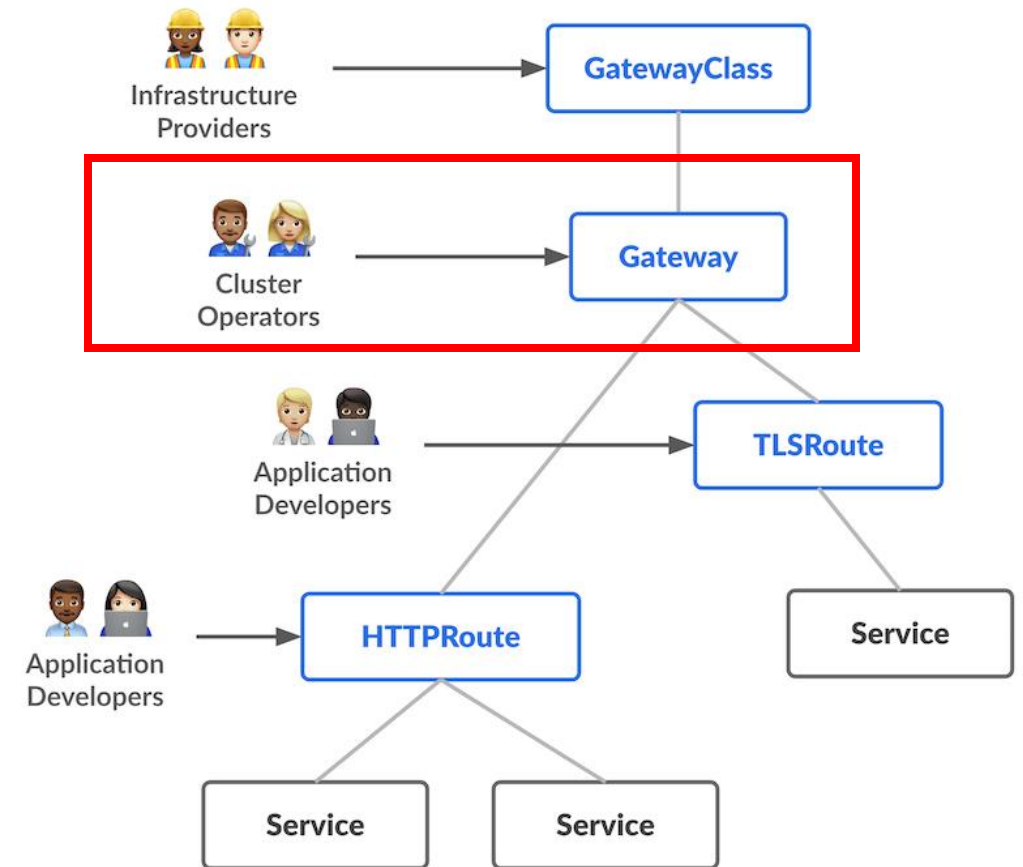
```
...
```

Beispiel: AWS GatewayClass

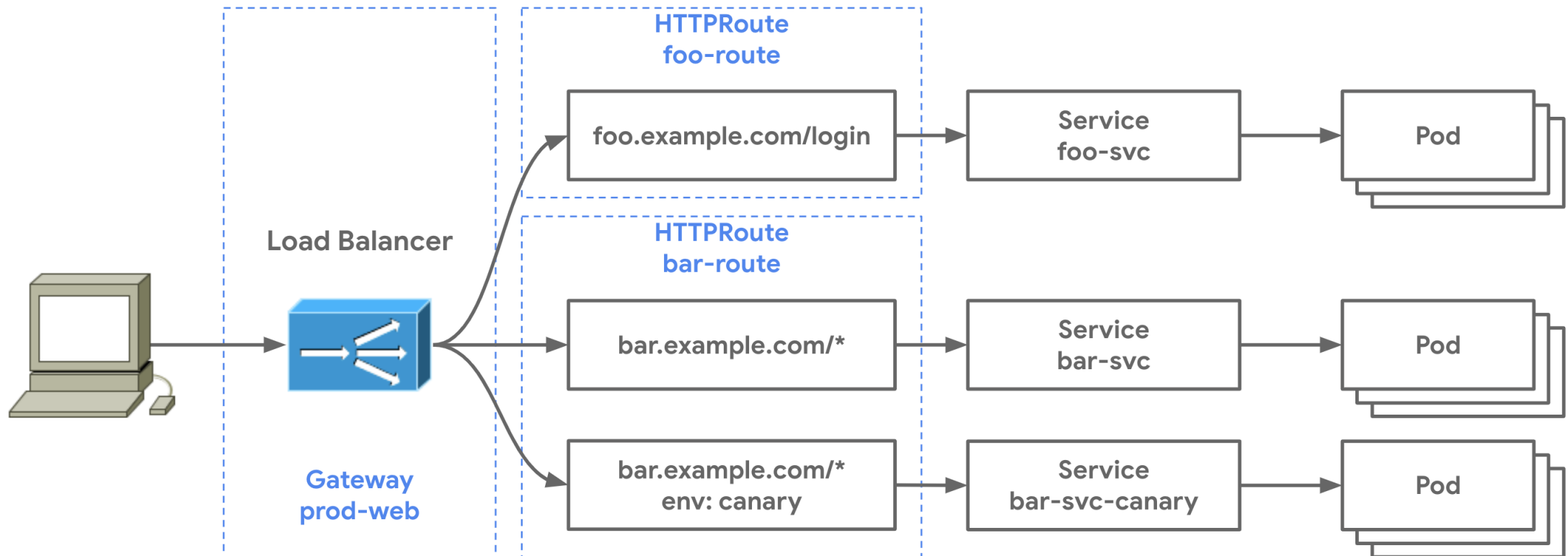
```
apiVersion: gateway.networking.k8s.io/v1beta1
kind: GatewayClass
metadata:
  name: amazon-vpc-lattice
spec:
  controllerName: application-networking.k8s.aws/gateway-api-controller
```

Gateways

Administratoren legen das Gateway an, welches dann von den Entwicklungsteams mit Routen bestückt wird.



Gateways und Routes

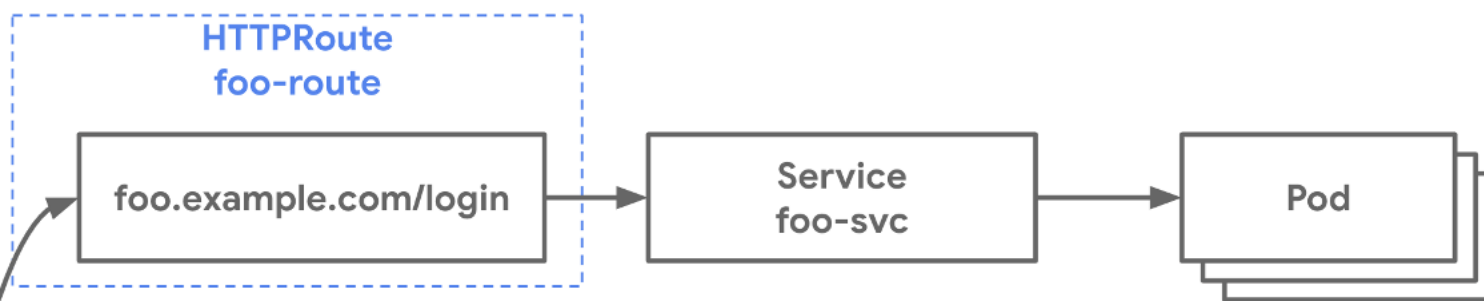


Gateway

Das Gateway definiert den Port und die Gatewayklasse.

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: example-gateway
spec:
  gatewayClassName: example-gateway-class
  listeners:
    - name: http
      protocol: HTTP
      port: 80
```

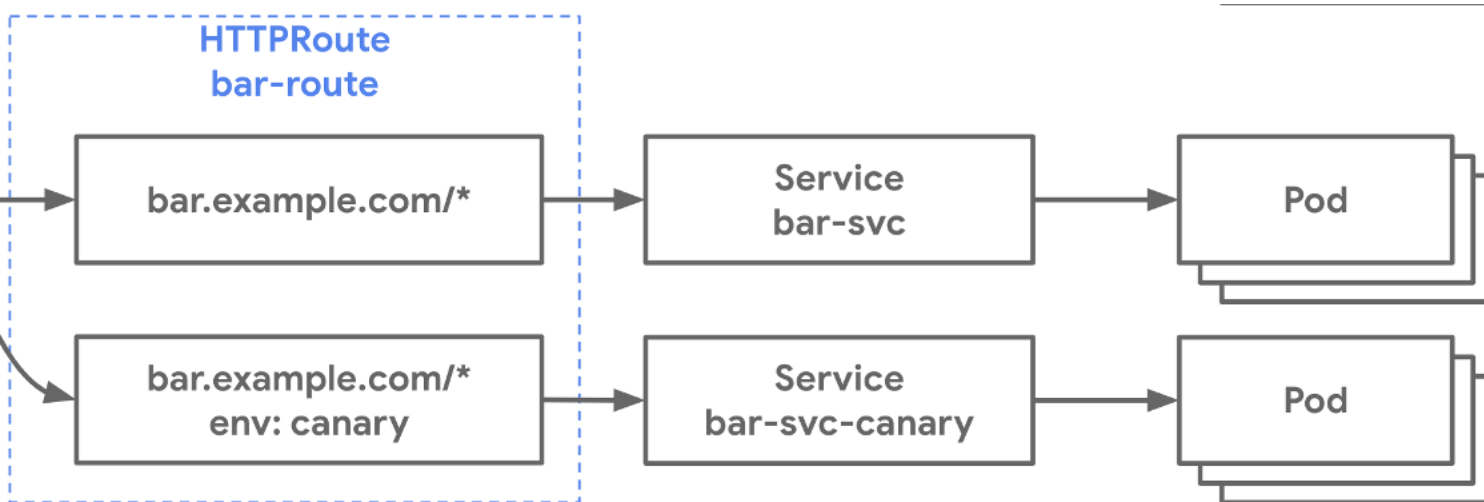
Routen



Eine einzelne Route enthält Regeln zum Matching auf Hostnames, Pfade und Header. Sie definieren den Ziel-Service.

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: foo-route
spec:
  parentRefs:
  - name: example-gateway
  hostnames:
  - "foo.example.com"
  rules:
  - matches:
    - path:
        type: PathPrefix
        value: /login
    backendRefs:
    - name: foo-svc
      port: 8080
```

Routen mit Header



In diesem Beispiel wird ein Header benutzt, um eine Canary-Variante des Services anzusteuern.

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: bar-route
spec:
  parentRefs:
  - name: example-gateway
  hostnames:
  - "bar.example.com"
  rules:
  - matches:
    - headers:
      - type: Exact
        name: env
        value: canary
    backendRefs:
    - name: bar-svc-canary
      port: 8080
  - backendRefs:
    - name: bar-svc
      port: 8080
```

Advanced Networking

Service Meshes

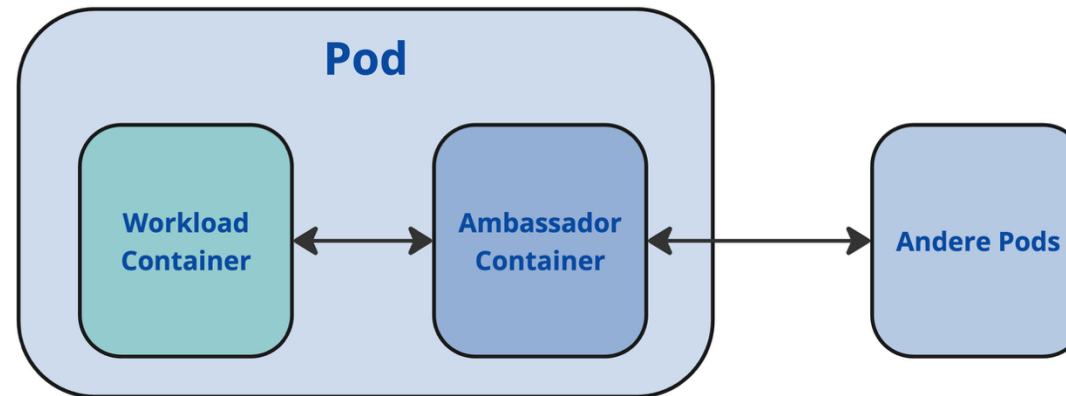
Service Meshes

- Service Meshes bieten eine weitere Netzwerkebene zur granularen Traffic-Kontrolle zwischen Pods
- Ermöglichen Rate-Limiting, Logging und Debugging von Netzwerkverkehr
- Namensverwirrung: Kein Bezug zu „echten“ Kubernetes-Services

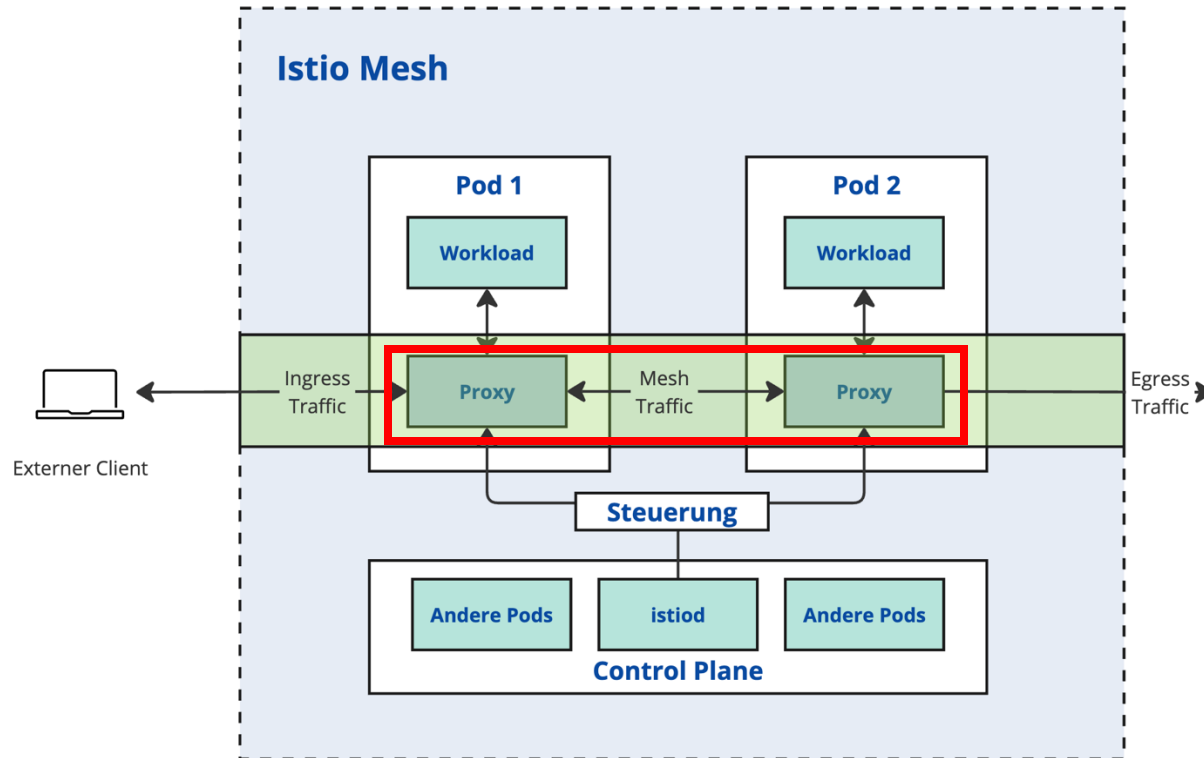


Service Meshes: Ambassador Pattern

Meshes werden realisiert über Proxies, die in jeden Pod injiziert werden. Man spricht vom Ambassador Pattern.

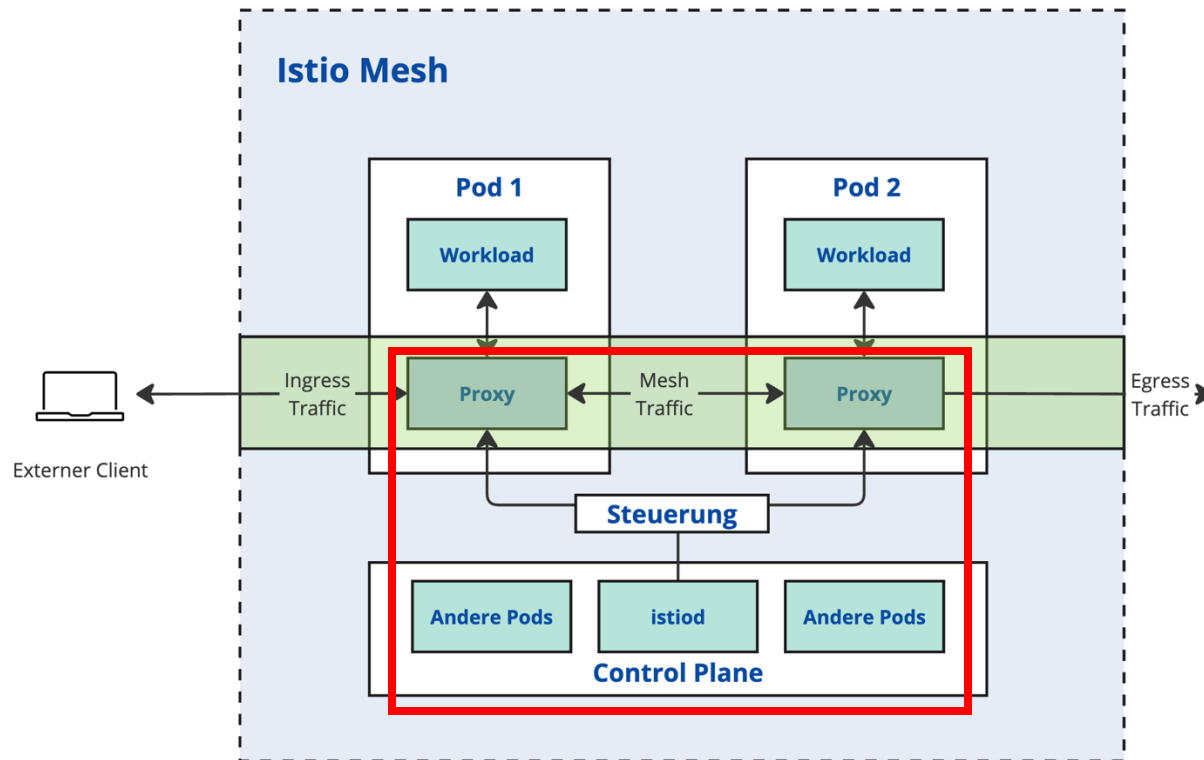


Istio Mesh-Aufbau



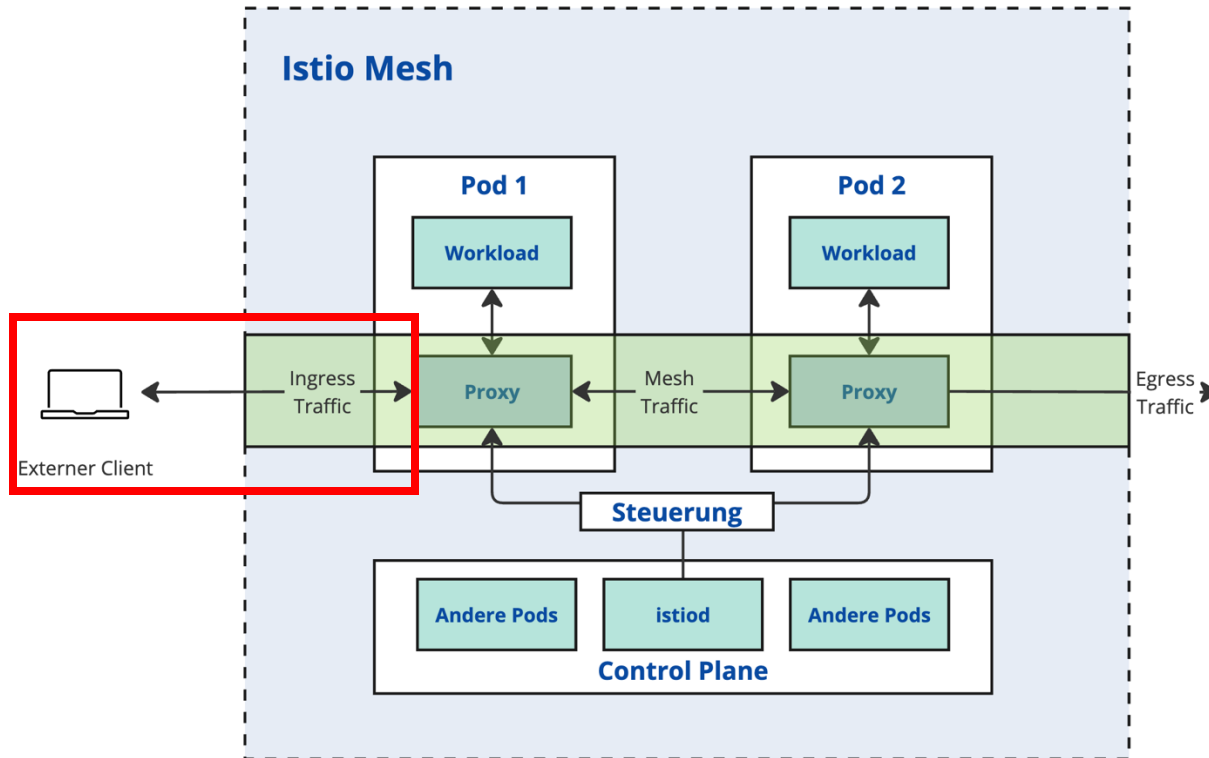
Jeglicher Traffic der eigentlichen Workload-Container läuft über Proxies. Diese kommunizieren über das Pod-Netzwerk miteinander.

Istio Mesh-Aufbau



Istiod ist ein Dienst, der in der Control Plane läuft. Er steuert die Proxies und stattet sie mit Zertifikaten aus, um sie miteinander zu authentifizieren.

Istio Mesh-Aufbau



Meshes werden nach außen über einen Kubernetes Ingress oder über ein Gateway angebunden.

VirtualServices

- Ein **VirtualService** in Istio ist eine Kubernetes-Ressource, die verwendet wird, um den Datenverkehr in einem Service Mesh zu steuern.
- In Istio können wir mithilfe von VirtualServices bestimmen, wie Anfragen an einen Dienst weitergeleitet werden,
- Regeln können basieren auf URI-Pfaden, Headern, HTTP-Methoden, usw.



Istio Virtual Services

hosts konfiguriert die (DNS-) Namen, unter denen der Service erreichbar ist im Istio-Netzwerk.

gateways bindet den Service an ein Kubernetes-Gateway für externen Traffic

http enthält konfigurierbare Regeln die bestimmen, welche Pfade und Ports auf welche Ziele geleitet werden.

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: httpbin
spec:
  hosts:
  - "httpbin.example.com"
  gateways:
  - httpbin-gateway
  http:
  - match:
    - uri:
        prefix: /status
    - uri:
        prefix: /delay
    route:
    - destination:
        port:
          number: 8000
        host: httpbin
```

Mesh: Ja oder nein?

- Services Meshes sind kein must have
- Nützlich sind sie besonders für Traffic Shaping
- Wenn einzelne Anwendungen durch Rate Limits intern geschützt werden müssen, sind Meshes stark
- Auch Auditing, Tracing und Debugging ist durch Meshes leichter
- Aber: Sie kosten Latenz und verbrauchen Ressourcen

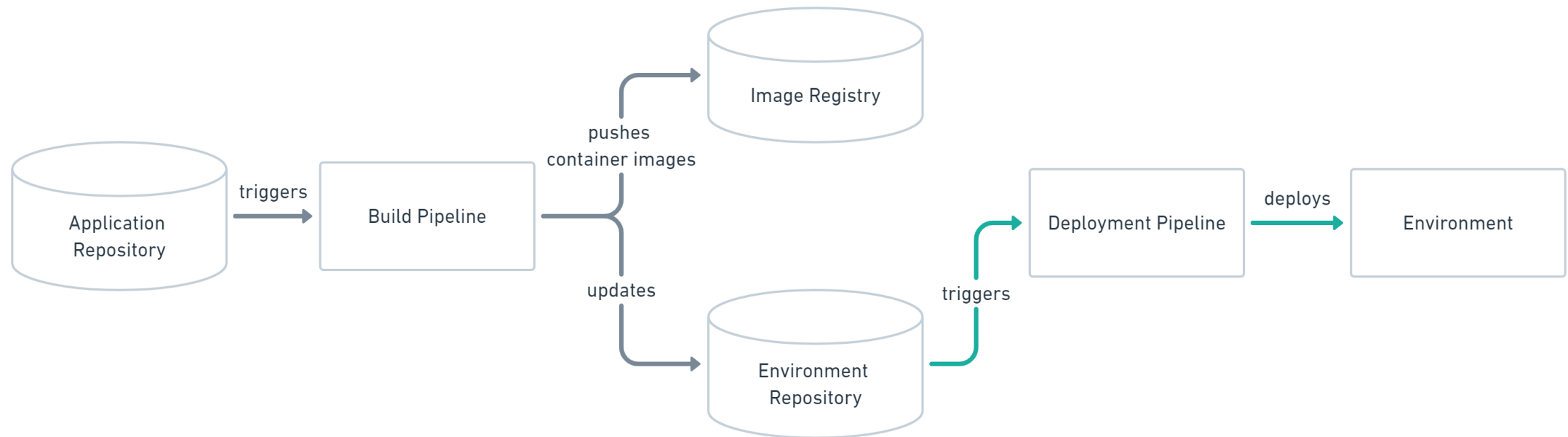
CI und CD in Kubernetes

GitOps

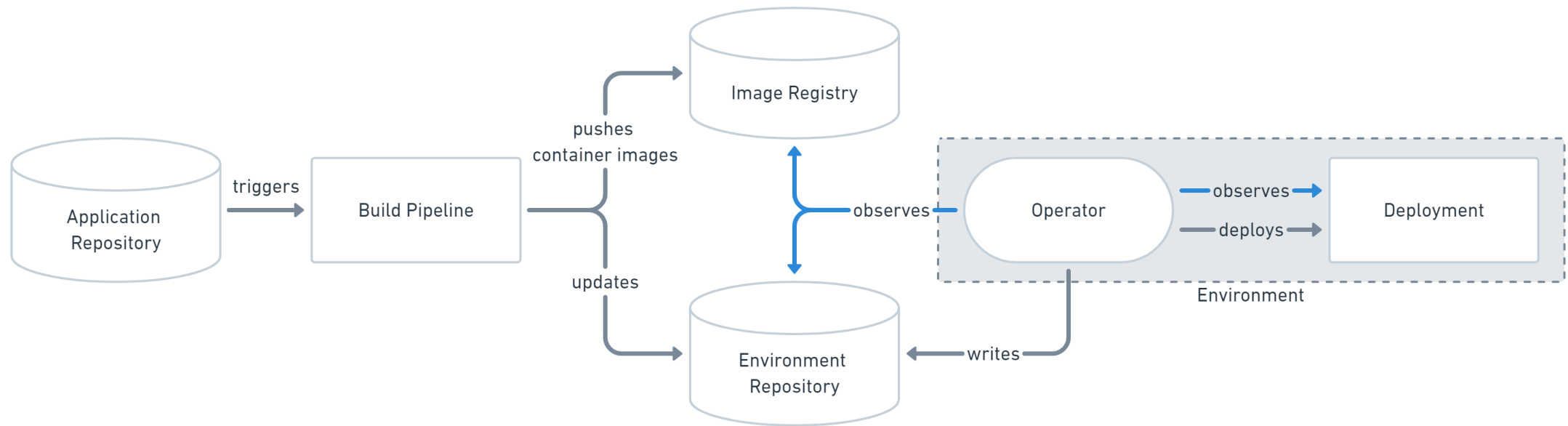
- Das GitOps-Konzept basiert auf der Idee von *zentralen Repositories*
- Enthalten Source-Code, Artefakte, Images, Manifeste und Konfigurationen
- Operatoren stellen sicher, dass die Builds vollständig, korrekt und getestet sind



Push Based Build & Deployment



Pull Based Build & Deployment



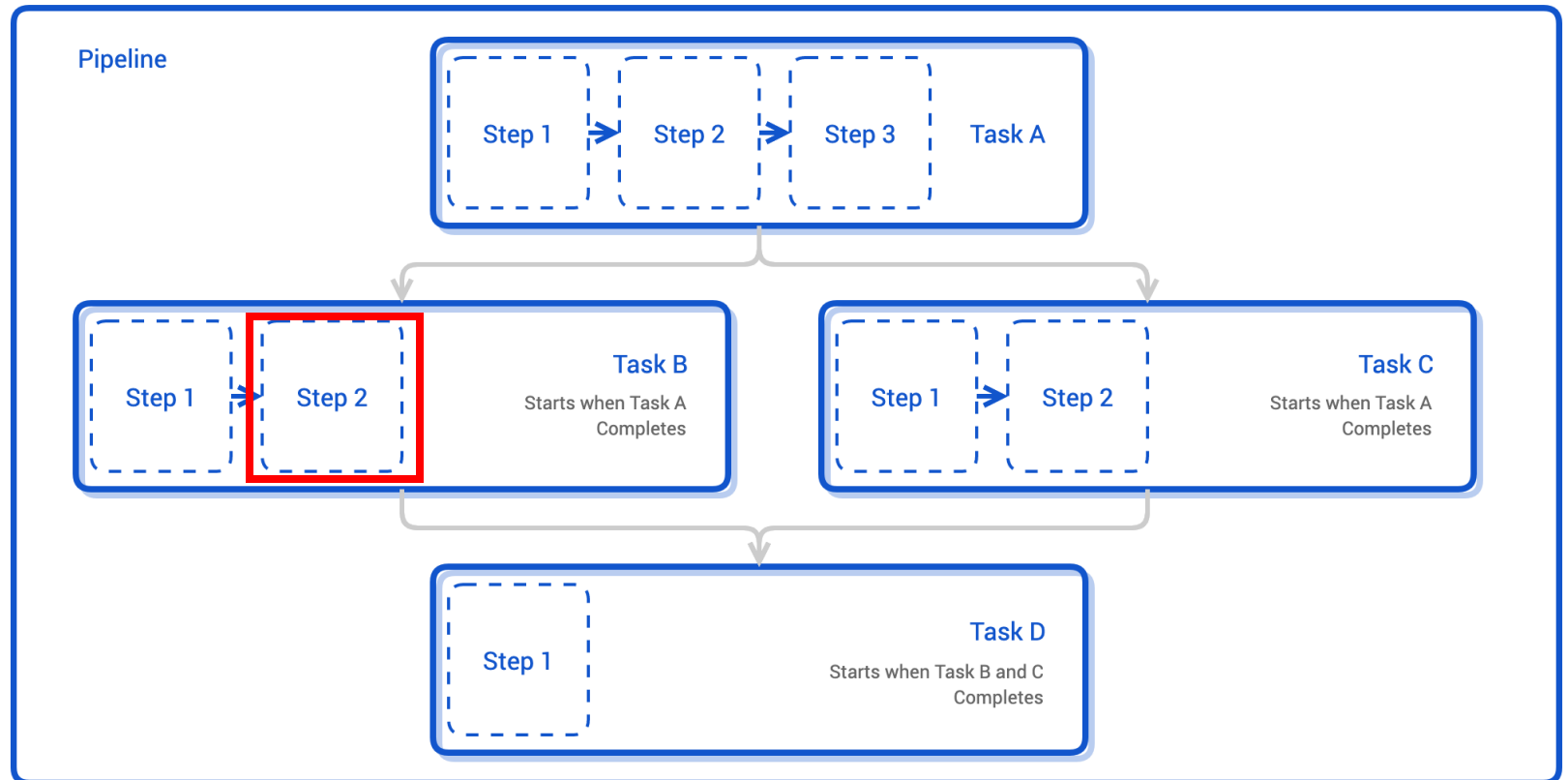
Tekton

- Tekton ist ein CI/CD-System, das in Kubernetes betrieben wird
- Tasks, Pipelines und Pipeline-Runs sind Kubernetes-Ressourcen
- Besteht aus CRDs und Operatoren zum Build und Deployment von Images und anderen Aufgaben



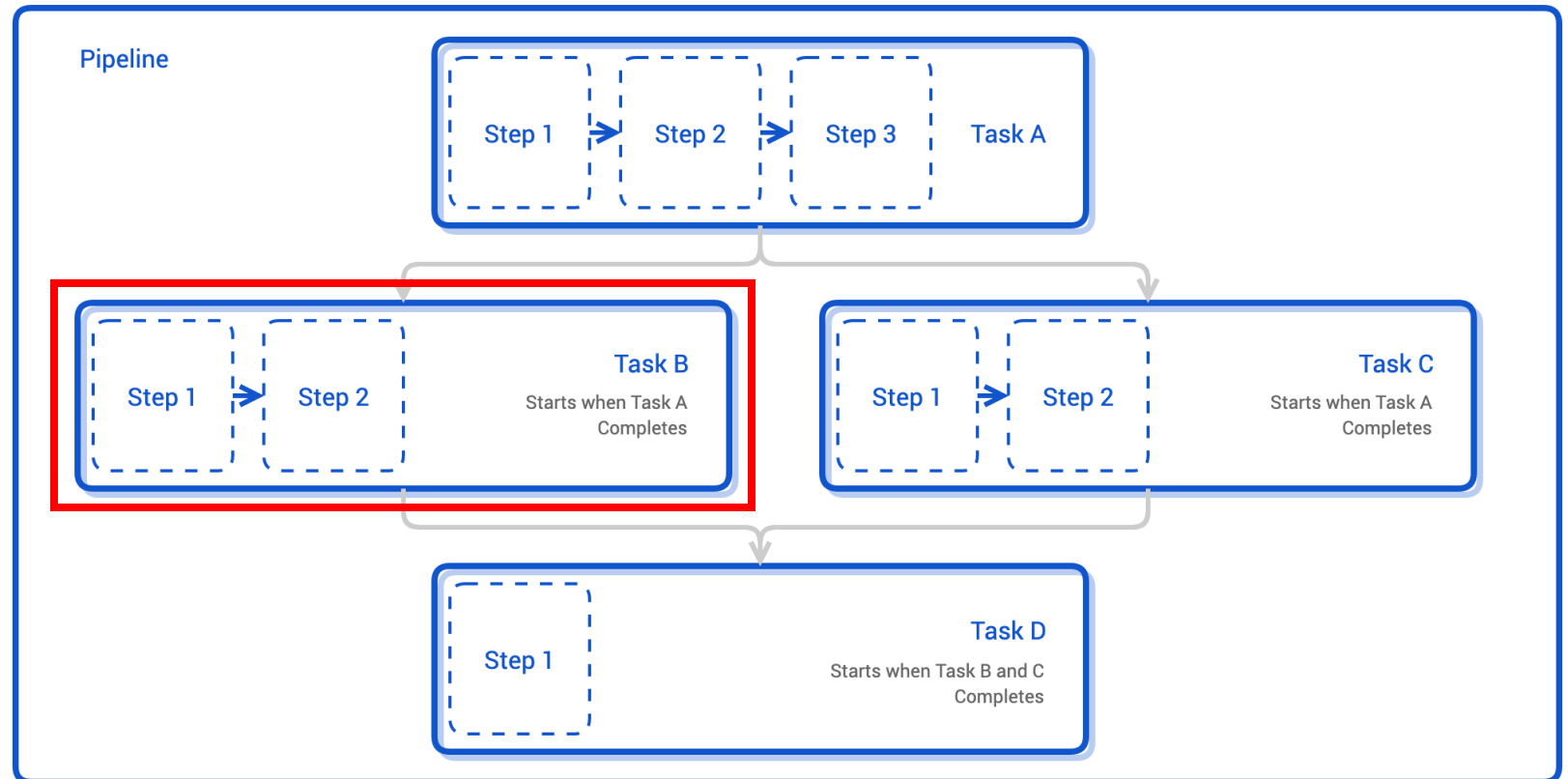
Tekton Task-Modell

Ein Step in Tekton ist eine granulare Aufgabe. Beispiel wäre die Ausführung von Unit Tests. Steps laufen in einem Container mit einem beliebigen Image.



Tekton Task-Modell

Ein Task ist eine Sammlung von Steps. Sie werden als Pod realisiert. Das hat den Vorteil, dass allen Tasks ein Volume zugeteilt werden kann.

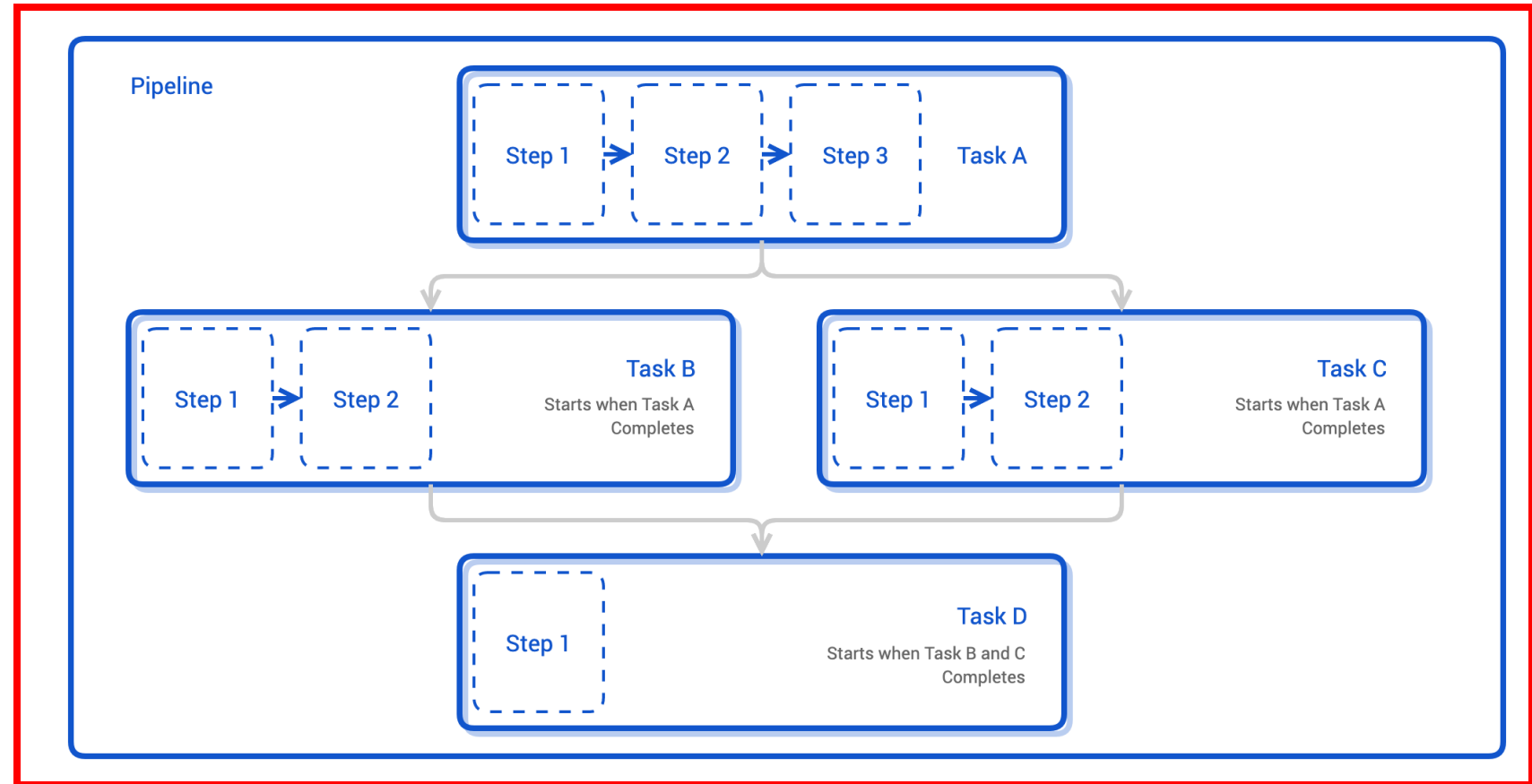


Task – Beispiel

```
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: goodbye
spec:
  params:
    - name: username
      type: string
  steps:
    - name: goodbye
      image: ubuntu
      script: |
        #!/bin/bash
        echo "Goodbye $(params.username)!"
```


Tekton Task-Modell

Eine Pipeline ist eine Sammlung von Tasks, die in einer Abhängigkeit zueinander stehen.

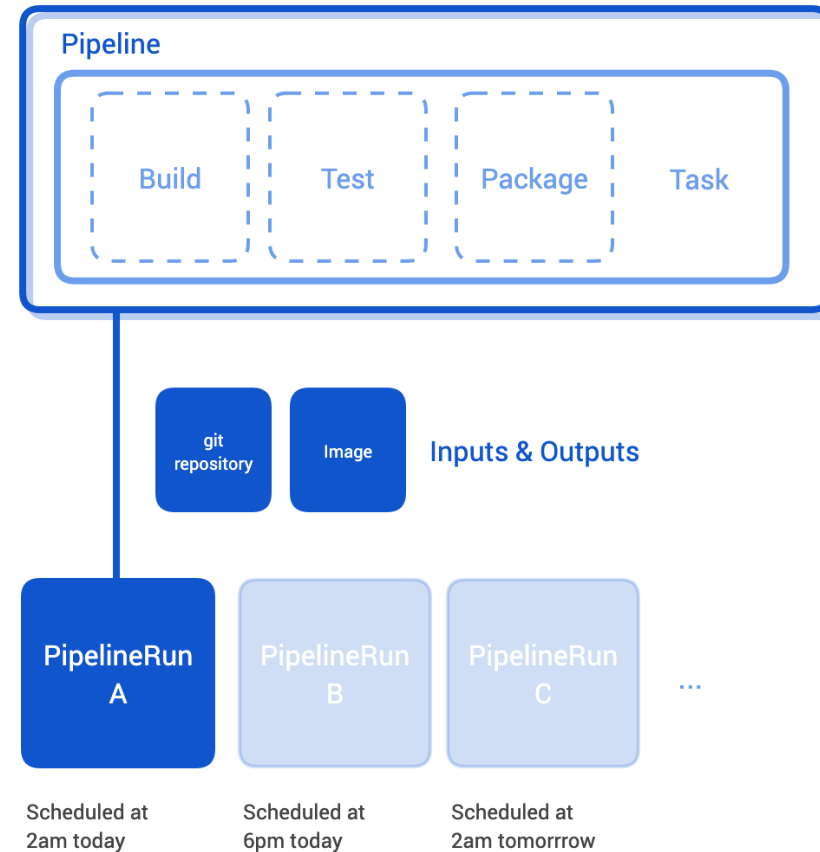


Pipeline – Beispiel

```
apiVersion: tekton.dev/v1beta1
kind: Pipeline
metadata:
  name: hello-goodbye
spec:
  params:
    - name: username
      type: string
  tasks:
    - name: hello
      taskRef:
        name: hello
    - name: goodbye
      runAfter:
        - hello
      taskRef:
        name: goodbye
      params:
        - name: username
          value: $(params.username)
```

Pipeline Run

Pipelines sind nur Vorlagen für die Ausführung eines Builds. Zur Ausführung kommen sie, wenn ein PipelineRun-Objekt erzeugt wird.

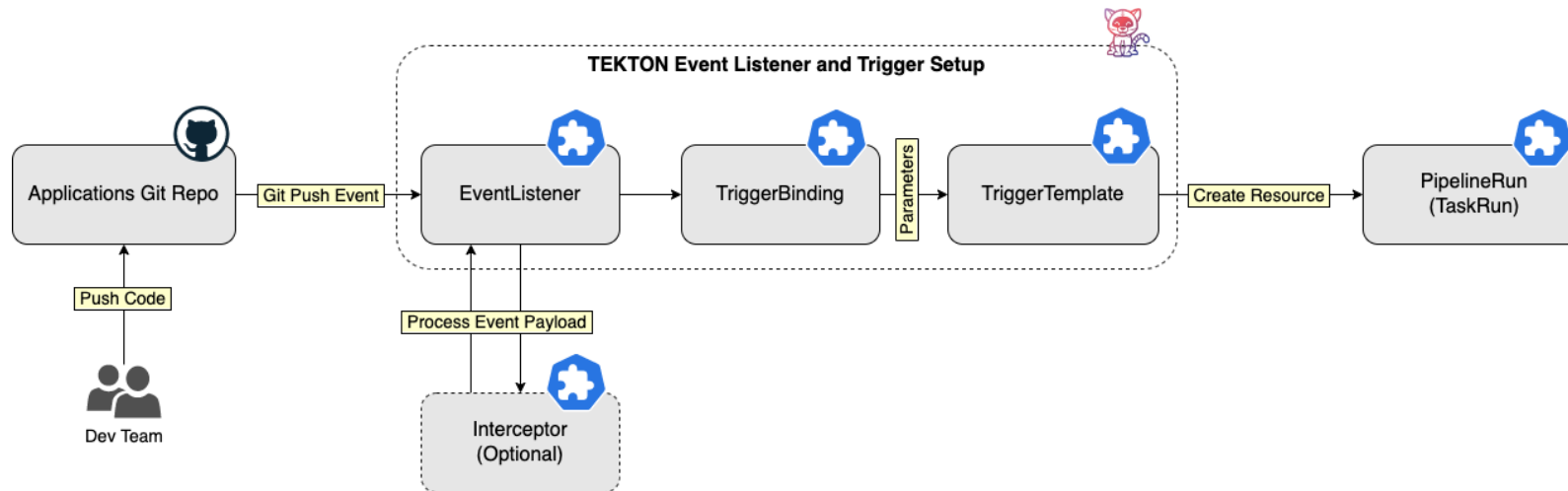


PipelineRun

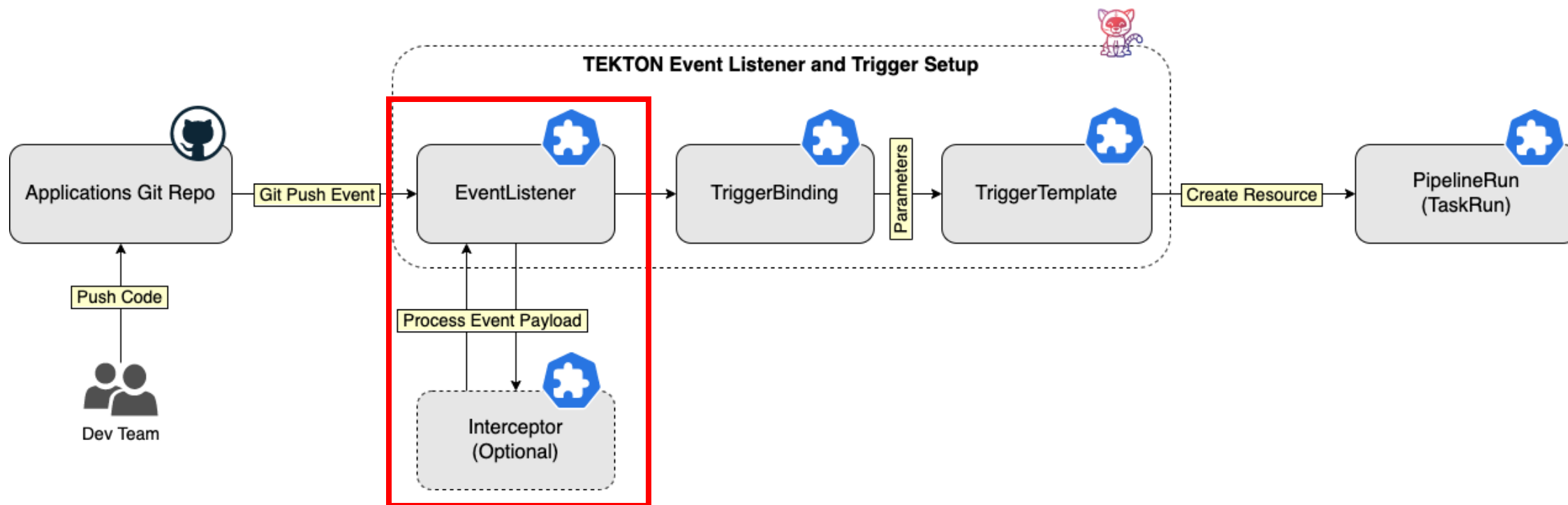
```
apiVersion: tekton.dev/v1beta1
kind: PipelineRun
metadata:
  name: hello-goodbye-run
spec:
  pipelineRef:
    name: hello-goodbye
  params:
    - name: username
      value: "Tekton"
```

Tekton Triggers

Tekton besitzt einen höchst flexiblen Trigger-Mechanismus. Mit ihm können Events aus verschiedenen Quellen entgegengenommen werden. Beispiele: GitHub, GitLab, CodeCommit, Slack...



EventListener



Der EventListener definiert die Quelle des Triggers.

EventListener

Ein Eventlistener wird automatisch mit einem **Service** vom Typ Loadbalancer nach außen verfügbar gemacht.

Er wird über einen **WebHook** aufgerufen z.B. aus GitHub.

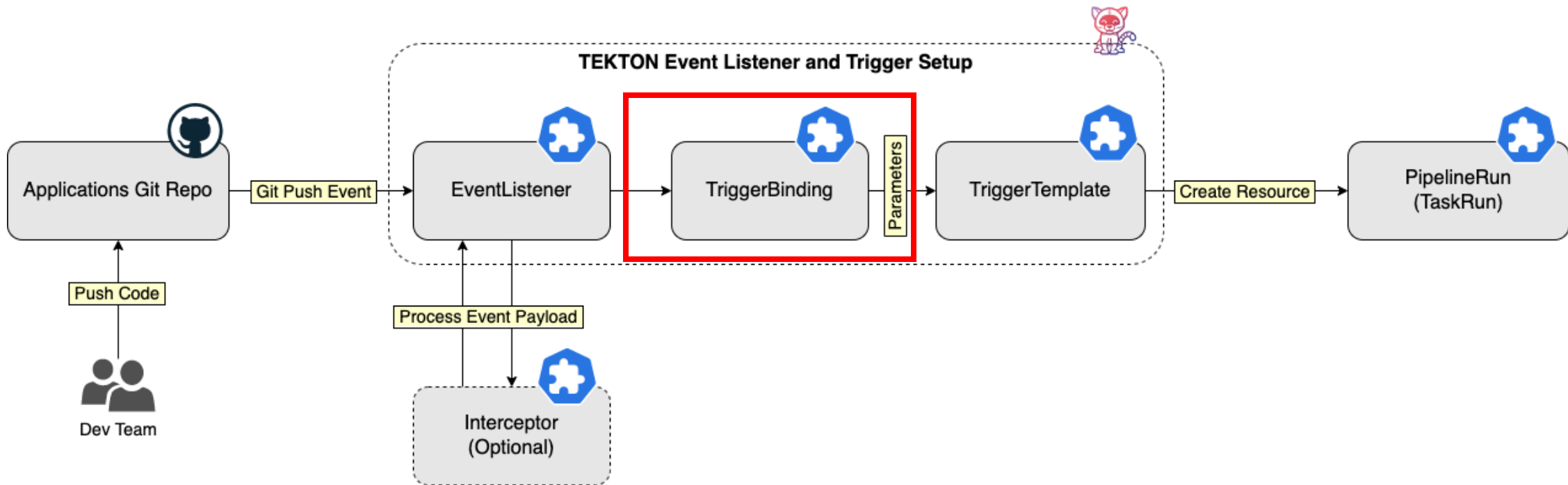
```
apiVersion: triggers.tekton.dev/v1beta1
kind: EventListener
metadata:
  name: github-listener
spec:
  triggers:
    - name: github-listener
      interceptors:
        - ref:
            name: "github"
          params:
            - name: "secretRef"
              value:
                secretName: github-secret
                secretKey: secretToken
            - name: "eventTypes"
              value: ["pull_request"]
        - name: "only when PRs are opened"
          ref:
            name: "cel"
          params:
            - name: "filter"
              value: "body.action in ['opened', 'synchronize', 'reopened']"
      bindings:
        - ref: github-pr-binding
      template:
        ref: github-template
```

EventListener

Interceptors
validieren und
filtern die
eingehenden
Requests.

```
apiVersion: triggers.tekton.dev/v1beta1
kind: EventListener
metadata:
  name: github-listener
spec:
  triggers:
  - name: github-listener
    interceptors:
    - ref:
        name: "github"
      params:
        - name: "secretRef"
          value:
            secretName: github-secret
            secretKey: secretToken
        - name: "eventTypes"
          value: ["pull_request"]
    - name: "only when PRs are opened"
      ref:
        name: "cel"
      params:
        - name: "filter"
          value: "body.action in ['opened', 'synchronize', 'reopened']"
    bindings:
    - ref: github-pr-binding
  template:
    ref: github-template
```


TriggersBinding

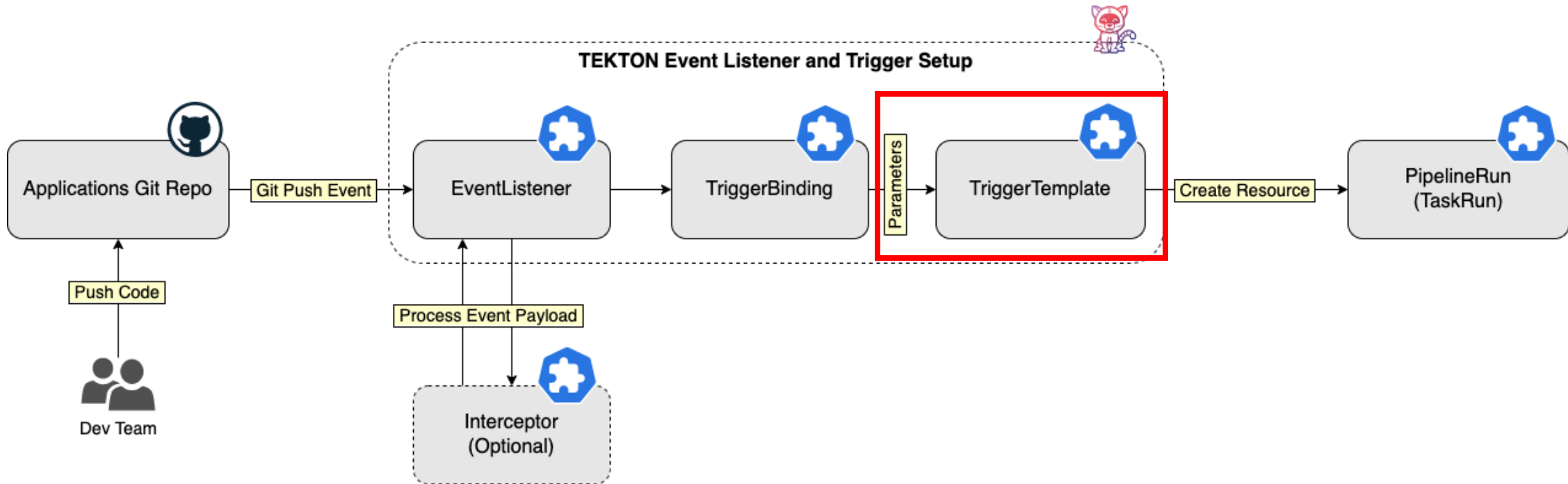


Das TriggerBinding bindet die Daten der eingehenden Events an Build-Variablen.

TriggerBinding

```
apiVersion: triggers.tekton.dev/v1beta1
kind: TriggerBinding
metadata:
  name: github-pr-binding
spec:
  params:
    - name: gitrevision
      value: $(body.pull_request.head.sha)
    - name: gitrepositoryurl
      value: $(body.repository.clone_url)
```

TriggerTemplate



Das TriggerTemplate definiert, wie welche Pipeline oder welcher Task ausgeführt werden soll.

TriggerBinding

```
apiVersion: triggers.tekton.dev/v1beta1
kind: TriggerTemplate
metadata:
  name: github-template
spec:
  params:
    - name: gitrevision
    - name: gitrepositoryurl
  resourcetemplates:
    - apiVersion: tekton.dev/v1beta1
      kind: TaskRun
      metadata:
        generateName: github-run-
      spec:
        taskSpec:
          steps:
            - image: ubuntu
              script: |
                #! /bin/bash
                echo "Revision is : $(tt.params.gitrevision). RepoURL is $(tt.params.gitrepositoryurl)"
```

Demo

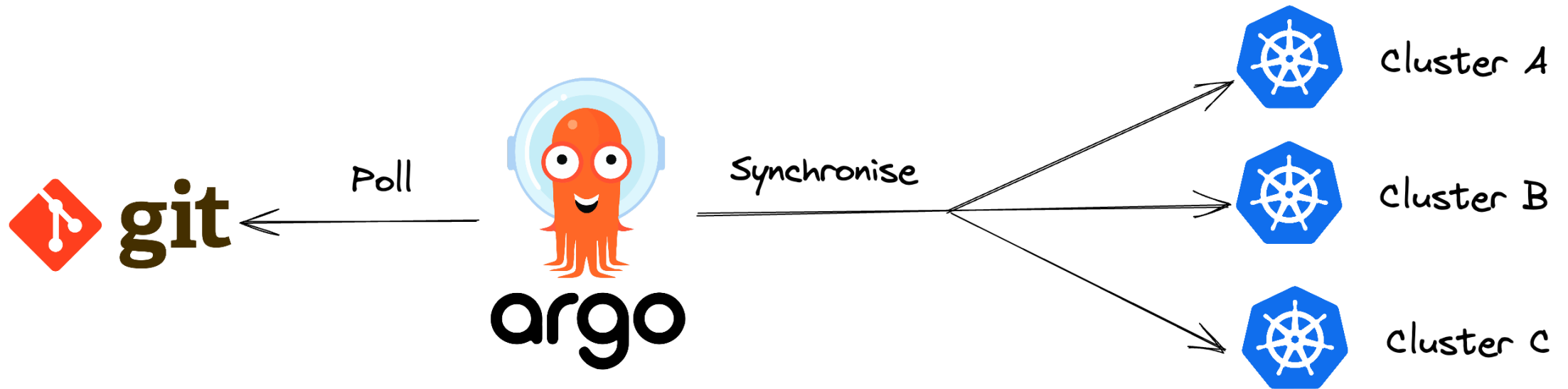
Tekton Pipelines

Pipeline-Workflows

- Tekton eignet sich ideal für CI.
- Der letzte Schritt ist dann üblicherweise, dass ein neues Image gepusht und das Kubernetes-Manifest z.B. eines Deployments angepasst wird.
- Für das eigentliche Ausspielen der Manifeste wird z.B. ArgoCD eingesetzt.

ArgoCD

ArgoCD ist wie Tekton ein Projekt mit CRDs und Operatoren. Es wird eingesetzt, um in git gespeicherte Kubernetes-Manifeste kontinuierlich auf Cluster auszuspielen.



Argo Application

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: my-application
spec:
  project: default
  source:
    repoURL: 'https://github.com/your-repo.git'
    targetRevision: HEAD
    path: 'path/to/manifests'
  destination:
    server: 'https://kubernetes.default.svc'
    namespace: your-namespace
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```


Tools

Helm

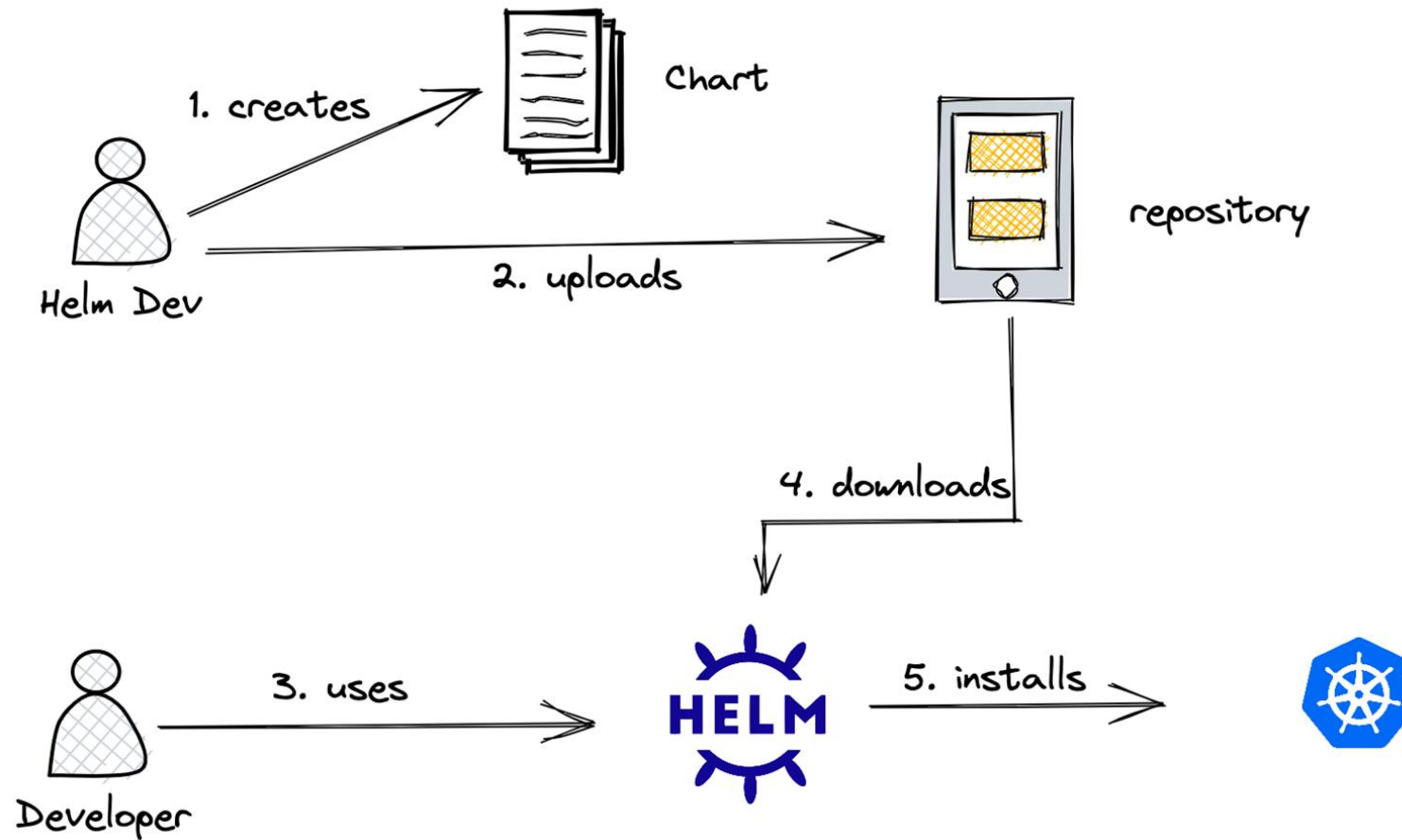
Was ist Helm?

- Paketmanager für Kubernetes („apt“ oder „yum“ für K8s)
- Verwaltet Deployments als **Charts** (Sammlung von YAML-Templates + Werte)
- Erleichtert Installation, Upgrade und Rollback komplexer Anwendungen

Warum Helm?

- **Wiederverwendbarkeit:** Standardisierte Charts für häufige Apps (Nginx, MySQL, Prometheus, ...)
- **Konfigurierbarkeit:** Werte können über values.yaml oder CLI überschrieben werden
- **Versionierung & Releases:** Einfaches Up- und Downgrade per helm upgrade und helm rollback

Helm Workflow



Helm in Aktion

```
$ helm repo add bitnami https://charts.bitnami.com/bitnami
```

```
$ helm install my-nginx bitnami/nginx
```

```
$ helm list
```

```
$ helm upgrade my-nginx bitnami/nginx
```

```
$ helm rollback my-nginx 1
```